PENJADWALAN MESIN CETAK FLEXOGRAFI DENGAN *SIMULATED ANNEALING* UNTUK EFISIENSI PRODUKSI PADA LINGKUNGAN STATIS DAN DINAMIS

SKRIPSI

Untuk memenuhi sebagian persyaratan
memperoleh gelar Sarjana Komputer

Disusun oleh:
Javier Aahmes Reansyah
NIM: 225150201111001

PROGRAM STUDI TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026

PENGESAHAN

PENJADWALAN MESIN CETAK FLEXOGRAFI DENGAN *SIMULATED ANNEALING* UNTUK EFISIENSI PRODUKSI PADA LINGKUNGAN STATIS DAN DINAMIS

SKRIPSI

Diajukan untuk memenuhi sebagian persyaratan
memperoleh gelar Sarjana Komputer

Disusun Oleh :
Javier Aahmes Reansyah
NIM: 225150201111001

Skripsi ini telah diuji dan dinyatakan lulus pada
8 Januari 2026
Telah diperiksa dan disetujui oleh:

Dosen Pembimbing I	Dosen Pembimbing 2
Dr., Ir. Diva Kurnianingtyas, S.Kom.	Ir. Agus Wahyu Widodo, S.T., M.Cs.
NIP: 2022019612132001	NIP: 197408052001121001

Mengetahui
Ketua Jurusan Teknik Informatika

Rekyan Regasari Mardi Putri, S.T., M.T., Ph.D.
NIP: 2011027704142001

PERNYATAAN ORISINALITAS

Saya menyatakan dengan sebenar-benarnya bahwa sepanjang pengetahuan saya, di dalam naskah skripsi ini tidak terdapat karya ilmiah yang pernah diajukan oleh orang lain untuk memperoleh gelar akademik di suatu perguruan tinggi, dan tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis disitasi dalam naskah ini dan disebutkan dalam daftar referensi.

Apabila ternyata didalam naskah skripsi ini dapat dibuktikan terdapat unsur-unsur plagiasi, saya bersedia skripsi ini digugurkan dan gelar akademik yang telah saya peroleh (sarjana) dibatalkan, serta diproses sesuai dengan peraturan perundang-undangan yang berlaku (UU No. 20 Tahun 2003, Pasal 25 ayat 2 dan Pasal 70).

Malang, 11 Januari 2026

Javier Aahmes Reansyah

NIM: 225150201111001

PRAKATA

Puji syukur ke hadirat Allah Subhanahu wa ta'ala atas limpahan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul "Penjadwalan Mesin Cetak Flexografi dengan *Simulated Annealing* untuk Efisiensi Produksi pada Lingkungan Statis dan Dinamis". Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Komputer pada Program Studi Teknik Informatika, Fakultas Ilmu Komputer, Universitas Brawijaya.

Penyusunan skripsi ini tidak terlepas dari bantuan, dukungan, serta bimbingan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan rasa terima kasih yang sebesar-besarnya kepada:

1. Kedua orang tua penulis yang senantiasa memberikan doa, dukungan, serta nasihat selama perkuliahan hingga penyusunan skripsi ini dapat diselesaikan.
2. Ibu Dr. Ir. Diva Kurnianingtyas, S.Kom. selaku dosen pembimbing I sekaligus dosen pembimbing akademik, serta Bapak Ir. Agus Wahyu Widodo, S.T., M.Cs. selaku dosen pembimbing II, yang telah meluangkan waktu dan memberikan bimbingan dengan penuh kesabaran selama proses penyusunan skripsi ini.
3. Bapak Bayu Priyambadha, S.Kom., M.Kom., Ph.D. selaku Ketua Program Studi Teknik Informatika Fakultas Ilmu Komputer Universitas Brawijaya.
4. Ibu Rekyan Regasari Mardi Putri, S.T., M.T., Ph.D. selaku Ketua Departemen Teknik Informatika Fakultas Ilmu Komputer Universitas Brawijaya.
5. Seluruh dosen dan civitas akademik Fakultas Ilmu Komputer Universitas Brawijaya yang telah memberikan ilmu, bimbingan, serta pengalaman berharga selama masa studi penulis.
6. Seluruh sahabat dan rekan penulis yang telah menemani dan memberikan dukungan selama perjalanan perkuliahan.
7. Seluruh pihak yang telah membantu penulis, baik secara langsung maupun tidak langsung, dalam proses penyusunan skripsi ini.

Penulis menyadari bahwa skripsi ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran sangat diharapkan demi perbaikan di masa mendatang. Semoga skripsi ini dapat memberikan manfaat dan berguna bagi pembaca.

Malang, 11 Januari 2026

Penulis

javierreansyah@student.ub.ac.id

ABSTRAK

Javier Aahmes Reansyah, Penjadwalan Mesin Cetak Flexografi dengan *Simulated Annealing* untuk Efisiensi Produksi pada Lingkungan Statis dan Dinamis

Pembimbing: Dr., Ir. Diva Kurnianingtyas, S.Kom. dan Ir. Agus Wahyu Widodo, S.T., M.Cs.

Proses produksi kemasan karton bergelombang menggunakan mesin cetak flexografi menghadapi inefisiensi tinggi akibat pergantian tinta yang bersifat *sequence-dependent*, terutama pada transisi dari tinta gelap ke tinta terang yang memerlukan pembersihan intensif yang berdampak pada waktu henti serta limbah. Kompleksitas ini semakin meningkat karena perbedaan kecepatan antar mesin dan potensi kedatangan pesanan secara bertahap, sehingga penjadwalan manual sulit menghasilkan solusi yang efisien. Penelitian ini bertujuan untuk memformulasi dan mengevaluasi penjadwalan produksi flexografi yang efektif pada lingkungan statis dan dinamis dengan memodelkan permasalahan sebagai *uniform parallel machine scheduling* dengan *sequence-dependent setup times*. Pendekatan yang diusulkan menggunakan *Simulated Annealing* sebagai metode optimasi utama dengan skema pendinginan geometrik, kriteria penerimaan probabilistik, berbagai operator *neighborhood*, dan inisialisasi jadwal menggunakan prosedur *greedy*. Untuk menangani kedatangan pekerjaan dinamis, algoritma diintegrasikan dalam kerangka *rolling horizon* dengan mekanisme *lock-unlock* serta penetapan *due date* berbasis beban kerja. Evaluasi dilakukan menggunakan dataset industri nyata yang terdiri dari 15 pekerjaan dan 3 mesin melalui berbagai skenario statis dan dinamis serta pengujian parameter dan bobot tujuan. Hasil eksperimen menunjukkan bahwa pendekatan yang diusulkan stabil dan mampu meningkatkan kinerja penjadwalan secara signifikan dibandingkan praktik industri, termasuk penurunan penalti pergantian tinta hingga 87,5%, pengurangan *makespan* sekitar 2,5 jam, serta peningkatan laba bersih sekitar Rp11,2 juta. Pada skenario dinamis, konfigurasi terbaik mampu menghilangkan keterlambatan pekerjaan dengan penalti pergantian tinta yang tetap terkendali. Hasil ini mengonfirmasi bahwa pendekatan *Simulated Annealing* dan *rolling horizon* yang diusulkan merupakan metode yang efektif, *robust*, dan efisien secara komputasi untuk mengoptimalkan kinerja operasional pada produksi flexografi.

Kata kunci: *uniform parallel machine scheduling*, percetakan flexografi, *Simulated Annealing*, *rolling horizon*, *sequence-dependent setup*, pergantian tinta

ABSTRACT

Javier Aahmes Reansyah, Flexographic Printing Machine Scheduling Using Simulated Annealing for Production Efficiency in Static and Dynamic Environments

Supervisors: Dr., Ir. Diva Kurnianingtyas, S.Kom. and Ir. Agus Wahyu Widodo, S.T., M.Cs.

The production of corrugated cardboard packaging using flexographic printing machines faces significant inefficiencies due to sequence-dependent ink changeovers, particularly during transitions from dark to light inks that require intensive cleaning, which results in significant downtime and waste. This complexity is further exacerbated by differences in machine speeds and the potential staggered arrival of orders, making manual scheduling difficult to achieve efficient solutions. This study aims to formulate and evaluate an effective flexographic production scheduling for both static and dynamic environments by modeling the problem as uniform parallel machine scheduling with sequence-dependent setup times. The proposed approach employs Simulated Annealing as the main optimization method, incorporating a geometric cooling scheme, probabilistic acceptance criteria, multiple neighborhood operators, and greedy-based schedule initialization. To address dynamic job arrivals, the algorithm is integrated within a rolling horizon framework using a lock-unlock mechanism and workload-based due date assignment. The evaluation is conducted using a real industrial dataset consisting of 15 jobs and 3 machines across various static and dynamic scenarios, along with parameter and objective-weight testing. Experimental results demonstrate that the proposed approach is stable and capable of significantly improving scheduling performance compared to industrial practice, including a reduction in ink changeover penalties of up to 87.5 percent, a makespan reduction of approximately 2.5 hours, and an increase in net profit of around IDR 11.2 million. In dynamic scenarios, the best configuration is able to eliminate job tardiness while maintaining controlled ink changeover penalties. These results confirm that the proposed Simulated Annealing and rolling horizon approach is an effective, robust, and computationally efficient method for optimizing operational performance in flexographic manufacturing.

Keywords: uniform parallel machine scheduling, flexographic printing, Simulated Annealing, rolling horizon, sequence-dependent setup, ink changeover

DAFTAR ISI

PENGESAHAN ..... **ii**
PERNYATAAN ORISINALITAS ..... **iii**
PRAKATA ..... **iv**
ABSTRAK ..... **v**
ABSTRACT ..... **vi**
DAFTAR ISI ..... **vii**
DAFTAR TABEL ..... **xiii**
DAFTAR GAMBAR ..... **xvii**
DAFTAR LAMPIRAN ..... **xix**
BAB 1 PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan 3
1.4 Manfaat 3
1.5 Batasan Masalah 3
1.6 Sistematika Pembahasan 4
BAB 2 LANDASAN KEPUSTAKAAN **5**
2.1 Kajian Pustaka 5
2.2 Proses Flexografi 9
2.2.1 Mesin Flexografi dan Prinsip Kerjanya 9
2.2.2 Proses Produksi pada Industri 9
2.3 *Uniform Parallel Machine Scheduling* 10
2.3.1 Klasifikasi Masalah 10
2.3.2 Kompleksitas Masalah 11
2.4 Warna dan Tingkat Kegelapan 11
2.5 *Simulated Annealing* 12
2.5.1 Inisialisasi Suhu Awal 13
2.5.2 Kriteria Penerimaan 14
2.5.3 Skema Pendinginan 14
2.5.4 Kriteria Penghentian 15

2.6 Tujuan Penjadwalan 15
2.6.1 *Sequence-Dependent Setup Times* 15
2.6.2 *Makespan* 15
2.6.3 Tardiness dan Due Dates 16
2.6.4 Fungsi Objektif 17
2.7 Penjadwalan Statis dan Dinamis 18
2.8 Penjadwalan *Rolling Horizon* 18
2.9 Metrik Evaluasi Kinerja 18
2.9.1 Profit 19
2.9.2 *Fitness Score* 19
2.9.3 Metrik Spesifik Tujuan 19
2.9.4 Utilisasi Mesin 20
2.9.5 *Robustness* 21
2.9.6 Efisiensi Komputasi 22
BAB 3 METODOLOGI PENELITIAN 23
3.1 Tipe Penelitian 23
3.2 Strategi Penelitian 23
3.3 Peralatan Pendukung 24
3.4 Studi Literatur 24
3.5 Dataset dan Praproses 25
3.6 Pemodelan Masalah 25
3.7 Kerangka *Simulated Annealing* 26
3.8 Kerangka *Rolling Horizon* 26
3.9 Rencana Eksperimen 27
3.10 Evaluasi dan Visualisasi 28
3.10.1 Metrik 28
3.10.2 Visualisasi 28
3.11 Penarikan Kesimpulan 29
BAB 4 PERANCANGAN 30
4.1 Dataset dan Praproses 30
4.1.1 Dataset Pekerjaan 30
4.1.2 Data Spesifikasi Mesin 30

4.1.3 Biaya dan Durasi SDST 31
4.1.4 Jadwal Industri *Baseline* 31
4.2 Pemodelan Masalah 31
4.2.1 Lingkungan Produksi 32
4.2.2 *Sequence Dependent Setup* 32
4.2.3 Fungsi Objektif 32
4.2.4 Penjadwalan Statis dan Dinamis 33
4.2.5 Kendala Penjadwalan 33
4.2.6 Asumsi dan Batasan 34
4.3 Representasi Solusi dan Konfigurasi 34
4.3.1 Representasi Solusi 34
4.3.2 *State* Mesin 35
4.3.3 Konfigurasi 36
4.4 Desain Algoritma *Simulated Annealing* 38
4.4.1 Hitung *Fitness* 39
4.4.2 Model *Changeover* 42
4.4.3 Inisialisasi Solusi 44
4.4.4 Struktur *Neighborhood* 46
4.4.5 Kriteria Penerimaan 53
4.4.6 Pendinginan 54
4.4.7 Inisialisasi Temperatur 55
4.4.8 Kriteria Penghentian 56
4.5 Perancangan Algoritma *Rolling Horizon* 57
4.5.1 Kebijakan *Lock/Unlock* 58
4.5.2 State Mesin 59
4.5.3 Pembangunan Data Mesin untuk Optimasi 61
4.5.4 Penetapan Due Date 63
4.6 Perhitungan Manual 68
4.6.1 Data untuk Perhitungan Manual 68
4.6.2 Persiapan *Rolling Horizon - Horizon 1* 69
4.6.3 *Simulated Annealing - Horizon* 1 70
4.6.4 Persiapan *Rolling Horizon - Horizon* 2 75

4.6.5 *Simulated Annealing - Horizon* 2 78
4.6.6 Jadwal Akhir dan Evaluasi *Fitness* 79
4.7 Rancangan Pengujian 80
4.7.1 Definisi Skenario 80
4.7.2 Pengujian Parameter *Simulated Annealing* 80
4.7.3 Pengujian Bobot Tujuan 82
4.7.4 Pengujian *Due Date Multiplier* 84
4.7.5 Pengujian Skenario 84
4.7.6 Evaluasi Jadwal Terbaik 85
4.7.7 Perbandingan Jadwal Optimasi dengan Jadwal Industri 87
BAB 5 IMPLEMENTASI 89
5.1 *Import Library*, Definisi Struktur Data, Dan Evaluasi Solusi 89
5.1.1 Kelas Konfigurasi Optimasi 89
5.1.2 Representasi Pekerjaan 90
5.1.3 Representasi Solusi Jadwal 91
5.1.4 Fungsi Perhitungan *Sequence Dependent Setup* 92
5.1.5 Fungsi Perhitungan *Fitness* Jadwal 93
5.2 Implementasi Algoritma SA 94
5.2.1 Pembuatan Solusi Awal 94
5.2.2 Struktur *Neighborhood* 96
5.2.3 Estimasi Suhu Awal 98
5.2.4 Algoritma *Simulated Annealing* 99
5.3 Implementasi *Rolling Horizon Scheduler* 100
5.3.1 State Mesin dan Mekanisme *Lock-Unlock* 100
5.3.2 Pembangunan Data Mesin untuk Optimasi Ulang 102
5.3.3 Perhitungan Beban Kerja dan Penetapan Due Date 103
5.3.4 Algoritma *Rolling Horizon* Utama 105
BAB 6 HASIL DAN PEMBAHASAN 108
6.1 Pengujian Parameter SA 108
6.1.1 Pengujian Parameter *Max Iteration* 108
6.1.2 Pengujian Parameter *Alpha* 110
6.1.3 Pengujian Parameter P0 111

6.1.4 Pengujian Parameter *Floor Ratio* 112
6.1.5 Pengujian Parameter *Critical Machine Bias* 113
6.1.6 Ringkasan dan Analisis Hasil Pengujian Parameter SA 114
6.2 Pengujian Bobot 115
6.2.1 Hasil Pengujian Bobot Skenario 3..... 115
6.2.2 Hasil Pengujian Bobot Skenario 4B 116
6.2.3 Ringkasan dan Analisis Hasil Pengujian Bobot 118
6.3 Pengujian *Due Date Multiplier*..... 119
6.3.1 Hasil Pengujian *Due Date Multiplier*..... 119
6.3.2 Analisis Hasil Pengujian *Due Date Multiplier*..... 120
6.4 Pengujian Skenario 120
6.4.1 Ringkasan Statistik Pengujian Skenario 121
6.4.2 Analisis Konvergensi Skenario..... 122
6.4.3 Analisis Hasil Pengujian Skenario 125
6.5 Pengujian Jadwal Terbaik 125
6.5.1 Hasil Pengujian Jadwal Terbaik Skenario 1..... 125
6.5.2 Hasil Pengujian Jadwal Terbaik Skenario 2..... 127
6.5.3 Hasil Pengujian Jadwal Terbaik Skenario 3..... 129
6.5.4 Hasil Pengujian Jadwal Terbaik Skenario 4A 130
6.5.5 Hasil Pengujian Jadwal Terbaik Skenario 4B..... 132
6.5.6 Hasil Pengujian Jadwal Terbaik Skenario 4C..... 134
6.5.7 Ringkasan dan Analisis Hasil Pengujian Jadwal Terbaik..... 136
6.6 Perbandingan Jadwal Hasil Optimasi dengan Jadwal Industri 136
6.6.1 Hasil Perbandingan Jadwal Hasil Optimasi dengan Jadwal Industri 137
6.6.2 Analisis Perbandingan Jadwal Optimasi dengan Jadwal Industri 138
6.7 Ringkasan Pengujian..... 140
6.8 Keterbatasan Model 141
BAB 7 PENUTUP..... 142
7.1 Kesimpulan..... 142
7.2 Saran 142

DAFTAR REFERENSI..... 143
LAMPIRAN..... 145

DAFTAR TABEL

Tabel 2.1 Kajian Pustaka 7
Tabel 3.1 Perangkat Keras Pendukung 24
Tabel 4.1 Data Pekerjaan dengan Warna yang Dikodekan 30
Tabel 4.2 Data Spesifikasi Mesin 30
Tabel 4.3 Data Jadwal Industri *Baseline* 31
Tabel 4.4 Representasi Solusi 34
Tabel 4.5 Representasi Pekerjaan 35
Tabel 4.6 Representasi State Mesin 35
Tabel 4.7 Konfigurasi *Optimizer* 36
Tabel 4.8 Data Pekerjaan untuk Perhitungan Manual 68
Tabel 4.9 Data Mesin untuk Perhitungan Manual 68
Tabel 4.10 Parameter untuk Perhitungan Manual 68
Tabel 4.11 *Visible Jobs* pada *Horizon* 1 69
Tabel 4.12 Data Mesin untuk Optimasi *Horizon* 1 69
Tabel 4.13 *Due Dates* Untuk Pekerjaan di *Horizon* 1 70
Tabel 4.14 Penugasan *Greedy* untuk *Horizon* 1 71
Tabel 4.15 Jadwal Awal *Greedy* untuk *Horizon* 1 71
Tabel 4.16 Rekonstruksi *Timeline* Jadwal *Greedy Horizon 1* 71
Tabel 4.17 Evaluasi *Tardiness* Jadwal *Greedy Horizon 1* 72
Tabel 4.18 Evaluasi *Changeover* Jadwal *Greedy Horizon 1* 72
Tabel 4.19 Komposisi Tinta antara *Job* 1 [6, 8] dan *Job* 2 [6, 6] 73
Tabel 4.20 Pasangan Penghapusan dan Penambahan antara *Job* 1 [6, 8] dan *Job* 2 [6, 6] 73
Tabel 4.21 Contoh *Neighbor Move* pada *Greedy Schedule Horizon* 1 73
Tabel 4.22 Perbandingan Nilai *Fitness* untuk Estimasi Temperatur Awal 74
Tabel 4.23 Log Iterasi SA *Horizon* 1 75
Tabel 4.24 Jadwal Akhir *Horizon* 1 75
Tabel 4.25 *Visible Jobs* pada *Horizon* 2 76
Tabel 4.26 *State* Mesin pada *Horizon* 2 76
Tabel 4.27 *Initial State* untuk *Optimizer* pada *Horizon* 2 76

Tabel 4.28 Data Mesin untuk Optimasi *Horizon* 2 77
Tabel 4.29 *Visible Jobs* pada *Horizon* 2 77
Tabel 4.30 Jadwal *Greedy* untuk *Horizon* 2 78
Tabel 4.31 Log Iterasi SA *Horizon* 2 78
Tabel 4.32 Jadwal Akhir *Horizon* 2 79
Tabel 4.33 Jadwal Akhir dari Seluruh *Horizon* 79
Tabel 4.34 Metrik Akhir dari Seluruh *Horizon* 79
Tabel 4.35 Definisi Skenario Eksperimen 80
Tabel 4.36 Rencana Pengujian Parameter SA 81
Tabel 4.37 Rencana Tata Letak Data Mentah untuk Pengujian Parameter SA 81
Tabel 4.38 Rencana Ringkasan Statistik untuk Pengujian Parameter SA 81
Tabel 4.39 Rencana Tata Letak Data Mentah untuk Pengujian Bobot Tujuan 82
Tabel 4.40 Rencana Ringkasan Statistik untuk Pengujian Bobot Tujuan S3 83
Tabel 4.41 Rencana Ringkasan Statistik untuk Pengujian Bobot Tujuan S4B 83
Tabel 4.42 Rencana Tata Letak Data Mentah untuk Pengujian *Due Date Multiplier* 84
Tabel 4.43 Rencana Ringkasan Statistik untuk Pengujian *Due Date Multiplier* 84
Tabel 4.44 Rencana Tata Letak Data Mentah untuk Pengujian Skenario 85
Tabel 4.45 Rencana Ringkasan Statistik untuk Pengujian Skenario 85
Tabel 4.46 Rencana Tata Letak Metrik Optimasi Jadwal Terbaik 86
Tabel 4.47 Rencana Tata Letak Metrik Utilisasi Mesin Jadwal Terbaik 86
Tabel 4.48 Rencana Tata Letak Metrik Keuangan Jadwal Terbaik 86
Tabel 4.49 Rencana Tata Letak Jadwal Rinci Jadwal Terbaik 87
Tabel 4.50 Rencana Tata Letak Perbandingan Metrik Optimasi 88
Tabel 4.51 Rencana Tata Letak Perbandingan Utilisasi Mesin 88
Tabel 4.52 Rencana Tata Letak Perbandingan Metrik Keuangan 88
Tabel 4.53 Rencana Tata Letak Jadwal Rinci Jadwal Industri 88
Tabel 5.1 Kode Sumber Importasi *Library* yang Diperlukan 89
Tabel 5.2 Kode Sumber Struktur Data *Optimizer Config* 89
Tabel 5.3 Kode Sumber Struktur Data Pekerjaan 90
Tabel 5.4 Kode Sumber Struktur Data Representasi Solusi 91
Tabel 5.5 Kode Sumber Algoritma Pergantian Tinta 92

Tabel 5.6 Kode Sumber Algoritma Hitung *Fitness* 93
Tabel 5.7 Kode Sumber Algoritma Buat Solusi Awal 94
Tabel 5.8 Kode Sumber Algortima Buat Solusi Tetangga 96
Tabel 5.9 Kode Sumber Algoritma Estimasi Temperatur Awal 98
Tabel 5.10 Kode Sumber Algoritma *Simulated Annealing* 99
Tabel 5.11 Kode Sumber Struktur Data *State* Mesin 100
Tabel 5.12 Kode Sumber Algoritma Buat *State* Mesin 101
Tabel 5.13 Kode Sumber Algoritma Buat Data Mesin 102
Tabel 5.14 Kode Sumber Hitung *Workload* 103
Tabel 5.15 Kode Sumber Algoritma Penetapan Due Date 104
Tabel 5.16 Kode Sumber Algoritma *Rolling Horizon* 105
Tabel 6.1 Statistik Hasil Pengujian Parameter *Max Iteration* 108
Tabel 6.2 Statistik Hasil Pengujian Parameter *Alpha* 110
Tabel 6.3 Statistik Hasil Pengujian Parameter P0 111
Tabel 6.4 Statistik Hasil Pengujian Parameter *Floor Ratio* 112
Tabel 6.5 Statistik Hasil Pengujian Parameter *Critical Machine Bias* 113
Tabel 6.6 Parameter SA Terbaik 115
Tabel 6.7 Statistik Hasil Pengujian Bobot Pada Skenario 3 115
Tabel 6.8 Statistik Sepuluh Kombinasi Bobot Terbaik pada Skenario 4B 116
Tabel 6.9 Bobot Terbaik Fungsi Objektif 119
Tabel 6.10 Statistik Hasil Pengujian Parameter *Due Date Multiplier S*4A-S4C ... 119
Tabel 6.11 Statistik Hasil Percobaan Skenario 121
Tabel 6.12 Metrik Optimasi Skenario 1 125
Tabel 6.13 Metrik Utilisasi Mesin Skenario 1 126
Tabel 6.14 Jadwal Skenario 1 126
Tabel 6.15 Metrik Optimasi Skenario 2 127
Tabel 6.16 Metrik Utilisasi Mesin Skenario 2 127
Tabel 6.17 Metrik Keuangan Skenario 2 127
Tabel 6.18 Jadwal Skenario 2 128
Tabel 6.19 Metrik Optimasi Skenario 3 129
Tabel 6.20 Metrik Utilisasi Mesin Skenario 3 129
Tabel 6.21 Metrik Keuangan Skenario 3 129

Tabel 6.22 Jadwal Skenario 3 129
Tabel 6.23 Metrik Optimasi Skenario 4A 130
Tabel 6.24 Metrik Utilisasi Mesin Skenario 4A 130
Tabel 6.25 Metrik Keuangan Skenario 4A 131
Tabel 6.26 Jadwal Skenario 4A 131
Tabel 6.27 Metrik Optimasi Skenario 4B 132
Tabel 6.28 Metrik Utilisasi Mesin Skenario 4B 132
Tabel 6.29 Metrik Keuangan Skenario 4B 132
Tabel 6.30 Jadwal Skenario 4B 133
Tabel 6.31 Metrik Optimasi Skenario 4C 134
Tabel 6.32 Metrik Utilisasi Mesin Skenario 4C 134
Tabel 6.33 Metrik Keuangan Skenario 4C 134
Tabel 6.34 Jadwal Skenario 4C 135
Tabel 6.35 Ringkasan Metrik Inti Pengujian Jadwal Terbaik Setiap Skenario 136
Tabel 6.36 Perbandingan Metrik Utama Jadwal Optimasi dengan Jadwal Industri 137
Tabel 6.37 Perbandingan Utilisasi Mesin Jadwal Optimasi dengan Jadwal Industri 137
Tabel 6.38 Perbandingan Metrik Keuangan Hasil Optimasi dengan Jadwal Industri 137
Tabel 6.39 Jadwal Industri 138

DAFTAR GAMBAR

Gambar 2.1 Proses Flexografi 9
Gambar 3.1 Strategi Penelitian 23
Gambar 4.1 *Flowchart* Algoritma SA 38
Gambar 4.2 *Flowchart* Algoritma Perhitungan *Fitness* 41
Gambar 4.3 *Flowchart* Algoritma Hitung Perubahan Tinta 43
Gambar 4.4 *Flowchart* Algoritma Inisialisasi Solusi *Greedy* 45
Gambar 4.5 *Flowchart* Algoritma Buat *Neighbor* 47
Gambar 4.6 *Flowchart* Algoritma Swap Within 48
Gambar 4.7 *Flowchart* Algoritma Swap *Between* 49
Gambar 4.8 *Flowchart* Algoritma *Relocate* 50
Gambar 4.9 *Flowchart* Algoritma *Exchange* 51
Gambar 4.10 *Flowchart* Algoritma *Block Insert* 52
Gambar 4.11 *Flowchart* Algoritma *Metropolis Acceptance Criterion* 53
Gambar 4.12 *Flowchart* Algoritma Pendinginan Geometrik 54
Gambar 4.13 *Flowchart* Algoritma Inisialisasi Temperatur 56
Gambar 4.14 *Flowchart* Algoritma *Rolling Horizon Scheduler* 57
Gambar 4.15 *Flowchart* Algoritma Buat State Mesin 60
Gambar 4.16 *Flowchart* Algoritma *Build Machine Data* 62
Gambar 4.17 *Flowchart* Algoritma Penetapan Due Date dengan Aturan TWK yang Dimodifikasi 64
Gambar 4.18 *Flowchart* Algoritma Perhitungan Beban Kerja yang Tersisa 66
Gambar 6.1 Pengaruh *Max Iteration* Terhadap Rata-rata *Fitness* 109
Gambar 6.2 Pengaruh *Max Iteration* Terhadap Rata-rata *Runtime* 109
Gambar 6.3 Pengaruh *Alpha* Terhadap Rata-rata *Fitness* 110
Gambar 6.4 Pengaruh P0 Terhadap Rata-rata *Fitness* 111
Gambar 6.5 Pengaruh *Floor Ratio* Terhadap Rata-rata *Fitness* 112
Gambar 6.6 Pengaruh *Critical Machine Bias* Terhadap Rata-rata *Fitness* 113
Gambar 6.7 Pengaruh Bobot Pada Nilai Objektif Skenario 3 116
Gambar 6.8 Pengaruh Bobot Terhadap Rata-rata *Makespan* 117
Gambar 6.9 Pengaruh Bobot Terhadap Rata-rata Penalti Tinta 117

Gambar 6.10 Pengaruh Bobot Terhadap Rata-rata Penalti *Tardy*..... 118
Gambar 6.11 Pengaruh Bobot Terhadap *Fitness* (2T + I + 0.1M) 118
Gambar 6.12 Konvergensi Skenario 1..... 122
Gambar 6.13 Konvergensi Skenario 2..... 122
Gambar 6.14 Konvergensi Skenario 3..... 123
Gambar 6.15 Konvergensi Skenario 4A..... 123
Gambar 6.16 Konvergensi Skenario 4B..... 124
Gambar 6.17 Konvergensi Skenario 4C..... 124
Gambar 6.18 *Gantt Chart* Jadwal Skenario 1..... 126
Gambar 6.19 *Gantt Chart* Jadwal Skenario 2..... 128
Gambar 6.20 *Gantt Chart* Jadwal Skenario 3..... 129
Gambar 6.21 *Gantt Chart* Jadwal Skenario 4A..... 131
Gambar 6.22 *Gantt Chart* Jadwal Skenario 4B..... 133
Gambar 6.23 *Gantt Chart* Jadwal Skenario 4C..... 135
Gambar 6.24 *Gantt Chart* Jadwal Industri 138

DAFTAR LAMPIRAN

Lampiran A Cuplikan Hasil Mentah Pengujian Parameter SA 145
Lampiran B Cuplikan Hasil Mentah Pengujian Bobot Tujuan 146
Lampiran C Cuplikan Hasil Mentah Pengujian *Due Date Multiplier* 147
Lampiran D Cuplikan Hasil Mentah Pengujian Skenario 148
Lampiran E Statistik Hasil Pengujian Bobot Pada Skenario 4B 149

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Penjadwalan merupakan proses pengambilan keputusan untuk mengalokasikan sumber daya terbatas kepada serangkaian tugas dalam kurun waktu tertentu dengan tujuan mengoptimalkan kinerja sistem. Dalam konteks manufaktur, keputusan penjadwalan menentukan urutan dan penempatan pekerjaan pada mesin sehingga berpengaruh langsung terhadap *throughput*, ketepatan waktu pengiriman, dan biaya operasional. Seiring meningkatnya kebutuhan industri akan efisiensi dan fleksibilitas, formulasi masalah penjadwalan menjadi semakin realistis dengan memperhatikan berbagai batasan operasional. Banyak dari formulasi tersebut bersifat *NP-hard*, sehingga metode eksak sulit diterapkan pada ukuran kasus yang merepresentasikan kondisi nyata di lantai produksi. Oleh karena itu, penyusunan jadwal yang efisien dan praktis menjadi tantangan penting dalam perancangan sistem produksi (Pinedo, 2022).

Mesin cetak flexografi merupakan contoh nyata penjadwalan manufaktur yang kompleks. Mesin ini merupakan mesin cetak *rotary* yang umum digunakan untuk produksi kemasan, seperti kemasan karton dan label, dengan memanfaatkan pelat relief fleksibel dan tinta cair yang cepat kering. Proses pencetakan dilakukan secara kontinu, di mana setiap warna dicetak melalui unit tinta terpisah (Kipphan, 2014). Dalam praktik industri, beberapa mesin flexografi dapat beroperasi secara paralel dengan kecepatan yang berbeda, sementara setiap pekerjaan memiliki kebutuhan warna tertentu. Kompleksitas utama muncul pada proses pergantian tinta yang bergantung pada urutan pekerjaan. Hutomo (2022) menjelaskan bahwa transisi dari tinta gelap ke tinta terang memerlukan proses pembersihan menyeluruh selama sekitar 30 hingga 60 menit untuk mencegah kontaminasi warna, yang menyebabkan *downtime* dan limbah tinta dalam jumlah signifikan. Sebaliknya, transisi dari tinta terang ke tinta gelap umumnya tidak memerlukan pembersihan intensif. Asimetri ini menyebabkan kualitas jadwal sangat ditentukan oleh keputusan pengurutan warna, karena setiap pasangan pekerjaan yang berurutan dapat menghasilkan waktu setup dan biaya yang berbeda. Dengan adanya mesin paralel yang berkecepatan berbeda dan setup yang bergantung urutan, masalah penjadwalan flexografi dapat dimodelkan sebagai *Uniform Parallel Machine Scheduling* dengan *Sequence-Dependent Setup Times* (UPMSP-SDST) (Pinedo, 2022).

Karakteristik UPMSP-SDST menghasilkan ruang solusi yang sangat besar karena penjadwalan harus menentukan sekaligus penugasan pekerjaan ke mesin dan urutan pekerjaan pada setiap mesin, dengan waktu setup yang bergantung pada pasangan pekerjaan yang berurutan. Kompleksitas ini membuat pendekatan eksak menjadi tidak praktis untuk kebutuhan operasional, terutama ketika penjadwalan harus menyeimbangkan berbagai tujuan yang saling bertentangan (Pinedo, 2022). Dalam konteks tersebut, pendekatan metaheuristik banyak digunakan karena mampu menghasilkan solusi berkualitas baik dalam waktu

komputasi yang wajar. *Simulated Annealing* (SA) merupakan salah satu metaheuristik yang telah luas diterapkan pada berbagai masalah penjadwalan *NP-hard* dan terbukti efektif dalam menangani ruang solusi yang kompleks serta fungsi objektif multi-komponen (Talbi, 2009; Pinedo, 2022). Selain itu, operasi produksi flexografi bersifat dinamis karena pesanan dapat datang secara bertahap sepanjang hari, sehingga jadwal perlu diperbarui ketika informasi baru tersedia. Kondisi ini mendorong penggunaan kerangka *rolling horizon*, di mana penjadwalan dilakukan secara periodik pada jendela waktu tertentu dengan memperbarui rencana berdasarkan kondisi produksi terkini, sambil mempertahankan keputusan jangka dekat agar jadwal tetap stabil dan layak diterapkan di lingkungan industri (Pinedo, 2022).

Penelitian sebelumnya mengenai penjadwalan flexografi menunjukkan potensi sekaligus keterbatasan pendekatan yang ada. Hutomo (2022) merumuskan masalah penjadwalan flexografi statis dengan biaya setup berbasis tingkat kegelapan menggunakan *Binary Integer Programming* dan menunjukkan bahwa pengurangan transisi gelap ke terang dapat menurunkan waktu setup dan limbah secara signifikan. Menggunakan data industri yang sama, Hanif et al. (2024) menerapkan metaheuristik *Puma Optimizer* untuk memaksimalkan profit, memperlihatkan bahwa metode metaheuristik mampu menangkap struktur biaya yang kompleks secara lebih fleksibel dibandingkan aturan penjadwalan sederhana. Namun, sebagian besar studi terkait pencetakan warna dan penjadwalan mesin paralel masih mengasumsikan kumpulan pekerjaan statis, model tinta yang disederhanakan, atau mesin identik, sehingga belum sepenuhnya mencerminkan kondisi operasional mesin flexografi multi warna yang bekerja dalam lingkungan dinamis (Schuurman & Vuuren, 2016; Iori, Locatelli & Locatelli, 2021; 2023; Moser et al., 2022; Hanif et al., 2024).

Dengan demikian terdapat kesenjangan antara model penjadwalan teoretis dan kebutuhan operasional pabrik flexografi yang mengoperasikan mesin heterogen, memproses pekerjaan multi warna, dan menerima pesanan secara bertahap sepanjang hari. Penelitian ini berupaya menjembatani kesenjangan tersebut melalui perumusan UPMSP SDST berbasis tingkat kegelapan pada mesin flexografi dan penerapan penjadwal *rolling horizon* berbasis SA yang bertujuan mengurangi transisi gelap ke terang yang mahal, menyeimbangkan beban kerja, serta mengendalikan jumlah pekerjaan terlambat dalam waktu komputasi yang sesuai untuk kebutuhan industri.

1.2 Rumusan Masalah

Penelitian ini bertujuan untuk menjawab tantangan yang telah diidentifikasi melalui pertanyaan penelitian utama berikut:

1. Bagaimana masalah penjadwalan pencetakan flexografi dapat diformulasikan dan diselesaikan dengan algoritma SA untuk meminimalkan inefisiensi produksi, baik pada lingkungan statis maupun dinamis?

2. Bagaimana efektivitas, *robustness,* dan efisiensi komputasi pendekatan penjadwalan yang diusulkan dapat dievaluasi serta dibandingkan dengan praktik industri saat ini?

1.3 Tujuan

Tujuan utama dari penelitian ini adalah:

1. Merumuskan dan mengimplementasikan model penjadwalan percetakan flexografi menggunakan algoritma SA untuk meminimalkan inefisiensi produksi, baik pada lingkungan statis maupun dinamis.
2. Mengevaluasi efektivitas, *robustness,* dan efisiensi komputasi pendekatan penjadwalan yang diusulkan melalui eksperimen komputasional dan membandingkan hasilnya dengan praktik penjadwalan industri yang ada.

1.4 Manfaat

Penelitian ini memberikan nilai praktis bagi industri percetakan dengan menawarkan pendekatan penjadwalan yang secara langsung menanggulangi inefisiensi mahal dalam pencetakan flexografi. Pendekatan ini menekan terjadinya transisi tinta dari gelap ke terang, yang dikenal paling mengganggu proses produksi, sehingga mampu mengurangi kebutuhan pencucian serta menurunkan limbah tinta, yang di lingkungan pabrik dapat mencapai ratusan kilogram per minggu. Pembagian pekerjaan yang merata pada mesin cetak heterogen meningkatkan *throughput* dan pemanfaatan sumber daya, sedangkan strategi *rolling-horizon* memungkinkan jadwal menyesuaikan diri secara efektif terhadap kedatangan pekerjaan bertahap. Secara keseluruhan, kontribusi ini memberikan perencana produksi alat untuk menekan biaya operasional, meningkatkan keandalan pengiriman, serta mendorong keberlanjutan dalam sektor kemasan karton bergelombang di Indonesia.

Dari sisi ilmiah, penelitian ini memperkaya kajian *UPMSP-SDST* dengan memperkenalkan formulasi *darkness-aware* yang diintegrasikan dengan optimasi multi-objektif berbobot. Kerangka SA yang diusulkan mengombinasikan *neighborhood operators*, *geometric cooling*, serta mekanisme reoptimasi dinamis, sehingga dapat diterapkan baik dalam lingkungan statis maupun dinamis. Penelitian ini juga berkontribusi dengan menjembatani teori dan praktik, yakni dengan mengubah klasifikasi tingkat kegelapan tinta menjadi biaya *setup* yang dapat dihitung secara kuantitatif. Berbekal dataset industri yang terdokumentasi serta uji *robustness* dari berbagai percobaan, penelitian ini menegaskan bahwa temuan tersebut dapat direplikasi sekaligus menghadirkan tolok ukur baru bagi kajian penjadwalan yang dipengaruhi faktor warna.

1.5 Batasan Masalah

Ruang lingkup penelitian ini ditentukan oleh batasan-batasan berikut:

1. Permasalahan dibatasi pada skenario penjadwalan pencetakan flexografi yang melibatkan 15 pekerjaan dan 3 mesin cetak heterogen berdasarkan studi kasus industri oleh Hutomo (2022).
2. Permasalahan diasumsikan berada pada kondisi operasi yang bersifat deterministik, di mana waktu pemrosesan setiap pekerjaan hanya bergantung pada jumlah produksi dan kecepatan mesin. Gangguan eksternal seperti kerusakan mesin maupun faktor manusia berada di luar cakupan penelitian ini.

1.6 Sistematika Pembahasan

BAB 1: PENDAHULUAN

Bab ini berisi uraian mengenai latar belakang, rumusan masalah, tujuan, batasan masalah, manfaat penelitian, serta sistematika penulisan tugas akhir yang disajikan untuk memberikan alur pembahasan yang jelas dan terstruktur.

BAB 2: LANDASAN KEPUSTAKAAN

Bab ini berisi kajian pustaka dan teori yang menjadi dasar dalam penelitian. Pembahasan mencakup teori UPMSP-SDST, konsep SA, serta hasil-hasil penelitian terdahulu yang relevan dan mendukung penyusunan tugas akhir ini.

BAB 3: METODOLOGI PENELITIAN

Bab ini menjelaskan metodologi penelitian yang digunakan, meliputi jenis dan strategi penelitian, teknik pengumpulan data, pemodelan masalah, serta tahapan eksperimen dan metode evaluasi.

BAB 4: PERANCANGAN

Bab ini membahas perancangan penelitian yang meliputi dataset dan langkah praproses, formulasi masalah, desain algoritma, perhitungan manual, definisi skenario, serta langkah pengujian dan evaluasi. Rancangan ini menjadi acuan dalam implementasi metode optimasi pada kasus penjadwalan mesin flexografi.

BAB 5: IMPLEMENTASI

Bab ini berisi implementasi metode SA dan pendekatan *rolling horizon* terhadap model penjadwalan yang telah dirancang.

BAB 6: HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil pengujian yang telah dilakukan. Hasil eksperimen dianalisis berdasarkan metrik kinerja seperti *makespan*, total penalti tinta, penalti *tardy*, dan profit. Analisis dilakukan untuk menilai efektivitas, *robustness*, dan efisiensi komputasi metode serta membandingkan kinerja dengan *baseline* industri.

BAB 7: PENUTUP

Bab ini berisi kesimpulan dari penelitian yang telah dilakukan, jawaban terhadap rumusan masalah, serta saran untuk penelitian selanjutnya.

BAB 2 LANDASAN KEPUSTAKAAN

2.1 Kajian Pustaka

Hutomo (2022) mengkaji penjadwalan produksi percetakan flexografi pada lingkungan multi-mesin, di mana inefisiensi utama muncul akibat *sequence-dependent setup* yang disebabkan oleh pergantian tinta. Masalah diformulasikan sebagai model statis dengan asumsi satu warna per pekerjaan, batasan kapasitas mesin, serta penalti setup berbasis transisi tingkat kegelapan tinta. Tujuan optimasi adalah memaksimalkan profit. Penyelesaian dilakukan menggunakan *Binary Integer Programming*. Pengujian pada data kasus industri menunjukkan berkurangnya transisi tinta yang tidak menguntungkan.

Hanif et al. (2024) membahas penjadwalan produksi percetakan flexografi pada lingkungan multi-mesin dengan *sequence-dependent setup* akibat pergantian tinta. Masalah diformulasikan sebagai model statis dengan asumsi satu warna per pekerjaan, batasan kapasitas mesin, serta penalti setup berbasis tingkat kegelapan tinta. Tujuan optimasi adalah memaksimalkan profit. Masalah diselesaikan menggunakan metaheuristik *Puma Optimizer*. Eksperimen pada dataset kasus industri yang sama menunjukkan kemampuan metode dalam menghasilkan jadwal berorientasi profit secara konsisten.

Iori, Locatelli & Locatelli (2021) mengkaji penjadwalan produksi flexografi pada mesin paralel heterogen dengan *sequence-dependent setup* yang dipengaruhi oleh pergantian warna dan komponen. Masalah diformulasikan sebagai model statis yang mencakup penugasan dan pengurutan *job, due date*, serta penalti setup. Tujuan optimasi adalah meminimalkan *weighted tardiness* dan *total setup time*. Penyelesaian dilakukan menggunakan heuristik konstruktif yang dilanjutkan dengan *local search*. Pengujian pada *instance* industri menunjukkan penurunan signifikan keterlambatan dan waktu setup dibandingkan solusi awal.

Iori, Locatelli & Locatelli (2023) membahas penjadwalan flexografi pada mesin paralel heterogen dengan *sequence-dependent setup* yang dipengaruhi oleh warna, alat *embossing*, dan perforasi. Masalah diformulasikan sebagai model statis yang mencakup penugasan *job*, pengurutan, serta manajemen alat. Tujuan optimasi adalah meminimalkan *weighted tardiness* dan *total setup time*. Masalah diselesaikan menggunakan metaheuristik GRASP yang dikombinasikan dengan *local search*. Hasil komputasi menunjukkan perbaikan substansial pada kedua metrik dibandingkan heuristik pembanding pada berbagai kasus industri.

Schuurman & Vuuren (2016) merumuskan *Parallel Machine Colour Printing Scheduling Problem (PMCPSP)* untuk penjadwalan mesin cetak paralel dengan *sequence-dependent setup times* akibat kesamaan warna antar-*job*. Waktu proses *job* bergantung pada mesin, sedangkan waktu setup ditentukan oleh perubahan komposisi warna dan kapasitas *magazin cartridge*. Tujuan optimasi adalah meminimalkan *makespan*. Penyelesaian dilakukan menggunakan aturan LPTF sebagai inisialisasi, diikuti *improving local search*, *tabu search*, dan *simulated annealing* dalam sebuah *decision support system*. Pengujian pada *instance* sintetik

dan satu studi kasus menunjukkan bahwa SA menghasilkan solusi terbaik pada sebagian besar kasus, meskipun dengan waktu komputasi rata-rata tertinggi.

Moser et al. (2022) mengkaji UPMSP-SDST dan batas *machine eligibility*. Masalah diformulasikan sebagai model statis yang mencakup *due date*, waktu proses yang bergantung pada mesin, serta waktu setup antar-*job* yang dipengaruhi urutan. Tujuan optimasi adalah meminimalkan *total tardiness* sebagai prioritas utama dan *makespan* sebagai tujuan sekunder. Penyelesaian dilakukan menggunakan beberapa varian SA dengan *neighborhood* berbeda dan dibandingkan dengan pendekatan MIP. Pengujian pada ribuan *instance* menunjukkan bahwa SA mampu menyamai solusi terbaik MIP pada mayoritas kasus dan menghasilkan solusi yang lebih baik pada sebagian *instance* berskala menengah dan besar.

Li et al. (2024) mengkaji penjadwalan mesin cetak warna paralel dengan *sequence-dependent setup* akibat penggantian tinta dan proses pembersihan. Masalah diformulasikan sebagai model statis dengan pengurutan *job* berdasarkan kemiripan urutan warna untuk mengurangi frekuensi setup. Tujuan optimasi adalah meminimalkan *makespan*. Metode yang diusulkan menggabungkan pengelompokan *job* berbasis kesamaan warna menggunakan *Moving Interpolation* atau *Color Sequence Mapping*, diikuti penjadwalan menggunakan *Iterated Greedy*. Evaluasi pada data uji percetakan menunjukkan penurunan *makespan* lebih dari 40% dibandingkan aturan *First-Come-First-Served*.

Li et al. (2011) membahas penjadwalan mesin paralel dengan waktu proses yang dapat dipercepat melalui alokasi sumber daya terbatas. Masalah diformulasikan sebagai model statis yang mengaitkan keputusan alokasi sumber daya dengan durasi proses *job*. Tujuan optimasi adalah meminimalkan *makespan*. Penyelesaian dilakukan menggunakan *Simulated Annealing* dengan strategi fokus pada *critical machine* untuk mempercepat konvergensi pencarian. Pengujian pada berbagai *instance benchmark* berskala besar menunjukkan solusi yang sangat mendekati batas bawah teoretis dengan waktu komputasi yang sangat singkat.

Alimian et al. (2022) membahas *lot-sizing* dan penjadwalan terintegrasi pada lini paralel dengan *sequence-dependent setup* serta *preventive maintenance*. Masalah diformulasikan untuk menangkap keputusan produksi, penjadwalan, dan *maintenance* dalam satu model statis berbasis periode. Tujuan optimasi adalah meminimalkan total biaya terintegrasi yang mencakup biaya produksi, setup, persediaan, keterlambatan, dan *maintenance*. Penyelesaian dilakukan menggunakan model MINLP dan dua heuristik *rolling horizon* berbasis MIP. Evaluasi pada berbagai *instance* sintetis menunjukkan bahwa pendekatan *rolling horizon* menghasilkan biaya total yang lebih rendah dibandingkan penyelesaian eksak dengan batas waktu.

Tabel 2.1 Kajian Pustaka

Penulis	Domain Masalah	Metode	Tujuan	Formulasi
Hutomo (2022)	Penjadwalan produksi flexografi multi-mesin dengan *sequence-dependent setup* akibat pergantian tinta	*Binary Integer Programming*	Memaksimalkan profit dengan meminimalkan jumlah pergantian tinta gelap ke terang	Model statis, satu warna per pekerjaan, batasan kapasitas mesin, penalti setup berbasis tingkat kegelapan tinta
Hanif et al. (2024)	Penjadwalan produksi flexografi multi-mesin dengan *sequence-dependent setup* akibat pergantian tinta	*Puma Optimizer*	Memaksimalkan profit dengan meminimalkan jumlah pergantian tinta gelap ke terang	Model statis, satu warna per pekerjaan, batasan kapasitas mesin, penalti setup berbasis tingkat kegelapan tinta
Iori, Locatelli & Locatelli (2021)	UPMSP-SDST pada percetakan flexografi	Heuristik konstruktif + *local search*	Meminimalkan *weighted tardiness* dan total setup time	Model statis, *due date*, *setup* bergantung urutan, bobot keterlambatan
Iori, Locatelli & Locatelli (2023)	UPMSP-SDST pada percetakan flexografi dengan *tool-dependent* setup	GRASP + *local search*	Meminimalkan *weighted tardiness* dan total setup time	Model statis, *due date*, setup warna dan alat, penugasan serta pengurutan job
Schuurman & Vuuren (2016)	PMCPSP berbasis warna	LPTF + improving *local search* + tabu *search + simulated annealing*	Meminimalkan *makespan*	Mesin heterogen, *sequence-dependent setup* berbasis kesamaan warna
Moser et al. (2022)	UPMSP-SDST dan *machine eligibility*	MIP + *Simulated Annealing (shift, swap, block)*	Meminimalkan total *tardiness* (utama) dan *makespan* (sekunder)	Model statis, *due date*, waktu proses bergantung mesin, setup bergantung pasangan *job, eligibility* dimodelkan eksplisit
Li et al. (2024)	PMSP-SDST pada *color printing* dengan *cleaning*/setup berbasis urutan	Pengelompokan MI/CSM + *Iterated Greedy*	Meminimalkan *makespan*	Model statis, pengurutan *job* berbasis kemiripan urutan warna, setup bergantung urutan

Penulis	Domain Masalah	Metode	Tujuan	Formulasi
Li et al. (2011)	PMSP dengan controllable processing time	*Simulated Annealing* berbasis *critical machine*	Meminimalkan *makespan*	Model statis, alokasi sumber daya memengaruhi waktu proses, fokus perbaikan pada mesin kritis
Alimian et al. (2022)	UPMSP-SDST terintegrasi dengan *lot-sizing* dan *preventive maintenance*	Model MINLP dan heuristik *rolling horizon* berbasis MIP	Meminimalkan total biaya terintegrasi	Model berbasis periode *rolling horizon*, keputusan produksi, *changeover*, dan *preventive maintenance*

Berdasarkan kajian pustaka, sebagian besar penelitian penjadwalan pada proses percetakan flexografi masih memodelkan lingkungan produksi sebagai statis, di mana seluruh job diasumsikan telah diketahui sejak awal perencanaan. Pendekatan ini terlihat pada studi berbasis formulasi eksak maupun metaheuristik, seperti Hutomo (2022), Hanif et al. (2024), serta Iori, Locatelli & Locatelli (2021), yang menyusun jadwal dalam satu tahap perencanaan tanpa mekanisme penyesuaian terhadap kedatangan *job* baru. Padahal, pada praktik industri, rilis *job* sering terjadi secara bertahap, sehingga diperlukan pendekatan penjadwalan yang mampu melakukan re-optimasi secara periodik untuk merefleksikan kondisi sistem yang terus berubah.

Selain asumsi statis, penelitian terdahulu juga menunjukkan keterbatasan dalam cakupan formulasi masalah. Studi flexografi yang mempertimbangkan *sequence-dependent setup* umumnya masih mengasumsikan satu warna per *job*, seperti pada penelitian Hutomo (2022) dan Hanif et al. (2024), sehingga belum merepresentasikan skenario *job* multi-warna yang lazim ditemui di lapangan.

Berdasarkan celah tersebut, penelitian ini memosisikan masalah penjadwalan flexografi dalam kerangka UPMSP-SDST yang mempertimbangkan *job* multi-warna, mesin heterogen, dan lingkungan produksi dinamis. Pendekatan yang diusulkan menggunakan SA sebagai metode optimasi utama yang dioperasikan dalam kerangka *rolling horizon*, sehingga jadwal dapat diperbarui secara adaptif seiring kedatangan *job* baru. Tujuan penjadwalan diarahkan untuk menyeimbangkan aspek keterlambatan pekerjaan, efisiensi pergantian tinta, dan *makespan*, agar solusi yang dihasilkan lebih representatif terhadap kebutuhan operasional industri flexografi nyata.

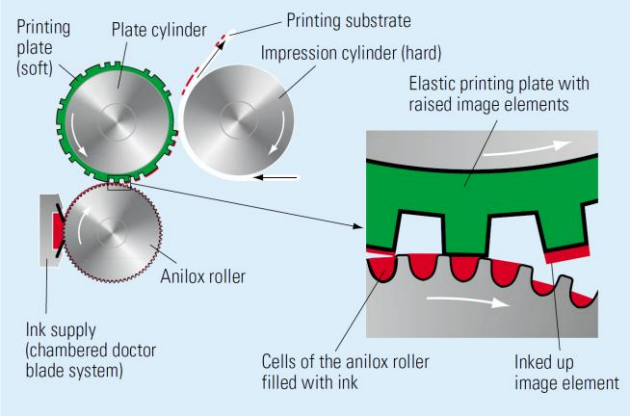
2.2 Proses Flexografi

2.2.1 Mesin Flexografi dan Prinsip Kerjanya

Flexografi didefinisikan sebagai metode cetak *rotary* yang menggunakan pelat cetak relief berbahan fleksibel dan tinta cair berviskositas rendah yang cepat kering (Kipphan, 2014). Teknologi ini dapat mencetak pada berbagai jenis substrat, mulai dari film plastik hingga material tebal seperti kardus gelombang, sehingga menjadi standar utama dalam industri kemasan.

Secara teknis, unit percetakan flexografi modern terdiri dari empat komponen utama yang bekerja secara berkesinambungan:

1. *Ink Fountain/Pan*: Tempat penampungan tinta cair.
2. *Anilox Roller*: Silinder keras yang diukir dengan sel-sel mikroskopis untuk mengambil tinta dalam jumlah yang terukur.
3. *Plate Cylinder*: Silinder tempat pelat cetak fleksibel ditempelkan.
4. *Impression Cylinder*: Silinder yang memberikan tekanan balik pada substrat.



Gambar 2.1 Cara Kerja Flexografi

Sumber: Kipphan (2014)

Mekanisme perpindahan tinta pada mesin flexografi, sebagaimana diilustrasikan pada Gambar 2.1, diawali dengan tinta yang dipompa dari bak penampungan ke sistem penintaan. Tinta tersebut mengisi sel-sel pada rol anilox, lalu kelebihan tinta dibersihkan oleh pisau *doctor blade*. Selanjutnya, rol anilox mentransfer lapisan tinta tersebut ke bagian relief pada pelat cetak. Akhirnya, substrat melewati celah antara silinder pelat dan silinder tekan untuk memindahkan gambar ke permukaan substrat melalui tekanan *kiss printing*. Mesin ini umumnya memiliki beberapa stasiun warna untuk mendukung pencetakan *multicolor* secara berurutan dalam satu lintasan produksi.

2.2.2 Proses Produksi pada Industri

Dalam konteks permasalahan yang dilaporkan oleh Hutomo (2022), penerapan flexografi diobservasi pada industri kemasan kotak karton gelombang (*corrugated*

carton box). Pada domain industri ini, mesin flexografi berperan sentral dalam tahap *converting* sebagai sarana pencetakan grafis. Namun, pada spesifikasi mesin tertentu, fungsinya terintegrasi secara *in-line* dengan proses pembentukan fisik karton, seperti pembuatan garis lipat dan pemotongan celah.

Alur produksi dalam studi kasus ini diawali dengan tahap *corrugating*, yaitu proses pengolahan gulungan kertas besar menjadi lembaran karton bergelombang. Lembaran-lembaran ini kemudian memasuki tahap *converting*, di mana mesin flexografi digunakan untuk mencetak grafis kemasan pada permukaan karton. Studi Hutomo (2022) mencatat bahwa mesin-mesin flexografi di lantai produksi bersifat heterogen dengan spesifikasi operasional yang berbeda. Beberapa mesin mampu beroperasi pada kecepatan yang tinggi, sedangkan mesin lainnya memiliki karakteristik operasional berkecepatan rendah. Variabilitas mesin ini menyebabkan waktu pemrosesan setiap pekerjaan sangat bergantung pada alokasi mesin spesifik yang digunakan, bukan hanya pada volume pesanan.

Tantangan operasional utama dalam lingkungan produksi ini muncul dari karakteristik tinta cair flexografi yang menuntut manajemen pergantian warna yang ketat. Hutomo (2022) menyoroti bahwa transisi urutan warna, khususnya dari tinta berwarna gelap ke terang, mewajibkan dilakukannya prosedur pencucian mesin yang menyeluruh untuk mencegah kontaminasi warna. Proses *setup* ini memakan waktu *downtime* yang signifikan, yaitu berkisar 30-60 menit, yang secara langsung mengurangi ketersediaan mesin dan menurunkan *throughput* produksi. Selain itu, aktivitas pencucian ini menghasilkan limbah tinta cair dalam volume besar, diperkirakan mencapai ratusan kilogram per minggu, yang menimbulkan beban biaya signifikan untuk penanganan dan pengolahan limbah.

Setelah pencetakan selesai, lembaran karton sering kali langsung melalui proses *finishing* seperti pembuatan celah, pembuatan garis lipat, hingga pengeleman dan penjahitan kawat untuk membentuk boks jadi. Rangkaian proses ini diakhiri dengan pemeriksaan kualitas untuk memastikan ketepatan dimensi, kekuatan sambungan, dan akurasi warna cetakan sebelum produk disimpan di gudang barang jadi. Kombinasi antara mesin yang heterogen, preferensi terang ke gelap untuk meminimalkan waktu cuci, dan volume pesanan yang fluktuatif menciptakan lingkungan penjadwalan yang sangat kompleks, yang melampaui kapasitas perencanaan manual sederhana.

2.3 *Uniform Parallel Machine Scheduling*

2.3.1 Klasifikasi Masalah

Pinedo (2022) mengklasifikasikan masalah penjadwalan menggunakan *three-field notation* α | β | γ. Berdasarkan kerangka tersebut, masalah penjadwalan mesin cetak flexografi dalam penelitian ini dapat dijelaskan sebagai berikut:

1. **Lingkungan mesin (α):** Masalah ini termasuk dalam *Uniform Parallel Machine Scheduling Problem* (UPMSP). Mesin-mesin cetak beroperasi secara paralel dengan fungsi yang sama, namun memiliki kecepatan produksi yang berbeda.

Sesuai dengan klasifikasi Pinedo (2022), lingkungan ini tergolong *uniform parallel machines* (Qm), di mana waktu pemrosesan pekerjaan ditentukan oleh beban pekerjaan dan kecepatan mesin yang digunakan.

2. **Karakteristik pekerjaan dan teknologi (β):** Masalah ini memiliki SDST, karena waktu penyiapan mesin bergantung pada urutan pekerjaan. Dalam pencetakan flexografi, hal ini terutama disebabkan oleh konfigurasi tinta, di mana transisi dari tinta gelap ke tinta terang memerlukan waktu pencucian yang lebih lama. Selain itu, pekerjaan dapat tiba secara bertahap, yang dimodelkan melalui *release times*.
3. **Tujuan (γ):** Pada formulasi statis, tujuan adalah meminimalkan *makespan* dan penalti pergantian tinta dalam bentuk *weighted-sum*. Pada formulasi dinamis, tujuan diperluas dengan memasukkan penalti *tardiness*, sehingga mencerminkan keseimbangan antara efisiensi produksi, biaya setup, dan ketepatan waktu pengiriman.

Dengan demikian, menurut klasifikasi Pinedo (2022), masalah ini merupakan permasalahan UPMSP-SDST dan tujuan multi-kriteria.

2.3.2 Kompleksitas Masalah

Masalah penjadwalan mesin dikenal memiliki tingkat kompleksitas komputasional yang tinggi. Pinedo (2022) menunjukkan bahwa bahkan pada lingkungan mesin paralel yang sederhana, seperti mesin identik dengan tujuan meminimalkan *makespan*, masalah penjadwalan sudah tergolong *NP-hard*. Kompleksitas ini meningkat secara signifikan ketika SDST diperhitungkan, karena pada mesin tunggal sekalipun varian ini telah tergolong *strongly NP-hard*.

Selain itu, penggunaan tujuan multi-kriteria, seperti kombinasi makespan, biaya setup, dan *tardiness*, memperluas ruang pencarian solusi secara kombinatorial. Oleh karena itu, pendekatan optimasi eksak menjadi tidak praktis untuk kasus berukuran realistis. Sebagai alternatif, penelitian ini menggunakan algoritma metaheuristik, yang secara luas direkomendasikan untuk masalah penjadwalan kompleks, termasuk SA, karena kemampuannya menjelajahi ruang solusi yang besar secara efisien (Delahaye, Chaimatanan & Mongeau, 2019; Pinedo, 2022).

2.4 Warna dan Tingkat Kegelapan

Warna merupakan atribut penting dalam pencetakan flexografi karena secara langsung memengaruhi SDST antar pekerjaan. Tingkat kegelapan tinta menentukan sejauh mana pembersihan diperlukan ketika beralih dari satu pekerjaan ke pekerjaan lainnya. Tanpa proses pencucian yang memadai, pigmen sisa dari tinta yang lebih gelap dapat mencemari tinta yang lebih terang, sehingga menimbulkan cacat cetak dan pekerjaan ulang yang mahal.

Said (2006) mengklasifikasikan sembilan tingkat kecerahan atau kegelapan sebagai berikut:

1. Putih (paling terang)
2. Kuning (sangat ringan)
3. Kuning-oranye / kuning-hijau (ringan)
4. Oranye / hijau (ringan-rendah)
5. Oranye kemerahan / hijau kebiruan (sedang)
6. Merah/biru (gelap-rendah)
7. Ungu kemerahan / ungu kebiruan (gelap)
8. Ungu (sangat gelap)
9. Hitam (paling gelap)

Kerangka ini sejak itu telah diterapkan dalam studi penjadwalan mesin cetak flexografi, termasuk oleh Hutomo (2022) dan Hanif et al. (2024), sebagai cara praktis untuk memodelkan kebutuhan setup. Dalam penerapannya, transisi dari tinta yang lebih gelap ke yang lebih terang, misalnya dari level 7 ke level 3, memerlukan operasi pencucian penuh. *Setup* semacam ini sangat memakan waktu dan menghasilkan limbah tinta dalam jumlah besar, sehingga menjadi faktor kritis dalam pencetakan flexografi, terutama karena tinta terang lebih rentan terhadap kontaminasi. Dengan memformalkan warna ke dalam skala numerik tingkat kegelapan, model penjadwalan dapat secara eksplisit memperhitungkan kompatibilitas tinta ketika mengevaluasi *sequence-dependent setups*.

2.5 *Simulated Annealing*

SA merupakan salah satu metode metaheuristik klasik yang digunakan untuk optimasi global. Metode ini terinspirasi dari proses *annealing* dalam bidang metalurgi, yaitu proses pendinginan bertahap pada logam panas hingga mencapai struktur kristal yang stabil dengan tingkat energi minimum. Konsep ini pertama kali diperkenalkan oleh Kirkpatrick, Gelatt Jr & Vecchi (1983), yang kemudian mengadaptasikannya ke dalam konteks optimasi matematis. Dalam analoginya, energi sistem fisik digantikan oleh fungsi objektif, sedangkan proses pendinginan dikaitkan dengan pengendalian parameter suhu yang secara bertahap diturunkan untuk menemukan solusi dengan nilai minimum global.

SA telah banyak diterapkan pada berbagai permasalahan penjadwalan yang tergolong *NP-hard* seperti *parallel machine scheduling* dengan SDST. SA telah terbukti mampu menghasilkan solusi berkualitas tinggi dengan struktur algoritma yang relatif sederhana (Schuurman & Vuuren, 2016). Fleksibilitasnya memungkinkan SA untuk dikombinasikan dengan heuristik khusus domain maupun metode hibrida lain, serta dapat disesuaikan untuk kondisi statis maupun dinamis di lingkungan manufaktur.

Pada prinsipnya, SA memperluas algoritma pencarian lokal dengan menambahkan aturan penerimaan probabilistik terhadap solusi yang lebih buruk. Dengan cara ini, algoritma memiliki kemampuan untuk keluar dari jebakan *local*

optima dan menjelajahi ruang solusi secara lebih luas. Pada tahap awal, ketika suhu masih tinggi, peluang untuk menerima solusi yang lebih buruk cukup besar sehingga proses pencarian bersifat eksploratif. Seiring turunnya suhu, peluang tersebut semakin kecil dan algoritma mulai berfokus pada solusi yang lebih baik hingga akhirnya konvergen pada solusi stabil (Delahaye, Chaimatanan & Mongeau, 2019).

Secara umum, prosedur SA terdiri atas beberapa komponen utama, yaitu:

1. Inisialisasi, yaitu penentuan solusi awal yang layak dan suhu awal.
2. Generasi tetangga, yaitu pembentukan solusi baru dengan melakukan perubahan kecil secara acak terhadap solusi saat ini.
3. Evaluasi, yaitu perhitungan nilai fungsi objektif untuk menilai kualitas solusi.
4. Kriteria penerimaan, yaitu mekanisme probabilistik untuk menentukan apakah solusi baru diterima atau ditolak.
5. Pendinginan, yaitu pengaturan penurunan suhu dari waktu ke waktu.
6. Kriteria penghentian, yaitu kondisi yang menandakan akhir dari proses pencarian, misalnya jumlah iterasi maksimum atau suhu minimum tercapai.

Pendekatan ini bersifat umum, sederhana, dan sangat fleksibel untuk diterapkan pada berbagai jenis permasalahan optimasi baik diskret maupun kontinu, termasuk penjadwalan, perutean, dan perancangan sistem industri (Delahaye, Chaimatanan & Mongeau, 2019).

2.5.1 Inisialisasi Suhu Awal

Suhu awal menentukan tingkat penerimaan solusi yang lebih buruk pada tahap awal pencarian. Salah satu pendekatan yang umum dipakai adalah menetapkan suhu awal berdasarkan probabilitas penerimaan awal p_0, yang dalam praktik SA biasanya berada pada rentang moderat sekitar 0.50 hingga 0.95 untuk menjaga keseimbangan eksplorasi dan stabilitas (Suman & Kumar, 2006). Dengan membalik aturan penerimaan Metropolis (Kirkpatrick, Gelatt Jr & Vecchi, 1983), suhu awal dihitung menggunakan:

$$T_0 = -\frac{\bar{\Delta}}{\ln(p_0)} \tag{2.1}$$

Di mana:

T_0 : suhu awal algoritma SA

$\bar{\Delta}$: rata-rata peningkatan nilai fungsi objektif dari beberapa langkah acak percobaan

p_0 : probabilitas awal untuk menerima solusi yang lebih buruk

Selain penentuan suhu awal, parameter penting lainnya dalam algoritma SA adalah suhu akhir atau suhu minimum ($T_{\min}$). Secara teoretis, $T_{\min}$ merepresentasikan kondisi di mana sistem dianggap telah 'membeku' atau mencapai keseimbangan termal pada energi rendah. Pada literatur standar,

kriteria penghentian sering kali dikaitkan dengan pencapaian suhu ini, di mana $T_{\min}$ biasanya ditetapkan mendekati nol. Pada kondisi ini, probabilitas penerimaan solusi yang lebih buruk menjadi sangat kecil, sehingga algoritma cenderung hanya menerima perbaikan solusi pada tahap akhir pencarian (Kirkpatrick, Gelatt Jr & Vecchi, 1983; Talbi, 2009).

2.5.2 Kriteria Penerimaan

Keputusan untuk menerima atau menolak solusi baru ditentukan oleh aturan Metropolis, yang merupakan inti dari mekanisme probabilistik pada SA. Probabilitas penerimaan solusi baru didefinisikan sebagai berikut:

$$P(\text{accept}) = \begin{cases} 1, \text{if } \Delta \leq 0 \\ \exp\left(-\frac{\Delta}{T}\right), \text{if } \Delta > 0 \end{cases} \quad (2.2)$$

Di mana:

$P(\text{accept})$: probabilitas penerimaan solusi baru,

$\Delta = f_{\text{new}} - f_{\text{current}}$: selisih nilai *fitness* antara solusi baru dan solusi saat ini,

T : temperatur saat ini dalam proses pendinginan.

Jika solusi baru lebih baik, maka solusi tersebut selalu diterima. Namun jika solusi baru lebih buruk, solusi tersebut masih dapat diterima dengan probabilitas, yang menurun seiring dengan menurunnya suhu. Aturan ini memungkinkan SA untuk keluar dari local optimum pada tahap awal dan secara bertahap berfokus pada pencarian solusi yang lebih baik di tahap akhir (Kirkpatrick, Gelatt Jr & Vecchi, 1983).

2.5.3 Skema Pendinginan

Skema pendinginan atau *cooling schedule* mengatur bagaimana suhu menurun selama proses iterasi berlangsung. Skema yang paling umum digunakan adalah pendinginan geometrik, yang dinyatakan dengan rumus berikut:

$$T_{\text{new}} = \max\left(T_{\text{min}}, T \times \alpha\right) \quad (2.3)$$

Di mana:

T_{new} : temperatur baru setelah pembaruan iterasi.

T_{min} : batas minimum temperatur.

T : temperatur saat ini sebelum pembaruan.

α : laju pendinginan (*cooling rate*), dengan nilai $0 < \alpha < 1$.

Nilai α umumnya berada pada rentang 0,90 hingga 0,99. Semakin besar nilai α, pendinginan berlangsung lebih lambat sehingga eksplorasi ruang solusi menjadi lebih luas namun memerlukan waktu komputasi lebih panjang. Sebaliknya, nilai α yang lebih kecil mempercepat konvergensi tetapi berisiko terjebak pada solusi lokal. Selain skema geometrik, variasi lain seperti linear cooling dan logarithmic

cooling juga dapat digunakan tergantung pada karakteristik permasalahan (Delahaye, Chaimatanan & Mongeau, 2019).

2.5.4 Kriteria Penghentian

Proses pencarian dihentikan apabila salah satu dari kondisi berikut terpenuhi:

1. Suhu telah mencapai nilai minimum $T_{\min}$,
2. Jumlah iterasi telah mencapai batas maksimum yang ditentukan ($k_{\max}$), atau
3. Tidak terjadi peningkatan signifikan pada nilai fungsi objektif setelah sejumlah iterasi tertentu.

2.6 Tujuan Penjadwalan

Dalam penjadwalan produksi, sering kali terdapat banyak tujuan yang perlu dioptimalkan secara bersamaan. Tujuan-tujuan ini terkadang saling bertentangan: peningkatan pada satu aspek kinerja dapat memperburuk aspek lainnya. Sebagai contoh, pengurangan waktu *setup* dapat meningkatkan *tardiness*. Untuk mengevaluasi jadwal kandidat secara sistematis, setiap tujuan dinyatakan dalam bentuk ukuran yang terukur.

2.6.1 *Sequence-Dependent Setup Times*

Dalam teori penjadwalan, SDST menggambarkan kondisi ketika waktu setup antara dua pekerjaan bergantung pada urutan pengerjaannya. Artinya, durasi setup dapat berbeda tergantung pekerjaan mana yang diproses sebelumnya. Fenomena ini umum terjadi pada sistem produksi yang memerlukan pergantian alat, pembersihan, atau penyesuaian konfigurasi mesin sebelum menjalankan pekerjaan berikutnya.

SDST meningkatkan kompleksitas penjadwalan karena urutan pekerjaan secara langsung memengaruhi total waktu produksi. Dalam industri seperti percetakan flexografi, transisi dari tinta gelap ke tinta terang, misalnya, memerlukan proses pembersihan yang lebih lama, sehingga urutan produksi yang tidak efisien dapat menyebabkan peningkatan waktu setup dan biaya operasional. Oleh karena itu, banyak penelitian berupaya meminimalkan efek SDST untuk meningkatkan efisiensi dan utilisasi mesin (Pinedo, 2022).

Selain memodelkan SDST secara eksplisit, penting juga untuk menghitung frekuensi terjadinya konfigurasi yang tidak diinginkan. Dalam pencetakan flexografi, hal ini dapat dilakukan dengan menghitung jumlah transisi warna yang memicu proses pembersihan. Setiap transisi tersebut diperlakukan sebagai satu kejadian setup yang menghasilkan penalti (pelanggaran tinta). Jumlah total setup yang dikenakan penalti ini kemudian digunakan baik sebagai komponen dalam fungsi objektif maupun sebagai metrik kinerja tersendiri dalam evaluasi.

2.6.2 *Makespan*

Tujuan fundamental lain adalah *makespan*. Dalam penjadwalan pencetakan UPMSP, *makespan* sering kali menjadi tujuan utama *shop level*. Pinedo (2022)

mendefinisikan *makespan* sebagai waktu penyelesaian maksimum di antara semua pekerjaan dalam jadwal, sebagaimana ditunjukkan dalam rumus berikut:

$$C_{max} = \max_j\{ C_j\} \tag{2.4}$$

Di mana:

C_{max} : *makespan*, panjang keseluruhan jadwal

C_j : waktu penyelesaian pekerjaan *j*

Definisi ini merepresentasikan total waktu yang dibutuhkan untuk menyelesaikan seluruh pekerjaan yang dijadwalkan. Dalam penjadwalan statis, ketika semua pekerjaan tersedia pada waktu nol, *makespan* sesuai dengan selang waktu sejak dimulainya jadwal hingga pekerjaan terakhir selesai. Pada pengaturan dinamis dengan *release times*, *makespan* tetap menjadi waktu penyelesaian maksimum, meskipun pekerjaan pertama tidak selalu dimulai pada waktu nol. Dalam konteks pencetakan UPMSP, *makespan* terbukti berfungsi sebagai metrik kinerja utama untuk optimasi tingkat *shop* (Pinedo, 2022).

Makespan yang lebih pendek menunjukkan *throughput* yang lebih baik dan distribusi beban kerja yang lebih seimbang antar mesin, sedangkan *makespan* yang lebih panjang mengindikasikan adanya *bottleneck* atau pemanfaatan sumber daya yang tidak merata.

2.6.3 Tardiness dan Due Dates

Pinedo (2022) mendefinisikan *tardiness* sebagai selisih waktu antara saat suatu pekerjaan selesai dengan *due date*-nya. Berbeda dengan *lateness*, yang dapat bernilai positif maupun negatif, *tardiness* selalu bernilai non-negatif dan hanya memberikan penalti pada pekerjaan yang terlambat:

$$T_i = \max\{ 0, C_i - D_i\} \tag{2.5}$$

Di mana:

T_i : keterlambatan pekerjaan ke-i

C_i : waktu penyelesaian pekerjaan ke-i

D_i : *due date* pekerjaan ke-i

Untuk mengevaluasi ketepatan jadwal terhadap *due date*, salah satu ukuran yang digunakan dalam penelitian ini adalah *number of tardy jobs*, yaitu jumlah pekerjaan yang memiliki *tardiness* lebih dari nol. Ukurannya dinyatakan sebagai berikut:

$$N_T = \text{jumlah pekerjaan } i \text{ dengan } T_i > 0 \tag{2.6}$$

Ukuran ini hanya menghitung banyaknya pekerjaan yang terlambat, tanpa memperhatikan besar keterlambatannya.

Salah satu metode penentuan *due date* yang paling sederhana dan sering digunakan dalam sistem penjadwalan adalah aturan *Total Work Content* (TWK) (Baker & Scudder, 1990). Aturan ini menetapkan batas waktu penyelesaian

pekerjaan berdasarkan total beban kerja yang diperlukan untuk menyelesaikan pekerjaan tersebut. Secara umum, TWK dapat dirumuskan sebagai berikut:

$$d_j = k\, p_j \tag{2.7}$$

Di mana:

d_j : *due date* yang ditetapkan untuk pekerjaan ke-j

p_j : waktu proses atau *processing time* dari pekerjaan ke-j

k : konstanta *due-date multiplier* yang mengontrol tingkat kelonggaran.

Aturan TWK mengasumsikan bahwa semakin besar beban kerja suatu pekerjaan, semakin lama waktu yang diberikan untuk menyelesaikannya. Konstanta k digunakan untuk menyesuaikan tingkat kelonggaran terhadap kondisi sistem produksi atau kebijakan pelayanan pelanggan. Jika nilai k lebih besar, maka *due date* akan lebih longgar. Sebaliknya, nilai k yang lebih kecil akan menghasilkan penjadwalan yang lebih ketat.

2.6.4 Fungsi Objektif

Dalam permasalahan penjadwalan produksi, sering kali terdapat lebih dari satu kriteria kinerja yang ingin dioptimalkan secara bersamaan. Salah satu pendekatan yang paling umum digunakan untuk menggabungkannya adalah pendekatan *weighted sum* atau fungsi objektif berbobot (Talbi, 2009; Pinedo, 2022).

Pendekatan ini mengubah masalah multiobjektif menjadi satu fungsi objektif tunggal dengan memberikan bobot tertentu pada setiap komponen tujuan. Secara umum, fungsi objektif berbobot dapat dinyatakan sebagai berikut:

$$F(S) = \sum_{i=1}^{n} w_i\, f_i(S) \tag{2.8}$$

Di mana:

$F(S)$: nilai total fungsi objektif untuk solusi S

$f_i(S)$: nilai fungsi objektif ke-i

w_i : bobot yang menunjukkan tingkat kepentingan relatif dari tujuan ke-i

n : jumlah total tujuan yang dipertimbangkan dalam optimasi

Pendekatan ini memberikan fleksibilitas kepada perancang sistem untuk menyesuaikan tingkat prioritas antar tujuan sesuai dengan kebijakan atau kondisi industri. Sebagai contoh, apabila keterlambatan pengiriman dianggap lebih kritis dibandingkan perubahan warna tinta, maka bobot untuk komponen *tardiness* dapat dibuat lebih besar. Sebaliknya, jika efisiensi pergantian mesin menjadi fokus utama, maka komponen setup dapat diberikan bobot yang lebih tinggi.

Metode *weighted* sum juga kompatibel dengan algoritma metaheuristik seperti SA, karena setiap solusi dapat dievaluasi menggunakan satu nilai agregat yang mewakili keseimbangan dari semua tujuan. Namun, hasil akhir tetap bergantung pada penetapan bobot yang mencerminkan preferensi atau kebijakan sistem yang dioptimalkan (Talbi, 2009).

2.7 Penjadwalan Statis dan Dinamis

Dalam literatur penjadwalan, istilah statis dan dinamis digunakan untuk membedakan apakah keputusan penjadwalan ditetapkan sepenuhnya di awal perencanaan atau dapat diperbarui selama proses produksi berlangsung. Pinedo (2022) menjelaskan pembedaan ini secara eksplisit dalam konteks aturan prioritas, bahwa aturan statis tidak bergantung pada waktu, sedangkan aturan dinamis bergantung pada waktu dan dapat berubah seiring perkembangan sistem produksi.

Pada lingkungan statis, seluruh pekerjaan diasumsikan telah diketahui dan tersedia pada awal *horizon* perencanaan sehingga penjadwal memiliki informasi lengkap sebelum optimasi dilakukan. Sebaliknya, pada lingkungan dinamis, pekerjaan dirilis secara bertahap sehingga informasi yang tersedia pada awal penyusunan jadwal bersifat parsial dan perlu diperbarui ketika pekerjaan baru tiba. Dengan demikian, pembedaan antara penjadwalan statis dan dinamis terutama berkaitan dengan perbedaan tingkat ketersediaan informasi serta kebutuhan pembaruan keputusan penjadwalan dari waktu ke waktu.

2.8 Penjadwalan *Rolling Horizon*

Dalam banyak lingkungan produksi nyata, pekerjaan tidak seluruhnya tersedia pada awal *horizon* perencanaan, melainkan tiba secara bertahap seiring waktu. Kondisi ini menyebabkan penjadwal tidak memiliki informasi lengkap sejak awal, sehingga keputusan yang dibuat perlu disesuaikan ketika informasi baru tersedia. Ovacik & Uzsoy (1995) menjelaskan bahwa karakteristik dinamis tersebut, khususnya pada masalah mesin paralel dengan kedatangan pekerjaan dari waktu ke waktu, memotivasi penggunaan pendekatan *rolling horizon*.

Pada pendekatan *rolling horizon*, proses penjadwalan dilakukan secara berulang pada titik keputusan tertentu. Pada setiap titik keputusan, sebuah submasalah penjadwalan diselesaikan dengan mempertimbangkan pekerjaan yang telah tersedia serta informasi prakiraan dalam jendela perencanaan ke depan. Jadwal yang dihasilkan tidak seluruhnya langsung dijalankan, melainkan hanya sebagian keputusan awal yang diimplementasikan, sementara keputusan untuk periode selanjutnya akan diperbarui pada titik keputusan berikutnya.

Melalui mekanisme ini, *rolling horizon* memungkinkan penjadwal untuk menyeimbangkan kebutuhan adaptasi terhadap perubahan sistem dengan upaya optimasi yang eksplisit terhadap tujuan penjadwalan. Kualitas solusi dan waktu komputasi dapat dikendalikan melalui pengaturan ukuran *horizon* dan frekuensi penjadwalan ulang, sehingga pendekatan ini tetap praktis untuk diterapkan pada lingkungan produksi dinamis.

2.9 Metrik Evaluasi Kinerja

Evaluasi suatu jadwal memerlukan metrik yang mampu menangkap kinerja operasional sekaligus perilaku algoritma. Tujuan serta ukuran kinerja dasar seperti *makespan* dan jumlah pekerjaan terlambat telah didefinisikan pada Subbab 2.6.

Dalam studi ini, ukuran-ukuran tersebut digunakan kembali dan dikelompokkan ke dalam lima aspek utama: profit, indikator kualitas solusi, utilisasi mesin, *robustness,* dan efisiensi komputasi. Metrik yang sama diterapkan secara konsisten dalam penalaan parameter, pengujian skenario, dan perbandingan dengan jadwal industri.

2.9.1 Profit

Profit mengukur nilai bisnis bersih yang dihasilkan oleh suatu jadwal. Nilai ini dihitung sebagai pendapatan produksi dikurangi biaya penalti akibat perubahan tinta yang tidak diinginkan:

$$\text{Profit} = (Q_{\text{total}} \times H) - (C_{\text{penalti}} \times P_{\text{tinta}}) \tag{2.9}$$

Di mana:

Q_{total} : total kuantitas seluruh pekerjaan.

H : harga jual per unit.

C_{penalti} : biaya penalti untuk satu kali penalti tinta.

P_{tinta} : jumlah penalti tinta yang terjadi pada jadwal.

2.9.2 *Fitness Score*

Dari perspektif optimasi, setiap kandidat solusi dievaluasi menggunakan suatu skor *fitness* tunggal. Skor ini menggabungkan beberapa tujuan ke dalam satu nilai menggunakan fungsi objektif berbobot yang didefinisikan pada 2.6.4. Nilai skalar ini memungkinkan algoritma SA membandingkan jadwal kandidat secara sistematis dan mengarahkan pencarian menuju solusi yang lebih baik.

2.9.3 Metrik Spesifik Tujuan

Selain skor *fitness* teragregasi, penting untuk menganalisis setiap tujuan secara terpisah guna memahami *trade-off* yang terjadi. Metrik kualitas solusi utama yang digunakan adalah:

1. ***Makespan*:** *Makespan* adalah waktu penyelesaian maksimum di antara seluruh pekerjaan dalam jadwal. Nilai ini merepresentasikan panjang total jadwal dan *throughput* sistem secara keseluruhan, sebagaimana didefinisikan pada Subbab 2.6.2.
2. **Pelanggaran tinta:** Pelanggaran tinta menghitung jumlah perubahan warna yang dianggap tidak diinginkan berdasarkan aturan penalti. Setiap perubahan tersebut diperlakukan sebagai satu penalti tinta dan digunakan baik dalam fungsi objektif maupun sebagai indikator kinerja tersendiri.
3. **Jumlah pekerjaan *tardy*:** Ketepatan waktu diukur menggunakan jumlah pekerjaan yang selesai melewati due date. Ukuran ini didefinisikan secara formal pada Subbab 2.6.3 sebagai jumlah pekerjaan dengan *tardiness* $T_i > 0$. Metrik ini hanya menghitung banyaknya pekerjaan yang terlambat, tanpa memperhitungkan besarnya keterlambatan.

2.9.4 Utilisasi Mesin

Utilisasi mesin menggambarkan intensitas pemakaian setiap mesin dalam suatu jadwal produksi. Evaluasi ini menggunakan empat indikator berbasis waktu, yaitu waktu pemrosesan, rentang operasi, efisiensi, dan cakupan. Keempat indikator tersebut dibangun di atas konsep beban kerja mesin dalam penjadwalan paralel, di mana total waktu proses pekerjaan yang dialokasikan ke suatu mesin merepresentasikan tingkat pemanfaatannya (Ouazene et al., 2014). Selain itu, pendekatan ini juga selaras dengan praktik pengukuran kinerja manufaktur yang menilai pemanfaatan kapasitas melalui rasio waktu produktif, seperti dibahas dalam Ramis Ferrer et al. (2018). Dengan demikian, rangkaian indikator ini memberikan dasar yang terstruktur untuk menilai efektivitas penggunaan mesin dalam berbagai skenario penjadwalan.

2.9.4.1 *Processing Time*

Waktu pemrosesan merepresentasikan total waktu yang digunakan mesin untuk memproses seluruh pekerjaan yang ditugaskan padanya. Konsep ini konsisten dengan definisi beban kerja mesin dalam penjadwalan mesin paralel, yaitu penjumlahan waktu proses setiap *job* pada mesin (Ouazene et al., 2014). Untuk mesin m, waktu kerja dirumuskan sebagai:

$$T_{\text{work},m} = \sum_{j \in J_m} p_{j,m} \tag{2.10}$$

Di mana:

$T_{\text{work},m}$: total waktu kerja mesin m.

$p_{j,m}$: waktu proses *job* j pada mesin m.

J_m : himpunan *job* yang dijadwalkan pada mesin m.

2.9.4.2 Rentang Waktu

Rentang waktu menunjukkan durasi aktif suatu mesin, yaitu interval sejak *job* pertama dimulai hingga *job* terakhir selesai. Secara umum dirumuskan:

$$T_{\text{span},m} = C_{\text{end},m} \tag{2.11}$$

Di mana:

$T_{\text{span},m}$: rentang operasi mesin m.

$C_{\text{end},m}$: waktu selesai *job* terakhir pada mesin m.

2.9.4.3 Efisiensi Mesin

Efisiensi mesin mengukur kepadatan aktivitas dengan membandingkan waktu pemrosesan terhadap rentang waktunya, mengikuti prinsip rasio waktu produktif yang digunakan dalam evaluasi kinerja manufaktur (Ramis Ferrer et al., 2018). Untuk mesin m:

$$E_m = \frac{T_{\text{work},m}}{T_{\text{span},m}} \tag{2.12}$$

Di mana:

E_m : efisiensi mesin m.

2.9.4.4 Cakupan mesin

Cakupan mesin merupakan indikator turunan yang menormalkan rentang operasi mesin terhadap *makespan* jadwal. Indikator ini membantu membandingkan tingkat keterlibatan mesin dalam *horizon* penjadwalan. Untuk mesin m:

$$C_m = \frac{T_{\text{span},m}}{M} \tag{2.13}$$

Di mana:

C_m: cakupan mesin m.

M: makespan jadwal.

2.9.5 *Robustness*

Robustness merupakan sifat penting dalam evaluasi algoritma optimasi stokastik seperti SA. Karena proses pencarian melibatkan elemen acak, kualitas solusi dapat bervariasi antar percobaan. Suatu algoritma dianggap *robust* apabila mampu menghasilkan kinerja yang konsisten pada banyak percobaan independen. Talbi (2009) menjelaskan bahwa pada algoritma stokastik, *robustness* dapat diamati melalui perilaku *average* dan *deviation* dari hasil yang diperoleh pada beberapa *independent runs* dalam instance yang sama, serta bahwa variabilitas yang lebih rendah menunjukkan *robustness* yang lebih baik.

Untuk memperoleh evaluasi yang reliabel, literatur metaheuristik merekomendasikan pelaksanaan sejumlah percobaan independen. Talbi (2009) menyatakan bahwa algoritma stokastik idealnya dievaluasi melalui banyak *trials*, dengan minimal sekitar 10 percobaan dan dapat mencapai lebih dari 100 apabila diperlukan untuk meningkatkan kekuatan analisis statistik.

Cara yang umum digunakan untuk mengukur *robustness* adalah menghitung nilai rata-rata dan standar deviasi dari suatu metrik kinerja berdasarkan beberapa *independent runs*. Menurut Talbi (2009), dari sekumpulan percobaan tersebut dapat dihitung ukuran ringkas seperti rata-rata dan standar deviasi, di samping ukuran lain seperti *median* dan rentang nilai.

Rata-rata

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{2.14}$$

Standar Deviasi

$$S = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{2.15}$$

Di mana:

$\bar{x}$: Rata-rata metrik evaluasi

S: Standar deviasi

n: Jumlah percobaan independen

x_i: Nilai metrik pada percobaan ke-i

Standar deviasi yang lebih rendah mencerminkan *robustness* yang lebih tinggi karena hasil algoritma menunjukkan sensitivitas yang lebih rendah terhadap unsur acak, sejalan dengan definisi *robustness* dalam evaluasi metaheuristik (Talbi, 2009).

2.9.6 Efisiensi Komputasi

Efisiensi komputasi merupakan aspek penting dalam aplikasi penjadwalan praktis karena solusi harus dihasilkan dalam batas waktu tertentu. Indikator efisiensi yang digunakan pada penelitian ini adalah waktu komputasi, diukur sebagai *wall clock time* yang diperlukan algoritma untuk menghasilkan solusi. Pengukuran ini merepresentasikan kelayakan metode untuk konteks operasional, di mana keputusan penjadwalan perlu dibuat secara cepat tanpa mengorbankan kualitas solusi. Dengan demikian, evaluasi waktu komputasi membantu memastikan bahwa pendekatan optimasi yang diusulkan tidak hanya efektif, tetapi juga praktis untuk diterapkan (Talbi, 2009).

BAB 3 METODOLOGI PENELITIAN

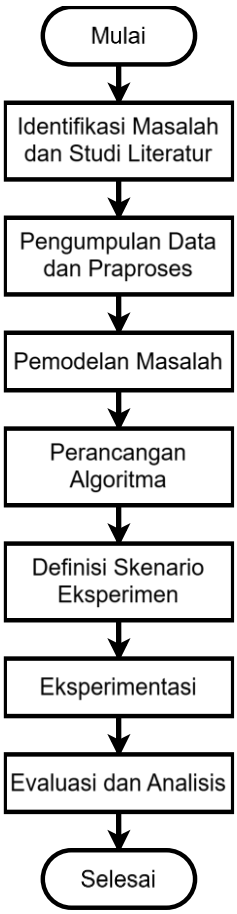
3.1 Tipe Penelitian

Penelitian ini menggunakan pendekatan non-implementatif analitik. Tujuan utama penelitian bukan untuk membangun sistem produksi nyata, melainkan untuk merancang, menjalankan, dan mengevaluasi suatu kerangka optimasi melalui eksperimen berbasis komputasi.

Metodologi penelitian menitikberatkan pada analisis perilaku algoritma SA yang diterapkan pada permasalahan UPMSP-SDST. Kinerja algoritma dievaluasi berdasarkan beberapa kriteria, antara lain biaya *setup*, *makespan*, *tardiness*, *robustness,* dan efisiensi komputasi.

Pendekatan penelitian ini memungkinkan dilakukan eksplorasi yang sistematis dan terkontrol untuk memahami bagaimana strategi optimasi dapat digunakan dalam menyelesaikan tantangan penjadwalan kompleks di industri percetakan flexografi secara efektif.

3.2 Strategi Penelitian



Gambar 3.1 Strategi Penelitian

Strategi penelitian dalam studi ini mengikuti proses yang terstruktur dan berurutan, dirancang agar setiap tahap memberikan kontribusi yang bermakna terhadap penyelesaian permasalahan penjadwalan. Proses dimulai dengan tinjauan pustaka, yang membangun landasan teoretis mengenai UPMSP-SDST, serta optimasi metaheuristik, khususnya SA.

Setelah dasar konseptual ditetapkan, dilakukan pengumpulan dan praproses data menggunakan data industri nyata, di mana pengodean tingkat kegelapan diterapkan untuk mengklasifikasikan warna tinta sebagai dasar pemodelan *setup*.

Tahap berikutnya adalah pemodelan permasalahan, di mana lingkungan penjadwalan didefinisikan melalui tujuan optimasi, batasan operasional, dan metrik kinerja. Setelah permasalahan terformulasi secara jelas, dilakukan tahap perancangan algoritma, yang mencakup konfigurasi kerangka kerja SA, struktur *neighborhood*, serta penerapan mekanisme *rolling horizon* untuk menangani kedatangan pekerjaan secara dinamis.

Selanjutnya, dilakukan pendefinisian skenario untuk membentuk kasus uji statis dan dinamis, sehingga memungkinkan evaluasi yang terarah terhadap berbagai kondisi operasional. Skenario-skenario tersebut kemudian dijalankan dalam tahap eksperimen komputasional, di mana sejumlah percobaan dilakukan untuk mengamati kinerja algoritmik dan variasi hasil optimasi.

Tahap terakhir adalah evaluasi dan analisis hasil, yang dilakukan dengan menilai performa sistem berdasarkan indikator kinerja operasional dan algoritmik seperti *makespan*, *tardiness*, penalti *setup*, dan profit produksi. Hasil yang diperoleh kemudian dibandingkan dengan jadwal industri untuk menilai efektivitas strategi optimasi yang diusulkan.

3.3 Peralatan Pendukung

Tabel 3.1 Perangkat Keras Pendukung

Perangkat Keras	**Spesifikasi**
CPU	Intel® Core™ i7-12700H
RAM	16 GB

3.4 Studi Literatur

Penelitian ini didasarkan pada empat aspek utama, yaitu pemodelan penjadwalan sebagai UPMSP-SDST, karakteristik penjadwalan berbasis warna pada proses pencetakan flexografi, penggunaan algoritma metaheuristik SA, serta penerapan pendekatan *rolling horizon* pada lingkungan produksi dinamis. Kajian pada Bab 2 menjadi landasan dalam memodelkan pekerjaan multi tinta, mesin dengan kecepatan heterogen, dan pergantian warna tinta yang bergantung urutan sebagai representasi kondisi industri nyata. Selain itu, literatur tersebut mendukung pemilihan SA dengan *geometric cooling* dan mekanisme penerimaan probabilistik, serta penggunaan *rolling horizon* untuk memungkinkan pembaruan keputusan penjadwalan seiring kedatangan pekerjaan dari waktu ke waktu.

3.5 Dataset dan Praproses

Penelitian ini didasarkan pada data sekunder yang bersumber dari laporan operasional sebuah fasilitas percetakan flexografi (Hutomo, 2022). Dataset ini merekam rencana produksi lengkap untuk satu hari operasi yang mencakup 15 pesanan berbeda pada tiga mesin paralel heterogen, memberikan dasar autentik untuk pemodelan dan eksperimen. Data mentah yang diperoleh meliputi daftar pesanan produksi, spesifikasi kapasitas mesin, dan parameter biaya operasional.

Sebelum digunakan dalam simulasi optimasi, dilakukan serangkaian tahapan praproses terhadap data mentah tersebut. Pertama, kapasitas produksi harian dikonversi menjadi laju produksi per menit untuk memungkinkan perhitungan waktu penyelesaian yang presisi dalam simulasi *continuous time*. Kedua, warna tinta dikodekan ke dalam skala numerik tingkat kegelapan 1 hingga 9 mengacu pada metode Said (2006) guna memodelkan logika *sequence-dependent setup*. Ketiga, penetapan biaya penalti tinta dan durasi *setup* disesuaikan dengan rata-rata industri untuk menjaga konservatisme model. Rincian lengkap mengenai nilai parameter, spesifikasi pekerjaan, dan konfigurasi mesin disajikan secara detail pada Subbab 4.1.

3.6 Pemodelan Masalah

Penelitian ini memformulasikan permasalahan penjadwalan pada mesin cetak flexografi sebagai UPMSP-SDST. Model ini secara eksplisit mengakomodasi keragaman mesin, ketergantungan waktu setup terhadap kombinasi tinta, serta beberapa tujuan penjadwalan.

Lingkungan produksi terdiri atas sejumlah mesin cetak flexografi yang heterogen, masing-masing beroperasi dengan kecepatan pemrosesan yang berbeda. Setiap pekerjaan dikarakterisasikan berdasarkan jumlah produksi dan kombinasi tinta yang diperlukan. Apabila dua pekerjaan berturut-turut memerlukan perubahan tinta dari warna yang lebih gelap ke warna yang lebih terang, maka dibutuhkan proses *setup* tambahan dengan waktu dan penalti tertentu.

Tujuan penjadwalan diformulasikan untuk meminimalkan inefisiensi operasional sekaligus memastikan pekerjaan selesai tepat waktu. Tujuan-tujuan tersebut meliputi:

1. Penalti perubahan tinta untuk merepresentasikan waktu henti mesin dan limbah material akibat pergantian warna.
2. *Makespan* untuk mengukur total waktu penyelesaian produksi.
3. Jumlah pekerjaan *tardy* untuk mengevaluasi tingkat ketepatan waktu penyelesaian pada skenario kedatangan pekerjaan dinamis.

Model ini juga mencakup sejumlah *constraints* kelayakan (*feasibility constraints*) yang berkaitan dengan penugasan pekerjaan, urutan produksi, kapasitas mesin, waktu rilis pekerjaan, dan *due date*. Kendala-kendala tersebut

memastikan bahwa jadwal yang dihasilkan dapat diimplementasikan secara operasional dan konsisten dengan kondisi nyata di lingkungan industri.

3.7 Kerangka *Simulated Annealing*

SA digunakan sebagai algoritma optimasi utama dalam penelitian ini untuk menyelesaikan permasalahan penjadwalan produksi pada tiga mesin dengan SDST. Kerangka SA yang diterapkan mengacu pada formulasi umum metaheuristik SA sebagaimana dibahas dalam literatur optimasi kombinatorial, khususnya pada Delahaye, Chaimatanan & Mongeau (2019) dan Talbi (2009). Kerangka dasar tersebut kemudian diadaptasi ke dalam konteks penjadwalan produksi dengan karakteristik khusus yang dibahas pada penelitian ini.

SA bekerja pada representasi jadwal tiga mesin dengan mengevaluasi setiap solusi menggunakan fungsi objektif berbobot yang menggabungkan *makespan*, jumlah perubahan tinta gelap ke terang, dan jumlah pekerjaan terlambat. Solusi awal dibentuk melalui heuristik sederhana yang bertujuan menghasilkan jadwal *feasible*, dengan menyeimbangkan beban mesin dan mengelompokkan pekerjaan berdasarkan karakteristik warna tinta.

Proses perbaikan solusi dilakukan melalui eksplorasi *neighborhood* menggunakan beberapa jenis *move*, yaitu *swap, relocate, exchange*, dan *block insert. Neighborhood move* tersebut memungkinkan perubahan urutan pekerjaan baik dalam satu mesin maupun lintas mesin, sehingga algoritma dapat mengeksplorasi ruang solusi yang lebih luas tanpa melanggar kendala jadwal pada bagian yang telah dikunci.

Kriteria penerimaan solusi mengikuti aturan Metropolis, di mana solusi dengan nilai fungsi objektif yang lebih buruk masih dapat diterima secara probabilistik. Mekanisme ini bertujuan menjaga keseimbangan antara eksplorasi dan eksploitasi selama proses pencarian. Suhu awal ditentukan berdasarkan estimasi rata-rata kenaikan nilai fungsi objektif pada solusi tetangga awal dan probabilitas penerimaan target P_0. Pendinginan dilakukan menggunakan skema geometrik dengan rasio α hingga mencapai batas iterasi maksimum atau suhu minimum yang telah ditentukan, sambil menyimpan solusi terbaik yang diperoleh sepanjang proses.

3.8 Kerangka *Rolling Horizon*

Untuk menangani lingkungan penjadwalan dinamis, SA diintegrasikan dengan strategi *rolling horizon*. Pendekatan *rolling horizon* digunakan secara luas dalam permasalahan penjadwalan dinamis untuk menjaga keseimbangan antara stabilitas jadwal dan kemampuan adaptasi terhadap kedatangan pekerjaan baru, sebagaimana dibahas dalam literatur penjadwalan produksi (Pinedo, 2022). *Rolling horizon* berfungsi sebagai mekanisme pengendali waktu penjadwalan ulang, sementara algoritma optimasi digunakan untuk menyusun kembali bagian jadwal yang masih fleksibel.

Pada setiap kejadian kedatangan *batch* pekerjaan baru, jadwal dibagi menjadi tiga bagian, yaitu pekerjaan yang telah selesai, pekerjaan yang sedang diproses, dan pekerjaan yang belum dimulai. Penjadwalan ulang hanya dilakukan pada bagian terakhir, sedangkan dua bagian sebelumnya dipertahankan untuk menjaga stabilitas operasional. Status mesin diperbarui meliputi waktu selesai sementara, konfigurasi tinta terakhir, dan beban kerja tersisa.

Due date untuk pekerjaan baru kemudian ditentukan menggunakan aturan *Total Work Content* (TWK) yang dimodifikasi agar sesuai dengan karakteristik waktu proses berbasis batch. Setelah itu, algoritma SA dijalankan kembali pada himpunan pekerjaan yang dapat dijadwal ulang untuk menghasilkan jadwal lokal terbaik yang konsisten dengan kondisi mesin saat ini.

Dengan pendekatan tersebut, *rolling horizon* berfungsi sebagai kerangka pengendali dinamika sistem penjadwalan, sedangkan SA bertindak sebagai mekanisme optimasi pada setiap horizon. Keseluruhan kerangka ini dibangun berdasarkan landasan teoretis yang telah dibahas pada Bab 2 dan akan diimplementasikan secara operasional pada Bab 4.

3.9 Rencana Eksperimen

Untuk mengevaluasi kinerja pendekatan penjadwalan yang diusulkan, empat skenario eksperimen didefinisikan sebagai berikut:

1. **Keseimbangan mesin:** Warna tinta diabaikan dan tujuan berfokus pada pemerataan beban kerja serta minimalisasi *makespan*.
2. **Penjadwalan warna:** Hanya penalti perubahan tinta yang diminimalkan berdasarkan aturan transisi warna.
3. **Penjadwalan statis kombinasi:** Penalti tinta dan *makespan* dioptimalkan secara bersamaan dengan seluruh pekerjaan tersedia pada waktu awal.
4. ***Rolling horizon* dinamis:** Menambahkan komponen *tardiness* dan melakukan optimasi ulang pada setiap kedatangan *batch*.

Setiap skenario dieksekusi dalam beberapa percobaan independen dengan *random seed* berbeda untuk menangkap variasi akibat sifat stokastik SA. Untuk skenario dinamis, evaluasi dilakukan pada setiap titik optimasi ulang dan hasil diagregasi sepanjang *horizon* perencanaan.

Secara keseluruhan, penelitian ini melakukan beberapa tahapan pengujian berikut:

1. **Pengujian Parameter SA:** menentukan konfigurasi algoritma terbaik melalui variasi jumlah iterasi, *cooling*, dan parameter SA lainnya.
2. **Pengujian Bobot Tujuan**: mengevaluasi *trade off* antara penalti tinta, *makespan*, dan *tardiness* pada skenario relevan.
3. **Pengujian *Due Date Multiplier*:** menyesuaikan tingkat keketatan *due date* untuk skenario dinamis guna mencerminkan kondisi beban kerja nyata.

4. **Pengujian Skenario:** mengevaluasi *robustness* dan perilaku konvergensi penjadwal pada berbagai skenario melalui pengujian berulang.
5. **Analisis Jadwal Terbaik:** mengevaluasi metrik hasil terbaik pada setiap skenario menggunakan indikator performa utama.
6. **Perbandingan dengan Jadwal Industri:** menilai peningkatan efisiensi produksi dengan membandingkan solusi SA terhadap jadwal *baseline* aktual.

3.10 Evaluasi dan Visualisasi

3.10.1 Metrik

Eksperimen dievaluasi menggunakan metrik kinerja yang telah didefinisikan pada Bab 2, dan dikelompokkan ke dalam aspek-aspek berikut:

1. **Profit:** Hasil bisnis bersih yang dihitung sebagai pendapatan dikurangi biaya penalti tinta. Profit dihitung pada tahap *post-processing* setelah proses optimasi dan terutama digunakan dalam analisis skenario serta perbandingan dengan jadwal industri.
2. ***Fitness score*:** Nilai fungsi objektif *weighted sum* yang digunakan dalam algoritma SA. Skor ini menggabungkan makespan, ink penalties, dan tardiness sesuai bobot yang dipilih, serta berfungsi sebagai indikator utama dalam proses optimasi.
3. **Metrik tujuan:** Metrik komponen yang menggambarkan setiap objektif secara terpisah. Termasuk di dalamnya *makespan*, jumlah penalti tinta, jumlah pekerjaan *tardy*.
4. ***Robustness*:** Stabilitas algoritma pada 30 *independent runs*. *Robustness* diukur menggunakan nilai rata-rata dan standar deviasi dari metrik pada berbagai *random seeds*.
5. **Efisiensi komputasi:** *Runtime* yang diperlukan untuk mencapai *stopping criterion*. *Runtime* dilaporkan untuk menilai apakah prosedur optimasi cukup cepat untuk penggunaan praktis.
6. **Metrik utilisasi mesin:** Metrik yang menggambarkan seberapa intensif setiap mesin digunakan. Metrik ini mencakup *processing time*, *time span*, efisiensi mesin, dan cakupan mesin, dan terutama digunakan dalam analisis skenario serta perbandingan dengan jadwal industri.

Metrik-metrik ini digunakan secara konsisten dalam penalaan parameter, penalaan bobot, eksperimen skenario, serta dalam perbandingan antara jadwal hasil optimasi dan *baseline* industri.

3.10.2 Visualisasi

Hasil eksperimen divisualisasikan untuk memberikan pemahaman yang lebih intuitif mengenai kinerja penjadwalan maupun perilaku algoritma. Beberapa jenis visualisasi digunakan secara saling melengkapi.

1. **Tabel ringkasan statistik:** Tabel digunakan untuk menyajikan ukuran kecenderungan sentral dan sebaran, seperti nilai rata-rata dan standar deviasi dari setiap metrik evaluasi pada berbagai pengulangan. Penyajian ini memberikan gambaran numerik yang ringkas dan informatif untuk setiap konfigurasi eksperimen.
2. **Diagram garis:** Diagram garis digunakan untuk memperlihatkan bagaimana metrik kinerja, seperti *fitness*, *makespan*, atau penalti tinta, berubah ketika suatu faktor desain divariasikan. Faktor tersebut dapat berupa parameter SA maupun kombinasi bobot tujuan. Visualisasi ini membantu mengidentifikasi pola hubungan dan *trade off* antar pengaturan.
3. **Plot *ternary*:** Untuk eksperimen yang melibatkan tiga bobot tujuan yang harus berjumlah satu, digunakan plot *ternary* dengan pewarnaan *heatmap*. Setiap titik pada *simplex* merepresentasikan satu kombinasi bobot, sedangkan warna menunjukkan nilai metrik kinerja tertentu. Cara ini memungkinkan identifikasi visual terhadap wilayah bobot yang memberikan performa terbaik.
4. **Plot konvergensi:** Perilaku konvergensi algoritma divisualisasikan melalui grafik *fitness* terhadap iterasi. Statistik agregat seperti median dan rentang *interkuartil* ditampilkan untuk menggambarkan pola perbaikan yang umum terjadi serta kestabilan proses pencarian sepanjang iterasi.
5. ***Gantt chart:*** *Gantt chart* digunakan untuk memvisualisasikan jadwal individual. Visualisasi ini menampilkan alokasi pekerjaan pada mesin dari waktu ke waktu, pergantian warna tinta, serta pekerjaan yang terlambat. Dengan demikian, berbagai metrik numerik seperti *makespan*, jumlah setup, dan *tardiness* dapat ditafsirkan dalam bentuk jadwal konkret di lini produksi.

3.11 Penarikan Kesimpulan

Untuk menarik kesimpulan dalam penelitian ini, seluruh hasil eksperimen dianalisis secara menyeluruh guna menjawab rumusan masalah mengenai efektivitas algoritma SA dalam menyelesaikan kasus penjadwalan UPMSP-SDST. Penarikan kesimpulan dilakukan dengan mengevaluasi stabilitas solusi pada setiap skenario pengujian serta membandingkan jadwal yang dihasilkan oleh sistem terhadap jadwal aktual yang diterapkan di industri. Perbandingan tersebut difokuskan pada pengukuran peningkatan efisiensi operasional, yang meliputi reduksi *makespan* dan penurunan frekuensi pergantian tinta, sehingga keunggulan dan kelayakan metode yang diusulkan dapat ditunjukkan secara objektif.

Sementara itu, saran untuk pengembangan penelitian selanjutnya disusun berdasarkan analisis terhadap keterbatasan dan perilaku algoritma yang diamati selama proses eksperimen. Evaluasi terhadap kendala teknis yang muncul digunakan sebagai dasar dalam merekomendasikan penyempurnaan strategi optimasi serta eksplorasi mekanisme algoritmik yang lebih adaptif pada penelitian mendatang.

BAB 4 PERANCANGAN

4.1 Dataset dan Praproses

4.1.1 Dataset Pekerjaan

Eksperimen menggunakan 15 pesanan produksi dengan variasi kuantitas dan kombinasi warna. Setiap warna telah dikonversi menjadi nilai numerik tingkat kegelapan untuk perhitungan setup. Setiap unit produk diasumsikan memiliki harga jual Rp1.500 per unit untuk keperluan perhitungan profitabilitas. Rincian pekerjaan disajikan pada Tabel 4.1.

Tabel 4.1 Data Pekerjaan dengan Warna yang Dikodekan

ID	Kuantitas	Warna	Tingkat Kegelapan Warna
1	16539	Biru, Ungu	6, 8
2	5020	Hijau, Cokelat, Merah	4, 5, 6
3	5110	Putih, Kuning, Ungu	1, 2, 8
4	6030	Merah, Biru	6, 6
5	6100	Hitam	9
6	24173	Biru, Ungu, Hitam	6, 8, 9
7	15740	Kuning, Hijau, Biru, Merah	2, 4, 6, 6
8	21165	Merah, Hijau, Biru	6, 4, 6
9	4640	Ungu	8
10	12100	Cokelat, Abu-abu, Hitam	5, 8, 9
11	8000	Ungu, Hitam	8, 9
12	14030	Merah, Cokelat, Biru, Ungu	6, 5, 6, 8
13	5180	Hijau, Abu-abu	4, 8
14	9150	Hitam	9
15	23979	Merah muda, Oranye, Biru	7, 4, 6

4.1.2 Data Spesifikasi Mesin

Lingkungan produksi terdiri atas tiga mesin heterogen (Flexo A, B, dan C). Berdasarkan data Hutomo (2022), waktu efektif produksi dalam satu *shift* (8 jam) setelah dikurangi rata-rata *downtime* adalah sekitar 348 menit. Kapasitas harian mesin telah dipraproses menjadi laju produksi per menit agar kompatibel dengan algoritma penjadwalan.

Tabel 4.2 Data Spesifikasi Mesin

ID Mesin	Kapasitas Harian	Kapasitas per Menit
A	39000	112

ID Mesin	Kapasitas Harian	Kapasitas per Menit
B	78000	224
C	61000	175

4.1.3 Biaya dan Durasi SDST

Model biaya dan waktu *setup* ditentukan berdasarkan karakteristik operasional berikut:

1. Biaya Pergantian Tinta ditetapkan sebesar Rp800.000 per kejadian pergantian dari warna gelap ke terang. Biaya ini merepresentasikan kerugian akibat waktu henti, penggunaan bahan pencuci, dan tinta yang terbuang.
2. Durasi SDST ditetapkan sebesar 45 menit pada setiap mesin yang mengalami pergantian tinta dengan SDST. Durasi ini diambil sebagai estimasi konservatif dari rentang data lapangan yaitu 30-60 menit.

4.1.4 Jadwal Industri *Baseline*

Untuk memvalidasi efektivitas algoritma, hasil optimasi akan dibandingkan dengan jadwal aktual yang diterapkan di industri. Jadwal ini disusun secara manual terutama berdasarkan ukuran pekerjaan tanpa optimasi urutan warna secara mendalam.

Tabel 4.3 Data Jadwal Industri *Baseline*

ID Mesin	Jadwal
A	4, 3, 5, 1, 2
B	9, 7, 8, 6, 10
C	11, 15, 14, 12, 13

4.2 Pemodelan Masalah

Penelitian ini membahas penjadwalan operasi pencetakan flexografi pada beberapa mesin cetak yang bekerja secara paralel. Setiap *job* dialokasikan tepat pada satu mesin dan ditempatkan dalam suatu urutan pemrosesan. Perbedaan laju produksi antar mesin serta variasi kombinasi tinta pada setiap *job* menyebabkan waktu proses bersifat heterogen dan menimbulkan *sequence-dependent setup* akibat pergantian tinta.

Tujuan penjadwalan adalah membangun jadwal produksi yang layak dan efisien dengan meminimalkan tiga kriteria utama, yaitu jumlah kejadian perubahan tinta yang memicu proses pencucian, *makespan* sebagai waktu penyelesaian total seluruh job, serta jumlah pekerjaan yang selesai melewati *due date*. Permasalahan ini diformulasikan sebagai UPMSP-SDST. Pendekatan metaheuristik digunakan untuk memperoleh solusi berkualitas tinggi secara efisien.

4.2.1 Lingkungan Produksi

Lingkungan produksi dimodelkan melalui tiga komponen utama, yaitu mesin, pekerjaan, dan setup. Setiap mesin cetak flexografi direpresentasikan sebagai satu unit dengan laju produksi tertentu, konfigurasi tinta aktif, serta garis waktu operasi. Setiap pekerjaan memiliki kuantitas produksi, kombinasi tinta yang dikodekan berdasarkan tingkat kegelapan warna, waktu kedatangan, dan *due date*. Pekerjaan dilepaskan ke sistem sesuai waktu kedatangannya dan dijadwalkan pada salah satu mesin yang tersedia.

4.2.2 *Sequence Dependent Setup*

Pergantian pekerjaan pada mesin yang sama dapat menuntut perubahan konfigurasi tinta. Jika pekerjaan berikutnya membutuhkan lebih sedikit tinta atau melibatkan transisi dari warna gelap ke terang, maka tinta yang tersisa harus dibersihkan terlebih dahulu untuk mencegah kontaminasi. Sebaliknya, transisi dari warna terang ke gelap tidak memerlukan pencucian.

Meskipun jumlah stasiun tinta yang dibersihkan dapat bervariasi, waktu setup dimodelkan secara biner. Jika tidak ada pencucian yang diperlukan, maka waktu setup tidak ditambahkan. Jika satu atau lebih pencucian diperlukan, maka disisipkan satu durasi setup tetap sebelum pekerjaan berikutnya dimulai. Dengan demikian, perhatian operasional difokuskan pada keberadaan proses pencucian, bukan jumlah stasiun tinta yang terlibat. Pendekatan ini mencerminkan praktik industri, di mana satu siklus pembersihan memiliki prosedur dan durasi yang sama.

Selain itu, setup bersifat *non-anticipatory*, yaitu tidak dapat dimulai sebelum pekerjaan berikutnya tersedia, meskipun mesin sedang menganggur. Asumsi ini mencegah penyusutan waktu setup secara tidak realistis melalui penyerapan ke dalam periode idle, sehingga jadwal yang dihasilkan tetap valid secara fisik dan operasional.

4.2.3 Fungsi Objektif

Kinerja suatu jadwal dievaluasi berdasarkan tiga tujuan utama, yaitu:

1. Penalti tinta: jumlah total kejadian pencucian yang disebabkan oleh transisi warna, khususnya dari tinta gelap ke tinta terang.
2. *Makespan*: waktu ketika mesin terakhir menyelesaikan seluruh pekerjaannya.
3. *Tardiness*: jumlah pekerjaan yang selesai melebihi batas waktu penyelesaiannya.

Ketiga komponen tersebut dikombinasikan ke dalam satu fungsi objektif berbentuk penjumlahan berbobot sebagai berikut:

$$\min F = w_t \cdot T + w_i \cdot I + w_m \cdot C_{max} \tag{4.1}$$

Di mana:

F : fungsi objektif yang akan diminimalkan

w_t : bobot penalti tardiness

w_i : bobot penalti perubahan tinta.

w_m : bobot makespan.

T : jumlah pekerjaan yang terlambat

I : jumlah total kejadian perubahan tinta

C_{max} : *makespan*.

Pada kasus statis, seluruh pekerjaan telah diketahui sejak awal perencanaan sehingga fungsi objektif difokuskan pada minimalisasi penalti pergantian tinta dan *makespan*. Pendekatan ini bertujuan memperoleh urutan produksi yang efisien secara operasional dengan waktu penyelesaian total yang minimal. Sebaliknya, pada kasus dinamis di mana pekerjaan masuk dalam beberapa *batch* pada waktu yang berbeda, komponen *tardiness* diaktifkan untuk mengendalikan ketepatan penyelesaian setiap *batch*. Penalti jumlah pekerjaan terlambat digunakan sebagai batas waktu penyelesaian agar penjadwalan tetap adil, sehingga pekerjaan yang datang lebih awal tidak terus tertunda dan pekerjaan baru tidak diprioritaskan secara tidak proporsional.

4.2.4 Penjadwalan Statis dan Dinamis

Penelitian ini mempertimbangkan dua mode penjadwalan, yaitu statis dan dinamis. Pada penjadwalan statis, seluruh *job* telah diketahui dan tersedia sejak awal *horizon* perencanaan. Algoritma bertujuan untuk meminimalkan *makespan* dan jumlah perubahan tinta untuk keseluruhan himpunan pekerjaan.

Sebaliknya, pada penjadwalan dinamis, *job* tiba secara bertahap seiring berjalannya waktu. Pendekatan *rolling-horizon* diterapkan untuk melakukan optimasi ulang setiap kali *job* baru tiba. Pada setiap titik optimasi ulang, *job* yang telah selesai atau sedang dikerjakan akan dikunci, sementara pekerjaan yang tersisa dan yang baru datang akan diurutkan ulang berdasarkan kondisi mesin terkini. *Due date* ditetapkan menggunakan aturan TWK untuk memastikan pembagian waktu yang adil di antara kelompok pekerjaan. Mekanisme ini memungkinkan sistem tetap stabil sekaligus responsif terhadap pesanan baru.

4.2.5 Kendala Penjadwalan

Suatu jadwal dianggap layak apabila memenuhi kendala operasional proses pencetakan berikut:

1. Setiap *job* harus muncul tepat satu kali dan hanya dapat dialokasikan pada satu mesin cetak.
2. Suatu *job* tidak dapat dimulai sebelum waktu kedatangannya.
3. *Job* yang berada pada mesin yang sama tidak boleh tumpang tindih dalam waktu pengerjaan.
4. Waktu *setup* disisipkan di antara dua pekerjaan pada mesin yang sama apabila konfigurasi tintanya menghasilkan SDST.

5. Mesin tidak diperbolehkan mengalami waktu menganggur apabila terdapat pekerjaan yang telah tersedia untuk diproses.

Kendala-kendala ini memastikan bahwa jadwal yang dihasilkan dapat langsung diterapkan di lantai produksi tanpa perlu penyesuaian manual tambahan.

4.2.6 Asumsi dan Batasan

Penelitian ini menerapkan beberapa asumsi penyederhanaan guna memfokuskan perhatian pada kompleksitas utama dari UPMSP-SDST, sekaligus menjaga agar model tetap terkelola secara komputasional, yaitu:

1. Mesin diasumsikan selalu tersedia, tidak ada gangguan *breakdown* maupun perawatan.
2. Pekerjaan bersifat *non-preemptive*, setelah dimulai, pekerjaan harus diselesaikan tanpa interupsi.
3. Waktu kedatangan dan waktu proses bersifat deterministik serta diketahui sebelumnya.
4. Durasi *setup* bersifat konstan untuk setiap kejadian dan hanya bergantung pada ada atau tidaknya proses pencucian.
5. Urutan posisi tinta pada mesin tidak berpengaruh, hanya himpunan tinta yang digunakan dalam setiap pekerjaan yang memengaruhi perilaku *setup*.
6. Setiap pergantian *job* memicu maksimal satu kejadian *setup*, terlepas dari berapa banyak stasiun tinta yang perlu dibersihkan.
7. Operasi *setup* bersifat *non-anticipatory,* artinya tidak dapat dimulai sebelum *job* berikutnya tersedia. Waktu tunggu dan *setup* diperlakukan sebagai dua keadaan operasional yang terpisah.

Asumsi-asumsi ini menyederhanakan model sehingga optimasi dapat berfokus pada keputusan urutan dan penugasan mesin tanpa diganggu oleh variasi operasional yang tidak terprediksi seperti pembersihan parsial, risiko kontaminasi tinta, atau kegagalan mesin.

4.3 Representasi Solusi dan Konfigurasi

4.3.1 Representasi Solusi

Sebuah solusi merepresentasikan keseluruhan jadwal produksi, pekerjaan mana yang dijalankan pada mesin mana, dan dalam urutan seperti apa. Struktur ini secara berulang diperbarui oleh algoritma optimasi, yang kemudian memilih hasil terbaik dari seluruh iterasi yang dilakukan.

Tabel 4.4 Representasi Solusi

Atribut	**Deskripsi**	**Contoh**
Schedule	Pemetaan antara mesin dan urutan *job* yang dikerjakan.	Flexo A: [J1, J4, J6]Flexo B: [J2, J5, J8]Flexo C: [J3, J7]

Atribut	Deskripsi	Contoh
Fitness Value	Nilai gabungan dari bobot *tardiness*, perubahan tinta, dan *makespan*. Nilai yang lebih rendah menunjukkan hasil yang lebih baik.	4,753
Tardy Count	Jumlah pekerjaan yang selesai melewati batas *due date*.	2 pekerjaan terlambat
Ink Change Count	Jumlah total kejadian pencucian antara dua pekerjaan berturutan.	5 perubahan
Makespan	Total waktu hingga seluruh mesin menyelesaikan pekerjaannya.	7,51 jam
Critical Machine	Mesin yang menyelesaikan pekerjaan paling akhir dan menentukan nilai *makespan*.	Flexo B

Daftar jadwal tersebut mudah diinterpretasikan, setiap mesin memiliki antrean pekerjaan, dan urutannya secara langsung menentukan kebutuhan setup serta waktu penyelesaian.

Tabel 4.5 Representasi Pekerjaan

Atribut	Deskripsi	Contoh
Job ID	Identitas unik untuk setiap pesanan.	1
Quantity	Jumlah unit yang akan dicetak.	16.539
Ink Set	Daftar tinta atau tingkat kegelapan warna yang digunakan.	[6, 8]
Arrival Time	Waktu kedatangan job ke sistem untuk dijadwalkan.	0 jam
Due Date	Batas waktu penyelesaian yang ditetapkan menggunakan aturan TWK.	3 jam
Start Time	Waktu mulai berdasarkan hasil simulasi jadwal.	2 jam
Completion Time	Waktu selesai berdasarkan hasil simulasi jadwal.	4 jam

4.3.2 *State* Mesin

Setiap mesin menyimpan catatan singkat mengenai apa yang sudah bersifat tetap dan apa yang masih dapat diubah. Hal ini mencegah optimasi baru mengubah pekerjaan yang telah dikerjakan sebelumnya.

Tabel 4.6 Representasi State Mesin

Atribut	Deskripsi	Contoh
Current Time	Waktu pada mesin setelah menyelesaikan pekerjaan terakhir yang terkunci.	6,0 jam
Current Inks	Set tinta yang saat ini terpasang pada mesin.	[6, 8]
Locked Jobs	Daftar pekerjaan yang sudah selesai atau sedang berlangsung (tidak dapat diubah).	[1, 3]

Atribut	Deskripsi	Contoh
Unlocked Jobs	Daftar pekerjaan yang masih dapat dioptimasi pada siklus saat ini.	[7]

Pemisahan ini menjamin bahwa setiap optimasi baru tidak akan memodifikasi pekerjaan yang telah dikunci, sehingga konsistensi antar *horizon* pada algoritma *rolling-horizon* tetap terjaga.

4.3.3 Konfigurasi

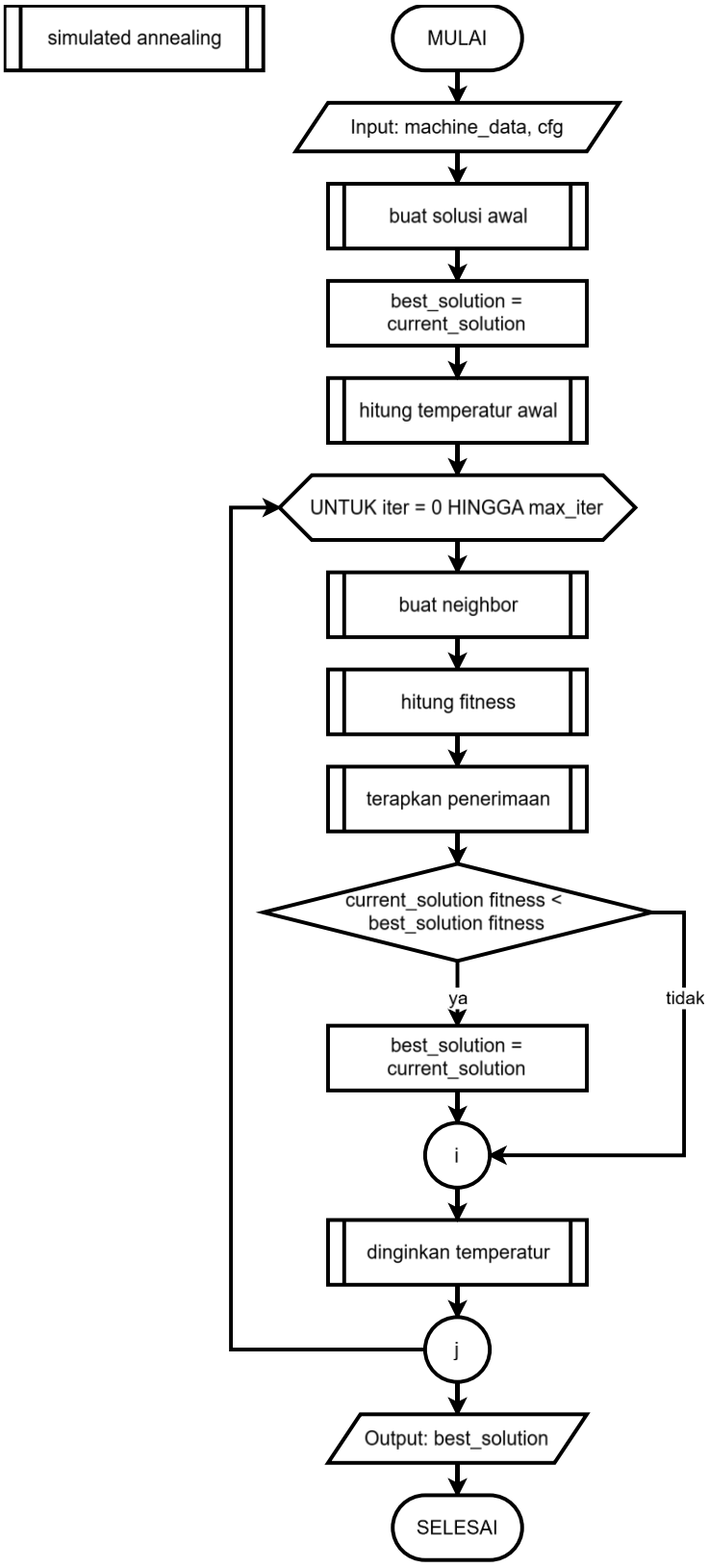
Bagian konfigurasi mendefinisikan lingkungan produksi, prioritas tujuan optimasi, serta perilaku pencarian algoritma. Pengaturan ini memastikan bahwa hasil eksekusi dapat direproduksi dan dibandingkan secara konsisten antar skenario.

Tabel 4.7 Konfigurasi *Optimizer*

Kategori	Nama Parameter	Deskripsi	Contoh
Lingkungan	Machine Rates	Kecepatan produksi masing-masing mesin (unit per menit).	Flexo A = 112 u/menit Flexo B = 224 u/menit Flexo C = 175 u/menit
	Setup Duration	Waktu tetap yang diperlukan saat terjadi pergantian tinta.	0,75 jam
	Initial State	Kondisi awal mesin pada setiap *horizon* penjadwalan (waktu terkini dan tinta yang terpasang).	Flexo A: time = 6,0 jam, inks = [6, 8]Flexo B: time = 5,2 jam, inks = [4, 5, 6]Flexo C: time = 4,8 jam, inks = [9]
Bobot Objektif	Tardiness Weight	Tingkat kepentingan dalam meminimalkan pekerjaan terlambat.	2
	Ink Change Weight	Tingkat kepentingan dalam meminimalkan jumlah pencucian.	1
	Makespan Weight	Tingkat kepentingan dalam meminimalkan total waktu penyelesaian.	0,1
Aturan Due Date	Due-Date Multiplier	Faktor pengali beban sistem untuk menentukan target due date.	1,0
Reproduksibilitas	Random Seed	Menjamin urutan acak yang identik pada setiap eksekusi ulang.	42
Simulated Annealing	Maximum Iterations	Batas jumlah iterasi per percobaan.	10.000

Kategori	Nama Parameter	Deskripsi	Contoh
	Critical Machine Bias	Peluang algoritma untuk memfokuskan pencarian pada mesin yang paling terbebani.	0,7
Pengaturan Suhu	Initial Acceptance Rate	Probabilitas awal untuk menerima solusi yang lebih buruk di tahap awal optimasi.	0,8
	Temperature Floor Ratio	Batas suhu terendah yang diizinkan sebagai proporsi dari suhu awal.	0,05
	Cooling Factor	Laju penurunan suhu pada setiap iterasi.	0,995

4.4 Desain Algoritma *Simulated Annealing*



Gambar 4.1 *Flowchart* Algoritma SA

Dalam penelitian ini, SA digunakan sebagai kerangka optimasi untuk menghasilkan jadwal produksi yang efisien pada lingkungan mesin cetak flexografi yang heterogen. Algoritma diawali dari jadwal awal yang layak, kemudian secara iteratif membangkitkan solusi tetangga melalui modifikasi acak yang terkontrol. Setiap solusi dievaluasi menggunakan fungsi *fitness* yang mempertimbangkan penalti perubahan tinta, *makespan*, dan keterlambatan pekerjaan sebagai representasi efisiensi keseluruhan jadwal.

Penerimaan solusi baru ditentukan oleh selisih nilai *fitness* dan tingkat temperatur saat itu. Pada temperatur tinggi, algoritma bersifat lebih eksploratif dengan peluang menerima solusi yang lebih buruk, sedangkan pada temperatur rendah pencarian menjadi lebih selektif dan berfokus pada perbaikan solusi. Mekanisme ini memungkinkan transisi bertahap dari eksplorasi menuju eksploitasi tanpa terjebak pada solusi lokal.

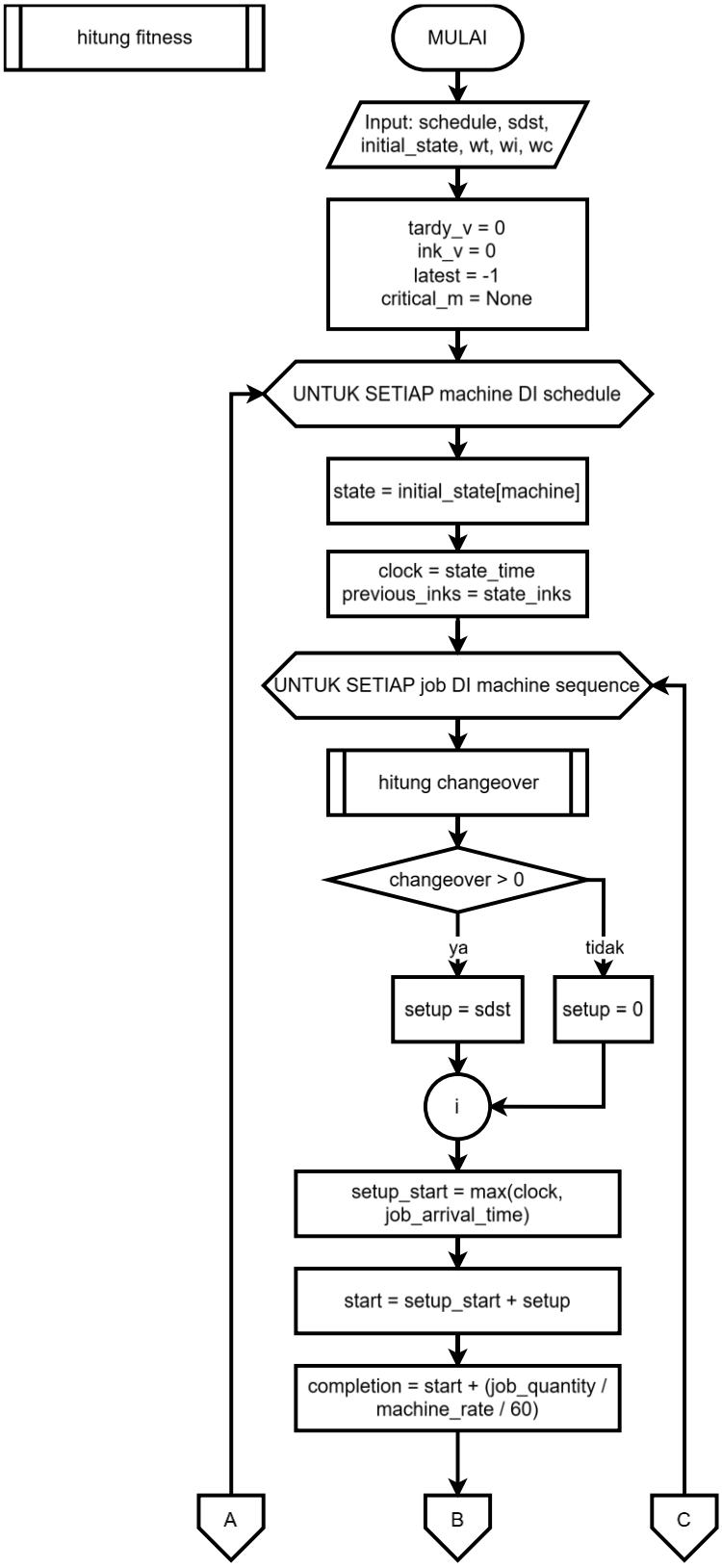
Secara umum, proses SA pada penelitian ini meliputi tahapan inisialisasi solusi, pembangkitan tetangga, evaluasi solusi, keputusan penerimaan, serta pendinginan temperatur, sebagaimana ditunjukkan pada Gambar 4.1. Rincian desain dan implementasi setiap tahapan dijelaskan pada subbab berikutnya.

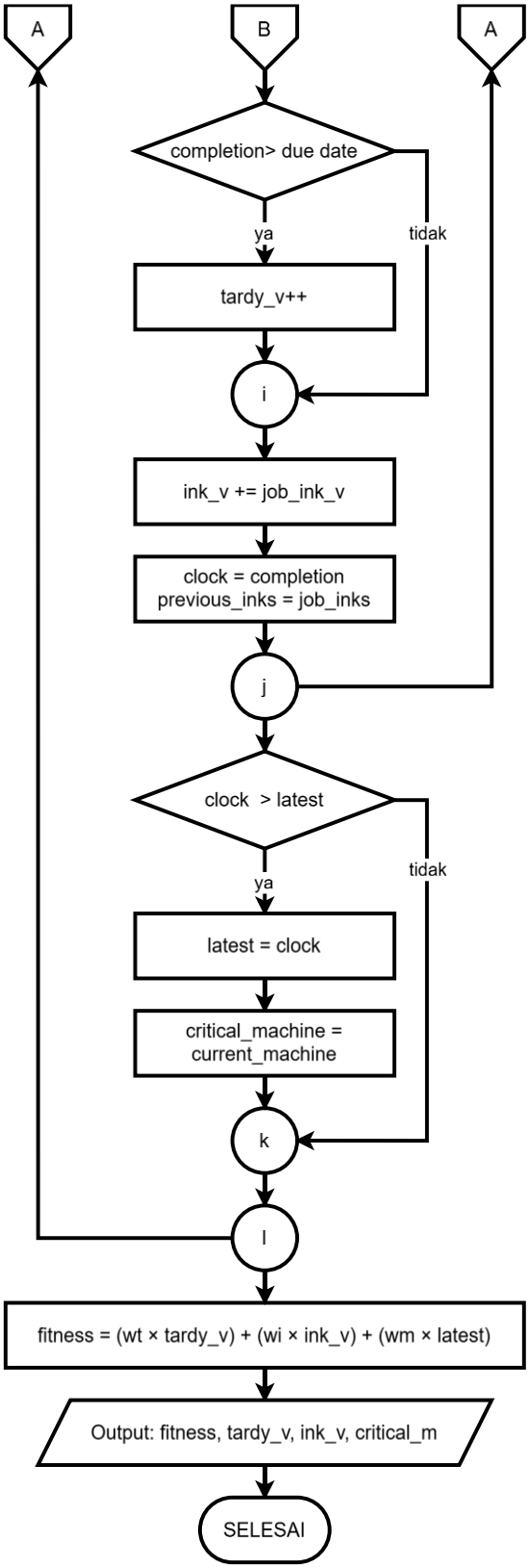
4.4.1 Hitung *Fitness*

Fungsi *calculate fitness* digunakan untuk mengevaluasi kualitas suatu jadwal kandidat terhadap tujuan optimasi yang telah ditetapkan. Evaluasi dilakukan dengan mensimulasikan urutan pengerjaan pada setiap mesin cetak sepanjang *horizon* produksi untuk menghitung total penalti perubahan tinta, keterlambatan pekerjaan, dan *makespan*, yang bersama-sama membentuk nilai *fitness* total.

Untuk setiap mesin, lini masa operasi direkonstruksi berdasarkan waktu ketersediaan mesin dan konfigurasi tinta terakhir. Kebutuhan setup antar pekerjaan berturut-turut dihitung menggunakan fungsi *changeover* dan ditambahkan ke waktu mesin apabila terjadi transisi tinta, khususnya dari warna gelap ke terang. Waktu mulai pekerjaan ditentukan sebagai nilai maksimum antara waktu mesin saat ini dan waktu kedatangan pekerjaan, sedangkan waktu penyelesaian dihitung berdasarkan kuantitas produksi dan laju proses mesin. Jika pekerjaan selesai melewati *due date*, maka penalti keterlambatan ditambahkan.

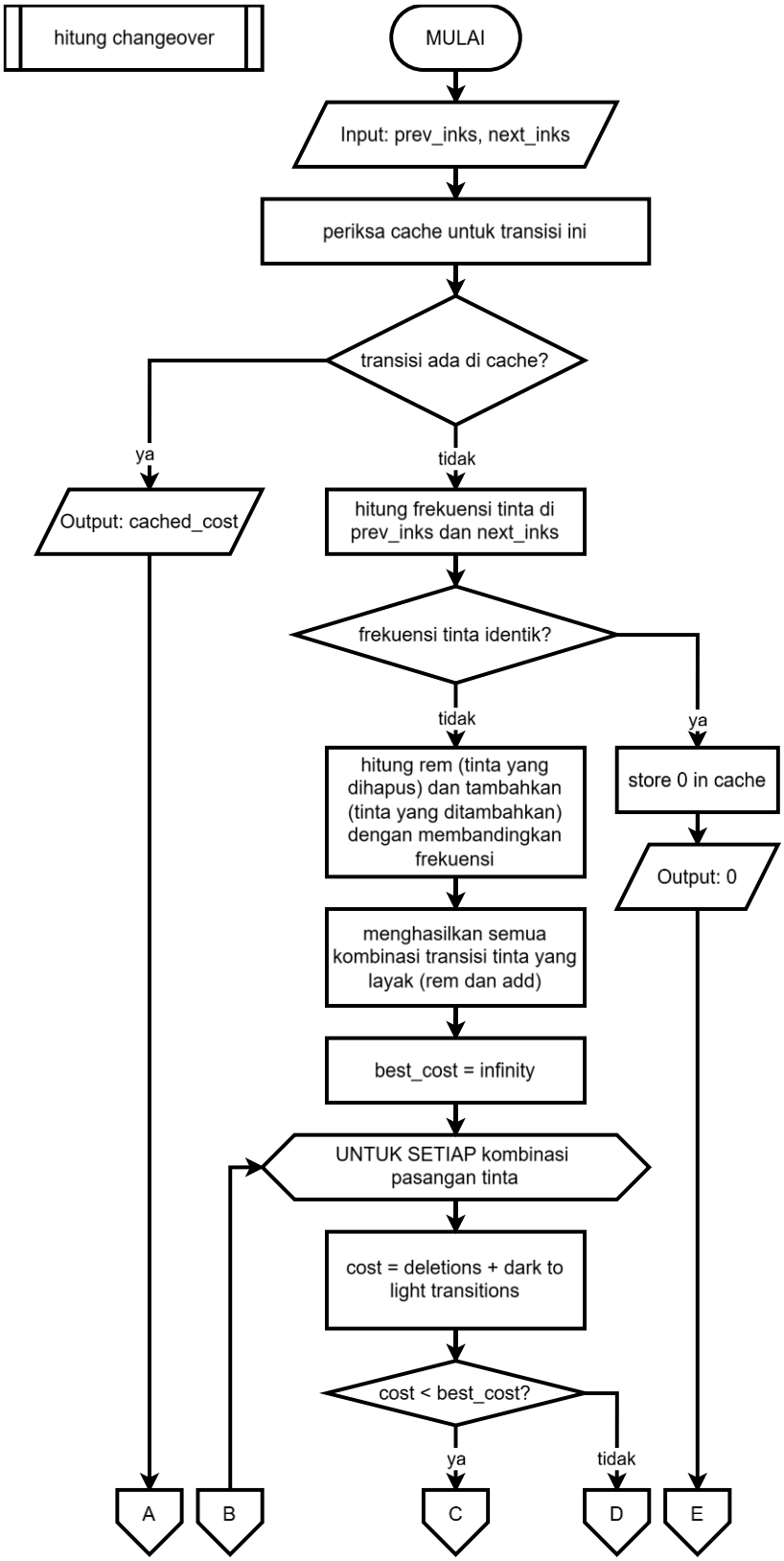
Setelah seluruh mesin disimulasikan, *makespan* diambil sebagai waktu penyelesaian terbesar, dan mesin yang mencapainya diidentifikasi sebagai mesin kritis. Nilai penalti perubahan tinta, keterlambatan, dan *makespan* kemudian digabungkan menggunakan bobot yang telah ditentukan untuk menghasilkan satu nilai *fitness*, di mana nilai yang lebih rendah menunjukkan jadwal yang lebih efisien dan seimbang. Nilai ini digunakan oleh algoritma SA sebagai dasar perbandingan dan pemilihan solusi selama proses optimasi.

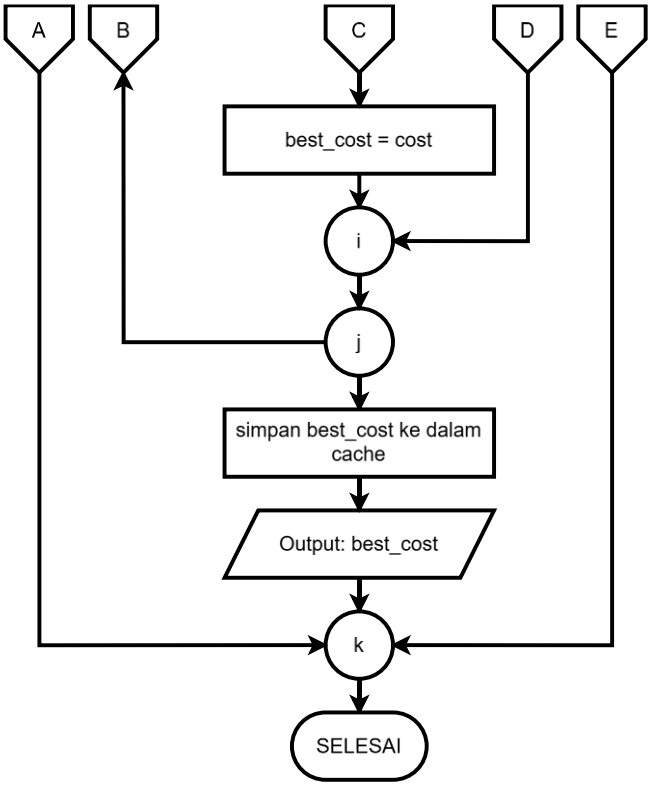

hitung fitness
MULAI
Input: schedule, sdst, initial_state, wt, wi, wc
tardy_v = 0
ink_v = 0
latest = -1
critical_m = None
UNTUK SETIAP machine DI schedule
state = initial_state[machine]
clock = state_time
previous_inks = state_inks
UNTUK SETIAP job DI machine sequence
hitung changeover
changeover > 0
ya
tidak
setup = sdst
setup = 0
i
setup_start = max(clock, job_arrival_time)
start = setup_start + setup
completion = start + (job_quantity / machine_rate / 60)
A
B
C



Gambar 4.2 *Flowchart* Algoritma Perhitungan *Fitness*

4.4.2 Model *Changeover*





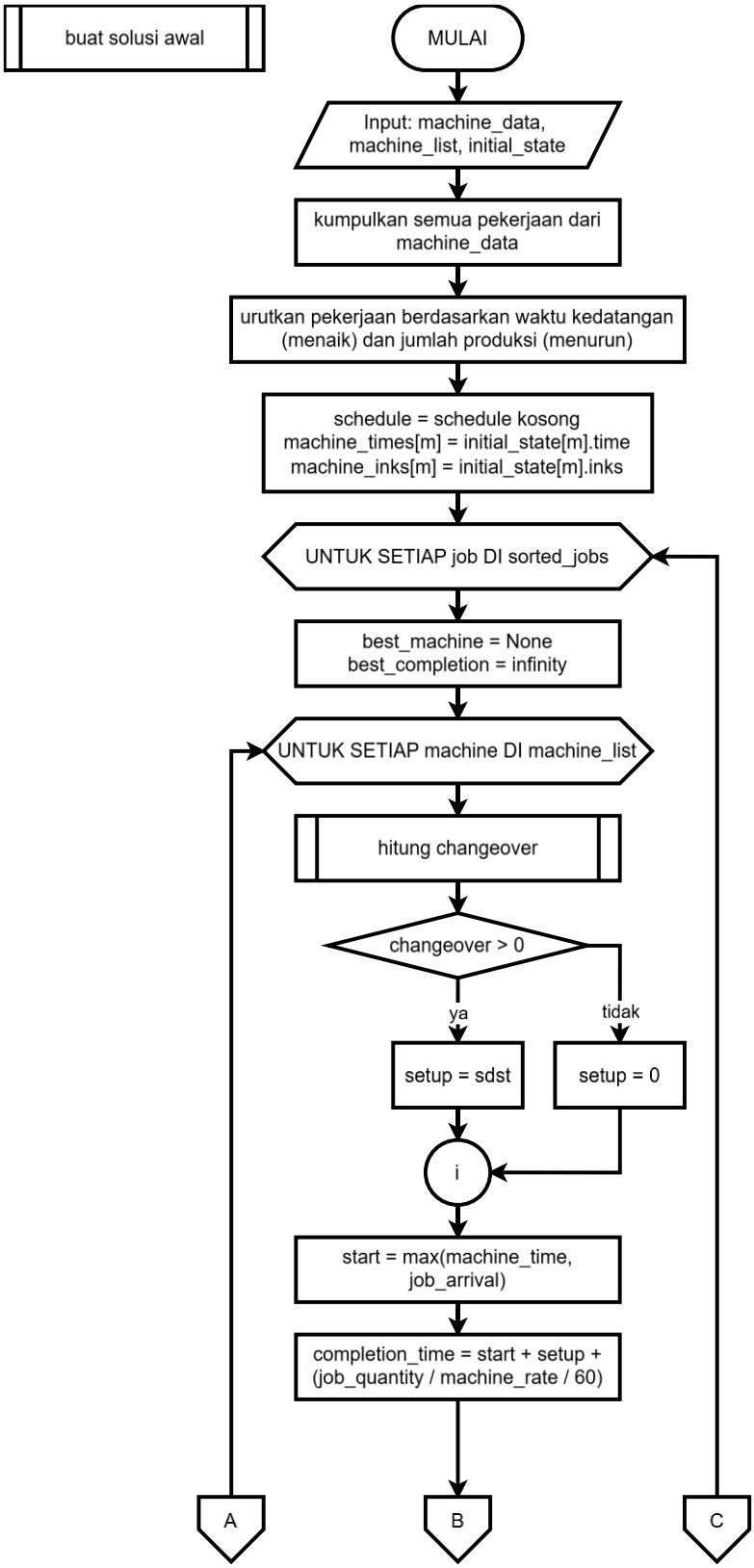
Gambar 4.3 *Flowchart* Algoritma Hitung Perubahan Tinta

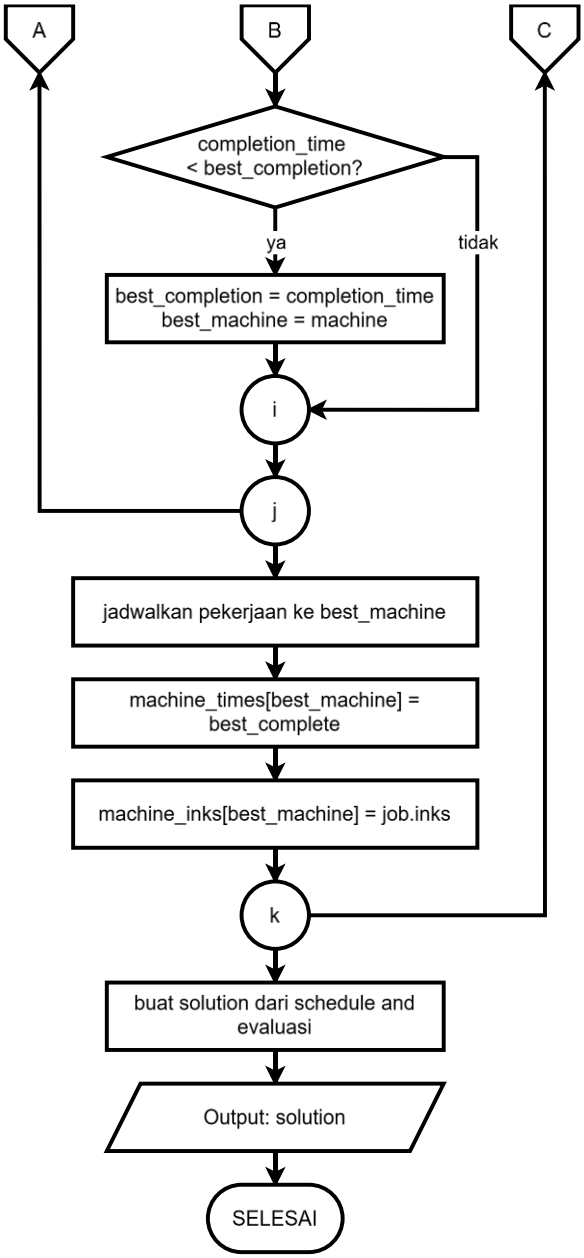
Model *changeover* digunakan untuk menghitung biaya setup akibat transisi antara dua pekerjaan berturut-turut pada mesin yang sama berdasarkan konfigurasi tinta masing-masing. Komponen ini berperan penting dalam penjadwalan flexografi, karena perubahan tinta, khususnya dari warna gelap ke terang, memerlukan proses pencucian yang lebih intensif sehingga menurunkan ketersediaan mesin dan meningkatkan limbah produksi.

Model menerima dua konfigurasi tinta yang mewakili pekerjaan sebelumnya dan pekerjaan berikutnya. Untuk meningkatkan efisiensi komputasi, sistem terlebih dahulu memeriksa *cache* untuk mengetahui apakah kombinasi transisi tinta tersebut telah dievaluasi sebelumnya. Jika tersedia, nilai *changeover* langsung digunakan. Apabila kedua konfigurasi tinta identik, tidak diperlukan proses pembersihan dan biaya *changeover* bernilai nol.

Jika konfigurasi tinta berbeda, algoritma menentukan tinta yang harus dihapus dan ditambahkan dengan membandingkan perbedaan frekuensi tingkat kegelapan antara kedua pekerjaan. Seluruh pasangan transisi yang mungkin antara tinta yang dihapus dan ditambahkan kemudian dievaluasi, dengan biaya dihitung berdasarkan jumlah penghapusan serta jumlah transisi dari warna gelap ke terang sebagai representasi aktivitas pencucian. Kombinasi dengan total biaya terendah dipilih sebagai nilai *changeover* optimal dan disimpan kembali ke dalam *cache* untuk penggunaan selanjutnya. Pendekatan ini memungkinkan estimasi biaya setup yang bergantung pada urutan secara efisien sekaligus merepresentasikan sifat asimetris perubahan tinta dalam praktik produksi nyata.

4.4.3 Inisialisasi Solusi





Gambar 4.4 *Flowchart* Algoritma Inisialisasi Solusi *Greedy*

Prosedur inisialisasi solusi bertujuan membangun jadwal awal yang layak sebagai titik awal bagi proses optimasi SA. Proses inisialisasi diawali dengan menggabungkan seluruh pekerjaan yang tersedia dari data input, kemudian mengurutkannya berdasarkan waktu kedatangan secara menaik dan kuantitas produksi secara menurun. Strategi ini memastikan bahwa pekerjaan yang datang lebih awal dan berukuran lebih besar diprioritaskan.

Setiap mesin diinisialisasi dengan waktu kerja nol dan konfigurasi tinta kosong. Algoritma kemudian secara iteratif menetapkan setiap pekerjaan ke mesin yang menghasilkan waktu penyelesaian tercepat, yang dihitung berdasarkan ketersediaan mesin saat ini, kebutuhan *changeover*, dan laju proses produksi.

Fungsi *changeover* menentukan apakah diperlukan waktu *setup* antara dua pekerjaan berturut-turut dan menambahkan durasi SDST jika relevan.

Setelah mesin terbaik bagi suatu pekerjaan ditemukan, pekerjaan tersebut ditambahkan ke urutan mesin tersebut, dan lini masa operasi serta konfigurasi tinta mesin diperbarui. Proses ini berlanjut hingga seluruh pekerjaan teralokasikan, menghasilkan jadwal awal yang seimbang antar mesin heterogen sambil tetap memperhitungkan transisi tinta dan batas waktu kedatangan.

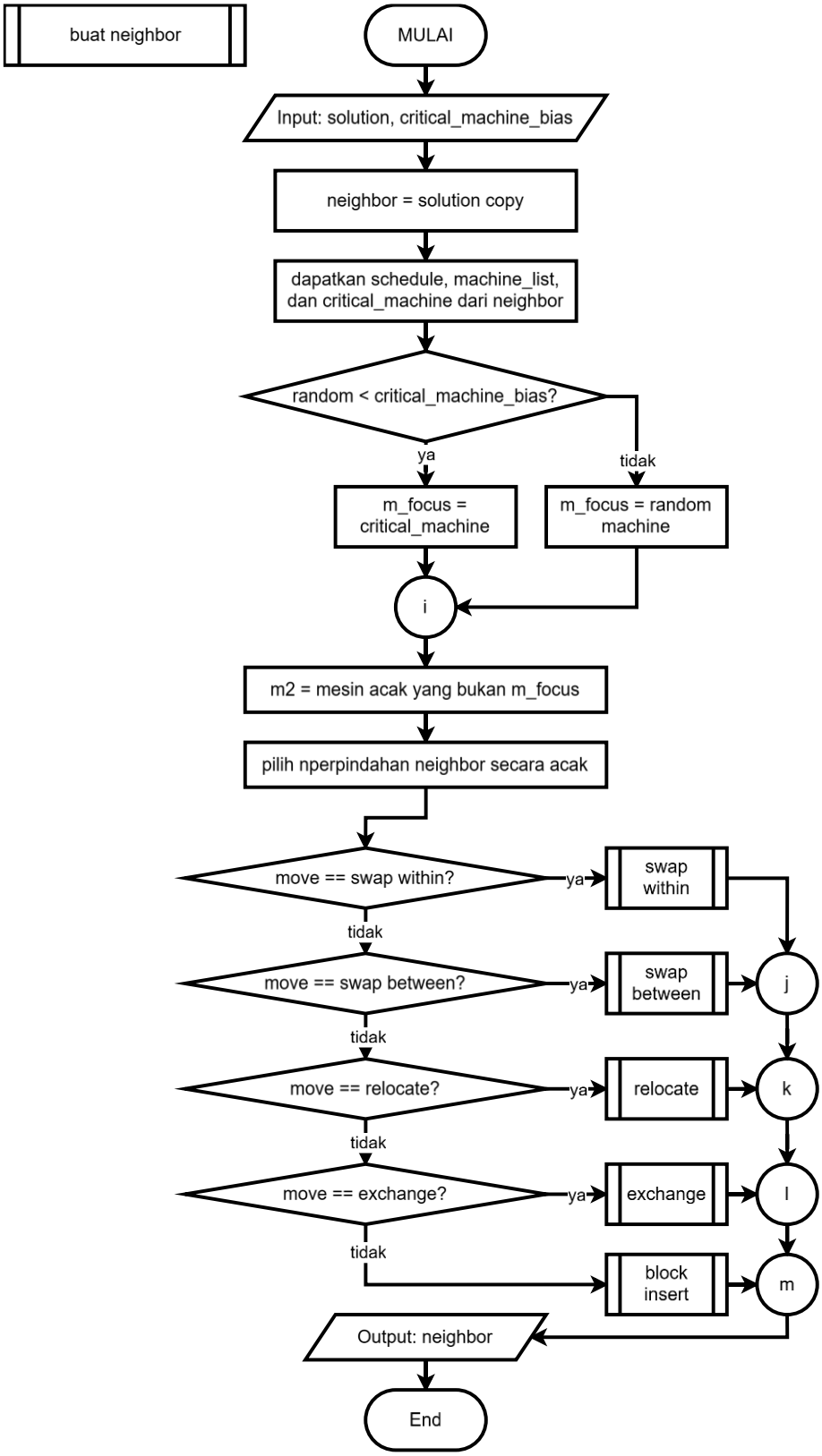
Jadwal yang dihasilkan kemudian dibuat dalam objek *Solution* dan dievaluasi menggunakan fungsi *fitness* untuk menghasilkan nilai acuan awal bagi algoritma SA. Meskipun pendekatan *greedy* ini tidak menjamin hasil optimal, ia menyediakan titik awal yang kuat dan layak, sehingga algoritma SA dapat mengeksplorasi konfigurasi yang lebih baik secara efisien selama proses iterasi selanjutnya.

4.4.4 Struktur *Neighborhood*

Struktur *neighborhood* menentukan bagaimana solusi kandidat baru dibuat dari jadwal saat ini selama proses optimasi. Dalam SA, mekanisme ini memungkinkan eksplorasi ruang pencarian melalui modifikasi acak yang terkontrol terhadap solusi saat ini. Setiap modifikasi menghasilkan jadwal *neighbor* yang berbeda secara lokal dari jadwal semula.

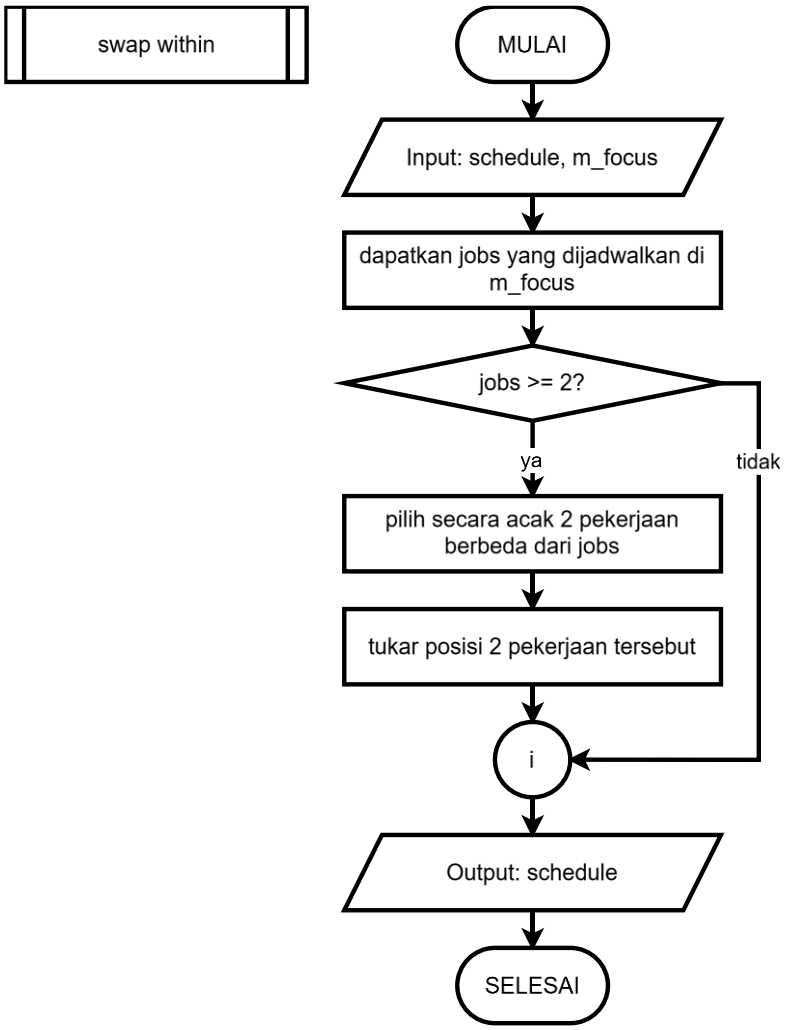
Proses pembuatan *neighbor* diawali dengan menyalin solusi saat ini, kemudian menentukan mesin yang akan dimodifikasi. Pemilihan mesin dilakukan menggunakan skema *critical machine bias*, yaitu sebuah parameter probabilitas dengan nilai antara 0 hingga 1. Jika nilai bias bernilai 0, mesin yang akan dimodifikasi dipilih sepenuhnya secara acak dari seluruh mesin yang tersedia. Sebaliknya, jika nilai bias bernilai 1, mesin yang dipilih selalu merupakan *mesin kritis*, yaitu mesin dengan waktu penyelesaian terbesar yang menentukan nilai *makespan* pada jadwal saat ini. Untuk nilai di antara kedua batas tersebut, pemilihan mesin dilakukan secara probabilistik antara mesin kritis dan mesin lain, sehingga sifat stokastik pencarian tetap terjaga.

Selanjutnya, apabila operator *neighborhood* yang dipilih melibatkan dua mesin, mesin kedua ditentukan secara acak dari mesin lain yang berbeda. Setelah mesin target ditetapkan, algoritma memilih salah satu operator *neighborhood* secara acak untuk memodifikasi urutan pekerjaan. Operator yang digunakan dalam penelitian ini meliputi *swap within*, *swap between*, *relocate*, *exchange*, dan *block insert*, yang masing-masing menghasilkan pola perubahan jadwal yang berbeda.



Gambar 4.5 *Flowchart* Algoritma Buat *Neighbor*

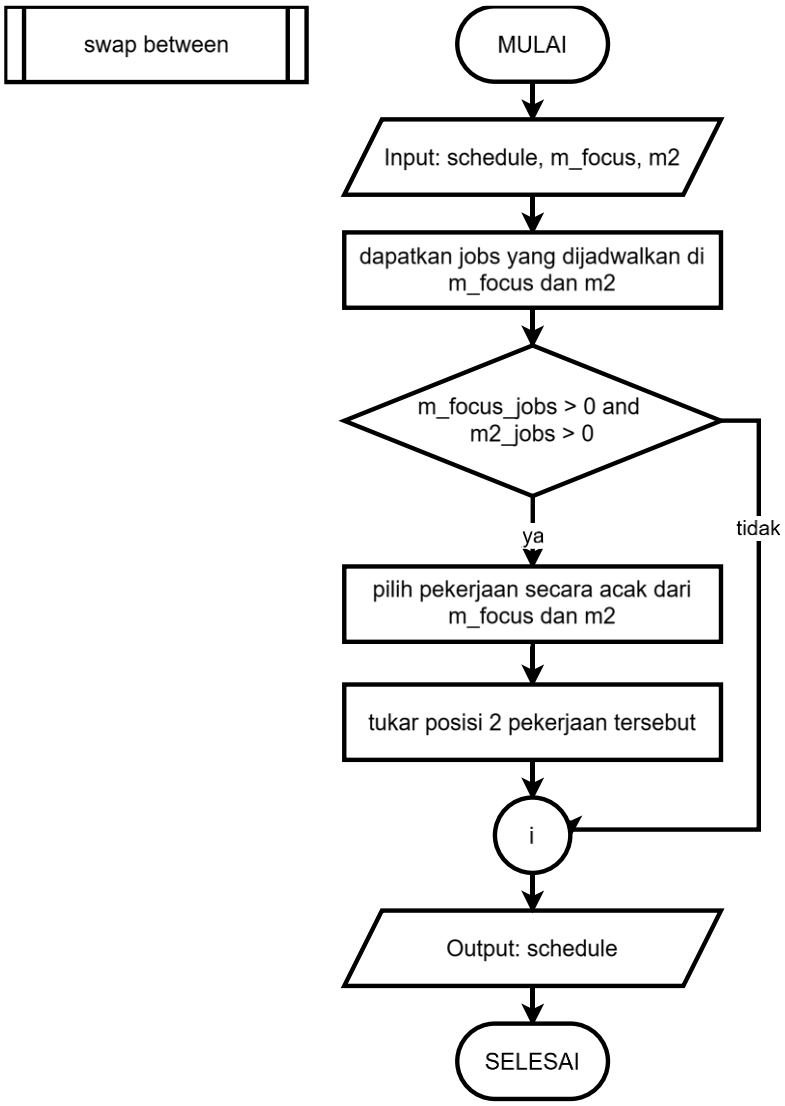
4.4.4.1 *Swap Within*



Gambar 4.6 *Flowchart* Algoritma Swap Within

Operator *swap within* melakukan modifikasi lokal pada satu mesin dengan menukar posisi dua pekerjaan yang dipilih secara acak dari urutan pekerjaan mesin yang sama. Operator ini hanya mengubah urutan internal tanpa memengaruhi alokasi pekerjaan antar mesin. Dampak perubahan yang dihasilkan relatif kecil dan terkontrol, sehingga sesuai untuk eksplorasi perbaikan urutan pekerjaan, khususnya yang berkaitan dengan pengurangan biaya setup atau keterlambatan lokal.

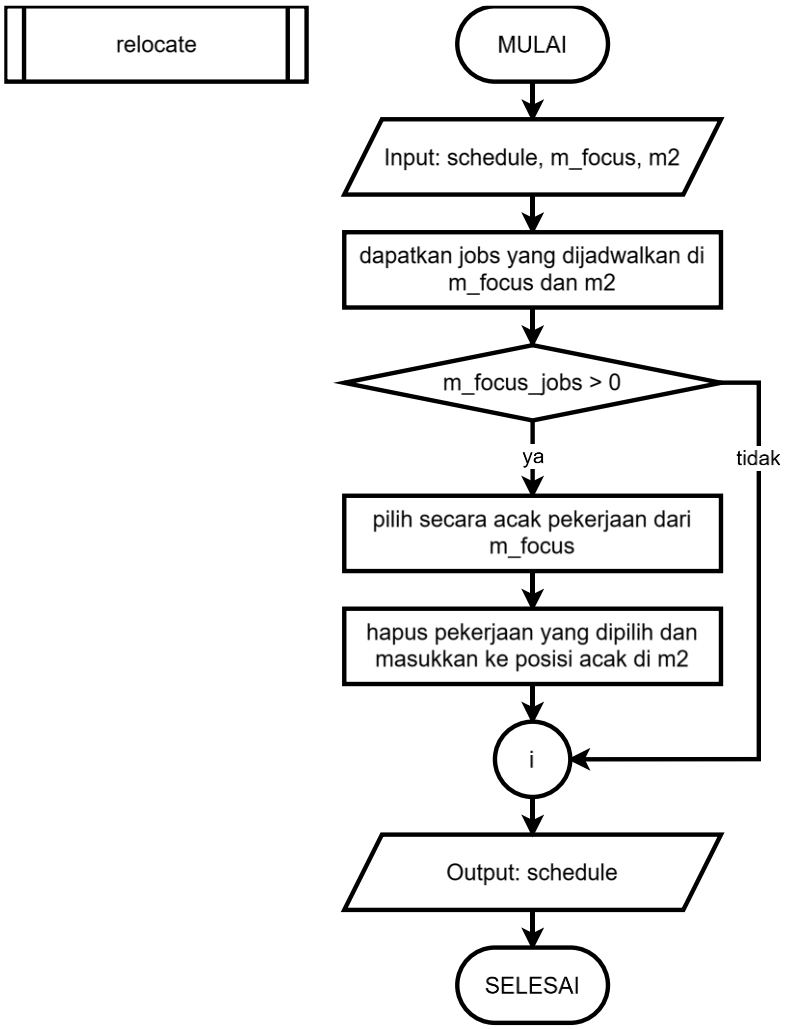
4.4.4.2 *Swap Between*



Gambar 4.7 *Flowchart* Algoritma Swap *Between*

Operator *swap between* menukar dua pekerjaan yang masing-masing berasal dari dua mesin yang berbeda. Satu pekerjaan dipilih dari mesin fokus dan satu dari mesin lain, kemudian keduanya dipertukarkan. Operator ini memengaruhi beban kerja dua mesin secara simultan dan memungkinkan perubahan distribusi pekerjaan antar mesin, sehingga berpotensi memperbaiki keseimbangan beban dan nilai *makespan*.

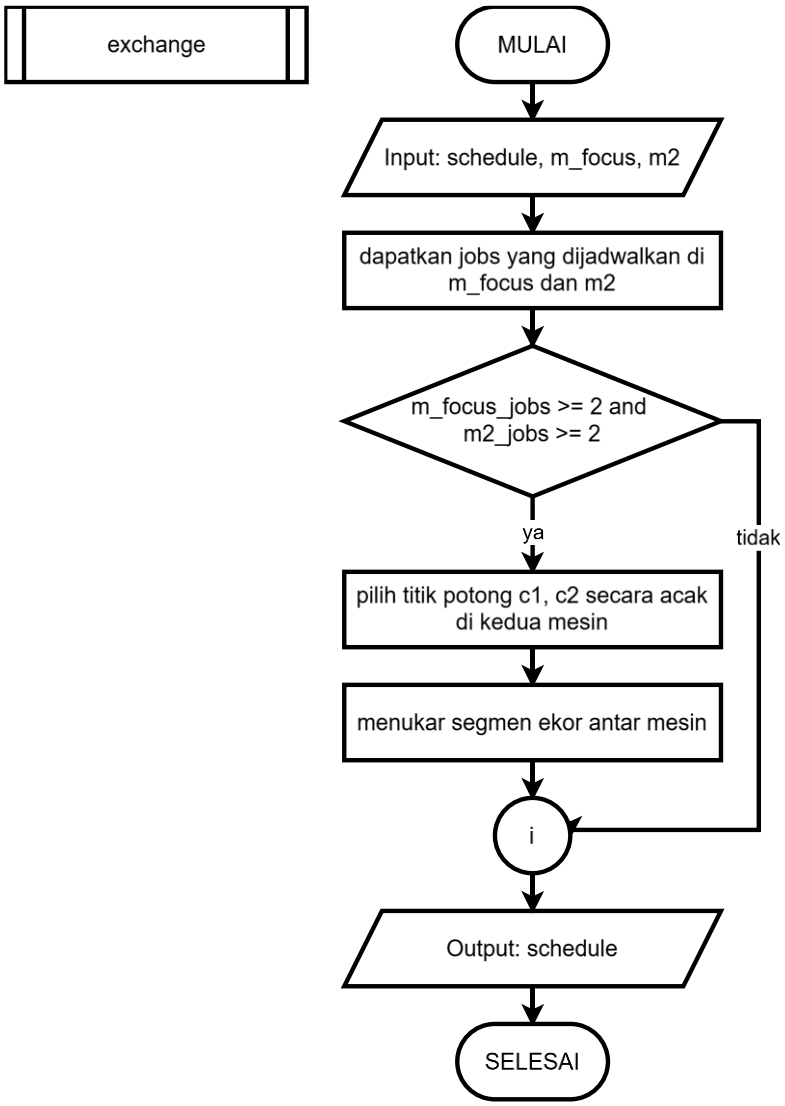
4.4.4.3 *Relocate*



Gambar 4.8 *Flowchart* Algoritma *Relocate*

Operator *relocate* memindahkan satu pekerjaan dari suatu mesin ke mesin lain dengan menyisipkannya pada posisi acak di urutan tujuan. Berbeda dari *swap between*, operator ini mengubah jumlah pekerjaan pada masing-masing mesin. Mekanisme ini efektif untuk menyesuaikan distribusi beban kerja antar mesin heterogen, terutama ketika terdapat ketimpangan waktu penyelesaian akibat perbedaan laju produksi.

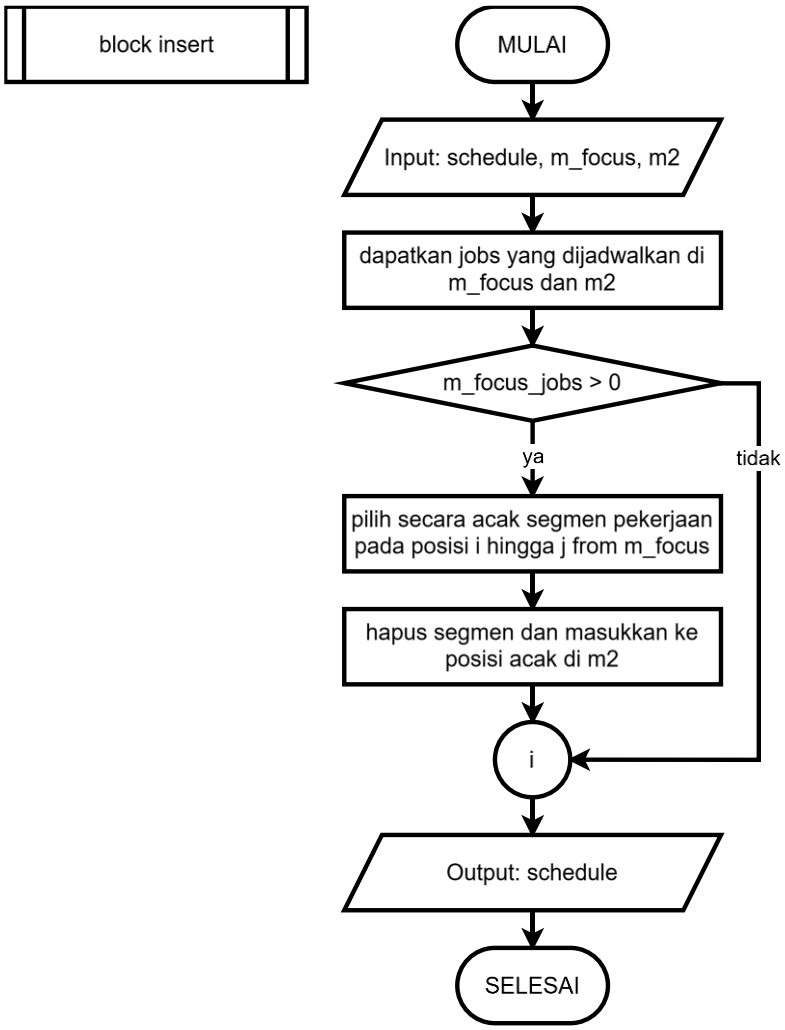
4.4.4.4 *Exchange*



Gambar 4.9 *Flowchart* Algoritma *Exchange*

Operator *exchange* melakukan pertukaran segmen urutan pekerjaan antara dua mesin. Titik potong dipilih secara acak pada masing-masing urutan, kemudian bagian setelah titik tersebut saling ditukar. Operator ini menghasilkan perubahan yang lebih besar dibandingkan operator lainnya, karena melibatkan beberapa pekerjaan sekaligus, sehingga memungkinkan eksplorasi solusi yang lebih jauh dari konfigurasi saat ini.

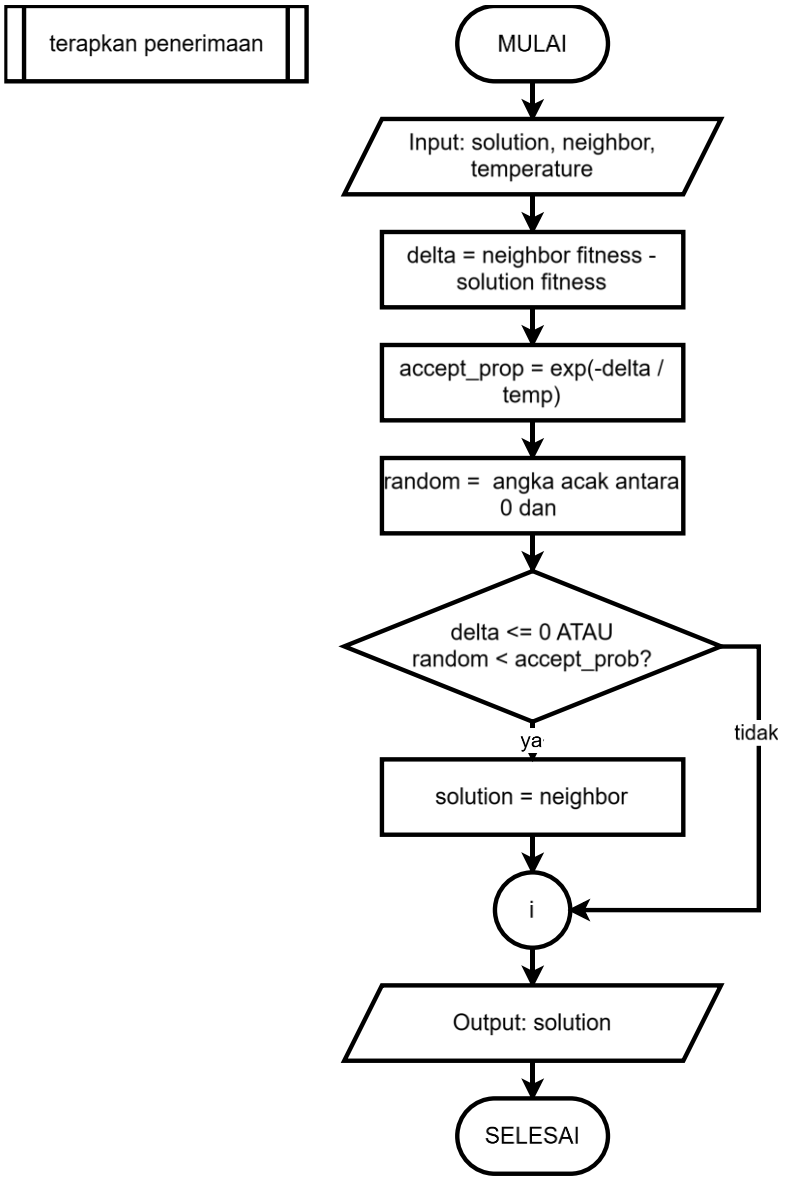
4.4.4.5 *Block Insert*



Gambar 4.10 *Flowchart* Algoritma *Block Insert*

Operator *block insert* memindahkan satu blok pekerjaan berurutan dari suatu mesin ke mesin lain dan menyisipkannya pada posisi acak. Dengan mempertahankan urutan internal blok, operator ini mengombinasikan perubahan lintas mesin dengan stabilitas urutan lokal. Pendekatan ini relevan untuk menguji alokasi alternatif bagi kelompok pekerjaan yang memiliki karakteristik serupa, seperti kombinasi tinta yang sama.

4.4.5 Kriteria Penerimaan



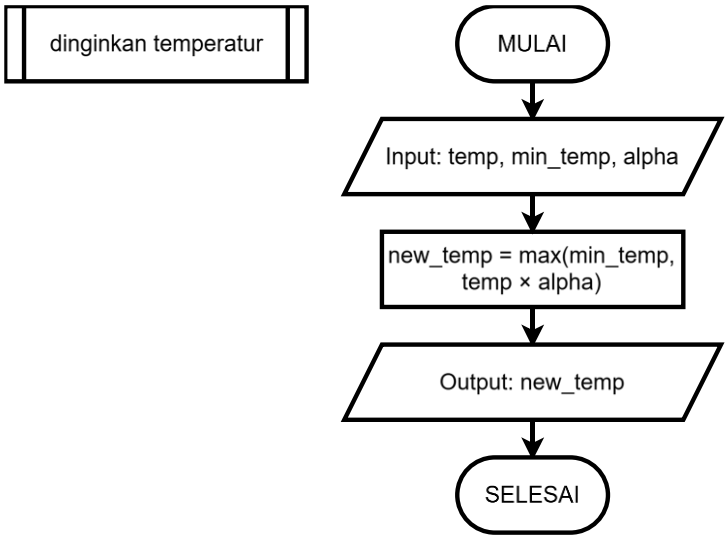
Gambar 4.11 *Flowchart* Algoritma *Metropolis Acceptance Criterion*

Kriteria penerimaan solusi pada algoritma SA mengikuti aturan Metropolis sebagaimana diformalkan pada Subbab 2.5.2. Pada setiap iterasi, algoritma menghitung selisih nilai *fitness* antara solusi kandidat dan solusi saat ini. Jika solusi baru memiliki nilai *fitness* yang lebih baik atau sama, solusi tersebut langsung diterima. Sebaliknya, jika solusi baru lebih buruk, solusi tersebut masih dapat diterima dengan probabilitas yang bergantung pada besarnya penurunan kualitas dan nilai temperatur saat itu.

Keputusan penerimaan dilakukan dengan membandingkan probabilitas Metropolis dengan sebuah bilangan acak seragam. Pada temperatur tinggi, peluang menerima solusi yang lebih buruk relatif besar sehingga pencarian bersifat eksploratif. Seiring penurunan temperatur, peluang tersebut semakin kecil dan algoritma menjadi lebih selektif. Dengan mekanisme ini, kriteria

penerimaan Metropolis dioperasionalkan sebagai sarana transisi bertahap dari fase eksplorasi menuju eksploitasi dalam proses pencarian SA.

4.4.6 Pendinginan

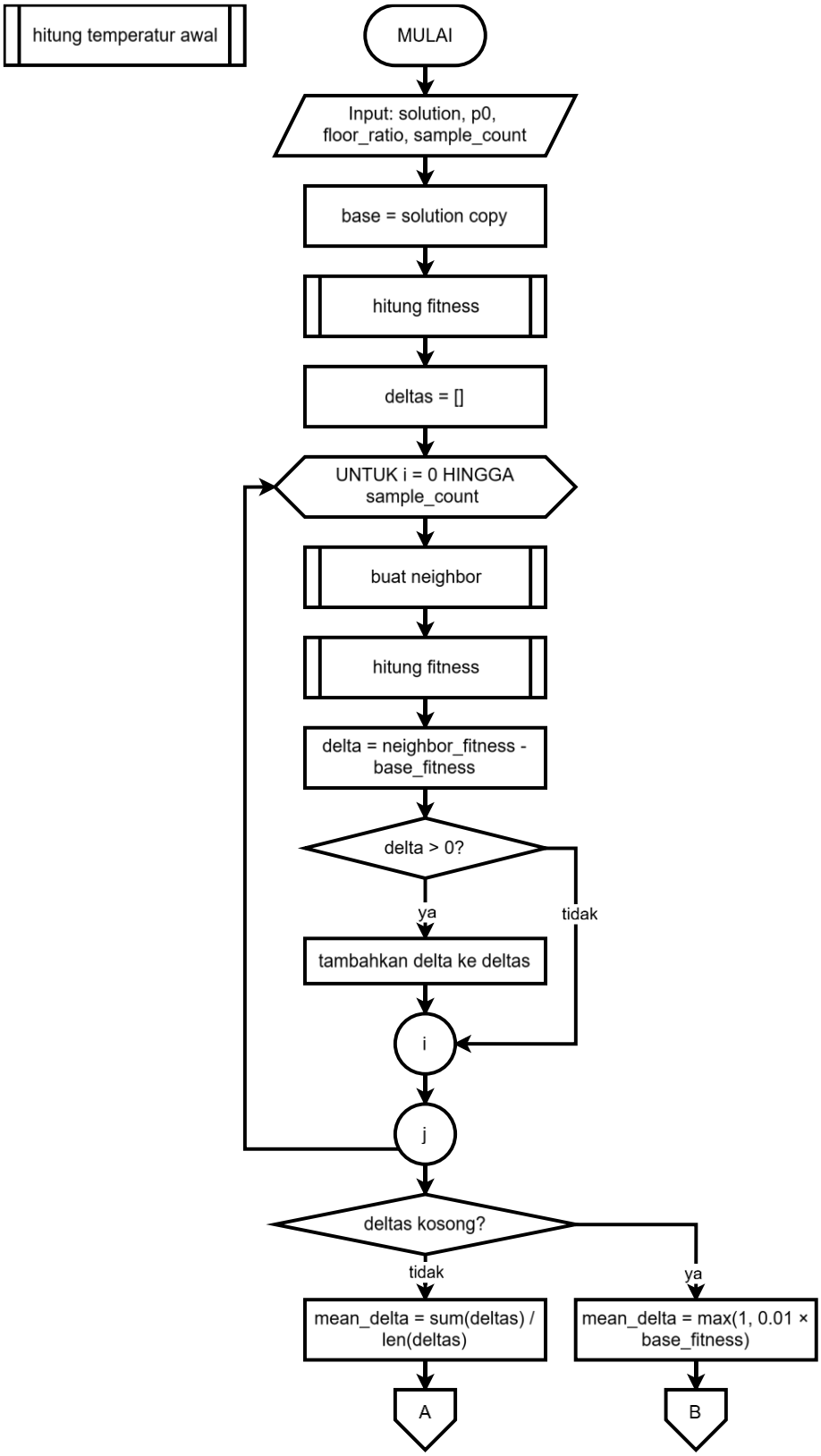


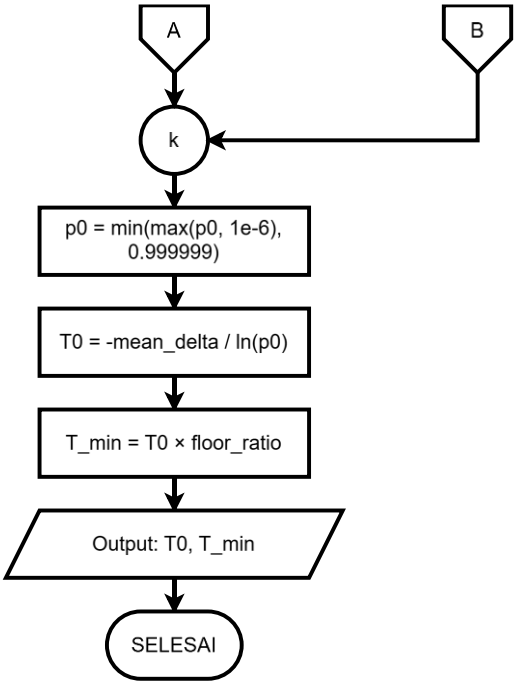
Gambar 4.12 *Flowchart* Algoritma Pendinginan Geometrik

Penelitian ini menggunakan skema pendinginan geometrik, sebagaimana dijelaskan secara matematis pada Subbab 2.5.3. Dalam implementasinya, temperatur diperbarui pada setiap iterasi dengan mengalikan temperatur saat ini dengan suatu faktor tetap (*cooling rate*), dengan pembatasan agar nilainya tidak turun di bawah temperatur minimum yang telah ditetapkan.

Nilai *cooling rate* mengendalikan laju penurunan temperatur. Nilai yang mendekati satu menghasilkan pendinginan yang lebih lambat sehingga eksplorasi ruang solusi berlangsung lebih luas, namun dengan konsekuensi waktu komputasi yang lebih panjang. Sebaliknya, nilai yang lebih kecil mempercepat konvergensi tetapi mengurangi kemampuan algoritma untuk keluar dari solusi lokal. Kombinasi antara *cooling rate* dan batas minimum temperatur memungkinkan proses pendinginan berlangsung stabil serta menjaga keseimbangan antara kualitas solusi dan efisiensi komputasi.

4.4.7 Inisialisasi Temperatur





Gambar 4.13 *Flowchart* Algoritma Inisialisasi Temperatur

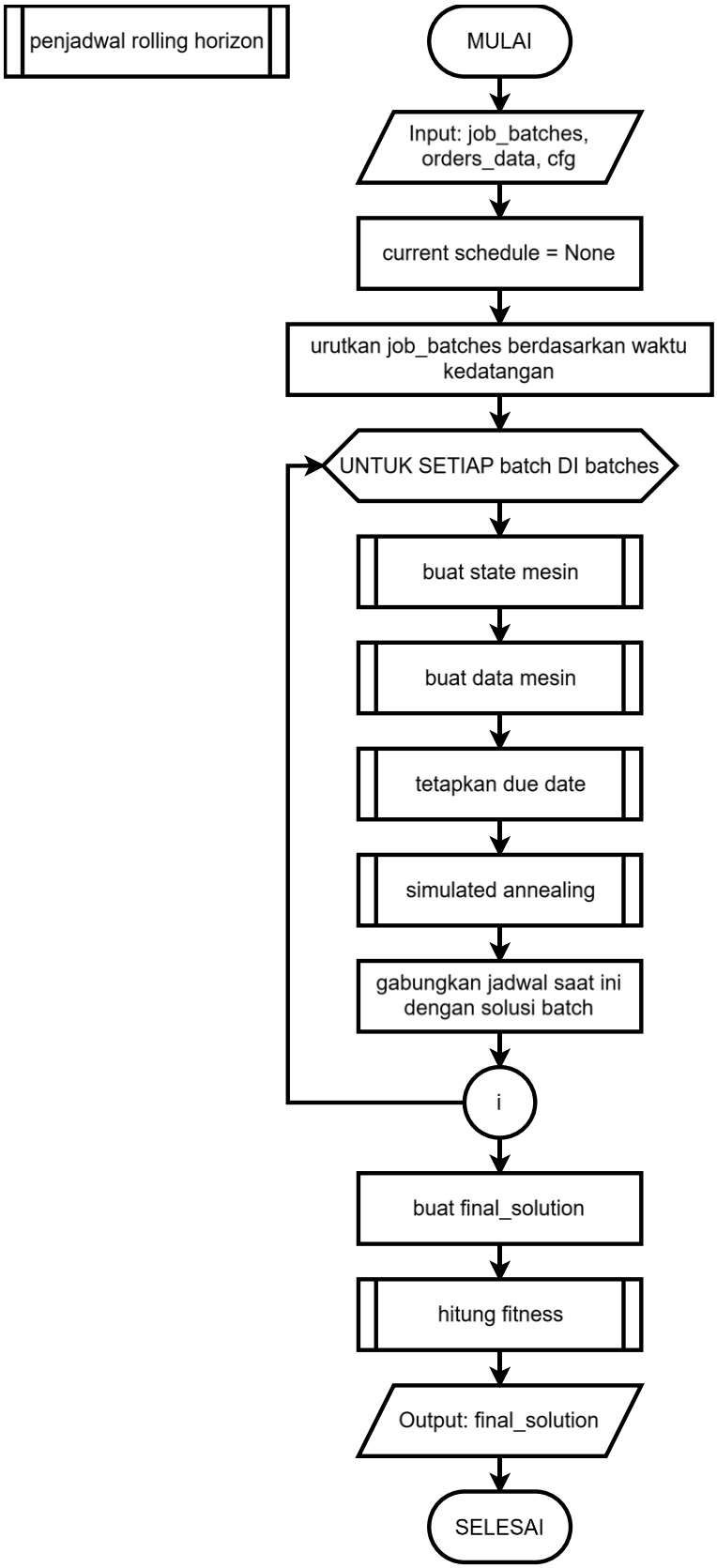
Inisialisasi temperatur awal dilakukan berdasarkan prinsip Metropolis yang dijelaskan pada Subbab 2.5.1, dengan fokus pada prosedur implementasinya. Algoritma menghasilkan sejumlah solusi tetangga acak dari solusi awal dan menghitung selisih nilai fitness terhadap solusi acuan. Hanya selisih yang bernilai positif dicatat sebagai representasi *uphill moves*. Nilai rata-rata dari selisih positif tersebut digunakan sebagai skala karakteristik perubahan fitness. Jika tidak ditemukan *uphill moves*, digunakan estimasi cadangan berupa fraksi kecil dari nilai fitness awal untuk menjaga kestabilan skala temperatur.

Berdasarkan nilai rata-rata selisih tersebut dan probabilitas penerimaan awal yang ditentukan, temperatur awal T_0 dihitung menggunakan persamaan Metropolis. Selanjutnya, temperatur minimum $T_{\min}$ ditetapkan sebagai fraksi dari T_0 melalui parameter *floor ratio* dan berfungsi sebagai batas bawah pendinginan, bukan sebagai kriteria penghentian. Apabila temperatur mencapai $T_{\min}$ sebelum iterasi maksimum terpenuhi, proses pencarian dilanjutkan pada temperatur konstan tersebut untuk mencegah *premature convergence* dan mempertahankan sifat stokastik algoritma hingga seluruh anggaran iterasi digunakan.

4.4.8 Kriteria Penghentian

Proses pencarian pada algoritma SA dihentikan berdasarkan batas maksimum iterasi yang telah ditentukan sebelumnya. Pendekatan ini dipilih untuk memastikan konsistensi waktu komputasi serta memudahkan perbandingan kinerja antar konfigurasi parameter dan skenario pengujian. Meskipun temperatur secara teoretis dapat diturunkan hingga $T_{\min}$, dalam penelitian ini mekanisme penghentian tidak didasarkan pada nilai temperatur minimum, melainkan sepenuhnya pada jumlah iterasi yang tercapai.

4.5 Perancangan Algoritma *Rolling Horizon*



Gambar 4.14 *Flowchart* Algoritma *Rolling Horizon Scheduler*

Algoritma *rolling horizon* digunakan untuk memperluas kerangka SA ke dalam lingkungan penjadwalan *online*, di mana pekerjaan produksi tiba secara bertahap seiring waktu. Pendekatan ini merepresentasikan kondisi industri yang realistis, di mana pesanan baru harus diintegrasikan ke dalam rencana produksi yang sedang berjalan tanpa mengganggu aktivitas yang telah dimulai.

Proses penjadwalan dilakukan secara berulang dalam jendela waktu yang disebut *horizon*, masing-masing merepresentasikan satu *batch* pekerjaan yang baru tersedia. Pada awal setiap *horizon*, algoritma merekonstruksi kondisi sistem terkini dengan mengidentifikasi pekerjaan yang telah terkunci, pekerjaan yang masih dapat dijadwal ulang, serta status mesin yang mencakup waktu ketersediaan dan konfigurasi tinta terakhir. Informasi ini disimpan dalam representasi *state* mesin untuk memastikan bahwa keputusan sebelumnya tetap dipatuhi sesuai dengan prinsip *non-anticipatory constraint*.

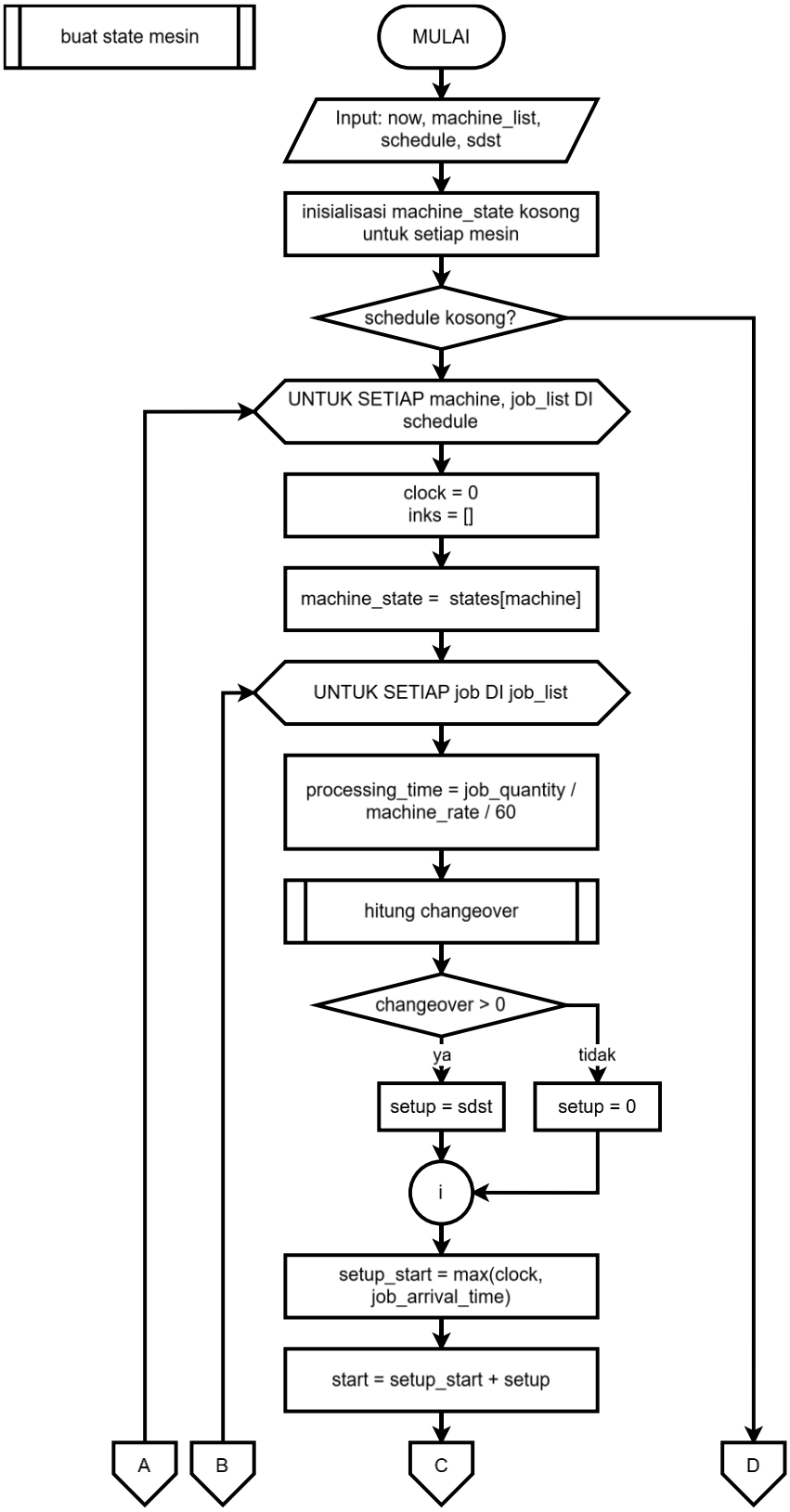
Berdasarkan kondisi tersebut, algoritma mengidentifikasi seluruh pekerjaan yang terlihat pada waktu saat ini, menyiapkan kembali data mesin, dan menetapkan *due date* menggunakan heuristik berbasis beban kerja. Selanjutnya, prosedur SA diterapkan untuk mengoptimasi penjadwalan pekerjaan yang belum terkunci dalam *horizon* tersebut, dengan tetap mempertahankan kompatibilitas terhadap bagian jadwal yang telah tetap. Hasil optimasi kemudian digabungkan ke dalam rencana produksi global, dan proses ini diulang untuk setiap *horizon* berikutnya hingga seluruh pekerjaan diproses. Kinerja jadwal akhir dievaluasi berdasarkan *makespan*, *tardiness*, dan perubahan tinta.

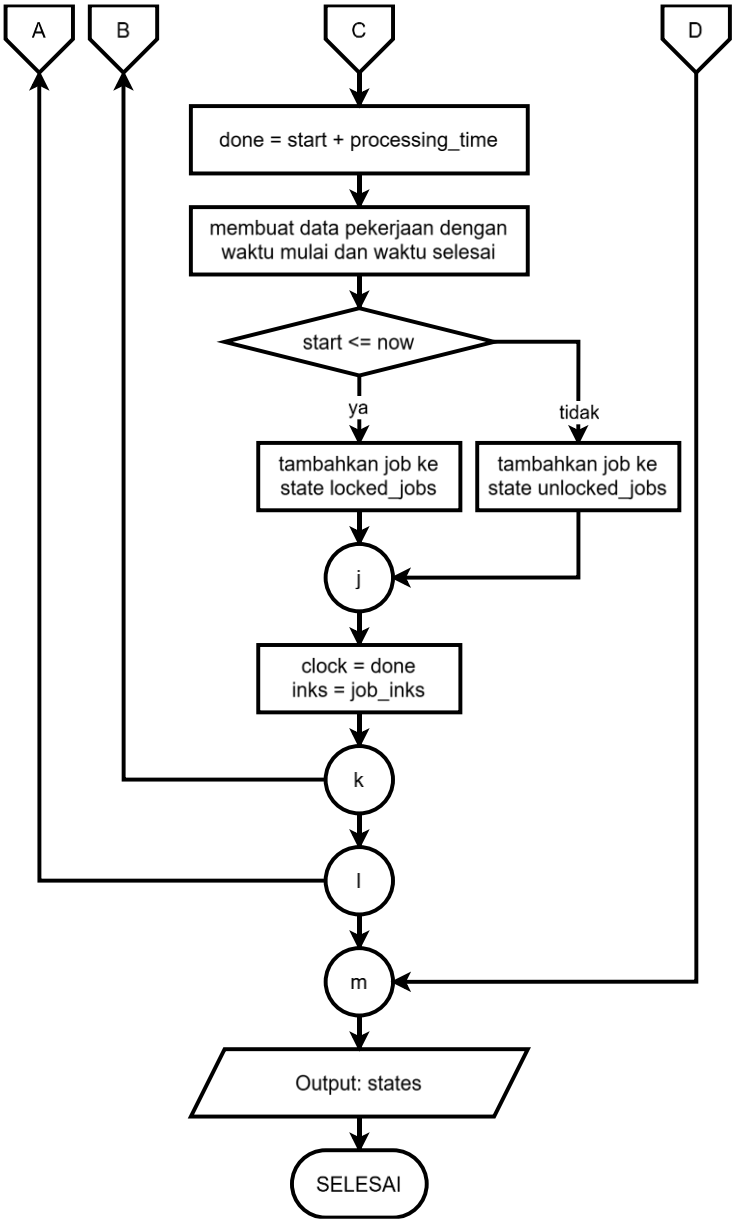
4.5.1 Kebijakan *Lock/Unlock*

Pada awal setiap *horizon*, pekerjaan yang telah dimulai atau sedang diproses pada waktu saat ini diklasifikasikan sebagai *locked jobs*. Pekerjaan ini tidak dapat diubah posisinya karena merepresentasikan komitmen produksi yang telah berlangsung secara fisik pada mesin. Sebaliknya, pekerjaan yang memiliki waktu mulai setelah waktu saat ini dikategorikan sebagai *unlocked jobs* dan masih dapat dijadwal ulang, diubah urutannya, atau dialihkan ke mesin lain oleh algoritma optimasi.

Seiring pergeseran *horizon*, status pekerjaan diperbarui secara bertahap, di mana sebagian pekerjaan yang sebelumnya *unlocked* dapat menjadi *locked*, dan pekerjaan baru yang masuk ke jendela perencanaan ditambahkan sebagai *unlocked jobs* baru. Mekanisme *lock/unlock* ini memungkinkan proses optimasi ulang dilakukan secara non-destruktif, menjaga kontinuitas jadwal dan konsistensi historis, sekaligus memberikan fleksibilitas adaptif terhadap kedatangan pesanan produksi baru.

4.5.2 State Mesin





Gambar 4.15 *Flowchart* Algoritma Buat State Mesin

Pembentukan *state* mesin bertujuan merekonstruksi kondisi aktual seluruh mesin pada awal setiap *rolling horizon* dengan menjaga kesinambungan jadwal produksi dan prinsip *non-anticipatory*. Prosedur ini menghasilkan representasi kondisi mesin yang konsisten secara temporal sebagai dasar proses *re-optimization*.

Pada setiap *horizon*, algoritma membangun *state* untuk setiap mesin yang mencakup waktu ketersediaan mesin, konfigurasi tinta terakhir, serta pembagian pekerjaan menjadi *locked jobs* dan *unlocked jobs*. Jika jadwal sebelumnya tersedia, urutan pekerjaan pada setiap mesin disimulasikan secara kronologis untuk memperbarui waktu mesin dan konfigurasi tinta dengan mempertimbangkan waktu proses dan kebutuhan *changeover*. Klasifikasi *locked* dan *unlocked*

dilakukan berdasarkan hubungan waktu mulai pekerjaan terhadap waktu simulasi saat ini.

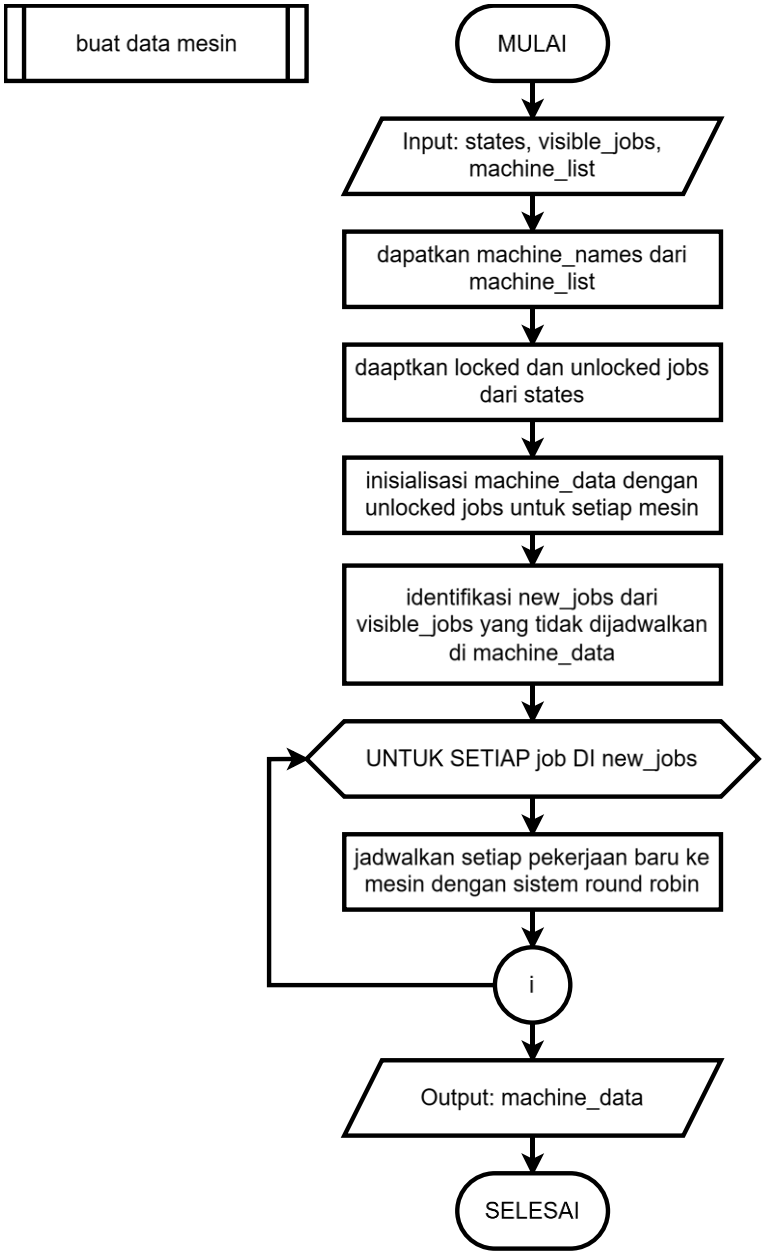
Hasil dari prosedur ini berupa *snapshot* sistem yang merepresentasikan kondisi mesin dan komitmen jadwal terkini. *State* mesin tersebut digunakan pada tahap berikutnya untuk memastikan bahwa proses optimasi dimulai dari kondisi sistem yang valid dan berkelanjutan antar *horizon*.

4.5.3 Pembangunan Data Mesin untuk Optimasi

Pembangunan data mesin bertujuan membentuk himpunan penjadwalan aktif pada *horizon* saat ini dengan mengintegrasikan *state* mesin dan pekerjaan baru yang telah *visible*. Tahap ini mendefinisikan ruang pencarian optimasi dengan mempertahankan *locked jobs* dan menyusun kembali *unlocked jobs* sebagai bagian dari masalah penjadwalan yang akan dioptimasi.

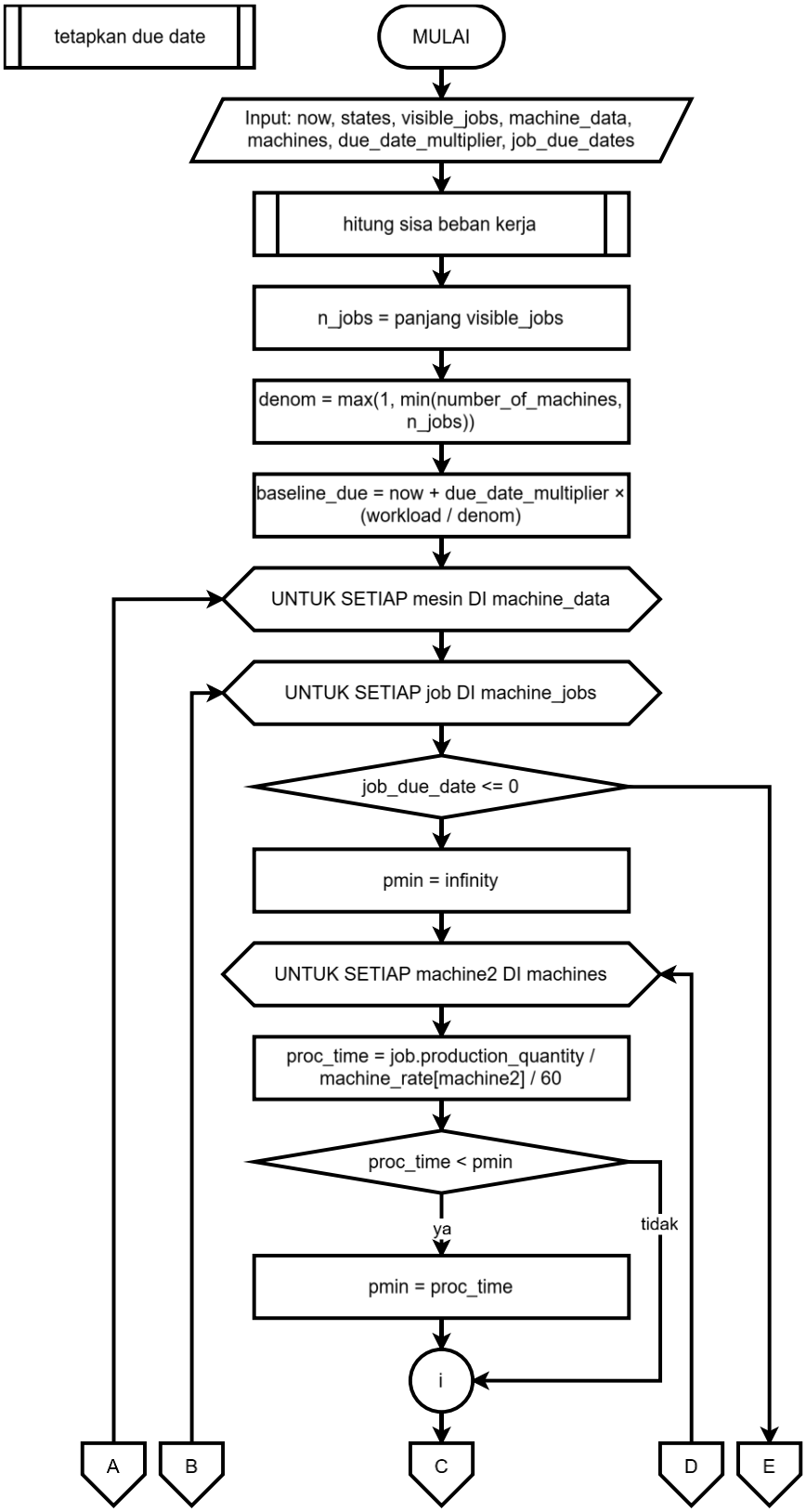
Pada awal *horizon*, algoritma memuat seluruh mesin beserta daftar *locked jobs* dan *unlocked jobs* dari *state* mesin. Selanjutnya, pekerjaan baru yang telah *visible* dan belum termasuk dalam jadwal dialokasikan ke mesin menggunakan aturan *round-robin* untuk membentuk konfigurasi awal yang seimbang. Struktur data mesin yang dihasilkan menjadi input utama bagi algoritma SA pada *horizon* tersebut.

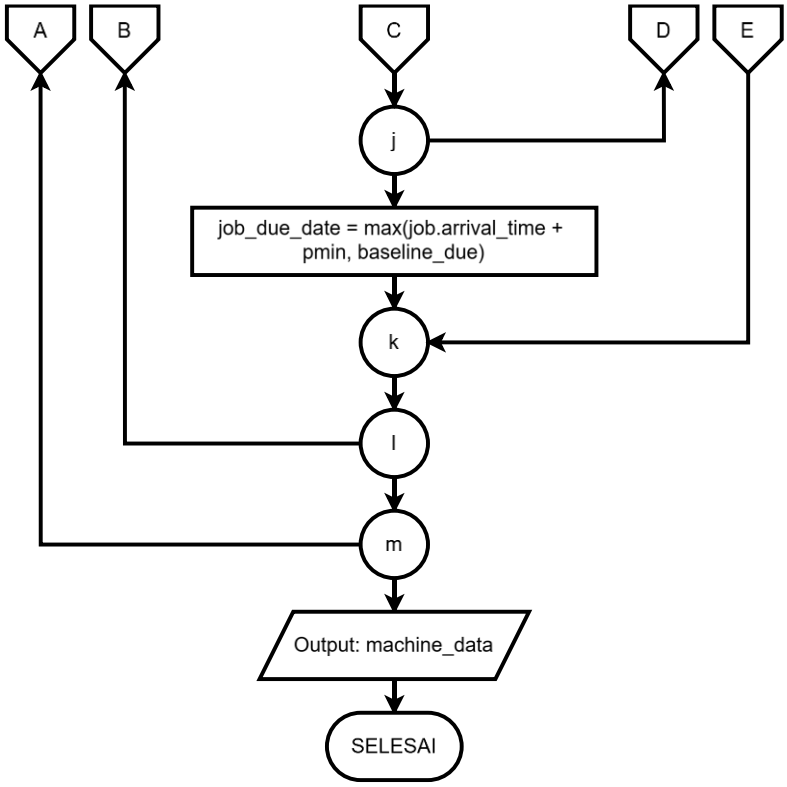
Melalui tahap ini, algoritma menjaga konsistensi jadwal masa lalu sekaligus membatasi proses *re-optimization* hanya pada bagian jadwal yang fleksibel. Pendekatan ini memastikan efisiensi komputasi dan stabilitas jadwal sepanjang siklus *rolling horizon*.



Gambar 4.16 *Flowchart* Algoritma *Build Machine Data*

4.5.4 Penetapan Due Date





Gambar 4.17 *Flowchart* Algoritma Penetapan Due Date dengan Aturan TWK yang Dimodifikasi

Pada waktu *horizon* saat ini, beban kerja sistem yang tersisa terlebih dahulu dihitung menggunakan prosedur perhitungan beban kerja. Misalkan N adalah jumlah *visible jobs* dan M adalah jumlah mesin yang aktif. Dalam konteks penjadwalan dinamis berbasis *horizon*, kumpulan pekerjaan yang *visible* pada saat t_{now} diperlakukan sebagai satu *batch* atau kelompok pekerjaan yang akan dijadwalkan bersama. Dengan demikian, konsep TWK klasik $d_j = k\,p_j$, yang pada awalnya diterapkan untuk satu pekerjaan tunggal, diadaptasikan sehingga p_j merepresentasikan total beban kerja *batch* yang harus diselesaikan pada *horizon* tersebut. Algoritma kemudian menghitung *baseline due date* menggunakan rumus berikut:

$$D_{\text{TWK}} = t_{\text{now}} + k \times \frac{W_{\text{rem}}}{\min(M,N)} \tag{4.2}$$

Di mana:

D_{TWK} : *Baseline due date* hasil perhitungan aturan TWK (jam).

t_{now} : waktu saat ini pada *horizon* penjadwalan.

k : pengali *due-date* yang mengontrol tingkat kelonggaran waktu.

W_{rem} : total beban kerja yang tersisa dalam sistem (jam).

M : jumlah mesin aktif dalam sistem produksi.

N : jumlah pekerjaan yang *visible* pada *horizon* saat ini.

Langkah selanjutnya, untuk setiap pekerjaan j, algoritma menghitung waktu proses minimum yang mungkin di antara semua mesin, menggunakan persamaan:

$$p_{min,j} = \min_{m} \left(\frac{q_j}{r_m} \right) \tag{4.3}$$

Di mana:

$p_{\min,j}$: waktu proses tercepat yang dapat dicapai pekerjaan j (jam).

q_j : jumlah unit produksi untuk pekerjaan j.

r_m : laju produksi mesin m(unit per jam).

Kemudian, *due date* akhir untuk pekerjaan $j$ditentukan sebagai:

$$D_j = \max\left(t_{\text{arr},j} + p_{min,j},\ D_{\text{TWK}}\right) \tag{4.4}$$

Di mana:

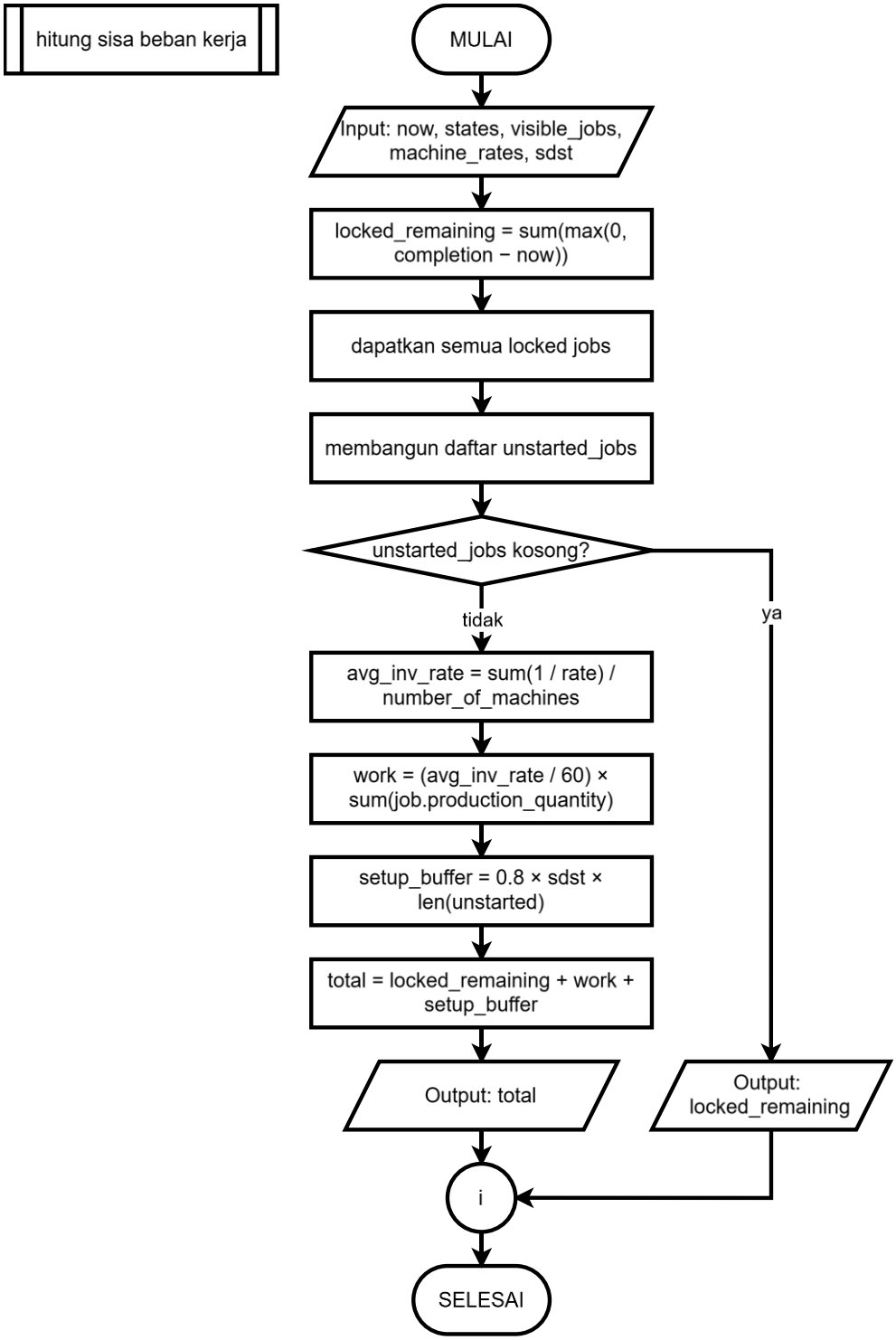
D_j : *due date* akhir yang ditetapkan untuk pekerjaan j.

$t_{\text{arr},j}$: waktu kedatangan pekerjaan j.

$p_{\min,j}$: waktu proses minimum pekerjaan $j$di antara seluruh mesin.

D_{TWK} : *baseline due date* berdasarkan beban kerja sistem.

Formulasi ini menjamin bahwa tidak ada pekerjaan yang diberikan *due date* lebih awal dari waktu penyelesaiannya yang paling cepat secara teoretis. Dengan demikian, aturan TWK yang dimodifikasi ini menggabungkan faktor spesifik per pekerjaan dan faktor global berbasis beban kerja sistem, menghasilkan target *due date* yang adaptif dan stabil sepanjang pembaruan *horizon* berturut-turut.



Gambar 4.18 *Flowchart* Algoritma Perhitungan Beban Kerja yang Tersisa

Total beban kerja tersisa dihitung sebagai jumlah dari seluruh operasi yang belum selesai serta perkiraan pekerjaan yang akan datang. Nilai ini mengintegrasikan tiga komponen utama sebagai berikut:

1. Beban kerja pekerjaan terkunci yang belum selesai

$$W_{\text{locked}} = \sum_{j \in J_{\text{locked}}} \max(0, C_j - t_{\text{now}}) \quad (4.5)$$

Di mana:

W_{locked} : sisa waktu proses untuk pekerjaan yang sedang berlangsung.

J_{locked} : himpunan pekerjaan yang terkunci (sedang berjalan).

C_j : waktu penyelesaian terjadwal untuk pekerjaan j.

t_{now} : waktu saat ini.

2. Beban kerja pekerjaan yang belum dimulai

Perkiraan waktu total dihitung menggunakan rata-rata kebalikan laju produksi, dengan rumus:

$$r_{\text{inv}} = \frac{1}{M} \sum_{m=1}^{M} \frac{1}{r_m} \quad (4.6)$$

$$W_{\text{unstarted}} = \frac{1}{60} r_{\text{inv}} \sum_{j=1}^{N} q_j \quad (4.7)$$

Di mana:

$W_{\text{unstarted}}$: total beban kerja dari pekerjaan yang belum dimulai (jam).

q_j : jumlah unit produksi untuk pekerjaan ke-j.

r_{inv} : rata-rata kebalikan laju produksi (menit per unit).

M : jumlah mesin aktif dalam sistem.

N : jumlah pekerjaan yang belum dimulai.

r_m : laju produksi mesin ke-m (unit per menit).

Faktor $\frac{1}{60}$: konversi satuan dari menit ke jam.

3. Kompensasi waktu setup

$$W_{\text{setup}} = 0.5 \times SDST \times N \quad (4.8)$$

Di mana:

W_{setup} : tambahan estimasi waktu untuk proses *setup*.

SDST : durasi SDST tetap untuk setiap transisi tinta.

N : jumlah pekerjaan yang belum dimulai.

Faktor 0.5 : konstanta untuk mengakomodasi *setup*.

Akhirnya, total beban kerja sistem yang digunakan dalam aturan TWK dihitung dengan menjumlahkan ketiga komponen tersebut:

$$W_{\text{rem}} = W_{\text{locked}} + W_{\text{unstarted}} + W_{\text{setup}} \quad (4.9)$$

Di mana:

W_{rem} : total beban kerja tersisa.

W_{locked} : sisa beban kerja dari pekerjaan yang sedang berlangsung.

$W_{\text{unstarted}}$: beban kerja yang belum dimulai.

W_{setup} : estimasi waktu *setup* tambahan.

Formulasi ini memberikan pendekatan terstruktur untuk memperkirakan beban kerja agregat sistem secara dinamis, yang kemudian digunakan oleh aturan TWK dalam menetapkan target *due date* yang realistis dan proporsional terhadap kondisi kapasitas mesin pada setiap *horizon* penjadwalan.

4.6 Perhitungan Manual

4.6.1 Data untuk Perhitungan Manual

Tabel 4.8 Data Pekerjaan untuk Perhitungan Manual

Job ID	Kuantitas	Kombinasi Tinta	Waktu Kedatangan
1	16.539	[6, 8]	0.0
2	6.030	[6, 6]	0.0
3	6.100	[9]	0.0
4	4.640	[8]	0.0
5	12.100	[5, 8, 9]	1.0
6	8.000	[8, 9]	1.0
7	5.180	[4, 8]	1.0
8	9.150	[9]	1.0

Tabel 4.9 Data Mesin untuk Perhitungan Manual

Mesin	Laju Produksi (unit/menit)
Flexo A	112
Flexo B	224

Tabel 4.10 Parameter untuk Perhitungan Manual

Parameter	Nilai
Waktu setup bergantung urutan (SDST)	0,75
Bobot tardiness	2,0
Bobot perubahan tinta	1,0
Bobot makespan	0,1
Pengali due date	1,0
Probabilitas temperatur awal	0,80
Laju pendinginan	0,995
Rasio batas temperatur bawah	0,05

Parameter	Nilai
Bias mesin kritis	0,7
Jumlah iterasi maksimum (untuk demo)	6

4.6.2 Persiapan *Rolling Horizon - Horizon 1*

4.6.2.1 *Visible Jobs* di *Horizon* 1

Seluruh pekerjaan dengan waktu kedatangan ≤ 0:

Tabel 4.11 *Visible Jobs* pada *Horizon* 1

Job ID	Kuantitas	Kombinasi Tinta	Waktu Kedatangan
1	16.539	[6, 8]	0.0
2	6.030	[6, 6]	0.0
3	6.100	[9]	0.0
4	4.640	[8]	0.0

(*Job* 5-8 tiba pada 1.0 jam → belum terlihat pada *horizon* ini.)

4.6.2.2 *State* Mesin

Pada awal *horizon* pertama, belum ada mesin yang mulai berproduksi. Dengan demikian, semua mesin memiliki:

1. **Waktu saat ini:** 0,0 jam
2. **Konfigurasi tinta:** [] (kosong)
3. **Pekerjaan terkunci:** 0

Bagian ini merupakan contoh penerapan langkah inisialisasi state mesin pada Gambar 4.15.

4.6.2.3 Penyusunan Data Mesin untuk Optimasi

Pekerjaan baru dialokasikan ke mesin menggunakan metode *round-robin*:

Tabel 4.12 Data Mesin untuk Optimasi *Horizon* 1

Mesin	Pekerjaan yang *Unlocked*
Flexo A	J1, J3
Flexo B	J2, J4

Penyusunan data mesin pada tahap ini mengikuti prosedur pada Gambar 4.16.

4.6.2.4 Penetapan *Due Date*

Due date ditentukan menggunakan aturan TWK. Total beban kerja sistem dihitung sebagai:

$$W_{\text{rem}} = W_{\text{locked}} + W_{\text{unstarted}} + W_{\text{setup}}$$

Karena pada *horizon* ini belum ada pekerjaan yang terkunci, maka:

$$W_{\text{rem}} = W_{\text{unstarted}} + W_{\text{setup}}$$

Beban Kerja Belum Dimulai ($W_{\text{unstarted}}$)

$$r_{\text{inv}} = \frac{1}{2}\left(\frac{1}{112} + \frac{1}{224}\right) = 0.0066964$$
$$W_{\text{unstarted}} = \frac{r_{\text{inv}}}{60} \times Q_{\text{visible}} = \frac{0.0066964}{60} \times 33309 = 3.72 \text{ jam}$$

Setup Buffer (W_{setup})

$$W_{\text{setup}} = 0.5 \times 0.75 \times 4 = 1.50 \text{ jam}$$

Total Beban Kerja (W_{rem})

$$W_{\text{rem}} = 3.72 + 1.50 = 5.22 \text{ jam}$$

Baseline dan Due Date Individu

Baseline due date dihitung dengan aturan TWK:

$$D_{\text{base}} = t_{\text{now}} + k \times \frac{W_{\text{rem}}}{\min(M,N)} = 0 + 1.0 \times \frac{5.22}{2} = 2.61 \text{ jam}$$

Setiap pekerjaan $j$kemudian memperoleh *due date* akhir:

$$D_j = \max\left(t_{\text{arr},j} + p_{\min,j},\ D_{\text{base}}\right)$$

Tabel 4.13 *Due Dates* Untuk Pekerjaan di *Horizon* 1

Job	Kuantitas	Waktu Proses Terpendek	Waktu Kedatangan	Baseline Due	Due Date Akhir
1	16.539	1.23	0.0	2.61	2.61
2	6.030	0.45	0.0	2.61	2.61
3	6.100	0.45	0.0	2.61	2.61
4	4.640	0.35	0.0	2.61	2.61

Perhitungan beban kerja dan *due date* pada bagian ini mengimplementasikan prosedur pada Gambar 4.17 serta Gambar 4.18.

4.6.3 *Simulated Annealing - Horizon* 1

4.6.3.1 Solusi Awal *Greedy*

Pada awal *Horizon-1*, empat pekerjaan (J1-J4) terlihat oleh algoritma optimasi. Semua pekerjaan tiba pada waktu 0,0 jam, dan kedua mesin (Flexo A dan Flexo B) berada dalam kondisi menganggur tanpa tinta terpasang. Prosedur inisialisasi *greedy* membentuk jadwal awal yang layak dengan menugaskan setiap pekerjaan secara berurutan ke mesin yang menghasilkan waktu penyelesaian paling awal, dengan mempertimbangkan waktu *setup* dan laju produksi. Konstruksi solusi awal pada subbagian ini mengikuti langkah pada Gambar 4.4.

Pengurutan Daftar Pekerjaan

Seluruh pekerjaan diurutkan berdasarkan waktu kedatangan, kemudian berdasarkan kuantitas secara menurun. Karena semuanya tiba secara bersamaan, urutan ditentukan oleh kuantitas: J1 (16.539 unit), J3 (6.100), J2 (6.030), J4 (4.640).

Keputusan Penugasan Berurutan

Tabel 4.14 Penugasan *Greedy* untuk *Horizon* 1

Job	Mesin	Tinta Saat Ini	Perubahan Tinta	Setup	Mulai	Proses	Selesai
J1 [6, 8]	Flexo A	[]	0	0.00	0.00	2.46	2.46
	Flexo B	[]	0	0.00	0.00	1.23	1.23 (terbaik)
J3 [9]	Flexo A	[]	0	0.00	0.00	0.91	0.91 (terbaik)
	Flexo B	[6, 8]	1	0.75	1.23	0.45	2.43
J2 [6, 6]	Flexo A	[9]	1	0.75	0.91	0.90	2.56
	Flexo B	[6, 8]	1	0.75	1.23	0.45	2.43 (terbaik)
J4 [8]	Flexo A	[9]	1	0.75	0.91	0.69	2.35 (terbaik)
	Flexo B	[6, 6]	1	0.75	2.43	0.35	3.52

Jadwal Awal *Greedy*

Tabel 4.15 Jadwal Awal *Greedy* untuk *Horizon* 1

Mesin	Urutan Pekerjaan
Flexo A	J3, J4
Flexo B	J1, J2

4.6.3.2 Evaluasi *Fitness* pada Jadwal Awal

Setelah jadwal *greedy* terbentuk, algoritma mengevaluasi kinerjanya menggunakan tiga komponen utama:

$$\min F = w_t \cdot T + w_i \cdot I + w_m \cdot C_{max}$$

Perhitungan *fitness* terintegrasi pada bagian ini mengikuti prosedur pada Gambar 4.2.

Rekonstruksi *Timeline* Mesin

Tabel 4.16 Rekonstruksi *Timeline* Jadwal *Greedy Horizon 1*

Mesin	Tinta Sebelumnya	Pekerjaan	Setup	Waktu Mulai	Mulai Proses	Durasi Proses	Selesai
Flexo A	-	J3 [9]	0.00	0.00	0.00	0.91	0.91
	[9]	J4 [8]	0.75	0.91	1.66	0.69	2.35
Flexo B	-	J1 [6, 8]	0.00	0.00	0.00	1.23	1.23
	[6, 8]	J2 [6, 6]	0.75	1.23	1.98	0.45	2.43

Evaluasi *Tardiness*

Setiap waktu penyelesaian pekerjaan dibandingkan dengan batas waktu *horizon* $D_j = 2.61$ jam.

Tabel 4.17 Evaluasi *Tardiness* Jadwal *Greedy Horizon 1*

Job	Selesai	Due Date	Terlambat?
J1	1.23	2.61	Tidak
J2	2.43	2.61	Tidak
J3	0.91	2.61	Tidak
J4	2.35	2.61	Tidak
Total			**0 pekerjaan terlambat**

Evaluasi Perubahan Tinta

Perubahan tinta terjadi setiap kali konfigurasi tinta berbeda dari pekerjaan sebelumnya. Untuk setiap transisi, sistem menghitung jumlah minimum perubahan warna yang diperlukan.

Tabel 4.18 Evaluasi *Changeover* Jadwal *Greedy Horizon 1*

Mesin	Transisi	Jumlah Changeover
Flexo A	[], [9]	0
Flexo A	[9], [8]	1
Flexo B	[], [6, 8]	0
Flexo B	[6, 8], [6, 6]	1
Total		**2**

***Makespan* dan Mesin Kritis**

Waktu penyelesaian total ditentukan oleh mesin yang selesai paling akhir:

$$C_{\max} = \max(2.35, 2.43) = 2.43 \text{ jam}$$

Dengan demikian, Flexo B merupakan mesin kritis.

Nilai *Fitness*

Ketiga komponen dikombinasikan sesuai bobot masing-masing:

$$F = 2(0) + 1(2) + 0.1(2.43) = 2.24$$

4.6.3.3 Perhitungan *Changeover*

Sebagai ilustrasi perhitungan biaya *changeover*, pertimbangkan transisi dari *Job* 1 [6, 8] ke *Job* 2 [6, 6].

Bandingkan komposisi tinta

Algoritme pertama-tama membandingkan jumlah tinta pada setiap level sebelum dan sesudah transisi:

Tabel 4.19 Komposisi Tinta antara *Job* 1 [6, 8] dan *Job* 2 [6, 6]

Level Tinta	Sebelum	Sesudah	Selisih	Interpretasi
6	1	2	−1	Tambah tinta 6
8	1	0	+1	Hapus tinta 8
Jumlah			0	Semua penghapusan dapat dipasangkan dengan penambahan

Karena jumlah selisihnya adalah 0, semua penghapusan dapat dipasangkan dengan penambahan, tidak ada penghapusan yang diperlukan.

Pasangkan penghapusan dengan penambahan

Setiap penghapusan dicocokkan dengan penambahan untuk mensimulasikan penggantian tinta. Untuk setiap pasangan, algoritma memeriksa apakah transisi berubah dari tinta yang lebih gelap menjadi lebih terang.

Tabel 4.20 Pasangan Penghapusan dan Penambahan antara *Job* 1 [6, 8] dan *Job* 2 [6, 6]

Pasangan	Transisi	Jenis	Biaya
1	8, 6	Gelap ke terang	1

Hitung total biaya

$$\begin{aligned}\text{Changeover} &= (\text{gelap ke terang}) + (\text{penghapusan tanpa pasangan}) \\ &= 1 + 0 = 1\end{aligned}$$

Contoh numerik pada bagian ini mengilustrasikan secara rinci langkah perhitungan *changeover* yang dirangkum dalam Gambar 4.3.

4.6.3.4 Operator *Neighborhood Move*

Algoritma SA mengeksplorasi solusi terdekat dengan melakukan modifikasi kecil pada jadwal saat ini. Setiap jenis modifikasi disebut operator *neighbor*. Tabel berikut merangkum kelima operator yang ditunjukkan pada jadwal awal.

Tabel 4.21 Contoh *Neighbor Move* pada *Greedy Schedule Horizon* 1

Jenis Gerakan	Aksi	Mesin	Sebelum				Sesudah			
			1	**2**	**3**	**4**	**1**	**2**	**3**	**4**
Swap Within	Menukar posisi Job 1 dan Job 2 pada Flexo B.	Flexo A	3	4	-	-	3	4	-	-
		Flexo B	1	2	-	-	**2**	**1**	-	-
Swap Between	Menukar Job 4 (dari A) dengan Job 2 (dari B).	Flexo A	3	4	-	-	3	**2**	-	-
		Flexo B	1	2	-	-	1	**4**	-	-
Relocate	Memindahkan Job 4 dari Flexo A ke akhir Flexo B.	Flexo A	3	4	-	-	3	**-**	-	-
		Flexo B	1	2	-	-	1	2	**4**	-

Jenis Gerakan	Aksi	Mesin	Sebelum				Sesudah			
			1	**2**	**3**	**4**	**1**	**2**	**3**	**4**
Exchange	Menukar bagian akhir Flexo A (setelah J4) dengan awal Flexo B (mulai J1).	Flexo A	3	4	-	-	3	**1**	**2**	-
		Flexo B	1	2	-	-	**4**	-	-	-
Block Insert	Memindahkan blok [3, 4] dari Flexo A ke bagian depan Flexo B.	Flexo A	3	4	-	-	-	-	-	-
		Flexo B	1	2	-	-	**3**	**4**	1	2

Kelima operator pada Tabel 4.18 merupakan contoh implementasi struktur tetangga pada Gambar 4.5 serta operator spesifik pada Gambar 4.6 sampai Gambar 4.10 (*Swap Within*, *Swap Between*, *Relocate*, *Exchange*, dan *Block Insert*).

4.6.3.5 Perhitungan Temperatur Awal

Estimasi Nilai *Fitness* Dasar dan *Neighbor*

Untuk memulai proses SA, algoritma memperkirakan besarnya rata-rata perburukan (*uphill moves*) dengan membandingkan nilai *fitness* dasar dengan beberapa solusi tetangga. Nilai *fitness* dasar yang digunakan adalah $F_{\text{base}} = 2.24$, sebagaimana diperoleh dari subbab sebelumnya.

Tabel 4.22 Perbandingan Nilai *Fitness* untuk Estimasi Temperatur Awal

Jadwal Tetangga	Base Fitness	Neighbor Fitness	ΔF
A: 3,4 / B: 2,1	2.24	1.23	−1.01
A: 3,2 / B: 1,4	2.24	2.26	0.02
A: 3 / B: 1,2,4	2.24	4.35	2.11
A: 3,1,2 / B: 4	2.24	6.58	4.34
A: - / B: 3,4,1,2	2.24	6.4	2.16
Jumlah *uphill* (ΔF > 0):			**10.62**
Rata-rata ΔF *uphill*:			**2.66**

Perhitungan Temperatur Awal

Nilai probabilitas penerimaan target digunakan sebesar $p_0 = 0.80$ dan rata-rata ΔF *uphill* = 2.66:

$$T_0 = -\frac{\Delta\bar{F}^+}{\ln(p_0)} = -\frac{2.66}{\ln(0.80)} = 11{,}90$$

Batas bawah temperatur ditetapkan sebesar 5% dari nilai ini:

$$T_{\min} = 0.05 \times T_0 = 0.05 \times 11{,}90 = 0.59$$

Perhitungan T_0dan T_{min}pada bagian ini merupakan implementasi langsung dari skema penetapan temperatur awal pada Gambar 4.13.

4.6.3.6 Iterasi *Simulated Annealing Horizon* 1

Pada setiap iterasi, SA mengevaluasi jadwal tetangga baru. Jika solusi tetangga lebih baik, maka langsung diterima; jika lebih buruk, solusi tersebut masih dapat diterima berdasarkan probabilitas Metropolis $e^{-\Delta F/T}$. Keseimbangan antara eksplorasi dan eksploitasi ini memungkinkan SA untuk keluar dari *local minima* pada tahap awal dan kemudian berkonvergensi ketika temperatur menurun.

Tabel 4.23 Log Iterasi SA *Horizon* 1

Iter	Neighbor	Jadwal	Base F	Neighbor F	ΔF	Temp.	Prob.	Diterima	Best
1	Swap Within	A: 3,4 B: 2,1	2.24	1.23	-1.01	11.90	1.09	Ya	1.23
2	Relocate	A: 3 B: 2,1,4	1.23	3.28	2.05	11.84	0.84	Ya	1.23
3	Swap Between	A: 1 B: 2,3,4	3.28	4.27	0.99	11.78	0.92	Ya	1.23
4	Exchange	A: 1,3,4 B: 2	4.27	6.56	2.29	11.72	0.82	Ya	1.23
5	Block Insert	A: 1 B: 3,4,2	6.56	4.28	-2.28	11.66	1.22	Ya	1.23
6	Swap Between	A: 2 B: 1,3,4	4.28	4.35	0.07	11.60	0.99	Ya	1.23

Aturan pendinginan: $T \leftarrow \max(T_{\min}, 0.995T)$

Jadwal Akhir *Horizon* 1

Tabel 4.24 Jadwal Akhir *Horizon* 1

Mesin	Urutan Pekerjaan Akhir
Flexo A	3, 4
Flexo B	2, 1

$F = 1.23$ dengan tidak ada keterlambatan dan hanya satu kali perubahan tinta. Jadwal akhir pada *Horizon* 1 ini menjadi jadwal terkunci untuk awal *horizon* berikutnya, konsisten dengan alur kerja pada Gambar 4.1 dan Gambar 4.14.

4.6.4 Persiapan *Rolling Horizon* - *Horizon* 2

4.6.4.1 *Visible Jobs* pada *Horizon* 2

Semua pekerjaan dengan waktu kedatangan ≤ 1:

Tabel 4.25 *Visible Jobs* pada *Horizon* 2

Job ID	Kuantitas	Kombinasi Tinta	Waktu Kedatangan
1	16 539	[6, 8]	0.0
2	6 030	[6, 6]	0.0
3	6 100	[9]	0.0
4	4 640	[8]	0.0
5	12 100	[5, 8, 9]	1.0
6	8 000	[8, 9]	1.0
7	5 180	[4, 8]	1.0
8	9 150	[9]	1.0

Daftar *visible jobs* mengikuti langkah pembentukan himpunan pekerjaan terlihat (*visible jobs*) pada Gambar 4.14.

4.6.4.2 Rekonstruksi *Timeline* Mesin dan Status Lock

Pada $t_{\text{now}} = 1.0\,\text{h}$, semua pekerjaan yang telah dimulai diklasifikasikan sebagai *locked*, sementara pekerjaan yang belum dimulai tetap *unlocked*. Rekonstruksi berikut ini berasal dari jadwal akhir yang diperoleh pada *Horizon* 1.

Mesin	Tinta Sebelumnya	Pekerjaan	Setup	Mulai Setup	Mulai	Proses	Selesai	Status
Flexo A	-	J3 [9]	No	0.00	0.00	0.91	0.91	Locked
	[9]	J4 [8]	Yes	0.91	1.66	0.69	2.35	Unlocked
Flexo B	-	J2 [6, 6]	No	0.00	0.00	0.45	0.45	Locked
	[6, 6]	J1 [6, 8]	No	0.45	0.45	1.23	1.68	Locked

Klasifikasi *locked* dan *unlocked* pada rekonstruksi *timeline* ini merupakan implementasi kebijakan *lock-unlock* pada Gambar 4.15.

4.6.4.3 State Mesin pada *Horizon* 2

Tabel 4.26 *State* Mesin pada *Horizon* 2

Mesin	Waktu Saat Ini	Tinta Saat Ini	Locked Jobs	Unlocked Jobs
Flexo A	0.91	[9]	J3	J4
Flexo B	1.68	[6, 8]	J2, J1	-

Tabel 4.27 *Initial State* untuk *Optimizer* pada *Horizon* 2

Mesin	Waktu Saat Ini	Tinta Saat Ini
Flexo A	0.91	[9]
Flexo B	1.68	[6, 8]

State mesin pada *Horizon* 2 dibentuk sesuai dengan prosedur pada Gambar 4.15 dan menjadi input awal bagi proses pembangunan data mesin pada Gambar 4.16.

4.6.4.4 Penyusunan Pekerjaan untuk Optimasi pada *Horizon* 2

Pada $t = 1.00\ \mathrm{h}$, pesanan baru J5-J8 tiba.

Daftar awal: J4

Tambahan baru: J5, J6, J7, J8

Tabel 4.28 Data Mesin untuk Optimasi *Horizon* 2

Mesin	Unlocked Jobs
Flexo A	J4, J5, J7
Flexo B	J6, J8

Distribusi pekerjaan *unlocked* ke masing-masing mesin mengikuti langkah penyusunan ulang data mesin pada Gambar 4.16.

4.6.4.5 Penetapan Due Date *Horizon* 2

Pada *horizon* ini, algoritma menerapkan aturan TWK yang sama seperti pada *Horizon* 1, namun hanya memberikan *due date* baru bagi pekerjaan yang baru tiba. Pekerjaan J1-J4 mempertahankan *due date* sebelumnya (2.61 jam), sedangkan pekerjaan baru (J5-J8) menerima *baseline due date* baru sebesar 4.01 jam.

Tabel 4.29 *Visible Jobs* pada *Horizon* 2

Pekerjaan	Waktu Kedatangan	Due Date
J1	0.00	2.61
J2	0.00	2.61
J3	0.00	2.61
J4	0.00	2.61
J5	1.00	4.01
J6	1.00	4.01
J7	1.00	4.01
J8	1.00	4.01

Penetapan *due date* J1-J8 merupakan implementasi lanjutan dari prosedur perhitungan beban kerja dan aturan TWK yang dimodifikasi pada Gambar 4.17 dan Gambar 4.18.

4.6.5 *Simulated Annealing - Horizon* 2

4.6.5.1 Jadwal Awal *Greedy Horizon* 2

Dengan mengikuti logika penugasan *greedy* yang sama seperti pada *Horizon* 1, algoritma menghasilkan jadwal awal yang layak berdasarkan kondisi mesin saat ini dan pekerjaan yang terlihat pada $t = 1.00\ \mathrm{h}$.

Tabel 4.30 Jadwal *Greedy* untuk *Horizon* 2

Mesin	Urutan Pekerjaan
Flexo A	J4, J8, J6
Flexo B	J5, J7

Jumlah pekerjaan terlambat = 1, Perubahan tinta = 3, *Makespan* = 4.9, *Fitness* = 5.49

Penyusunan jadwal awal *greedy* pada Horizon 2 ini mengikuti langkah konstruksi solusi awal pada Gambar 4.4, dengan state awal mesin yang diambil dari hasil pembentukan state pada Gambar 4.15.

4.6.5.2 Inisialisasi Temperatur *Horizon* 2

1. $T_0 = 15.86$
2. $T_{\min} = 0.79$

Estimasi T_0 dan penentuan T_{min} pada *Horizon* 2 mengikuti prosedur inisialisasi temperatur yang dirancang pada Gambar 4.13.

4.6.5.3 Iterasi *Simulated Annealing Horizon* 2

Tabel 4.31 Log Iterasi SA *Horizon* 2

Iter	Neighbor	Jadwal	Base F	Neighbor F	ΔF	Temp	Prob	Diterima	Best
1	Swap Within	A: 4,6,8 / B: 5,7	5.49	6.57	1.08	15.86	0.93	Ya	5.49
2	Relocate	A: 4,8 / B: 6,5,7	6.57	5.51	1.06	15.78	1.07	Ya	5.49
3	Swap Between	A: 4,6 / B: 8,5,7	5.51	8.51	3.00	15.71	0.83	Ya	5.49
4	Exchange	A: 4,5,7 / B: 8,6	8.51	8.57	0.06	15.63	1.00	Ya	5.49
5	Block Insert	A: 4 / B: 8,5,7,6	8.57	10.57	2.00	15.55	0.88	Ya	5.49
6	Swap Between	A: 5 / B: 8,4,7,6	10.57	8.52	2.05	15.47	1.14	Ya	5.49

Rangkaian iterasi dan keputusan penerimaan solusi mengikuti *loop* utama SA pada Gambar 4.1, dengan mekanisme penerimaan Metropolis yang dirinci pada Gambar 4.11 dan pembaruan temperatur geometrik pada Gambar 4.12.

Tabel 4.32 Jadwal Akhir *Horizon* 2

Mesin	Urutan Pekerjaan
Flexo A	J4, J8, J6
Flexo B	J5, J7

Jumlah pekerjaan terlambat = 1, Perubahan tinta = 3, *Makespan* = 4.90 jam, *Fitness* = 5.49. Jadwal akhir Horizon 2 merupakan solusi terbaik yang dipertahankan oleh *loop* SA untuk horizon ini, sebelum jadwal tersebut digabungkan oleh mekanisme *rolling horizon* pada Gambar 4.14.

4.6.6 Jadwal Akhir dan Evaluasi *Fitness*

Setelah kedua *horizon* selesai, algoritma membentuk jadwal akhir dengan menggabungkan pekerjaan yang telah terkunci dari *Horizon* 1 dengan urutan hasil optimasi pada *Horizon* 2. Jadwal gabungan ini menjadi rencana produksi final untuk seluruh delapan pekerjaan.

Tabel 4.33 Jadwal Akhir dari Seluruh *Horizon*

Mesin	Urutan Pekerjaan
Flexo A	J3, J4, J8, J6
Flexo B	J2, J1, J5, J7

Jadwal keseluruhan kemudian dievaluasi menggunakan fungsi objektif berbobot yang sama, yaitu mempertimbangkan penalti keterlambatan, penalti perubahan tinta, dan *makespan*.

Tabel 4.34 Metrik Akhir dari Seluruh *Horizon*

Metrik	Nilai
Penalti tardy	1
Penalti tinta	3
Makespan	4.9 jam
Fitness	5.49

Evaluasi jadwal akhir menggunakan fungsi objektif berbobot menghasilkan nilai *fitness* 5,49, dengan 1 penalti *tardiness*, penalti tinta sebesar 3, dan *makespan* 4,90 jam. *Makespan* ditentukan oleh waktu penyelesaian terakhir pada mesin Flexo A. Penalti tinta masih muncul akibat adanya perubahan tinta yang tidak sepenuhnya tereliminasi, yang dipengaruhi oleh keterbatasan jumlah iterasi SA pada setiap *horizon*. Dengan iterasi yang lebih banyak, algoritma berpotensi menurunkan penalti tinta dan meningkatkan kualitas jadwal secara keseluruhan.

4.7 Rancangan Pengujian

Bagian ini menjelaskan konfigurasi eksperimen yang dilakukan pada seluruh skenario. Setiap skenario dievaluasi menggunakan dataset dan pengaturan algoritma yang sama, dengan perbedaan hanya pada formulasi tujuan dan struktur kedatangan pekerjaan.

4.7.1 Definisi Skenario

Tabel 4.35 Definisi Skenario Eksperimen

Kode	Skenario	Deskripsi	Horizon	Distribusi Pekerjaan
S1	Keseimbangan mesin	Warna tinta diabaikan; tujuan meminimalkan ketidakseimbangan beban kerja dan *makespan*. SDST dinonaktifkan.	Satu *horizon*	Semua 15 pekerjaan
S2	Penjadwalan warna	Tujuan meminimalkan perubahan tinta saja (*setup costs*).	Satu *horizon*	Semua 15 pekerjaan
S3	Penjadwalan statis kombinasi	Pengaturan tinta dan *makespan* dioptimalkan secara bersamaan; semua pekerjaan diketahui pada (t=0).	Satu *horizon*	Semua 15 pekerjaan
S4A	Dinamis (2 *batch*)	Penjadwalan *rolling horizon*; kedatangan pada 0 jam dan 3 jam.	Kedatangan [0, 3]	0: Job 1-8 3: Job 9-15
S4B	Dinamis (2 *batch* interval lebih pendek)	Sama seperti S4A namun batch kedua tiba lebih awal.	Kedatangan [0, 1]	0: Job 1-8 1: Job 9-15
S4C	Dinamis (3 *batch*)	Kedatangan lebih sering untuk menguji re-optimalisasi.	Kedatangan [0, 1.5, 3]	0: Job 1-5 1.5: Job 6-10 3: Job 11-15

Pekerjaan diurutkan berdasarkan identifikasi (*ID*), di mana ID yang lebih kecil selalu termasuk dalam *batch* yang lebih awal. Semua mesin, kecepatan pemrosesan, dan parameter *setup* tetap identik di seluruh skenario.

4.7.2 Pengujian Parameter *Simulated Annealing*

Pada tahap pertama, parameter algoritma SA di-*tuning* menggunakan desain *one-factor-at-a-time* (OFAT). Pengujian parameter dilakukan pada Skenario S3 yang digunakan sebagai *baseline* representatif karena semua *job* tersedia pada waktu nol dan baik *makespan* maupun penalti tinta diaktifkan.

Konfigurasi parameter *baseline* terlebih dahulu ditetapkan. Setelah itu, untuk setiap parameter SA, beberapa nilai alternatif diuji dengan mempertahankan parameter lainnya pada nilai *baseline*. Parameter yang dipertimbangkan meliputi:

1. jumlah iterasi maksimum
2. faktor pendinginan
3. probabilitas penerimaan awal
4. *floor ratio*
5. *critical machine bias*

Setiap konfigurasi dievaluasi dengan 30 percobaan independen menggunakan *random seed* yang berbeda. Untuk setiap percobaan, hasil mentah seperti *fitness, makespan*, jumlah *ink violation*, jumlah pekerjaan terlambat, dan *runtime* dicatat.

Tabel 4.36 Rencana Pengujian Parameter SA

Parameter	**Nilai yang Diuji**	**Nilai Awal**	**Deskripsi**
Iterasi maksimum	5000 / 10000 / 25000 / 50000 / 100000 / 250000 / 500.000	-	Mengontrol waktu eksekusi dan panjang konvergensi
Faktor pendinginan	0.990 / 0.991 / 0.992 / 0.993 / 0.994 / 0.995 / 0.996 / 0.997 / 0.998 / 0.999	0.995	Penurunan temperatur per iterasi
Probabilitas penerimaan awal	0.6 / 0.65 / 0.7 / 0.75 / 0.8 / 0.85 / 0.9	0.8	Peluang menerima solusi lebih buruk di awal
Rasio batas bawah (*floor ratio*)	0.01 / 0.025 / 0.04 / 0.05 / 0.06 / 0.075 / 0.10	0.05	Fraksi temperatur minimum
Bias mesin kritis	0.1 / 0.2 / 0.3 / 0.4 / 0.5 / 0.6 / 0.7 / 0.8 / 0.9 / 1.0	0.7	Peluang memilih mesin dengan beban tertinggi

Tabel 4.37 Rencana Tata Letak Data Mentah untuk Pengujian Parameter SA

Parameter	**Nilai**	**Seed**	**Fitness**	**Makespan**	**Penalti Tinta**	**Penalti Tardy**	**Runtime**
Max iter	5000	42					
Max iter	5000	43					
…	…	…	…	…	…	…	…

Tabel 4.38 Rencana Ringkasan Statistik untuk Pengujian Parameter SA

Parameter	**Nilai**	**Rata-rata Fitness**	**Standar Deviasi Fitness**	**Rata-rata Makespan**	**Rata-rata Penalti Tinta**	**Rata-rata Runtime**
Max iter	5000					
Max iter	10000					
…	…	…	…	…	…	…

Tabel 4.37 dan Tabel 4.38 mendefinisikan struktur pencatatan data yang direncanakan untuk pengujian parameter. Nilai mentah untuk setiap *seed* akan

disimpan dalam Tabel 4.37, sedangkan statistik ringkas seperti rata-rata, dan standar deviasi, akan dihitung dan disajikan dalam Tabel 4.38. Konfigurasi dengan rata-rata *fitness* terendah dan variasi paling stabil akan dipilih untuk eksperimen penyesuaian bobot berikutnya.

4.7.3 Pengujian Bobot Tujuan

Setelah konfigurasi parameter SA ditetapkan, tahap berikutnya adalah menyesuaikan bobot pada fungsi objektif agar diperoleh keseimbangan yang wajar antara berbagai tujuan penjadwalan. Pengujian ini menggunakan *grid* bobot terstruktur. Secara umum, seluruh kombinasi bobot yang dipertimbangkan memenuhi dua ketentuan utama. Pertama, setiap bobot berada pada rentang 0 sampai 1. Kedua, jumlah seluruh bobot dalam satu kombinasi sama dengan 1 dan tidak ada bobot yang bernilai 0. Kombinasi dibuat dengan *step* 0,05 sehingga ruang bobot dieksplorasi secara sistematis dengan resolusi yang cukup halus.

4.7.3.1 Kasus statis (Skenario S3)

Pada konteks statis, fungsi objektif hanya mengaktifkan dua komponen tujuan, yaitu *makespan* dan penalti tinta. Dengan demikian, kombinasi bobot terdiri atas bobot untuk *makespan* dan bobot untuk penalti tinta. Keduanya dibangkitkan dengan langkah 0,05 dengan syarat keduanya positif dan jumlahnya sama dengan 1. Pasangan seperti (0,05; 0,95), (0,10; 0,90), dan seterusnya disertakan, sedangkan kombinasi yang mengandung bobot 0 tidak digunakan.

Setiap kombinasi diuji pada Skenario 3 melalui beberapa percobaan independen dengan *seed* yang berbeda. Untuk setiap kombinasi dihitung rata-rata *makespan* dan rata-rata penalti tinta, kemudian divisualisasikan menggunakan diagram garis untuk melihat pengaruh perubahan bobot terhadap kedua metrik tersebut.

Sebagai pelengkap, disusun indikator gabungan sederhana berupa penjumlahan rata-rata *makespan* dan rata-rata penalti tinta yang ditulis sebagai M + I. Indikator ini digunakan untuk membantu memilih kombinasi bobot yang paling seimbang dalam menekan waktu penyelesaian dan jumlah pergantian tinta. Kombinasi terbaik pada Skenario 3 dipilih dari nilai rata-rata M + I terendah dengan variasi yang relatif stabil.

Tabel 4.39 Rencana Tata Letak Data Mentah untuk Pengujian Bobot Tujuan

Skenario	wt	wi	wm	Seed	Fitness	Makespan	Penalti Tinta	Penalti Tardy
S3	0.05	0.05	0.9	1				
S3	0.05	0.05	0.9	2				
…	…	…	…	…	…	…	…	…

Tabel 4.40 Rencana Ringkasan Statistik untuk Pengujian Bobot Tujuan S3

Bobot		Rata-rata			Standar Deviasi (M + I)
Makespan	Penalti Tinta	Makespan	Penalti Tinta	(M + I)	
0.95	0.05				
0.9	0.1				
...	…	…	…	…	…

4.7.3.2 Kasus dinamis (Skenario S4B)

Pada konteks dinamis, pengujian bobot dilakukan pada Skenario 4B yang merepresentasikan lingkungan *rolling horizon* dengan keberadaan pekerjaan *leftover*. Tiga komponen tujuan diaktifkan secara bersamaan, yaitu *makespan*, penalti tinta, dan penalti *tardy*. Setiap kombinasi bobot terdiri atas tiga bobot positif yang jumlahnya bernilai 1.

Kombinasi bobot dibangkitkan pada *grid simplex* tiga dimensi dengan langkah 0,05 sehingga seluruh kombinasi yang memenuhi ketentuan rentang dan jumlah bobot tercakup dalam pengujian. Untuk setiap kombinasi, algoritma diuji melalui beberapa percobaan independen, kemudian diringkas dalam bentuk rata-rata *fitness*, rata-rata *makespan*, rata-rata penalti tinta, dan rata-rata jumlah pekerjaan terlambat.

Pada lingkungan dinamis ini digunakan indikator gabungan 2T + I + 0,1M yang mengintegrasikan penalti *tardy*, penalti tinta, dan *makespan* ke dalam satu nilai. Koefisien 2 pada *tardy* mencerminkan prioritas tertinggi pada pemenuhan *due date* dan keadilan pemrosesan order, komponen tinta diberi prioritas menengah, sedangkan *makespan* diberi prioritas lebih rendah melalui koefisien 0,1. Dengan demikian, indikator gabungan ini selaras dengan kebijakan bahwa penurunan keterlambatan merupakan fokus utama, diikuti stabilisasi pergantian tinta, dan terakhir pemendekan *makespan*.

Kombinasi bobot divisualisasikan dalam bentuk *ternary heatmap* untuk membantu interpretasi sensitivitas performa terhadap variasi bobot. Setiap titik pada segitiga mewakili satu kombinasi bobot positif dengan total 1, sementara warna menunjukkan nilai rata-rata suatu metrik, termasuk indikator gabungan 2T + I + 0,1M. Wilayah dengan nilai indikator gabungan rendah diidentifikasi sebagai kandidat bobot yang memberikan kompromi terbaik antara ketiga tujuan. Kombinasi terpilih dari wilayah ini kemudian digunakan pada seluruh eksperimen berikutnya.

Tabel 4.41 Rencana Ringkasan Statistik untuk Pengujian Bobot Tujuan S4B

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.9	0.05	0.05					

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.85	0.1	0.05					
...	…	…	…	…	…	…	…

4.7.4 Pengujian *Due Date Multiplier*

Dengan parameter SA dan bobot tujuan sudah tetap, tahap ketiga berfokus pada penyesuaian pengali *due date* yang digunakan dalam aturan TWK untuk skenario dinamis. Pengali *due date* mengendalikan tingkat ketatnya *due date* dan karenanya memengaruhi jumlah pekerjaan terlambat serta *trade off* antara *tardiness* dan tujuan lainnya.

Beberapa nilai pengali diuji (misalnya 0,8, 1,0, dan 1,2) pada skenario dinamis S4A, S4B, dan S4C. Untuk setiap skenario dan setiap nilai pengali, dilakukan 30 percobaan independen menggunakan himpunan *seed* yang sama. Nilai *fitness*, *makespan*, penalti tinta, jumlah pekerjaan terlambat, dan profit dicatat, lalu diringkas secara statistik.

Tabel 4.42 Rencana Tata Letak Data Mentah untuk Pengujian *Due Date Multiplier*

Skenario	Seed	Due Date Multiplier	Fitness	Makespan	Penalti Tinta	Penalti Tardy	Runtime
S4A	42	0.8					
S4A	42	0.8					
…	…	…	…	…			

Tabel 4.43 Rencana Ringkasan Statistik untuk Pengujian *Due Date Multiplier*

Skenario	Due Date Multiplier	Rata-rata				Std Fitness	Rata-rata Runtime
		Makespan	Penalti Tinta	Penalti Tardy	Fitness		
S4A	0.8						
S4A	1						
S4A	1.2						
…	…	…	…	…	…	…	…

4.7.5 Pengujian Skenario

Setelah parameter SA, bobot tujuan, dan *due date multiplier* ditetapkan, tahap berikutnya adalah mengevaluasi performa algoritma pada enam skenario yang telah dirumuskan. Struktur skenario dianggap tetap, sehingga pengujian difokuskan pada perbandingan hasil secara konsisten. Setiap skenario dijalankan sebanyak 30 kali dengan *seed* berbeda untuk menangkap sifat stokastik SA,

menghasilkan metrik *fitness*, *makespan*, penalti tinta, jumlah pekerjaan terlambat, dan *runtime*. Seluruh data mentah beserta ringkasan statistiknya kemudian disusun dalam dua jenis tabel sebagai dasar analisis pada bab berikutnya.

Tabel 4.44 Rencana Tata Letak Data Mentah untuk Pengujian Skenario

Skenario	Seed	Fitness	Makespan	Penalti Tinta	Penalti Tardy	Runtime
S1	42					
S1	42					
…	…	…	…	…	…	…

Tabel 4.45 Rencana Ringkasan Statistik untuk Pengujian Skenario

Skenario	Metrik	Rata-rata	Standar Deviasi	Min	Max
S1	Fitness				
	Makespan				
	Runtime				
...	...	...	...	...	...
S4C	Fitness				
	Makespan				
	Ink Violation				
	Tardy Violation				
	Runtime				

Tabel ringkasan ini memberikan gambaran kompak mengenai kinerja tipikal dan variasi hasil pada setiap skenario. Pada Bab 6, tabel tersebut diisi untuk seluruh skenario dan digunakan untuk membandingkan lingkungan statis dan dinamis, serta membahas dampak pola kedatangan *job* terhadap kualitas solusi, kestabilan, dan kebutuhan waktu komputasi.

4.7.6 Evaluasi Jadwal Terbaik

Selain analisis agregat, penting juga untuk mengkaji jadwal individu secara lebih rinci. Untuk setiap skenario, satu jadwal “terbaik” dipilih dari 30 percobaan dan dianalisis secara mendalam.

Pemilihan jadwal terbaik mengikuti prioritas optimasi utama sebagai berikut:

1. Untuk skenario tanpa keterlambatan (S1-S3), jadwal terbaik dipilih terutama berdasarkan nilai *fitness*, dengan *makespan* dan jumlah penalti tinta sebagai kriteria pembeda tambahan jika diperlukan.
2. Untuk skenario yang melibatkan *tardiness* (S4A-S4B), pemilihan konsisten dengan prioritas komposit yang digunakan pada tahap pengujian bobot, yaitu keadilan dan pemenuhan *due date* ditempatkan pada prioritas tertinggi, diikuti oleh penalti tinta, dan *makespan* sebagai prioritas relatif paling rendah.

Setelah percobaan terbaik diidentifikasi untuk setiap skenario, beberapa kelompok metrik dihitung dan dilaporkan:

1. Metrik optimasi mencakup *fitness, makespan*, jumlah penalti tinta, jumlah pekerjaan terlambat, dan *runtime* untuk jadwal terpilih.
2. Metrik utilisasi mesin untuk setiap mesin dihitung waktu kerja, time span, efisiensi, dan *coverage* sesuai definisi pada Subbab 2.8, sehingga distribusi beban kerja antar mesin dapat dianalisis.
3. Metrik keuangan meliputi pendapatan, biaya penalti tinta, dan laba bersih yang dihasilkan oleh jadwal tersebut.
4. Struktur jadwal secara rinci berupa urutan *job* pada setiap mesin, meliputi ID *job*, *batch*, level tinta, waktu mulai dan selesai, *due date* (untuk skenario dinamis), serta penanda apakah terjadi penalti (tinta maupun keterlambatan).

Selain tabel, untuk setiap jadwal terbaik juga disusun *Gantt chart* yang memvisualisasikan pemakaian mesin sepanjang waktu, transisi warna, dan *job* yang terlambat. Visualisasi ini membantu menafsirkan metrik numerik dan menunjukkan bagaimana algoritma menyeimbangkan tujuan yang saling berkonflik dalam bentuk jadwal konkret.

Desain tabel yang digunakan untuk evaluasi jadwal terbaik adalah sebagai berikut.

Tabel 4.46 Rencana Tata Letak Metrik Optimasi Jadwal Terbaik

Metrik	Nilai
Fitness	
Penalti Tinta	
Penalti Tardy	
Makespan (h)	
Runtime (s)	

Tabel 4.47 Rencana Tata Letak Metrik Utilisasi Mesin Jadwal Terbaik

Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Flexo A				
Flexo B				
Flexo C				

Tabel 4.48 Rencana Tata Letak Metrik Keuangan Jadwal Terbaik

Metrik Keuangan	Nilai
Pendapatan	
Penalti Tinta	
Laba Bersih	

Tabel 4.49 Rencana Tata Letak Jadwal Rinci Jadwal Terbaik

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Deadline	Penalti
Flexo A	1							
Flexo A	2							
...	...	...	...	...	...	...	...	...

4.7.7 Perbandingan Jadwal Optimasi dengan Jadwal Industri

Tahap terakhir dari rancangan eksperimen adalah membandingkan jadwal terbaik hasil optimasi dengan jadwal aktual yang digunakan di industri. Perbandingan ini bertujuan untuk mengukur secara kuantitatif potensi perbaikan apabila perusahaan menerapkan pendekatan penjadwalan berbasis SA yang diusulkan.

Pada penelitian ini, jadwal optimasi yang digunakan sebagai pembanding diambil dari Skenario 3, karena:

1. seluruh *job* diketahui sejak awal (t = 0),
2. tujuan yang dioptimalkan (*makespan* dan penalti tinta) selaras dengan kondisi aktual di perusahaan,
3. struktur data (*job*, mesin, kapasitas, dan parameter proses) sama dengan yang digunakan pada jadwal industri.

Dengan demikian, kedua jadwal (optimasi dan industri) menggunakan himpunan *job*, mesin, dan parameter proses yang identik, sehingga selisih kinerja dapat ditafsirkan langsung sebagai potensi perbaikan tanpa perubahan kapasitas fisik.

Perbandingan dilakukan dalam tiga kelompok metrik utama:

1. Metrik optimasi Mencakup *fitness*, penalti tinta, dan *makespan*. Metrik ini digunakan untuk menunjukkan perbedaan kualitas jadwal secara langsung dari sisi fungsi objektif dan kinerja waktu.
2. Metrik utilisasi mesin Mencakup waktu kerja, time span, efisiensi, dan cakupan setiap mesin. Metrik ini digunakan untuk menilai seberapa padat dan seimbang beban kerja pada jadwal optimasi dibanding jadwal industri.
3. Metrik keuangan Mencakup pendapatan, penalti tinta dalam rupiah, dan laba bersih. Metrik ini digunakan untuk mengonversi perbedaan teknis (setup dan *makespan*) ke dalam implikasi finansial yang mudah dipahami manajemen.

Selain tabel numerik, *Gantt chart* jadwal industri dan jadwal optimasi akan ditampilkan untuk memberikan ilustrasi visual mengenai urutan *job*, pola perubahan tinta, dan celah *idle* antar pekerjaan. Visualisasi ini membantu menjelaskan sumber inefisiensi pada jadwal industri dan bagaimana jadwal optimasi memperbaikinya.

Tabel 4.50 Rencana Tata Letak Perbandingan Metrik Optimasi

Metrik	Jadwal Optimasi	Jadwal Industri
Fitness		
Penalti Tinta		
Makespan (h)		

Tabel 4.51 Rencana Tata Letak Perbandingan Utilisasi Mesin

Jadwal	Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Optimasi	Flexo A				
	Flexo B				
	Flexo C				
Industri	Flexo A				
	Flexo B				
	Flexo C				

Tabel 4.52 Rencana Tata Letak Perbandingan Metrik Keuangan

Metrik Keuangan	Jadwal Optimasi	Jadwal Industri
Pendapatan		
Penalti Tinta		
Laba Bersih		

Tabel 4.53 Rencana Tata Letak Jadwal Rinci Jadwal Industri

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Deadline	Penalti
Flexo A	1							
Flexo A	2							
...	...	...	...	...	...	...	...	...

BAB 5 IMPLEMENTASI

5.1 *Import Library*, Definisi Struktur Data, Dan Evaluasi Solusi

Tabel 5.1 Kode Sumber Importasi *Library* yang Diperlukan

```
Import Library
1  import time
2  import math
3  import random
4  from collections import Counter
5  from dataclasses import dataclass, field, replace
6  from collections import Counter
7  from itertools import combinations, permutations
```

Bagian ini mengimplementasikan komponen dasar yang diperlukan untuk keseluruhan sistem penjadwalan. *Library* yang digunakan meliputi modul standar *Python* (seperti *math, random,* dan *dataclasses*). Seluruh *library* diperlukan untuk komputasi waktu proses, operasi probabilistik pada SA, pengacakan struktur tetangga, penyimpanan state, serta penanganan array.

1. Baris 1-7: Bagian ini mengimpor seluruh pustaka *Python* standar dan modul eksternal yang diperlukan untuk mendukung sistem penjadwalan, seperti modul *math* untuk komputasi waktu, *random* untuk operasi probabilistik pada algoritma SA, serta *dataclasses* untuk manajemen struktur data yang efisien.

Selain *import*, bagian ini juga mencakup definisi struktur data fundamental berupa konfigurasi optimasi, entitas pekerjaan, serta struktur solusi. Seluruh elemen ini sesuai dengan desain representasi yang dibahas pada Bab 4.2.

5.1.1 Kelas Konfigurasi Optimasi

Tabel 5.2 Kode Sumber Struktur Data *Optimizer Config*

```
Struktur Data Optimizer Config
1   @dataclass(frozen=True, kw_only=True)
2   class OptimizerConfig:
3       machine_production_rates: dict[str, float]
4       sequence_dependent_setup_time: float = 0.75
5       initial_state: dict | None = None
6
7       tardiness_weight: float = 2.0
8       ink_weight: float = 1.0
9       makespan_weight: float = 0.1
10      due_date_multiplier: float = 1.0
11
12      seed: int | None = None
13      max_iter: int = 100_000
14
15      critical_machine_bias: float = 0.7
16      p0: float = 0.80
17      floor_ratio: float = 0.05
18      alpha: float = 0.995
19
20      machines: tuple[str, ...] = field(init=False)
21
22      def __post_init__(self):
23          object.__setattr__(
```

```
24              self,
25              "machines",
26              tuple(self.machine_production_rates),
27          )
28
29      def proc_time(self, machine: str, qty: float) -> float:
30          return (
31              qty
32              / self.machine_production_rates[machine]
33              / 60.0
34          )
35
36      def seed_rng(self, extra_seed: int | None = None):
37          if self.seed is not None:
38              s = self.seed + (extra_seed or 0)
39              random.seed(s)
```

Kelas ini menyimpan seluruh parameter yang digunakan dalam proses optimasi, termasuk konfigurasi mesin, parameter SA (seperti suhu awal, α, p0, dan *floor ratio*), serta bobot tujuan. Implementasi ini merupakan bentuk konkret dari Tabel 4.7 pada BAB 4. Metode tambahan seperti *proc_time()* digunakan menghitung waktu produksi berdasarkan kecepatan mesin, sedangkan *seed_rng()* memastikan proses *random* dapat direplikasi.

1. Baris 1-20: Mendefinisikan kelas *OptimizerConfig* sebagai struktur data yang menyimpan parameter optimasi, termasuk kecepatan produksi mesin, bobot tujuan untuk fungsi *fitness* (*tardiness*, tinta, dan *makespan*), serta parameter spesifik SA seperti suhu awal (p0) dan laju pendinginan (*alpha*).
2. Baris 22-27: Mengimplementasikan metode *post_init* yang berfungsi untuk melakukan inisialisasi otomatis pada daftar nama mesin segera setelah objek konfigurasi dibuat.
3. Baris 29-34: Mendefinisikan fungsi untuk menghitung waktu produksi berdasarkan kuantitas pekerjaan dibagi dengan kecepatan mesin terkait, yang kemudian dikonversi ke dalam satuan waktu jam (dibagi 60.0).
4. Baris 36-39: Menyediakan fungsi *seed_rng* untuk mengatur nilai *seed* pada generator angka acak guna memastikan bahwa hasil pengujian algoritma dapat direplikasi dengan hasil yang sama.

5.1.2 Representasi Pekerjaan

Tabel 5.3 Kode Sumber Struktur Data Pekerjaan

```
Struktur Data Job
1   @dataclass(slots=True)
2   class Job:
3       order_code: str
4       inks: list[int] = field(default_factory=list)
5       production_quantity: int = 0
6       arrival_time: float = 0.0
7       due_date: float = 0.0
8
9       start_time: float = 0.0
10      completion_time: float = 0.0
```

Kelas ini merupakan implementasi langsung dari struktur pekerjaan pada Tabel 4.5. Setiap pekerjaan menyimpan daftar tinta, kuantitas produksi, waktu kedatangan, *due date*, serta informasi waktu aktual (*start* dan *completion*) yang dihasilkan setelah proses penjadwalan. Representasi ini memungkinkan sistem untuk melacak setiap *job* secara lengkap di seluruh *horizon* penjadwalan.

1. Baris 1-10: Mendefinisikan kelas *Job* yang merepresentasikan satu entitas pesanan. Struktur ini menyimpan atribut statis (kode pesanan, daftar tinta, kuantitas) dan atribut dinamis yang diisi saat penjadwalan (waktu mulai, waktu selesai, kedatangan, dan tenggat waktu).

5.1.3 Representasi Solusi Jadwal

Tabel 5.4 Kode Sumber Struktur Data Representasi Solusi

Struktur Data Solution

```
1   @dataclass(slots=True)
2   class Solution:
3       schedule: dict[str, list[Job]]
4       fitness: float = float("inf")
5       tardiness_violations: float = 0.0
6       ink_violations: float = 0.0
7       makespan: float = 0.0
8       critical_machine: str | None = None
9
10      def evaluate(self, cfg: OptimizerConfig):
11          (
12              self.fitness,
13              self.tardiness_violations,
14              self.ink_violations,
15              self.makespan,
16              self.critical_machine,
17          ) = calculate_fitness(self.schedule, cfg)
18          return self.fitness
19
20      def copy(self):
21          return Solution(
22              {m: lst[:] for m, lst in self.schedule.items()},
23              self.fitness,
24              self.tardiness_violations,
25              self.ink_violations,
26              self.makespan,
27              self.critical_machine,
28          )
```

Kelas ini menyimpan jadwal yang dihasilkan untuk tiap mesin, serta nilai evaluasi mencakup *fitness* total, *makespan, ink violations, tardiness violations*, dan *critical machine*. Fungsi *evaluate()* memanggil kalkulasi *fitness* untuk memperbarui seluruh informasi tersebut, sedangkan *copy()* digunakan untuk menduplikasi solusi tanpa efek samping.

1. Baris 1-8: Mendefinisikan kelas Solution untuk menampung hasil jadwal. Kelas ini memiliki atribut *schedule* (kamus berisi daftar pekerjaan per mesin) dan atribut metrik evaluasi seperti *fitness*, tardiness_violations, dan *makespan*.
2. Baris 10-18: Metode *evaluate* berfungsi sebagai penghubung ke fungsi kalkulasi *fitness*. Metode ini memperbarui nilai *fitness* dan komponen penaltinya dalam objek solusi berdasarkan konfigurasi yang diberikan.

3. Baris 20-28: Metode *copy* membuat salinan mendalam *(deep copy)* dari objek solusi. Fungsi ini penting untuk algoritma optimasi agar solusi baru dapat dimodifikasi tanpa merusak solusi sebelumnya.

5.1.4 Fungsi Perhitungan *Sequence Dependent Setup*

Tabel 5.5 Kode Sumber Algoritma Pergantian Tinta

Algoritma Changeover

```
1   CHANGEOVER_CACHE: dict[tuple, int] = {}
2
3   def changeover(prev_inks, next_inks) -> int:
4       key = (tuple(prev_inks), tuple(next_inks))
5       hit = CHANGEOVER_CACHE.get(key)
6       if hit is not None:
7           return hit
8
9       A, B = Counter(prev_inks), Counter(next_inks)
10      if A == B:
11          CHANGEOVER_CACHE[key] = 0
12          return 0
13
14      rem, add = [], []
15      for lvl in set(A) | set(B):
16          d = A.get(lvl, 0) - B.get(lvl, 0)
17          (
18              (rem if d > 0 else add).extend([lvl] * abs(d))
19              if d
20              else None
21          )
22
23      m = min(len(rem), len(add))
24      deletions = len(rem) - m
25      best = 10**9
26
27      rem_choices = (
28          [tuple(range(len(rem)))]
29          if len(rem) <= m
30          else combinations(range(len(rem)), m)
31      )
32      add_choices = (
33          [tuple(range(len(add)))]
34          if len(add) <= m
35          else combinations(range(len(add)), m)
36      )
37
38      for ridx in rem_choices:
39          r = [rem[i] for i in ridx]
40          for aidx in add_choices:
41              sel = [add[i] for i in aidx]
42              for p in permutations(sel):
43                  cost = deletions + sum(
44                      1 for x, y in zip(r, p) if x > y
45                  )
46                  if cost < best:
47                      best = cost
48
49      CHANGEOVER_CACHE[key] = best
50      return best
```

Fungsi ini mengimplementasikan model penalti kegelapan tinta seperti pada Bab 4.3.2. Perhitungan dilakukan dengan membandingkan komposisi tinta antara

dua pekerjaan menggunakan mekanisme *deletions* dan *substitutions* yang dihitung secara kombinatorial. Fungsi ini juga memanfaatkan cache untuk mempercepat evaluasi berulang.

1. Baris 1: Penyimpanan map *key* dan *value* untuk *cache* biaya tinta.
2. Baris 3-9: Fungsi *changeover* dimulai dengan pengecekan *cache*. Jika kombinasi tinta pernah dihitung sebelumnya, hasil langsung diambil dari memori untuk menghemat waktu komputasi.
3. Baris 10-12: Penanganan kasus dasar di mana komposisi tinta antara dua pekerjaan identik. Fungsi langsung mengembalikan nilai 0.
4. Baris 14-21: Menghitung perbedaan himpunan warna antara pekerjaan sebelumnya dan pekerjaan saat ini. Bagian ini mengidentifikasi warna mana yang harus dihapus (*rem*) dan mana yang harus ditambahkan (*add*).
5. Baris 23-36: Persiapan logika kombinatorial. Kode ini menentukan jumlah *slot* yang tersedia untuk substitusi dan membangkitkan semua kemungkinan kombinasi penghapusan dan penambahan tinta.
6. Baris 38-50: *Loop* utama yang mengevaluasi setiap permutasi urutan penggantian tinta. Algoritma mencari urutan dengan biaya penalti terendah (misalnya menghindari warna terang diganti warna gelap tanpa pencucian) dan mengembalikan nilai minimal tersebut.

5.1.5 Fungsi Perhitungan *Fitness* Jadwal

Tabel 5.6 Kode Sumber Algoritma Hitung *Fitness*

Algoritma Calculate Fitness

```
1  def calculate_fitness(schedule: dict, cfg: OptimizerConfig):
2      tardy_v = ink_v = 0.0
3      latest = -1.0
4      critical_m = None
5      setup_base = cfg.sequence_dependent_setup_time
6      init_state = cfg.initial_state or {}
7
8      for m, jobs in schedule.items():
9          state = init_state.get(m, {})
10         t0 = float(state.get("time", 0.0))
11         p_inks = list(state.get("inks", []))
12         t = t0
13
14         for job in jobs:
15             ink_chg = changeover(p_inks, job.inks)
16             setup = setup_base if ink_chg > 0 else 0.0
17             setup_start = max(t, job.arrival_time)
18             start = setup_start + setup
19             done = start + cfg.proc_time(
20                 m, job.production_quantity
21             )
22
23             if done > job.due_date:
24                 tardy_v += 1.0
25             ink_v += ink_chg
26
27             t = done
28             p_inks = job.inks
```

```
29
30          if t > latest:
31              latest = t
32              critical_m = m
33
34      fit = (
35          cfg.tardiness_weight * tardy_v
36          + cfg.ink_weight * ink_v
37          + cfg.makespan_weight * latest
38      )
39      return fit, tardy_v, ink_v, latest, critical_m
```

Fungsi ini mengimplementasikan perhitungan *fitness* sesuai rancangan pada Bab 4.3.1. Evaluasi mencakup penalti tinta, dihitung dari total perubahan tinta antar pekerjaan, penalti *tardy*, dihitung berdasarkan jumlah pekerjaan yang selesai lebih dari *due date*, *makespan*, ditentukan oleh mesin yang menyelesaikan pekerjaan paling akhir. Seluruh komponen digunakan dalam fungsi objektif terintegrasi berbobot.

1. Baris 1-6: Inisialisasi variabel lokal untuk menampung total penalti *tardiness*, penalti tinta, dan *makespan*.
2. Baris 8-12: Iterasi dimulai untuk setiap mesin. Kondisi awal mesin (waktu dan tinta terakhir) diambil dari state sebelumnya atau diatur ke nol jika awal horizon.
3. Baris 14-21: *Loop* pemrosesan pekerjaan di dalam mesin. Menghitung waktu setup berdasarkan perubahan tinta, menentukan waktu mulai (mempertimbangkan waktu kedatangan), dan menghitung waktu selesai.
4. Baris 23-28: Akumulasi penalti. Jika pekerjaan selesai terlambat, penalti *tardiness* ditambahkan. Biaya pergantian tinta juga dijumlahkan ke total ink_v.
5. Baris 30-32: Pembaruan *makespan*. Jika waktu selesai mesin ini lebih lambat dari mesin lain, nilai *latest* diperbarui dan mesin ditandai sebagai *critical_machine*.
6. Baris 34-39: Menghitung dan mengembalikan nilai *fitness* total. Nilai ini merupakan hasil penjumlahan terbobot dari komponen *tardiness*, tinta, dan *makespan* sesuai konfigurasi.

5.2 Implementasi Algoritma SA

Bagian ini mengimplementasikan seluruh komponen algoritma SA yang telah dirancang pada Bab 4.3, mencakup pembuatan solusi awal, struktur tetangga, estimasi temperatur, hingga *loop* utama SA. Sesuai implementasi, mekanisme penerimaan solusi dan pendinginan temperatur berada dalam satu fungsi dan tidak dipisahkan.

5.2.1 Pembuatan Solusi Awal

Tabel 5.7 Kode Sumber Algoritma Buat Solusi Awal

Algoritma Create Initial Solution	
1	def create_initial_solution(

```
2       machine_data: dict[str, list[Job]], cfg: OptimizerConfig
3   ):
4       all_jobs = [
5           j for jobs in machine_data.values() for j in jobs
6       ]
7       all_jobs.sort(
8           key=lambda j: (
9               j.arrival_time,
10              -j.production_quantity,
11          )
12      )
13
14      init = cfg.initial_state or {}
15      schedule = {m: [] for m in cfg.machines}
16      machine_times = {
17          m: float(init.get(m, {}).get("time", 0.0))
18          for m in cfg.machines
19      }
20      machine_inks = {
21          m: list(init.get(m, {}).get("inks", []))
22          for m in cfg.machines
23      }
24
25      for job in all_jobs:
26          best_m, best_complete = None, float("inf")
27          for m in cfg.machines:
28              ink_chg = changeover(machine_inks[m], job.inks)
29              setup = (
30                  cfg.sequence_dependent_setup_time
31                  if ink_chg > 0
32                  else 0.0
33              )
34              start = max(machine_times[m], job.arrival_time)
35              complete = (
36                  start
37                  + setup
38                  + cfg.proc_time(m, job.production_quantity)
39              )
40
41              if complete < best_complete:
42                  best_m, best_complete = m, complete
43
44          schedule[best_m].append(job)
45          machine_times[best_m] = best_complete
46          machine_inks[best_m] = job.inks
47
48      sol = Solution(schedule)
49      sol.evaluate(cfg)
50      return sol
```

Fungsi ini mengimplementasikan *flowchart* pada Gambar 4.4. Solusi awal dibangun dengan pendekatan *greedy* berdasarkan *arrival time* dan estimasi waktu penyelesaian tercepat pada masing-masing mesin. Pendekatan ini memberikan titik awal yang layak sebelum proses eksplorasi SA dimulai.

1. Baris 1-3: Mendefinisikan fungsi *create_initial_solution*. Fungsi ini menerima data pekerjaan dan konfigurasi, lalu bertujuan menghasilkan jadwal awal yang valid sebagai titik berangkat optimasi.
2. Baris 4-12: Mengumpulkan seluruh pekerjaan dari berbagai mesin ke dalam satu daftar *all_jobs*. Daftar ini kemudian diurutkan secara *greedy* berdasarkan

waktu kedatangan dan jumlah produksi, untuk memprioritaskan pekerjaan yang datang lebih awal.

3. Baris 14-23: Menginisialisasi struktur jadwal kosong dan mengambil status awal mesin (waktu dan tinta) dari konfigurasi. Ini memastikan penjadwalan dimulai dengan memperhatikan kondisi aktual mesin saat ini.
4. Baris 25-42: Melakukan iterasi untuk setiap pekerjaan dalam daftar terurut. Algoritma mencari mesin terbaik dengan menghitung estimasi waktu selesai pada setiap mesin, memperhitungkan waktu *setup* jika terjadi pergantian tinta.
5. Baris 44-50: Menetapkan pekerjaan ke mesin yang memberikan waktu penyelesaian tercepat. Status mesin diperbarui, objek Solution dibuat, dievaluasi skor *fitness*-nya, dan dikembalikan sebagai hasil akhir.

5.2.2 Struktur *Neighborhood*

Tabel 5.8 Kode Sumber Algortima Buat Solusi Tetangga

Algortima Generate Neighbor

```
1  def generate_neighbor(
2      solution: Solution, cfg: OptimizerConfig
3  ):
4      neighbor = solution.copy()
5      schedule = neighbor.schedule
6      machines = list(schedule)
7      n_m = len(machines)
8
9      m_focus = (
10         solution.critical_machine
11         if random.random() < cfg.critical_machine_bias
12         else machines[random.randrange(n_m)]
13     )
14
15     other_machines = [m for m in machines if m != m_focus]
16     m2 = random.choice(other_machines)
17
18     def swap_within():
19         jobs = schedule[m_focus]
20         if len(jobs) >= 2:
21             schedule[m_focus] = jobs = list(jobs)
22             i, j = random.sample(range(len(jobs)), 2)
23             jobs[i], jobs[j] = jobs[j], jobs[i]
24
25     def swap_between():
26         j1, j2 = schedule[m_focus], schedule[m2]
27         if j1 and j2:
28             schedule[m_focus], schedule[m2] = list(
29                 j1
30             ), list(j2)
31             i, j = random.randrange(
32                 len(j1)
33             ), random.randrange(len(j2))
34             schedule[m_focus][i], schedule[m2][j] = (
35                 schedule[m2][j],
36                 schedule[m_focus][i],
37             )
38
39     def relocate():
40         j1, j2 = schedule[m_focus], schedule[m2]
```

```
41          if j1:
42              schedule[m_focus], schedule[m2] = list(
43                  j1
44              ), list(j2)
45              job = schedule[m_focus].pop(
46                  random.randrange(len(j1))
47              )
48              pos = random.randrange(len(j2) + 1)
49              schedule[m2].insert(pos, job)
50
51      def exchange():
52          j1, j2 = schedule[m_focus], schedule[m2]
53          if len(j1) >= 2 and len(j2) >= 2:
54              schedule[m_focus], schedule[m2] = list(
55                  j1
56              ), list(j2)
57              c1, c2 = random.randrange(
58                  len(j1)
59              ), random.randrange(len(j2))
60              seg1, seg2 = (
61                  schedule[m_focus][c1:],
62                  schedule[m2][c2:],
63              )
64              schedule[m_focus][c1:], schedule[m2][c2:] = (
66                  seg2,
67                  seg1,
68              )
69
70      def block_insert():
71          j1, j2 = schedule[m_focus], schedule[m2]
72          if j1:
73              schedule[m_focus], schedule[m2] = list(
74                  j1
75              ), list(j2)
76              i = random.randrange(len(j1))
77              j = random.randrange(i + 1, len(j1) + 1)
78              block = schedule[m_focus][i:j]
79              del schedule[m_focus][i:j]
80              pos = random.randrange(len(j2) + 1)
81              schedule[m2][pos:pos] = block
82
83      ops = [
84          swap_within,
85          swap_between,
86          relocate,
87          exchange,
88          block_insert,
89      ]
90      random.choice(ops)()
91
92      return neighbor
```

Fungsi ini mengimplementasikan seluruh operator tetangga yang dijelaskan pada Bab 4.3.4 dan Gambar 4.5-Gambar 4.10, yaitu: *swap within, swap between, relocate, exchange, block insert.* Fungsi juga menerapkan bias terhadap mesin kritis untuk mempercepat penurunan *makespan*.

1. Baris 1-7: Fungsi *generate_neighbor* dimulai dengan menyalin solusi saat ini. Sistem kemudian memilih satu mesin utama (*m_focus*) untuk dimodifikasi, dengan bias probabilitas memilih mesin kritis guna memperbaiki *makespan*.

2. Baris 9-16: Memilih mesin kedua (m2) secara acak dari daftar mesin yang tersisa. Mesin kedua ini diperlukan untuk operasi pemindahan pekerjaan antar-mesin.
3. Baris 18-23: Mendefinisikan operator *swap_within*. Operator ini menukar posisi dua pekerjaan yang berada di dalam satu mesin yang sama (*m_focus*) untuk mengubah urutan produksi lokal.
4. Baris 25-37: Mendefinisikan operator *swap_between*. Operator ini menukar satu pekerjaan dari mesin fokus dengan satu pekerjaan dari mesin kedua, memungkinkan pertukaran beban kerja antar mesin.
5. Baris 39-49: Mendefinisikan operator *relocate*. Operator ini memindahkan satu pekerjaan dari mesin fokus ke posisi acak di mesin kedua, efektif untuk menyeimbangkan beban jika satu mesin terlalu padat.
6. Baris 51-69: Mendefinisikan operator *exchange*. Operator ini memotong segmen pekerjaan (dari titik potong tertentu hingga akhir) pada kedua mesin dan menukarnya, mirip dengan operasi *crossover* pada algoritma genetika.
7. Baris 70-81: Mendefinisikan operator *block_insert*. Operator ini mengambil sekumpulan pekerjaan berurutan (blok) dari mesin fokus dan menyisipkannya ke mesin kedua, memindahkan beban kerja dalam jumlah besar sekaligus.
8. Baris 83-92: Mengumpulkan seluruh operator ke dalam daftar *ops*, memilih salah satu secara acak, mengeksekusinya, dan mengembalikan solusi tetangga yang baru terbentuk.

5.2.3 Estimasi Suhu Awal

Tabel 5.9 Kode Sumber Algoritma Estimasi Temperatur Awal

Algoritma Compute Initial Temperature

```
1  def compute_initial_temperature(
2      solution, cfg: OptimizerConfig, samples: int = 512
3  ) -> tuple[float, float]:
4      base = solution.copy()
5      base.evaluate(cfg)
6
7      deltas = []
8      for _ in range(samples):
9          nei = generate_neighbor(base, cfg)
10         nei.evaluate(cfg)
11         d = nei.fitness - base.fitness
12         if d > 0:
13             deltas.append(d)
14
15     if deltas:
16         dbar = float(sum(deltas) / len(deltas))
17     else:
18         dbar = max(1.0, 0.01 * max(base.fitness, 1.0))
19
20     p0 = min(max(cfg.p0, 1e-6), 0.999999)
21     T0 = -dbar / math.log(p0)
22     T_min = T0 * cfg.floor_ratio
23     return T0, T_min
```

Fungsi ini mengimplementasikan Gambar 4.13. Dengan mengambil beberapa sampel perubahan solusi, fungsi ini menghitung Δ rata-rata yang digunakan untuk menentukan temperatur awal dan temperatur minimum. Perhitungan ini mengikuti formula Kirkpatrick, Gelatt Jr & Vecchi (1983).

1. Baris 1-5: Mendefinisikan fungsi *compute_initial_temperature*. Fungsi menyalin solusi basis dan memastikan nilai *fitness*-nya sudah terhitung sebagai referensi dasar.
2. Baris 7-18: Melakukan *sampling* dengan membangkitkan sejumlah tetangga acak. *Loop* ini menghitung selisih *fitness* (*delta*) yang memburuk (positif) dan menghitung rata-ratanya (*dbar*) untuk mengestimasi seberapa besar kenaikan energi rata-rata yang terjadi.
3. Baris 20-23: Menghitung temperatur awal (T0) menggunakan rumus invers dari probabilitas penerimaan awal (p0). Fungsi juga menetapkan batas suhu minimum (*T_min*) berdasarkan rasio tertentu dan mengembalikan kedua nilai tersebut.

5.2.4 Algoritma *Simulated Annealing*

Tabel 5.10 Kode Sumber Algoritma *Simulated Annealing*

Algortima Simulated Annealing

```
1  def simulated_annealing(
2      machine_data: dict[str, list[Job]], cfg: OptimizerConfig
3  ):
4      cfg.seed_rng()
5      current_solution = create_initial_solution(
6          machine_data, cfg
7      )
8      best_solution = current_solution.copy()
9
10     initial_temp, min_temp = compute_initial_temperature(
11         current_solution, cfg
12     )
13     temp = initial_temp
14     convergence_log = []
15
16     for iteration in range(cfg.max_iter):
17         if iteration % 100 == 0:
18             convergence_log.append(
19                 {
20                     "iteration": iteration + 1,
21                     "best_fitness": best_solution.fitness,
22                     "current_fitness": current_solution.fitness,
23                 }
24             )
25
26         neighbor = generate_neighbor(current_solution, cfg)
27         neighbor.evaluate(cfg)
28         delta = neighbor.fitness - current_solution.fitness
29
30         if delta <= 0 or random.random() < math.exp(
31             -delta / max(temp, 1e-12)
32         ):
33             current_solution = neighbor
34
35         if current_solution.fitness < best_solution.fitness:
36             best_solution = current_solution.copy()
```

```
37
38          temp = max(min_temp, temp * cfg.alpha)
39
40      return best_solution, convergence_log
```

Fungsi ini mengimplementasikan keseluruhan algoritma SA pada Gambar 4.1, termasuk: penggunaan solusi awal, perhitungan temperatur awal, perulangan iterasi SA sampai batas maksimum, pembangkitan *neighbor*, evaluasi delta *fitness*, penerimaan Metropolis (bagian dari *loop*), pendinginan geometrik (juga bagian dari *loop*), pembaruan solusi terbaik, pencatatan log konvergensi. Mekanisme penerimaan dan pendinginan tidak dipisahkan menjadi fungsi terpisah karena algoritma cukup sederhana untuk diimplementasikan.

1. Baris 1-8: Fungsi utama *simulated_annealing* dimulai dengan inisialisasi *seed* acak dan pembuatan solusi awal. Solusi awal ini langsung disimpan sebagai solusi terbaik sementara (*best_solution*).
2. Baris 10-13: Memanggil fungsi estimasi suhu untuk mendapatkan *initial_temp* dan *min_temp*. Variabel temperatur saat ini (*temp*) diatur ke nilai awal tersebut.
3. Baris 16-24: Memulai *loop* utama optimasi yang berjalan hingga *max_iter*. Bagian ini juga menangani pencatatan log konvergensi setiap 100 iterasi untuk memantau performa algoritma.
4. Baris 25-28: Membangkitkan solusi tetangga baru (*neighbor*) dan menghitung selisih *fitness* (delta) dibandingkan solusi saat ini.
5. Baris 30-33: Menerapkan Kriteria Metropolis. Solusi baru diterima jika lebih baik (*delta* <= 0) atau berdasarkan probabilitas eksponensial jika lebih buruk. Jika diterima, *current_solution* diperbarui.
6. Baris 35-36: Memeriksa apakah solusi yang baru diterima lebih baik daripada rekor terbaik sejauh ini. Jika ya, *best_solution* diperbarui.
7. Baris 38-40: Menurunkan temperatur menggunakan skema pendinginan geometris (*temp* × *alpha*) dan mengembalikan solusi terbaik beserta log riwayat konvergensi setelah iterasi selesai.

5.3 Implementasi *Rolling Horizon Scheduler*

Bagian ini mengimplementasikan seluruh konsep pada Bab 4.4, mencakup state mesin, mekanisme *lock-unlock* pada setiap *horizon*, pembangunan ulang data mesin, penetapan *due date* berbasis TWK, dan pemanggilan SA secara berulang di setiap batch kedatangan pekerjaan.

5.3.1 State Mesin dan Mekanisme *Lock-Unlock*

Tabel 5.11 Kode Sumber Struktur Data *State* Mesin

Struktur Data Machine State	
1	`@dataclass(frozen=True)`
2	`class MachineState:`
3	`    name: str`

```
4       current_time: float = 0.0
5       current_inks: list = field(default_factory=list)
6       locked_jobs: list[Job] = field(default_factory=list)
7       unlocked_jobs: list[Job] = field(default_factory=list)
8
9       def add_locked_job(self, job: Job):
10          self.locked_jobs.append(job)
11          object.__setattr__(
12              self, "current_time", job.completion_time
13          )
14          object.__setattr__(self, "current_inks", job.inks)
```

State mesin mengimplementasikan Gambar 4.15.

1. Baris 1-8: Mendefinisikan kelas *MachineState* sebagai *dataclass* yang bersifat *frozen*. Kelas ini menyimpan status terkini mesin, meliputi nama mesin, waktu saat ini (*current_time*), daftar tinta yang sedang digunakan, serta pemisahan daftar pekerjaan menjadi *locked_jobs* dan *unlocked_jobs*.
2. Baris 9-14: Metode *add_locked_job* digunakan untuk memasukkan pekerjaan ke dalam daftar terkunci (*locked*). Metode ini secara otomatis memperbarui waktu mesin dan komposisi tinta terakhir berdasarkan pekerjaan yang baru saja dikunci tersebut.

Tabel 5.12 Kode Sumber Algoritma Buat *State* Mesin

```
Algortima Create Machine State
1   def create_machine_states(
2       now: float, cfg: OptimizerConfig, schedule=None
3   ):
4       states = {m: MachineState(m) for m in cfg.machines}
5       if not schedule:
6           return states
7
8       for m, jobs in schedule.items():
9           clock, inks = 0.0, []
10          state = states[m]
11          for job in jobs:
12              proc = cfg.proc_time(m, job.production_quantity)
13              if changeover(inks, job.inks) > 0:
14                  setup = cfg.sequence_dependent_setup_time
15              else:
16                  setup = 0.0
17
18              setup_start = max(clock, job.arrival_time)
19              start = setup_start + setup
20              done = start + proc
21              job_with_times = Job(
22                  order_code=job.order_code,
23                  inks=job.inks,
24                  production_quantity=job.production_quantity,
25                  arrival_time=job.arrival_time,
26                  due_date=job.due_date,
27                  start_time=start,
28                  completion_time=done,
29              )
30              if start <= now:
31                  state.add_locked_job(job_with_times)
32              else:
33                  state.unlocked_jobs.append(job_with_times)
34
35              clock, inks = done, job.inks
```

```
36      return states
```

Fungsi *create_machine_states()* mengatur pekerjaan menjadi dua kategori: *locked jobs*, yaitu pekerjaan yang telah mulai diproses sebelum waktu *horizon*, *unlocked jobs*, yaitu pekerjaan yang belum dimulai dan dapat dijadwalkan ulang. Mekanisme ini memungkinkan proses *rolling horizon* berjalan secara inkremental.

1. Baris 1-5: Fungsi *create_machine_states* diinisialisasi untuk membangun *snapshot* kondisi pabrik pada waktu *now*. Jika belum ada jadwal sebelumnya, fungsi mengembalikan state kosong untuk setiap mesin.
2. Baris 6-16: Melakukan iterasi pada jadwal yang ada. Untuk setiap pekerjaan, kode menghitung durasi proses dan durasi *setup* (pergantian tinta) secara simulasi dari awal.
3. Baris 29-35: Menghitung waktu mulai (start) dan waktu selesai (*done*) yang presisi. Objek pekerjaan baru (*job_with_times*) dibuat dengan atribut waktu yang sudah terisi lengkap.
4. Baris 36-39: Logika pemisahan *Lock/Unlock*. Jika pekerjaan dimulai sebelum atau tepat pada waktu *now*, pekerjaan tersebut dianggap *locked* (sudah terjadi). Sebaliknya, jika dimulai setelah *now*, pekerjaan masuk ke daftar *unlocked* untuk dioptimasi ulang.

5.3.2 Pembangunan Data Mesin untuk Optimasi Ulang

Tabel 5.13 Kode Sumber Algoritma Buat Data Mesin

```
Algortima Build Machine Data
1   def build_machine_data(states, visible_jobs, cfg):
2       machine_names = list(cfg.machines)
3       already_scheduled = {
4           job.order_code
5           for s in states.values()
6           for job in s.locked_jobs + s.unlocked_jobs
7       }
8       machine_data = {
9           m: s.unlocked_jobs[:] for m, s in states.items()
10      }
11
12      new_jobs = [
13          j
14          for j in visible_jobs.values()
15          if j.order_code not in already_scheduled
16      ]
17      for i, job in enumerate(new_jobs):
18          machine = machine_names[i % len(machine_names)]
19          machine_data[machine].append(job)
20      return machine_data
```

Sesuai Gambar 4.16, fungsi ini menggabungkan pekerjaan *unlocked* dengan pekerjaan baru yang sudah *released*. Distribusi pekerjaan ke mesin dilakukan secara *round-robin* agar setiap mesin menerima pekerjaan untuk *horizon* tersebut.

1. Baris 1-7: Fungsi *build_machine_data* dimulai dengan mendata pekerjaan yang sudah dijadwalkan untuk menghindari duplikasi saat penggabungan data.

2. Baris 8-16: Mengumpulkan pekerjaan yang berstatus *unlocked* dari seluruh mesin. Pekerjaan ini ditarik kembali dari jadwal lama untuk dimasukkan ke dalam *pool* optimasi yang baru.
3. Baris 17-20: Mendistribusikan pekerjaan baru (*visible_jobs*) yang belum pernah dijadwalkan. Distribusi awal dilakukan dengan mekanisme *Round-Robin* (bergiliran) ke setiap mesin untuk memastikan keseimbangan beban awal sebelum optimasi dimulai.

5.3.3 Perhitungan Beban Kerja dan Penetapan Due Date

Dua komponen utama *TWK-based due date assignment* diimplementasikan: Perhitungan *Remaining Workload* (Gambar 4.18), Penetapan Due Date menggunakan *baseline* dan koreksi berdasarkan *arrival time* serta *processing* time terpendek (Gambar 4.17). Fungsi ini memastikan setiap pekerjaan baru pada tiap *horizon* memiliki *due date* yang proporsional terhadap beban sistem.

Tabel 5.14 Kode Sumber Hitung *Workload*

Algoritma Compute Remaining Workload

```
1  def compute_remaining_workload(
2      now: float, states, visible_jobs, cfg
3  ):
4      locked_remaining = sum(
5          max(0, j.completion_time - now)
6          for s in states.values()
7          for j in s.locked_jobs
8      )
9      locked_ids = {
10         j.order_code
11         for s in states.values()
12         for j in s.locked_jobs
13     }
14     unstarted = [
15         j
16         for j in visible_jobs.values()
17         if j.order_code not in locked_ids
18     ]
19
20     if not unstarted:
21         return locked_remaining
22
23     avg_inv_rate = sum(
24         1 / r for r in cfg.machine_production_rates.values()
25     ) / len(cfg.machines)
26     work = (avg_inv_rate / 60.0) * sum(
27         j.production_quantity for j in unstarted
28     )
29     setup_buf = (
30         0.5
31         * cfg.sequence_dependent_setup_time
32         * len(unstarted)
33     )
34     return locked_remaining + work + setup_buf
```

1. Baris 1-8: Fungsi *compute_remaining_workload* menghitung sisa beban kerja dari pekerjaan yang sedang berjalan (*locked* tetapi belum selesai). Beban dihitung dari selisih waktu selesai dengan waktu sekarang (*now*).

2. Baris 9-21: Mengidentifikasi pekerjaan yang belum dimulai sama sekali (*unstarted*). Kode ini menyaring pekerjaan yang ada di *visible_jobs* tetapi tidak termasuk dalam daftar pekerjaan yang dikunci.

3. Baris 23-34: Mengestimasi total durasi yang dibutuhkan untuk menyelesaikan pekerjaan *unstarted*. Perhitungan menggunakan rata-rata kecepatan produksi mesin ditambah *buffer* waktu untuk estimasi *setup* (pergantian tinta), lalu dijumlahkan dengan beban sisa pekerjaan *locked*.

Tabel 5.15 Kode Sumber Algoritma Penetapan Due Date

Algoritma Assign Due Date

```
1  def assign_due_dates(
2      now: float,
3      states: dict,
4      visible_jobs: dict,
5      machine_data: dict,
6      cfg: OptimizerConfig,
7  ):
8      workload = compute_remaining_workload(
9          now, states, visible_jobs, cfg
10     )
11
12     n_jobs = len(visible_jobs)
13     denom = max(1, min(len(cfg.machines), n_jobs))
14     baseline_due = now + cfg.due_date_multiplier * (
15         workload / denom
16     )
17
18     for m, jobs in machine_data.items():
19         for job in jobs:
20             if job.due_date <= 0:
21                 pmin = min(
22                     cfg.proc_time(
23                         m2, job.production_quantity
24                     )
25                     for m2 in cfg.machines
26                 )
27                 job.due_date = max(
28                     job.arrival_time + pmin, baseline_due
29                 )
```

1. Baris 1-10: Fungsi *assign_due_dates* memanggil fungsi perhitungan *workload* untuk mendapatkan gambaran total beban sistem saat ini.

2. Baris 12-16: Menghitung *baseline_due* (tenggat waktu dasar) menggunakan prinsip *Total Work Content* (TWK). Nilai ini didapat dari waktu sekarang ditambah beban kerja yang dinormalisasi dengan jumlah mesin dan faktor pengali (*due_date_multiplier*).

3. Baris 18-29: Menetapkan *due date* untuk setiap pekerjaan yang belum memilikinya. Tenggat waktu diambil dari nilai maksimum antara waktu kedatangan ditambah durasi proses minimal, atau nilai *baseline_due*. Ini menjamin tenggat waktu yang realistis dan proporsional terhadap kepadatan jadwal.

5.3.4 Algoritma *Rolling Horizon* Utama

Tabel 5.16 Kode Sumber Algoritma *Rolling Horizon*

Algortima Rolling Horizon

```
def online_rolling_horizon_scheduler(
    job_batches, orders_data, cfg: OptimizerConfig
):
    cfg.seed_rng()
    order_lookup = {o["id"]: o for o in orders_data}
    all_jobs = {
        oid: Job(
            order_code=oid,
            inks=order_lookup[oid]["inks"],
            production_quantity=order_lookup[oid][
                "quantity"
            ],
            arrival_time=batch["arrival_time"],
        )
        for batch in job_batches
        for oid in batch["orders"]
    }

    current_schedule = None
    all_convergence_logs = []

    for idx, batch in enumerate(
        sorted(
            job_batches, key=lambda b: b["arrival_time"]
        ),
        start=1,
    ):
        now = batch["arrival_time"]
        visible_jobs = {
            oid: j
            for oid, j in all_jobs.items()
            if j.arrival_time <= now
        }

        states = create_machine_states(
            now, cfg, current_schedule
        )
        machine_data = build_machine_data(
            states, visible_jobs, cfg
        )

        assign_due_dates(
            now, states, visible_jobs, machine_data, cfg
        )

        unlocked = sum(
            len(v) for v in machine_data.values()
        )

        if not unlocked:
            continue

        cfg_for_batch = replace(
            cfg,
            initial_state={
                m: {
                    "time": s.current_time,
                    "inks": s.current_inks,
                }
                for m, s in states.items()
```

```
61              },
62          )
63
64          start = time.time()
66          batch_solution, convergence_log = (
67              simulated_annealing(machine_data, cfg_for_batch)
68          )
69          runtime = time.time() - start
70
71          current_schedule = {
72              m: s.locked_jobs
73              + batch_solution.schedule.get(m, [])
74              for m, s in states.items()
75          }
76
77          for entry in convergence_log:
78              if (
79                  "iteration" in entry
80                  and "best_fitness" in entry
81              ):
82                  entry["batch"] = idx
83
84          convergence_log.append(
85              {"runtime_s": runtime, "batch": idx}
86          )
87          all_convergence_logs.extend(convergence_log)
88
89      final_solution = Solution(current_schedule)
90      final_solution.evaluate(cfg)
91
92      return final_solution, all_convergence_logs
```

Fungsi ini mengimplementasikan seluruh alur pada Gambar 4.14, dimulai dari membaca *batch* pekerjaan, membangun state mesin, menghitung *due date*, mengoptimalkan pekerjaan *unlocked* menggunakan SA, hingga menggabungkan hasilnya dengan pekerjaan *locked*. Fungsi ini berjalan untuk setiap *batch* secara berurutan sehingga membentuk jadwal dinamis berdasarkan *arrival time*.

1. Baris 1-5: Fungsi utama *online_rolling_horizon_scheduler* dimulai. Fungsi ini menginisialisasi *Random Number Generator* dan membuat kamus pencarian (*lookup*) untuk data pesanan.
2. Baris 6-17: Mengonversi data pesanan mentah menjadi objek *Job*. Setiap objek diisi dengan detail tinta, kuantitas, dan waktu kedatangan sesuai dengan *batch* kedatangannya.
3. Baris 22-27: Mengurutkan *batch* pekerjaan berdasarkan waktu kedatangan dan memulai *loop* utama simulasi *rolling horizon*.
4. Baris 28-33: Memfilter pekerjaan yang terlihat (*visible_jobs*), yaitu pekerjaan yang waktu kedatangannya kurang dari atau sama dengan waktu simulasi saat ini (*now*).
5. Baris 35-40: Membangun ulang status mesin dan data pekerjaan. Fungsi *create_machine_states* dipanggil untuk memisahkan pekerjaan lama, diikuti *build_machine_data* untuk menggabungkan pekerjaan baru.

6. Baris 42-44: Memanggil *assign_due_dates* untuk memberikan tenggat waktu dinamis pada pekerjaan-pekerjaan baru berdasarkan beban kerja aktual sistem.
7. Baris 53-62: Memperbarui konfigurasi optimasi (*cfg*). Bagian ini sangat krusial karena menetapkan *initial_state* (waktu dan tinta awal) bagi algoritma SA agar sesuai dengan kondisi akhir dari pekerjaan yang terkunci (*locked jobs*).
8. Baris 64-69: Menjalankan algoritma *simulated_annealing* pada data mesin yang telah disiapkan untuk mendapatkan jadwal optimal bagi *batch* ini.
9. Baris 71-75: Memperbarui *current_schedule*. Jadwal akhir dibentuk dengan menggabungkan pekerjaan yang terkunci (sebagai riwayat tetap) dengan hasil optimasi SA (sebagai rencana masa depan).
10. Baris 77-87: Melakukan *logging* konvergensi setiap *batch*.
11. Baris 89-92: Membungkus hasil akhir ke dalam objek Solution, melakukan evaluasi performa terakhir, dan mengembalikan solusi lengkap beserta log konvergensi.

BAB 6 HASIL DAN PEMBAHASAN

6.1 Pengujian Parameter SA

Pengujian parameter pada algoritma SA dilakukan untuk memperoleh konfigurasi yang memberikan performa terbaik dan cukup *robust* sebelum digunakan pada seluruh skenario pengujian utama. Seluruh proses *tuning* dilakukan pada Skenario 3, karena skenario ini memuat semua pekerjaan dalam satu *horizon* perencanaan statis. Kondisi tersebut memberikan lingkungan yang stabil untuk mengevaluasi perilaku konvergensi algoritma tanpa dipengaruhi dinamika kedatangan pekerjaan sebagaimana pada skenario online.

Metode pengujian yang digunakan adalah *One-Factor-At-a-Time* (OFAT), yaitu setiap parameter diubah dan diuji secara terpisah sementara parameter lainnya dipertahankan pada nilai terbaik sementara. Pendekatan ini dipilih untuk mengisolasi pengaruh marginal masing-masing parameter terhadap kinerja SA.

Penentuan nilai terbaik mengikuti hierarki kriteria berikut:

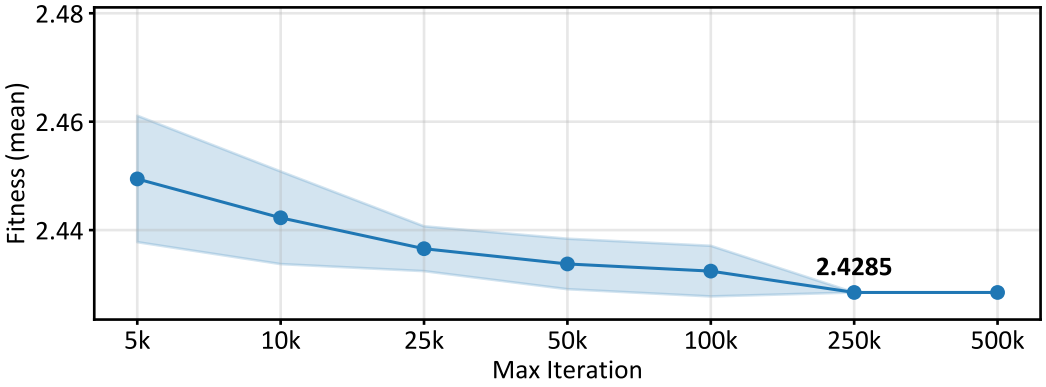
1. Rata-rata *fitness* (berkaitan langsung dengan kombinasi *makespan* dan penalti tinta) sebagai kriteria utama; semakin kecil nilai *fitness*, semakin baik kualitas jadwal.
2. Stabilitas (standar deviasi *fitness*) sebagai indikator kekokohan algoritma terhadap variasi *random seed*.
3. Rata-rata *runtime* sebagai kriteria sekunder untuk membedakan konfigurasi dengan kualitas solusi yang setara.

Berikut disajikan pengaruh masing-masing parameter terhadap performa SA dalam menyelesaikan masalah UPMSP-SDST.

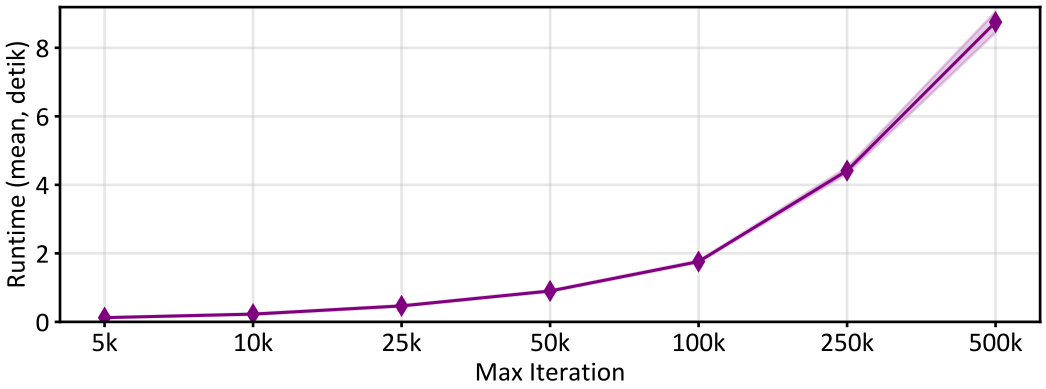
6.1.1 Pengujian Parameter *Max Iteration*

Tabel 6.1 Statistik Hasil Pengujian Parameter *Max Iteration*

Nilai Parameter	Rata-rata Fitness	Standar Deviasi Fitness	Rata-rata Makespan	Rata-rata Ink Changeover	Rata-rata Runtime (detik)
5000	2.4494	0.0117	6.4944	2.00	0.12
10000	2.4423	0.0085	6.4227	2.00	0.23
25000	2.4366	0.0041	6.3657	2.00	0.47
50000	2.4338	0.0047	6.3375	2.00	0.90
100000	2.4324	0.0047	6.3243	2.00	1.76
250000	2.4285	0.0000	6.2850	2.00	4.41
500000	2.4285	0.0000	6.2850	2.00	8.75



Gambar 6.1 Pengaruh *Max Iteration* Terhadap Rata-rata *Fitness*



Gambar 6.2 Pengaruh *Max Iteration* Terhadap Rata-rata *Runtime*

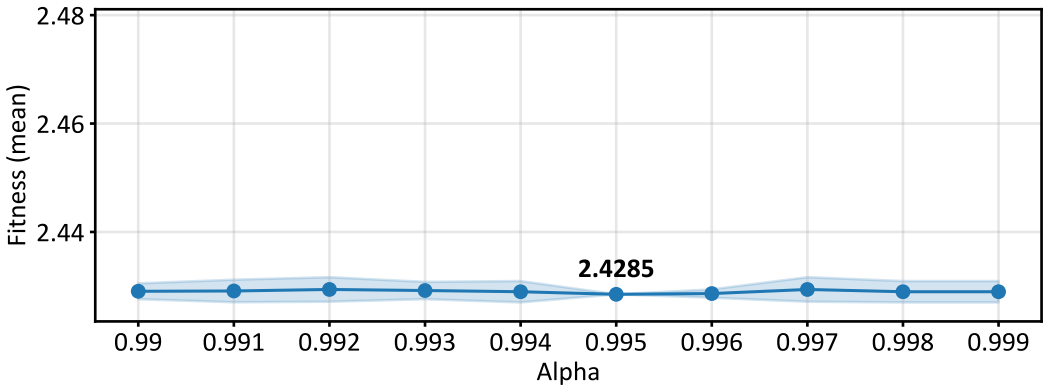
Tabel 6.1 serta Gambar 6.1 dan Gambar 6.2 menunjukkan pengaruh nilai *max iteration* terhadap rata-rata *fitness* dan *runtime*. Peningkatan jumlah iterasi menghasilkan penurunan *fitness* yang konsisten, dari 2,4494 pada 5.000 iterasi menjadi 2,4285 pada 250.000 dan 500.000 iterasi. Mulai 250.000 iterasi, nilai *fitness* mencapai titik terendah dengan standar deviasi 0, yang berarti solusi yang diperoleh konsisten di seluruh *seed*.

Di sisi lain, *runtime* meningkat hampir linear terhadap jumlah iterasi, dari 0,12 detik (5.000 iterasi) hingga 8,75 detik (500.000 iterasi). Karena tidak terdapat perbaikan *fitness* yang berarti setelah 250.000 iterasi sementara biaya komputasi meningkat cukup besar, maka *max iteration* = 250.000 dipilih sebagai nilai terbaik. Pertimbangan ini sejalan dengan Talbi (2009) yang menegaskan bahwa *stopping criteria* merupakan elemen penting dalam desain metaheuristik dan umumnya ditetapkan berdasarkan *trade-off* antara kualitas solusi dan biaya komputasi, termasuk penghentian ketika perbaikan tambahan menjadi tidak signifikan.

6.1.2 Pengujian Parameter *Alpha*

Tabel 6.2 Statistik Hasil Pengujian Parameter *Alpha*

Nilai Parameter	Rata-rata Fitness	Standar Deviasi Fitness	Rata-rata Makespan	Rata-rata Ink Changeover	Rata-rata Runtime (detik)
0.99	2.4291	0.0015	6.2905	2.00	4.37
0.991	2.4291	0.0021	6.2911	2.00	4.34
0.992	2.4294	0.0023	6.2939	2.00	4.38
0.993	2.4292	0.0016	6.2919	2.00	4.39
0.994	2.4290	0.0020	6.2897	2.00	4.33
0.995	2.4285	0.0000	6.2850	2.00	4.35
0.996	2.4286	0.0008	6.2864	2.00	4.40
0.997	2.4294	0.0023	6.2939	2.00	4.33
0.998	2.4290	0.0020	6.2897	2.00	4.35
0.999	2.4290	0.0020	6.2897	2.00	4.37



Gambar 6.3 Pengaruh *Alpha* Terhadap Rata-rata *Fitness*

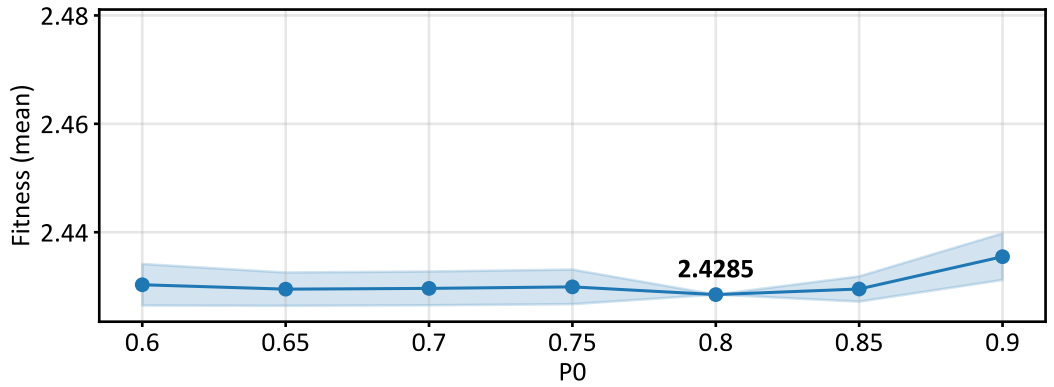
Parameter *alpha* mengatur laju pendinginan suhu pada SA. Nilai yang terlalu kecil membuat suhu turun terlalu cepat sehingga eksplorasi ruang solusi menjadi terbatas, sedangkan nilai yang terlalu besar menyebabkan proses pendinginan terlalu lambat dan berpotensi menimbulkan *stalling* tanpa peningkatan kualitas solusi.

Meskipun literatur seperti Delahaye, Chaimatanan & Mongeau (2019) merekomendasikan *alpha* pada kisaran 0,8-0,99 untuk kasus berskala kecil, masalah penjadwalan industri dengan iterasi besar memerlukan pendinginan yang lebih lambat sehingga nilai α yang lebih tinggi lazim digunakan. Hasil menunjukkan bahwa *alpha* = 0,995 memberikan *fitness* terbaik dengan stabilitas tinggi, dan nilai ini konsisten dengan penerapan SA pada konteks manufaktur seperti pada Shirazi et al. (2014) yang juga menggunakan *alpha* = 0,995. Oleh karena itu, *alpha* = 0,995 ditetapkan sebagai nilai optimal dalam penelitian ini.

6.1.3 Pengujian Parameter P0

Tabel 6.3 Statistik Hasil Pengujian Parameter P0

Nilai Parameter	Rata-rata Fitness	Standar Deviasi Fitness	Rata-rata Makespan	Rata-rata Ink Changeover	Rata-rata Runtime (detik)
0.6	2.4303	0.0038	6.3031	2.00	4.33
0.65	2.4295	0.0031	6.2950	2.00	4.33
0.7	2.4296	0.0031	6.2964	2.00	4.39
0.75	2.4299	0.0032	6.2992	2.00	4.39
0.8	2.4285	0.0000	6.2850	2.00	4.42
0.85	2.4295	0.0023	6.2953	2.00	4.58
0.9	2.4355	0.0043	6.3549	2.00	4.45



Gambar 6.4 Pengaruh P0 Terhadap Rata-rata *Fitness*

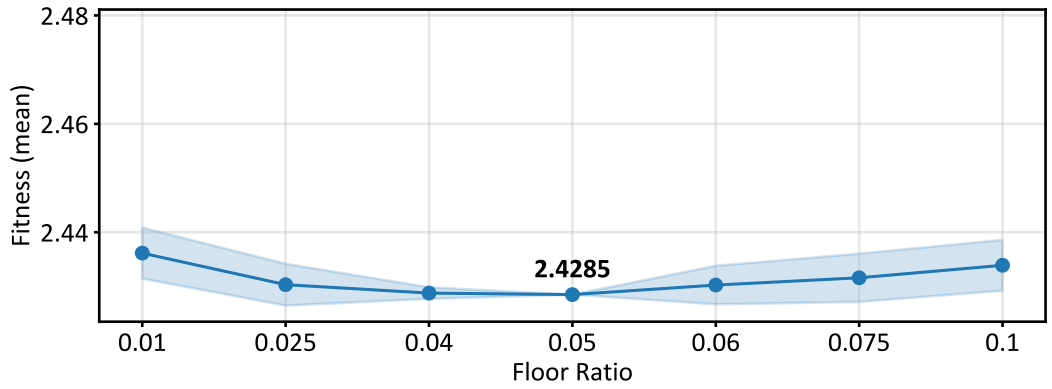
Parameter p_0 menyatakan peluang awal untuk menerima solusi yang lebih buruk pada awal proses, sehingga mengontrol kemampuan SA untuk melakukan eksplorasi agresif sebelum suhu menurun.

Hasil menunjukkan bahwa $p_0 = 0{,}8$ menghasilkan rata-rata *fitness* terbaik (2,4285) dengan standar deviasi nol. Nilai p_0 yang lebih kecil (0,6-0,7) cenderung membatasi eksplorasi pada tahap awal sehingga kualitas solusi sedikit menurun, sedangkan nilai yang terlalu besar meningkatkan variasi hasil tanpa memberikan perbaikan rata-rata yang konsisten. Pemilihan probabilitas penerimaan awal sendiri merupakan praktik yang lazim dalam SA dan umumnya berada pada rentang moderat, dengan literatur melaporkan kisaran probabilitas penerimaan awal sekitar 0,50 hingga 0,95 untuk mengatur keseimbangan eksplorasi dan stabilitas (Suman & Kumar, 2006). Oleh karena itu, berdasarkan hasil pengujian empiris pada data dan konfigurasi penelitian ini, $p_0 = 0{,}8$ ditetapkan sebagai nilai terbaik untuk eksperimen selanjutnya.

6.1.4 Pengujian Parameter *Floor Ratio*

Tabel 6.4 Statistik Hasil Pengujian Parameter *Floor Ratio*

Nilai Parameter	Rata-rata Fitness	Standar Deviasi Fitness	Rata-rata Makespan	Rata-rata Ink Changeover	Rata-rata Runtime (detik)
0.01	2.4361	0.0047	6.3615	2.00	4.34
0.025	2.4303	0.0039	6.3032	2.00	4.50
0.04	2.4288	0.0011	6.2878	2.00	4.37
0.05	2.4285	0.0000	6.2850	2.00	4.43
0.06	2.4303	0.0035	6.3026	2.00	4.42
0.075	2.4316	0.0044	6.3160	2.00	4.64
0.1	2.4339	0.0047	6.3390	2.00	4.33



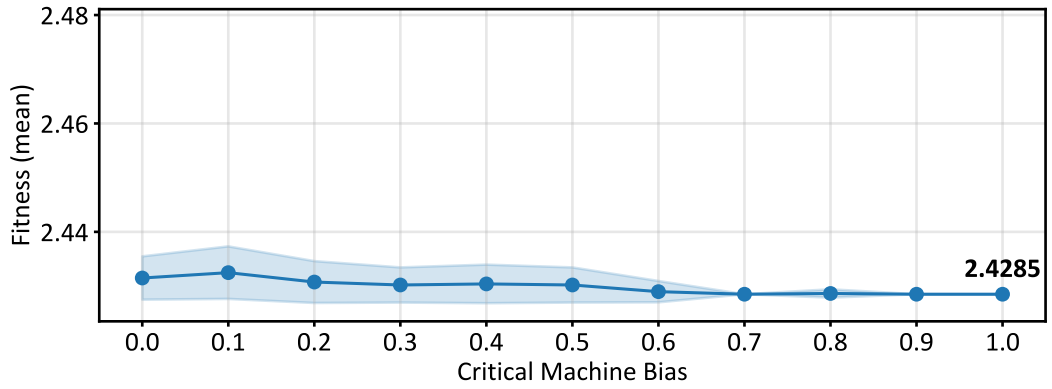
Gambar 6.5 Pengaruh *Floor Ratio* Terhadap Rata-rata *Fitness*

Floor ratio mengatur suhu minimum sebagai rasio dari suhu awal. Berbeda dari formulasi SA klasik yang umumnya menghentikan pencarian ketika suhu mencapai *final temperature* yang sangat rendah karena peluang penerimaan solusi memburuk dianggap sudah dapat diabaikan, penelitian ini tidak menggunakan *Tmin* sebagai *stopping criterion*. Setelah suhu mencapai Tmin, pencarian tetap dilanjutkan hingga *max iter* dengan suhu konstan, sehingga fase *exploitation* pada suhu rendah dapat dimanfaatkan secara lebih intensif. Pendekatan ini berbeda dari praktik umum yang dijelaskan oleh Talbi (2009), namun hasil pada Tabel 6.4 dan Gambar 6.5 menunjukkan adanya titik keseimbangan yang jelas. Nilai *floor ratio* 0.05 menghasilkan rata-rata *fitness* terendah sebesar 2.4285 dengan standar deviasi nol, sedangkan nilai yang lebih kecil cenderung menghasilkan kualitas solusi yang lebih buruk akibat eksplorasi berlebihan, dan nilai yang lebih besar menyebabkan penurunan kualitas karena pendinginan terlalu cepat. Oleh karena itu, *floor ratio* 0.05 dipilih sebagai konfigurasi terbaik karena memberikan keseimbangan antara intensitas eksploitasi pada suhu rendah dan kualitas solusi yang stabil.

6.1.5 Pengujian Parameter *Critical Machine Bias*

Tabel 6.5 Statistik Hasil Pengujian Parameter *Critical Machine Bias*

Nilai Parameter	Rata-rata Fitness	Standar Deviasi Fitness	Rata-rata Makespan	Rata-rata Ink Changeover	Rata-rata Runtime (detik)
0	2.4315	0.0040	6.3149	2.00	4.40
0.1	2.4325	0.0048	6.3248	2.00	4.36
0.2	2.4307	0.0039	6.3074	2.00	4.37
0.3	2.4302	0.0032	6.3020	2.00	4.43
0.4	2.4304	0.0036	6.3040	2.00	4.35
0.5	2.4302	0.0032	6.3020	2.00	4.36
0.6	2.4290	0.0020	6.2897	2.00	4.40
0.7	2.4285	0.0000	6.2850	2.00	4.34
0.8	2.4286	0.0008	6.2864	2.00	4.34
0.9	2.4285	0.0000	6.2850	2.00	4.37
1	2.4285	0.0000	6.2850	2.00	4.28



Gambar 6.6 Pengaruh *Critical Machine Bias* Terhadap Rata-rata *Fitness*

Parameter *critical machine* bias mengatur kecenderungan SA untuk memfokuskan pembangkitan *neighborhood move* pada mesin kritis dengan makespan terbesar. Hasil pada Tabel 6.5 dan Gambar 6.6 menunjukkan bahwa peningkatan nilai bias secara konsisten menurunkan rata-rata *fitness* dan *makespan*, dengan nilai 0,7 hingga 1,0 menghasilkan *fitness* terendah sebesar 2,4285 dan standar deviasi nol, yang menandakan konvergensi solusi yang stabil pada seluruh seed. Fokus eksplorasi pada mesin *bottleneck* mempercepat perbaikan ketidakseimbangan beban, sehingga berdampak langsung pada pemendekan *makespan*, sejalan dengan temuan Li et al. (2011) yang menunjukkan bahwa *neighborhood search* berbasis mesin kritis efektif untuk masalah penjadwalan berorientasi *makespan*. Karena kualitas solusi pada rentang tersebut identik, pemilihan konfigurasi akhir didasarkan pada efisiensi komputasi, di mana

critical machine bias = 1,0 memberikan *runtime* rata-rata terendah dan dipilih sebagai konfigurasi terbaik.

6.1.6 Ringkasan dan Analisis Hasil Pengujian Parameter SA

Pengujian parameter SA menunjukkan pola yang konsisten bahwa peningkatan iterasi dan pendinginan yang lebih lambat menghasilkan kualitas solusi yang lebih baik hingga titik jenuh tertentu. Max iteration = 250.000 dipilih karena menjadi batas di mana perbaikan fitness berhenti meningkat secara signifikan, sementara floor ratio = 0,05 memberikan titik henti temperatur yang efisien tanpa menurunkan kualitas solusi. Parameter eksplorasi awal p0 = 0,8 dan laju pendinginan alpha = 0,995 memberikan kombinasi performa dan stabilitas terbaik pada data penelitian. Selain itu, critical machine bias = 1,0 terbukti efektif mempercepat perbaikan pada mesin dengan beban tertinggi tanpa menambah variasi hasil.

Selain ringkasan tersebut, hasil *tuning* juga memperlihatkan karakteristik penting terkait hubungan antara pendinginan dan kebutuhan jumlah iterasi. Dengan konfigurasi akhir yang dipilih (*alpha* = 0,995 dan *floor ratio* = 0,05), suhu minimum telah tercapai relatif cepat, yaitu sekitar 598 iterasi pertama. Namun, peningkatan kualitas solusi tetap terus terjadi jauh setelah titik tersebut, dan baru mencapai kondisi stabil sepenuhnya pada 250.000 iterasi. Temuan ini menunjukkan bahwa pada fase suhu rendah, masih sering terjadi perubahan kecil pada struktur jadwal, terutama pada distribusi beban antar mesin dan penyesuaian urutan pekerjaan, sehingga *makespan* dapat berkurang secara bertahap. Proses ini tidak bersifat monoton, melainkan ditandai oleh osilasi solusi di sekitar nilai *fitness* yang baik.

Osilasi yang berlangsung relatif lama terjadi karena perbaikan *makespan* umumnya memerlukan kombinasi *move neighborhood* yang spesifik, sementara *neighbor* pada SA dibangkitkan secara acak. Banyak *move* yang dievaluasi pada fase ini bersifat netral atau tidak menghasilkan perbaikan langsung, dan hanya sebagian kecil yang benar-benar mampu memperbaiki keseimbangan beban mesin tanpa meningkatkan setup akibat *sequence-dependent setup time*. Akibatnya, dibutuhkan jumlah iterasi yang besar agar *move* yang tepat dapat ditemukan berulang kali hingga menghasilkan solusi terbaik yang konsisten pada seluruh *seed*. Pada penelitian ini, *critical machine bias* terbukti cukup efektif untuk mengurangi dampak sifat acak tersebut dengan memfokuskan pembangkitan *move* pada mesin kritis yang berkontribusi langsung terhadap *makespan*. Namun, pendekatan ini tidak sepenuhnya menghilangkan karakter pencarian yang tetap bergantung pada pembangkitan *neighbor* secara acak, sehingga osilasi dan kebutuhan iterasi besar masih tetap muncul, meskipun dalam bentuk yang lebih terarah.

Secara keseluruhan, konfigurasi berikut dipilih sebagai parameter final karena menghasilkan *fitness* terendah, stabilitas tinggi, dan *runtime* yang tetap efisien.

Tabel 6.6 Parameter SA Terbaik

Nama Parameter	Nilai Parameter
Max Iteration	250000
Alpha	0.995
P0	0.8
Floor Ratio	0.05
Critical Machine Bias	1

Konfigurasi ini digunakan sebagai *baseline* pada seluruh pengujian bobot, *due date multiplier*, serta skenario statis dan dinamis di bagian berikutnya.

6.2 Pengujian Bobot

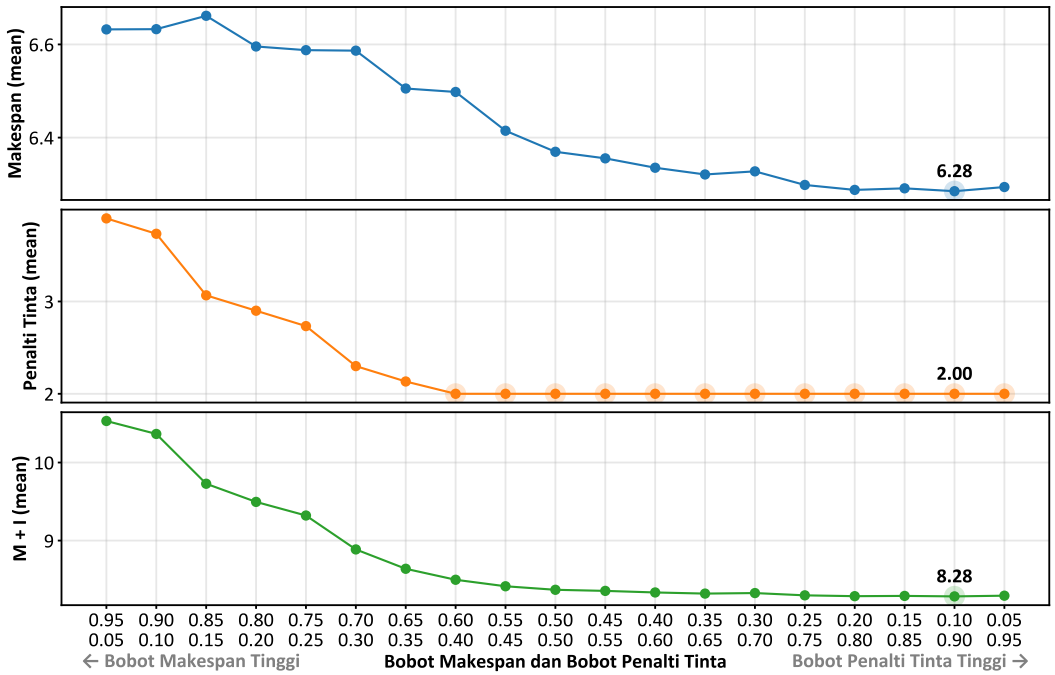
Pengujian bobot bertujuan menentukan kombinasi bobot terbaik pada fungsi objektif multi-kriteria yang digunakan oleh SA. Prosedur pembentukan *grid*, jumlah percobaan per kombinasi, dan ringkasan statistik mengikuti Subbab 4.7.3. Evaluasi dilakukan pada lingkungan statis menggunakan Skenario 3 dengan tujuan *makespan* dan penalti tinta serta lingkungan dinamis menggunakan Skenario 4B dengan tujuan penalti *tardy*, *makespan*, dan penalti tinta.

6.2.1 Hasil Pengujian Bobot Skenario 3

Tabel 6.7 Statistik Hasil Pengujian Bobot Pada Skenario 3

Bobot		Rata-rata			Standar Deviasi (M + I)
Makespan	Penalti Tinta	Makespan	Penalti Tinta	(M + I)	
0.05	0.95	6.29	2.00	8.29	0.02
0.1	**0.9**	**6.28**	**2.00**	**8.28**	**0.00**
0.15	0.85	6.29	2.00	8.29	0.02
0.2	0.8	6.29	2.00	8.29	0.01
0.25	0.75	6.30	2.00	8.30	0.03
0.3	0.7	6.33	2.00	8.33	0.05
0.35	0.65	6.32	2.00	8.32	0.05
0.4	0.6	6.34	2.00	8.34	0.04
0.45	0.55	6.36	2.00	8.36	0.05
0.5	0.5	6.37	2.00	8.37	0.05
0.55	0.45	6.41	2.00	8.41	0.08
0.6	0.4	6.50	2.00	8.50	0.12
0.65	0.35	6.51	2.13	8.64	0.37
0.7	0.3	6.59	2.30	8.89	0.50
0.75	0.25	6.59	2.73	9.32	0.46

Bobot		Rata-rata			Standar Deviasi (M + I)
Makespan	Penalti Tinta	Makespan	Penalti Tinta	(M + I)	
0.8	0.2	6.60	2.90	9.50	0.62
0.85	0.15	6.66	3.07	9.73	0.65
0.9	0.1	6.63	3.73	10.37	0.70
0.95	0.05	6.63	3.90	10.53	0.81



Gambar 6.7 Pengaruh Bobot Pada Nilai Objektif Skenario 3

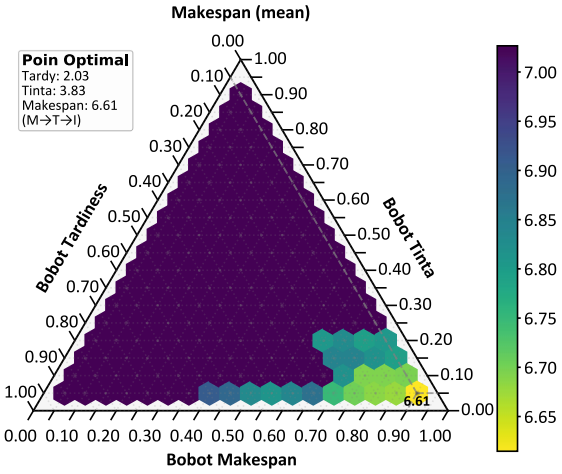
Pada Skenario 3, fungsi objektif terdiri dari *makespan* dan penalti tinta dengan total bobot satu. Hasil pada Tabel 6.7 dan Gambar 6.7 menunjukkan bahwa pada bobot *makespan* 0,05 hingga 0,60, penalti tinta berada pada nilai minimum 2,00 dan *makespan* stabil pada kisaran 6,28 hingga 6,37 jam. Nilai objektif gabungan terbaik diperoleh pada bobot *makespan* 0,10 dan penalti tinta 0,90 dengan rata-rata 8,28 serta stabilitas sempurna antar seed. Ketika bobot *makespan* melebihi 0,60, penalti tinta mulai meningkat dan nilai gabungan memburuk. Berdasarkan hasil tersebut, bobot final untuk konteks statis ditetapkan *makespan* 0,10 dan penalti tinta 0,90.

6.2.2 Hasil Pengujian Bobot Skenario 4B

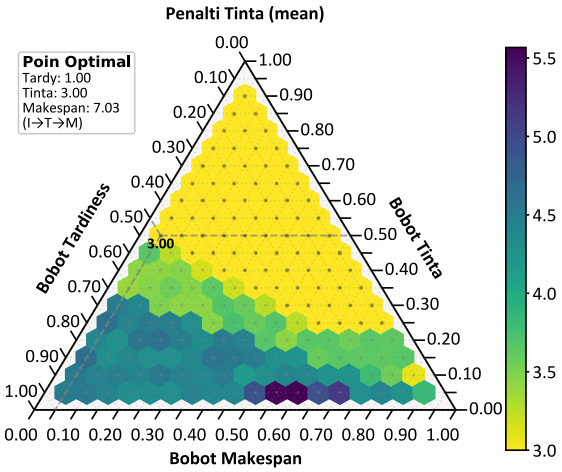
Tabel 6.8 Statistik Sepuluh Kombinasi Bobot Terbaik pada Skenario 4B

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Makespan	Penalti Tinta	Penalti Tardy	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.45	**0.1**	**0.45**	**7.03**	**4.17**	**0.00**	**4.87**	**0.38**

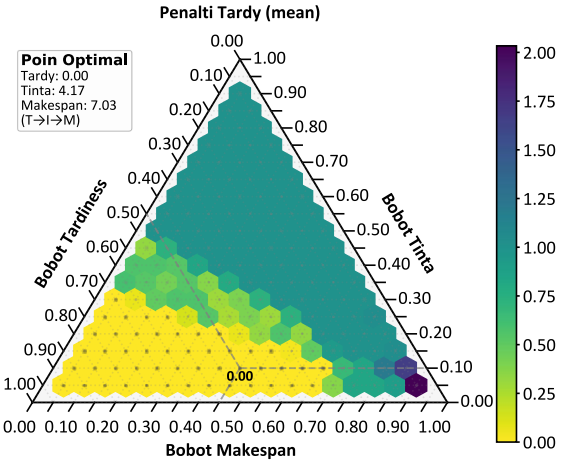
Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Makespan	Penalti Tinta	Penalti Tardy	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.65	0.1	0.25	7.03	4.23	0.00	4.94	0.43
0.35	0.25	0.4	7.03	3.73	0.27	4.97	1.01
0.1	0.2	0.7	7.03	4.27	0.00	4.97	0.45
0.5	0.15	0.35	7.03	4.27	0.00	4.97	0.45
0.25	0.15	0.6	7.03	4.30	0.00	5.00	0.47
0.3	0.05	0.65	7.03	4.30	0.00	5.00	0.47
0.1	0.15	0.75	7.03	4.30	0.00	5.00	0.47
0.45	0.25	0.3	7.03	3.70	0.30	5.00	1.04
0.3	0.3	0.4	7.03	3.70	0.30	5.00	1.04



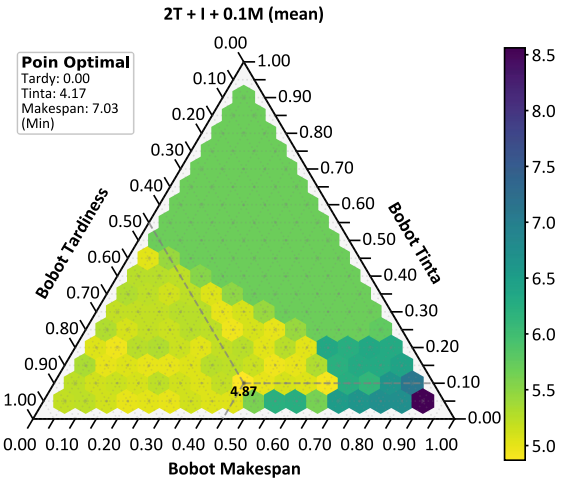
Gambar 6.8 Pengaruh Bobot Terhadap Rata-rata *Makespan*



Gambar 6.9 Pengaruh Bobot Terhadap Rata-rata Penalti Tinta



Gambar 6.10 Pengaruh Bobot Terhadap Rata-rata Penalti *Tardy*



Gambar 6.11 Pengaruh Bobot Terhadap *Fitness* (2T + I + 0.1M)

Pada Skenario 4B, fungsi objektif mencakup penalti *tardy*, penalti tinta, dan *makespan* dengan total bobot satu. Evaluasi terhadap 171 kombinasi bobot menunjukkan bahwa interaksi utama terjadi antara penalti *tardy* dan penalti tinta. Bobot *tardy* yang terlalu kecil mendorong pengurangan pergantian tinta tetapi mengorbankan ketepatan waktu, sedangkan bobot *tardy* yang terlalu besar menghilangkan keterlambatan dengan konsekuensi peningkatan penalti tinta akibat urutan warna yang kurang efisien. Dalam rentang bobot ini, *makespan* relatif stabil dan tidak menjadi faktor pembeda utama kinerja jadwal. Kombinasi bobot (0,45; 0,10; 0,45) memberikan keseimbangan terbaik, menghasilkan *tardy* nol dengan penalti tinta rata-rata 4,17 dan *makespan* 7,03 jam yang stabil antar *seed*, sehingga dipilih sebagai konfigurasi bobot final untuk pengujian skenario dinamis.

6.2.3 Ringkasan dan Analisis Hasil Pengujian Bobot

Perbedaan tren antara Skenario 3 dan Skenario 4B menunjukkan bahwa struktur *trade-off* berubah ketika konteks berpindah dari offline statis ke dinamis. Pada Skenario 3, penalti tinta dapat ditekan hingga minimum tanpa

mengorbankan *makespan* pada rentang bobot yang lebar. Hal ini konsisten dengan karakter masalah yang memiliki *sequence-dependent setup times*, karena pengurangan pergantian tinta menurunkan komponen setup dan secara langsung mengurangi waktu penyelesaian efektif, sehingga dapat membantu menekan *makespan*, sebagaimana ditekankan dalam survei setup bergantung urutan oleh Allahverdi et al. (2008).

Sebaliknya, pada Skenario 4B, keberadaan *leftover* dan tekanan ketepatan waktu membuat kompromi utama bergeser menjadi penalti *tardy* versus penalti tinta, sementara *makespan* relatif stabil. Pola ini selaras dengan literatur penjadwalan mesin paralel dengan setup bergantung urutan yang menempatkan performa berbasis *due date*, seperti *weighted tardiness*, sebagai tujuan utama, sehingga keputusan urutan yang mengejar ketepatan waktu dapat bertentangan dengan keputusan yang meminimalkan pergantian setup (Lee & Pinedo, 1997). Karena itu, bobot dinamis dipilih untuk menjaga penalti *tardy* tetap nol dengan penalti tinta yang masih wajar secara operasional, bukan untuk menekan penalti tinta semaksimal mungkin seperti pada konteks statis.

Tabel 6.9 Bobot Terbaik Fungsi Objektif

Konteks	Skenario Acuan	Bobot Makespan	Bobot Penalti Tinta	Bobot Penalti Tardy
Statis	Skenario 3	0.10	0.90	-
Dinamis	Skenario 4B	0.45	0.10	0.45

Bobot pada Tabel 6.9 digunakan sebagai konfigurasi final untuk seluruh pengujian lanjutan.

6.3 Pengujian *Due Date Multiplier*

6.3.1 Hasil Pengujian *Due Date Multiplier*

Tabel 6.10 Statistik Hasil Pengujian Parameter *Due Date Multiplier S*4A-S4C

Skenario	Due Date Multiplier	Rata-rata				Std Fitness	Rata-rata Runtime
		Makespan	Penalti Tinta	Penalti Tardy	Fitness		
S4A	0.8	7.72	6	2.53	5.21	0.05	6.16
	1	7.01	5.77	0	3.73	0.09	5.67
	1.2	7.02	5.07	0	3.67	0.03	5.61
S4B	0.8	7.51	4.53	1.47	4.49	0.03	6.29
	1	7.03	4.17	0	3.58	0.04	6.38
	1.2	6.86	4	0	3.49	0.05	6.52
S4C	0.8	7.97	5.93	2.7	5.4	0.18	7.83
	1	7.5	6	1	4.42	0	8.05
	1.2	7.92	8	0	4.36	0	7.6

Pada skenario dinamis, *due date* setiap pekerjaan ditentukan menggunakan aturan *workload-based dynamic due date assignment* dengan tiga tingkat *due date multiplier*, yaitu 0,8 sebagai *due date* ketat, 1,0 sebagai baseline, dan 1,2 sebagai *due date* longgar. Hasil pada Tabel 6.10 menunjukkan bahwa pada Skenario 4A dan 4B, *multiplier* 0,8 menghasilkan penalti *tardy* yang lebih tinggi, sedangkan *multiplier* 1,0 dan 1,2 mampu menghilangkannya. *Multiplier* 1,2 hanya memberikan perbaikan kecil pada beberapa metrik dan mengindikasikan *due date* yang terlalu longgar, khususnya untuk pekerjaan *leftover*. Pada Skenario 4C yang paling berat, *multiplier* 0,8 kembali menunjukkan performa terburuk, *multiplier* 1,0 menurunkan penalti *tardy* ke tingkat menengah, sedangkan *multiplier* 1,2 menghilangkannya namun disertai kenaikan penalti tinta.

6.3.2 Analisis Hasil Pengujian *Due Date Multiplier*

Secara konseptual, *due date multiplier* mengendalikan tingkat keketatan *due date* atau kelonggaran *flow allowance*, sehingga nilai yang lebih kecil meningkatkan tekanan ketepatan waktu dan cenderung menaikkan *tardiness*, sedangkan nilai yang lebih besar menurunkannya, sejalan dengan pembahasan Baker & Scudder (1990). Hasil pengujian menunjukkan bahwa *multiplier* 0,8 terlalu ketat untuk seluruh variasi skenario, sementara *multiplier* 1,2 cenderung terlalu longgar. Meskipun mampu menghilangkan *tardy*, kelonggaran yang berlebihan berpotensi menurunkan *fairness* terhadap pekerjaan yang datang lebih awal karena tekanan tenggat melemah dan optimasi bergeser ke tujuan lain seperti minimalisasi penalti tinta, yang pada beban kerja tinggi justru dapat meningkat. Oleh karena itu, dengan mempertimbangkan keseimbangan performa jadwal, stabilitas hasil, dan kelonggaran *due date* yang masih wajar secara operasional, penelitian ini menetapkan *due date multiplier* 1,0 sebagai *baseline* untuk seluruh pengujian skenario dinamis berikutnya.

6.4 Pengujian Skenario

Setelah parameter SA dan bobot tujuan ditetapkan, tahap selanjutnya adalah mengevaluasi performa algoritma pada enam skenario eksperimen yang telah didefinisikan pada Subbab 4.7.1. Keenam skenario tersebut merepresentasikan variasi tingkat ketersediaan informasi, mulai dari lingkungan statis hingga lingkungan dinamis berbasis *batch arrival*, serta perbedaan tujuan penjadwalan.

Setiap skenario dijalankan sebanyak 30 *seed* berbeda untuk menangkap sifat stokastik SA dan menilai konsistensi hasil. Evaluasi difokuskan pada perilaku agregat algoritma menggunakan metrik penalti pergantian tinta, *makespan*, *tardiness*, *fitness*, serta *runtime*, dengan penekanan pada nilai rata-rata, variasi, dan pola konvergensi. Dengan demikian, subbab ini menilai *robustness* dan karakter pencarian SA secara sistematis, sementara interpretasi operasional dan struktur jadwal terbaik dibahas secara terpisah pada Subbab 6.5.

6.4.1 Ringkasan Statistik Pengujian Skenario

Tabel 6.11 Statistik Hasil Percobaan Skenario

Skenario	Metrik	Rata-rata	Standar Deviasi	Min	Max
S1	Fitness	5.77	0.0008	5.77	5.78
	Makespan	5.77	0.0008	5.77	5.78
	Runtime	6.05	0.77	4.97	7.04
S2	Fitness	2	0	2	2
	Penalti Tinta	2	0	2	2
	Runtime	5.52	0.29	4.95	6.09
S3	Fitness	2.43	0	2.43	2.43
	Makespan	6.29	0	6.29	6.29
	Penalti Tinta	2	0	2	2
	Runtime	5.66	0.36	5.07	6.24
S4A	Fitness	3.73	0.09	3.66	3.95
	Makespan	7.01	0.04	6.91	7.03
	Penalti Tinta	5.77	1.02	5	8
	Penalti Tardy	0	0	0	0
	Runtime	7.51	0.48	6.86	8.62
S4B	Fitness	3.58	0.04	3.56	3.66
	Makespan	7.03	0	7.03	7.03
	Penalti Tinta	4.17	0.37	4	5
	Penalti Tardy	0	0	0	0
	Runtime	8.94	0.55	7.78	9.63
S4C	Fitness	4.42	0	4.42	4.42
	Makespan	7.5	0	7.5	7.5
	Penalti Tinta	6	0	6	6
	Penalti Tardy	1	0	1	1
	Runtime	10.5	0.58	9.52	11.29

Ringkasan statistik kinerja untuk setiap skenario disajikan pada Tabel 6.11, yang memuat nilai rata-rata, standar deviasi, serta nilai minimum dan maksimum dari metrik evaluasi utama berdasarkan 30 kali pengulangan.

Hasil statistik menunjukkan bahwa seluruh skenario statis (S1-S3) memiliki tingkat stabilitas yang sangat tinggi. Pada S2 dan S3, nilai *fitness* dan metrik utama lainnya memiliki standar deviasi nol, yang mengindikasikan bahwa SA secara konsisten menghasilkan solusi identik pada seluruh *seed*. Kondisi ini mencerminkan bahwa pada lingkungan statis dengan informasi pekerjaan yang

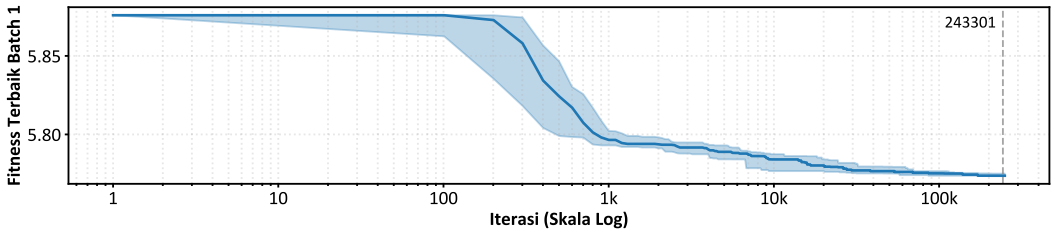
lengkap, lanskap pencarian relatif sederhana dan algoritma cenderung konvergen ke solusi yang sama.

Sebaliknya, skenario dinamis menunjukkan nilai rata-rata *fitness*, *makespan*, dan penalti tinta yang lebih tinggi dibandingkan skenario statis, serta variasi yang lebih besar. Hal ini mencerminkan meningkatnya kompleksitas pencarian akibat keterbatasan informasi awal dan kebutuhan optimasi ulang ketika pekerjaan datang secara bertahap. Perbedaan kinerja antar skenario dinamis mencerminkan sensitivitas algoritma terhadap pola kedatangan pekerjaan dan tidak dianalisis pada tingkat struktur jadwal pada subbab ini.

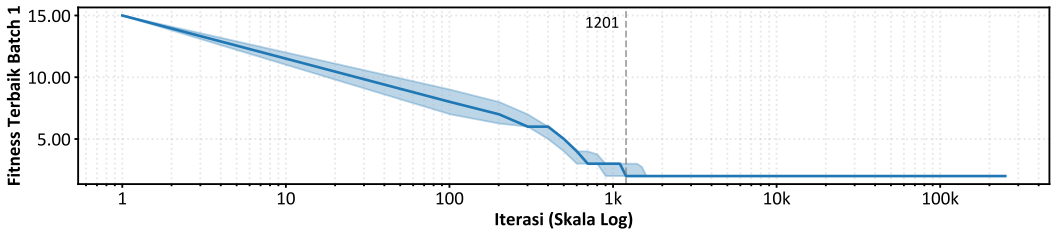
Ditinjau dari efisiensi komputasi, *runtime* pada skenario statis relatif rendah dan stabil karena optimasi dilakukan satu kali dengan informasi lengkap. Pada skenario dinamis, *runtime* meningkat seiring bertambahnya jumlah *batch* dan frekuensi optimasi ulang dalam pendekatan *rolling horizon*. Kenaikan runtime ini tetap menunjukkan variasi yang kecil antar *seed*, sehingga mencerminkan perilaku komputasi yang terkontrol dan dapat diprediksi.

6.4.2 Analisis Konvergensi Skenario

Analisis konvergensi digunakan untuk mengamati dinamika perbaikan kualitas solusi sepanjang iterasi SA pada setiap skenario. Gambar 6.12 hingga Gambar 6.17 menampilkan perubahan *fitness* terbaik dalam skala log iterasi, dengan garis median dari 30 *seed*, rentang interkuartil (Q25-Q75), serta *last median improvement* sebagai indikator iterasi terakhir dengan perbaikan.



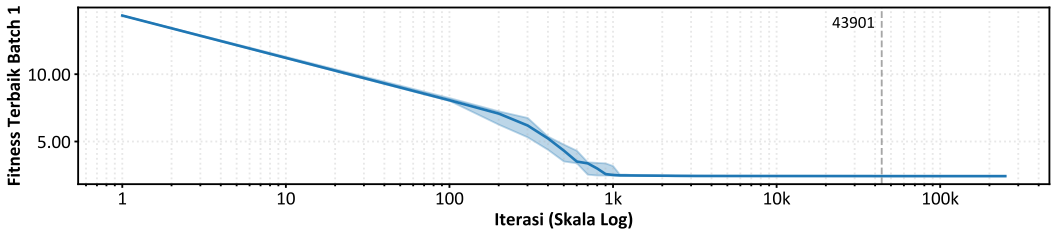
Gambar 6.12 Konvergensi Skenario 1



Gambar 6.13 Konvergensi Skenario 2

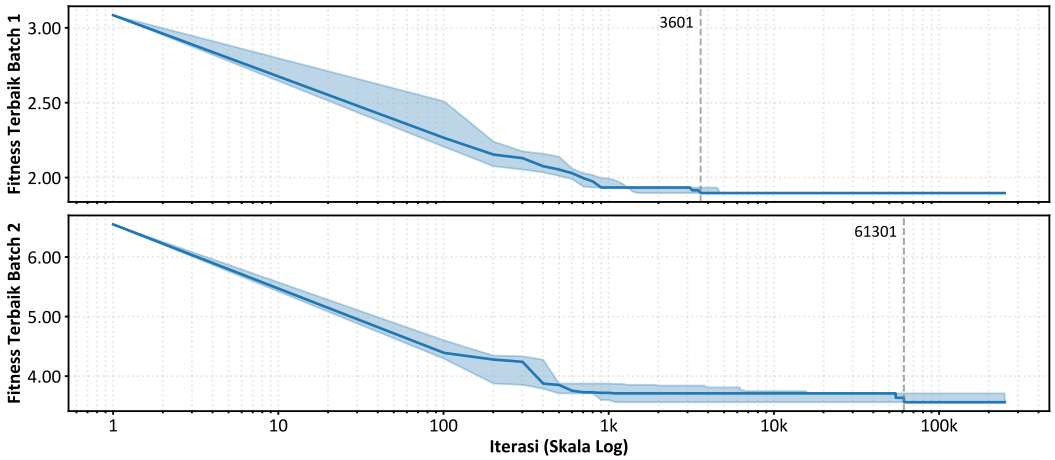
Pada Skenario 1, yang hanya meminimalkan *makespan*, perbaikan terbesar terjadi pada iterasi awal, namun median *fitness* terus membaik secara perlahan hingga mendekati batas iterasi maksimum. Hal ini mencerminkan bahwa *minimasi makespan* bersifat sulit dan dipenuhi perbaikan marginal kecil, sehingga memerlukan proses pencarian yang panjang meskipun solusi antar *seed* sangat konsisten. Sebaliknya, pada Skenario 2, median *fitness* turun sangat cepat dan

mencapai *plateau* jauh sebelum iterasi maksimum, dengan IQR menyusut menjadi nol. Pola ini menunjukkan bahwa *minimasi* pergantian tinta merupakan masalah yang relatif mudah dan cepat konvergen karena ruang solusi yang diskret.

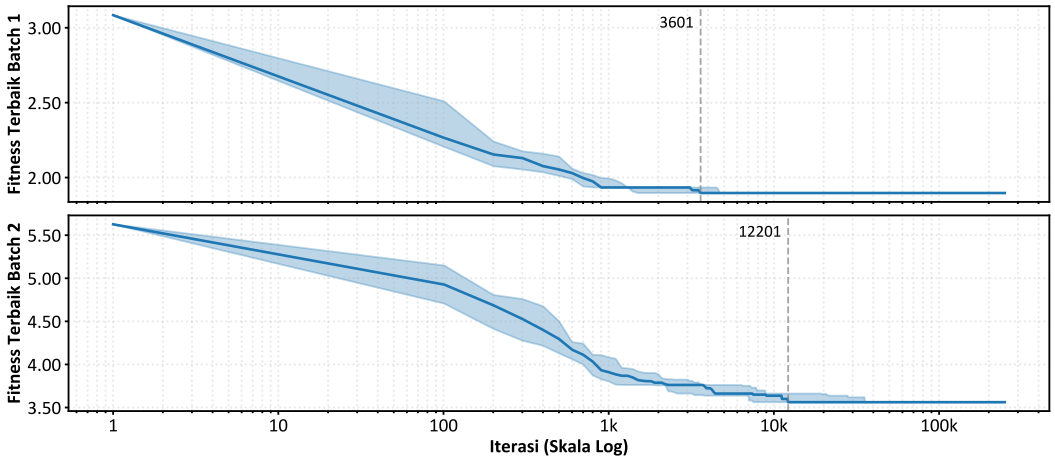


Gambar 6.14 Konvergensi Skenario 3

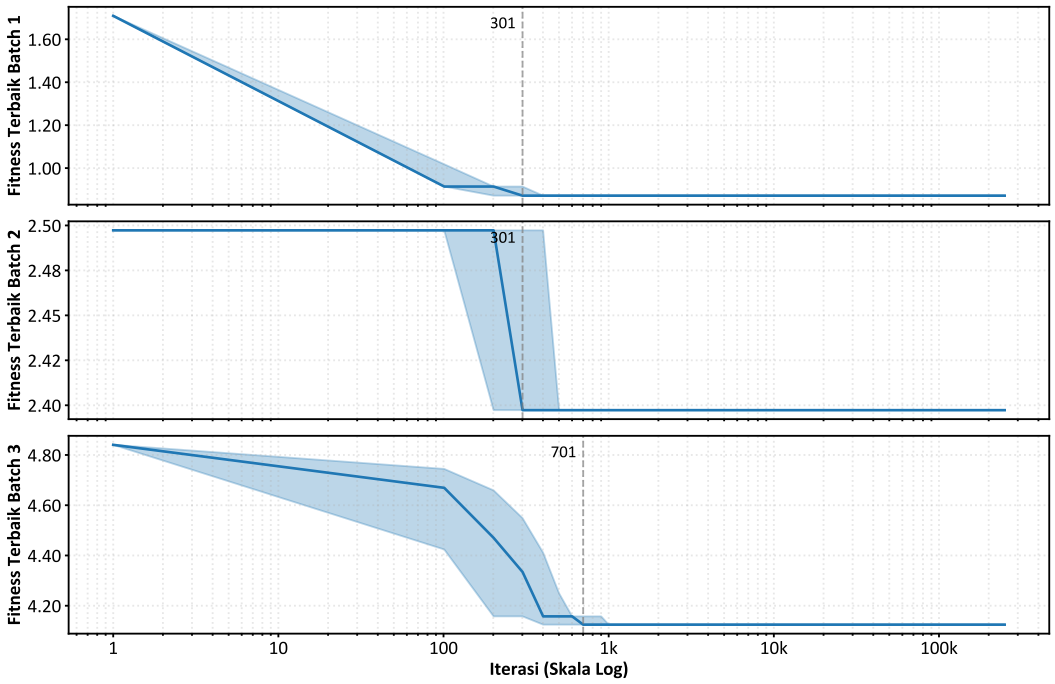
Pada Skenario 3, yang menggabungkan *makespan* dan penalti tinta, median *fitness* turun tajam pada fase awal dan kemudian menjadi hampir datar dengan IQR yang sangat sempit. Komponen penalti tinta konvergen lebih awal, sedangkan perbaikan lanjutan hingga *last median improvement* terutama berasal dari penyempurnaan *makespan*. Konsistensi nilai *fitness* antar *seed* mengindikasikan bahwa konfigurasi bobot statis menghasilkan solusi yang sangat stabil dan konvergen ke pola jadwal yang sama.



Gambar 6.15 Konvergensi Skenario 4A



Gambar 6.16 Konvergensi Skenario 4B



Gambar 6.17 Konvergensi Skenario 4C

Pada skenario dinamis S4A-S4C, konvergensi dianalisis secara terpisah untuk setiap batch. Pada S4A, *Batch* 1 menunjukkan konvergensi yang relatif cepat, sementara *Batch* 2 membutuhkan iterasi yang lebih panjang sebelum mencapai kondisi stabil, yang tercermin dari penurunan *fitness* yang lebih gradual. Pada S4B, konvergensi Batch 2 berlangsung lebih cepat dan stabil dibandingkan S4A, dengan penyempitan IQR yang lebih dini. Pada S4C, seluruh *batch* menunjukkan konvergensi yang sangat cepat dengan IQR yang hampir nol, mengindikasikan konsistensi solusi antar *run*. Pola ini selaras dengan hasil statistik yang menunjukkan variabilitas yang sangat rendah pada skenario tersebut.

6.4.3 Analisis Hasil Pengujian Skenario

Secara keseluruhan, hasil pengujian menunjukkan bahwa perbedaan kinerja antara lingkungan statis dan dinamis terutama dipengaruhi oleh struktur informasi yang tersedia selama proses penjadwalan. Pada skenario statis, ketersediaan informasi pekerjaan secara penuh sejak awal menghasilkan proses pencarian yang stabil, sehingga algoritma SA secara konsisten konvergen ke solusi yang sama dengan variasi antar *seed* yang sangat rendah serta *runtime* yang relatif singkat.

Pada skenario dinamis, pendekatan *rolling horizon* menyebabkan keputusan penjadwalan diambil secara bertahap berdasarkan informasi yang terbatas pada setiap *horizon*. Kedatangan *batch* baru menambah interaksi antar tujuan yang perlu dipertimbangkan kembali, sehingga kualitas solusi agregat dan *runtime* cenderung lebih tinggi dibandingkan skenario statis. Pola ini sejalan dengan karakteristik umum penjadwalan dinamis berbasis *rolling horizon*, di mana keputusan dibuat dengan visibilitas parsial terhadap pekerjaan yang akan datang (Ovacik & Uzsoy, 1995).

Meskipun demikian, hasil pengujian tidak menunjukkan indikasi ketidakstabilan algoritma. Pola konvergensi yang konsisten dan variasi antar *seed* yang tetap rendah menegaskan bahwa SA mampu beradaptasi terhadap perubahan informasi dan tetap menghasilkan solusi yang konvergen. Dengan demikian, perbedaan kinerja antar skenario dinamis lebih merefleksikan keterbatasan sistem terhadap pola kedatangan pekerjaan dan konflik tujuan yang dihadapi, bukan keterbatasan mekanisme pencarian SA itu sendiri. Analisis lebih lanjut mengenai struktur jadwal dan implikasi operasional masing-masing skenario dibahas pada Subbab 6.5.

6.5 Pengujian Jadwal Terbaik

Subbab ini membahas jadwal terbaik pada setiap skenario sebagai contoh representatif untuk menginterpretasikan struktur urutan pekerjaan, pola utilisasi mesin, dan implikasi operasionalnya. Analisis merujuk pada tabel metrik, utilisasi mesin, keuangan, serta *Gantt chart* masing-masing skenario. Berbeda dari Subbab 6.4 yang menilai performa secara agregat antar *seed*, subbab ini menekankan penjelasan berbasis struktur jadwal.

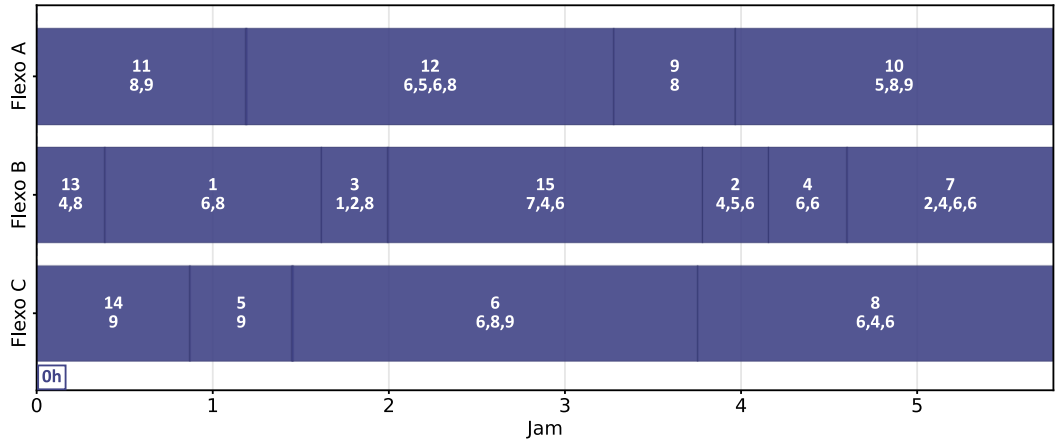
6.5.1 Hasil Pengujian Jadwal Terbaik Skenario 1

Tabel 6.12 Metrik Optimasi Skenario 1

Metrik	Nilai
Fitness	5.774
Makespan (h)	5.77
Runtime (s)	3.24

Tabel 6.13 Metrik Utilisasi Mesin Skenario 1

Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Flexo A	5.77	5.77	100.00	99.93
Flexo B	5.77	5.77	100.00	100.00
Flexo C	5.77	5.77	100.00	99.94



Gambar 6.18 *Gantt Chart* Jadwal Skenario 1

Tabel 6.14 Jadwal Skenario 1

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Penalti
Flexo A	1	11	0h	8, 9	0.00	1.19	Tidak ada
	2	12	0h	6, 5, 6, 8	1.19	3.28	Tidak ada
	3	9	0h	8	3.28	3.97	Tidak ada
	4	10	0h	5, 8, 9	3.97	5.77	Tidak ada
Flexo B	1	13	0h	4, 8	0.00	0.39	Tidak ada
	2	1	0h	6, 8	0.39	1.62	Tidak ada
	3	3	0h	1, 2, 8	1.62	2.00	Tidak ada
	4	15	0h	7, 4, 6	2.00	3.78	Tidak ada
	5	2	0h	4, 5, 6	3.78	4.15	Tidak ada
	6	4	0h	6, 6	4.15	4.60	Tidak ada
	7	7	0h	2, 4, 6, 6	4.60	5.77	Tidak ada
Flexo C	1	14	0h	9	0.00	0.87	Tidak ada
	2	5	0h	9	0.87	1.45	Tidak ada
	3	6	0h	6, 8, 9	1.45	3.75	Tidak ada
	4	8	0h	6, 4, 6	3.75	5.77	Tidak ada

Pada Skenario 1, fungsi objektif memprioritaskan minimalisasi *makespan* tanpa mempertimbangkan konsekuensi pergantian tinta. Mengacu pada Tabel

6.12 dan Tabel 6.13, jadwal terbaik menunjukkan pembagian beban yang sangat merata di seluruh mesin dengan cakupan hampir penuh. Visualisasi pada Gambar 6.18 memperlihatkan bahwa urutan pekerjaan tidak membentuk pengelompokan berdasarkan level tinta, sehingga transisi warna terjadi secara acak. Pola ini merepresentasikan *baseline* kecepatan murni, yang optimal dari sisi waktu penyelesaian, tetapi berpotensi menghasilkan biaya setup tinggi apabila aspek tinta diperhitungkan.

6.5.2 Hasil Pengujian Jadwal Terbaik Skenario 2

Tabel 6.15 Metrik Optimasi Skenario 2

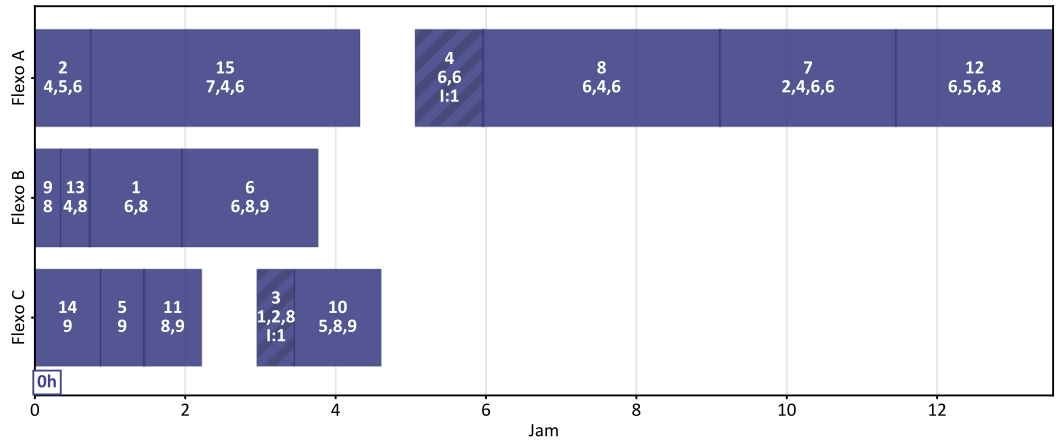
Metrik	Nilai
Fitness	2.000
Penalti Tinta	2.0
Runtime (s)	3.31

Tabel 6.16 Metrik Utilisasi Mesin Skenario 2

Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Flexo A	12.79	13.54	94.46	100.00
Flexo B	3.76	3.76	100.00	27.76
Flexo C	3.85	4.60	83.71	33.99

Tabel 6.17 Metrik Keuangan Skenario 2

Metrik Keuangan	Nilai
Pendapatan	Rp 265,434,000
Penalti Tinta	Rp 1,600,000
Laba Bersih	Rp 263,834,000



Gambar 6.19 *Gantt Chart* Jadwal Skenario 2

Tabel 6.18 Jadwal Skenario 2

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Penalti
Flexo A	1	2	0h	4, 5, 6	0.00	0.75	Tidak ada
	2	15	0h	7, 4, 6	0.75	4.32	Tidak ada
	3	4	0h	6, 6	5.07	5.96	1 Tinta
	4	8	0h	6, 4, 6	5.96	9.11	Tidak ada
	5	7	0h	2, 4, 6, 6	9.11	11.45	Tidak ada
	6	12	0h	6, 5, 6, 8	11.45	13.54	Tidak ada
Flexo B	1	9	0h	8	0.00	0.35	Tidak ada
	2	13	0h	4, 8	0.35	0.73	Tidak ada
	3	1	0h	6, 8	0.73	1.96	Tidak ada
	4	6	0h	6, 8, 9	1.96	3.76	Tidak ada
Flexo C	1	14	0h	9	0.00	0.87	Tidak ada
	2	5	0h	9	0.87	1.45	Tidak ada
	3	11	0h	8, 9	1.45	2.21	Tidak ada
	4	3	0h	1, 2, 8	2.96	3.45	1 Tinta
	5	10	0h	5, 8, 9	3.45	4.60	Tidak ada

Pada Skenario 2, optimasi diarahkan untuk meminimalkan penalti pergantian tinta. Mengacu Tabel 6.15 dan Tabel 6.16, jadwal terbaik memperlihatkan konsolidasi pekerjaan pada mesin tertentu untuk menjaga konsistensi warna. Strategi ini menghasilkan ketimpangan utilisasi yang signifikan antar mesin, sebagaimana tampak pada *Gantt chart* Gambar 6.19. Dari sisi finansial, penurunan penalti tinta meningkatkan laba bersih, namun implikasinya adalah *makespan* yang lebih panjang dan penggunaan kapasitas sistem yang tidak seimbang. Skenario ini merepresentasikan ekstrem efisiensi setup, bukan solusi operasional yang seimbang.

6.5.3 Hasil Pengujian Jadwal Terbaik Skenario 3

Tabel 6.19 Metrik Optimasi Skenario 3

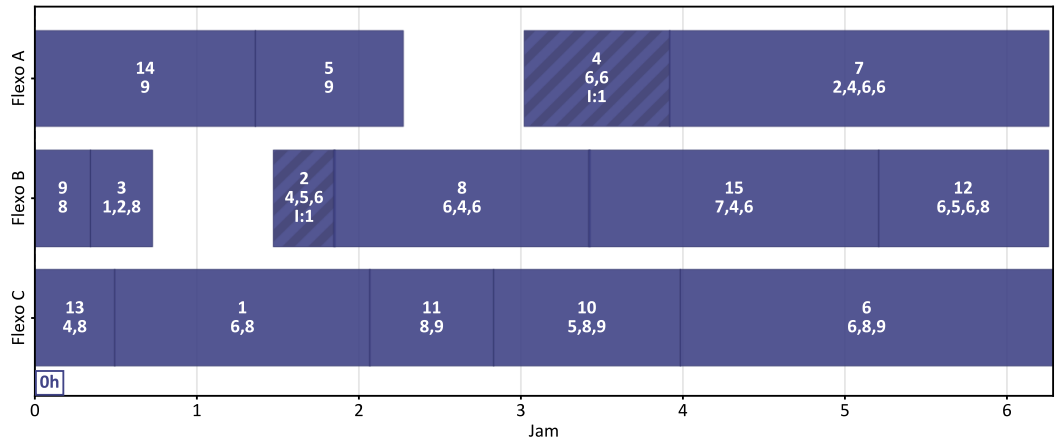
Metrik	Nilai
Fitness	2.428
Penalti Tinta	2.0
Makespan (h)	6.28
Runtime (s)	3.18

Tabel 6.20 Metrik Utilisasi Mesin Skenario 3

Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Flexo A	5.51	6.26	88.02	99.59
Flexo B	5.50	6.25	88.00	99.47
Flexo C	6.28	6.28	100.00	100.00

Tabel 6.21 Metrik Keuangan Skenario 3

Metrik Keuangan	Nilai
Pendapatan	Rp 265,434,000
Penalti Tinta	Rp 1,600,000
Laba Bersih	Rp 263,834,000



Gambar 6.20 *Gantt Chart* Jadwal Skenario 3

Tabel 6.22 Jadwal Skenario 3

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Penalti
Flexo A	1	14	0h	9	0.00	1.36	Tidak ada
	2	5	0h	9	1.36	2.27	Tidak ada
	3	4	0h	6, 6	3.02	3.92	1 Tinta

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Penalti
	4	7	0h	2, 4, 6, 6	3.92	6.26	Tidak ada
Flexo B	1	9	0h	8	0.00	0.35	Tidak ada
	2	3	0h	1, 2, 8	0.35	0.73	Tidak ada
	3	2	0h	4, 5, 6	1.48	1.85	1 Tinta
	4	8	0h	6, 4, 6	1.85	3.42	Tidak ada
	5	15	0h	7, 4, 6	3.42	5.21	Tidak ada
	6	12	0h	6, 5, 6, 8	5.21	6.25	Tidak ada
Flexo C	1	13	0h	4, 8	0.00	0.49	Tidak ada
	2	1	0h	6, 8	0.49	2.07	Tidak ada
	3	11	0h	8, 9	2.07	2.83	Tidak ada
	4	10	0h	5, 8, 9	2.83	3.98	Tidak ada
	5	6	0h	6, 8, 9	3.98	6.28	Tidak ada

Skenario 3 menggabungkan tujuan minimalisasi *makespan* dan penalti tinta secara simultan. Mengacu pada Tabel 6.19 dan Tabel 6.20, jadwal terbaik menunjukkan distribusi beban yang relatif seimbang dengan pengelompokan tinta yang masih terjaga. Visualisasi *Gantt chart* pada Gambar 6.20 memperlihatkan bahwa transisi warna dapat dikendalikan tanpa mengorbankan kecepatan penyelesaian secara signifikan. Pola ini menjadikan Skenario 3 sebagai kompromi statis yang paling realistis, karena mampu menyeimbangkan kecepatan, biaya setup tinta, dan utilisasi mesin.

6.5.4 Hasil Pengujian Jadwal Terbaik Skenario 4A

Tabel 6.23 Metrik Optimasi Skenario 4A

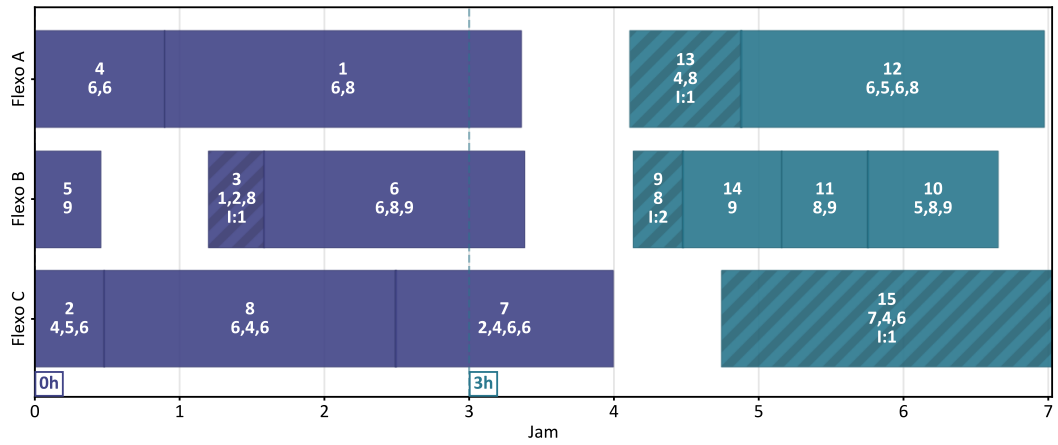
Metrik	Nilai
Fitness	3.662
Penalti Tinta	5.0
Penalti Tardy	0.0
Makespan (h)	7.03
Runtime (s)	4.36

Tabel 6.24 Metrik Utilisasi Mesin Skenario 4A

Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Flexo A	6.22	6.97	89.24	99.15
Flexo B	5.15	6.65	77.46	94.70
Flexo C	6.28	7.03	89.33	100.00

Tabel 6.25 Metrik Keuangan Skenario 4A

Metrik Keuangan	Nilai
Pendapatan	Rp 265,434,000
Penalti Tinta	Rp 4,000,000
Laba Bersih	Rp 261,434,000



Gambar 6.21 *Gantt Chart* Jadwal Skenario 4A

Tabel 6.26 Jadwal Skenario 4A

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Deadline	Penalti
Flexo A	1	4	0h	6, 6	0.00	0.90	4.53	Tidak ada
	2	1	0h	6, 8	0.90	3.36	4.53	Tidak ada
	3	13	3h	4, 8	4.11	4.88	7.18	1 Tinta
	4	12	3h	6, 5, 6, 8	4.88	6.97	7.18	Tidak ada
Flexo B	1	5	0h	9	0.00	0.45	4.53	Tidak ada
	2	3	0h	1, 2, 8	1.20	1.58	4.53	1 Tinta
	3	6	0h	6, 8, 9	1.58	3.38	4.53	Tidak ada
	4	9	3h	8	4.13	4.48	7.18	2 Tinta
	5	14	3h	9	4.48	5.16	7.18	Tidak ada
	6	11	3h	8, 9	5.16	5.75	7.18	Tidak ada
	7	10	3h	5, 8, 9	5.75	6.65	7.18	Tidak ada
Flexo C	1	2	0h	4, 5, 6	0.00	0.48	4.53	Tidak ada
	2	8	0h	6, 4, 6	0.48	2.49	4.53	Tidak ada
	3	7	0h	2, 4, 6, 6	2.49	3.99	4.53	Tidak ada
	4	15	3h	7, 4, 6	4.74	7.03	7.18	1 Tinta

Pada Skenario 4A, penjadwalan dilakukan dalam lingkungan dinamis dengan dua *batch* kedatangan. Mengacu pada Tabel 6.23 dan Tabel 6.24, jadwal terbaik

tetap mampu menyelesaikan seluruh pekerjaan tanpa keterlambatan, namun dengan penalti tinta yang lebih tinggi dibanding skenario statis. Visualisasi pada Gambar 6.21 menunjukkan adanya tambahan setup akibat transisi antar *batch*. Pola ini mencerminkan konsekuensi keputusan awal yang dibuat sebelum seluruh informasi pekerjaan tersedia, sehingga menghasilkan jadwal online yang aman tetapi kurang efisien dari sisi setup tinta.

6.5.5 Hasil Pengujian Jadwal Terbaik Skenario 4B

Tabel 6.27 Metrik Optimasi Skenario 4B

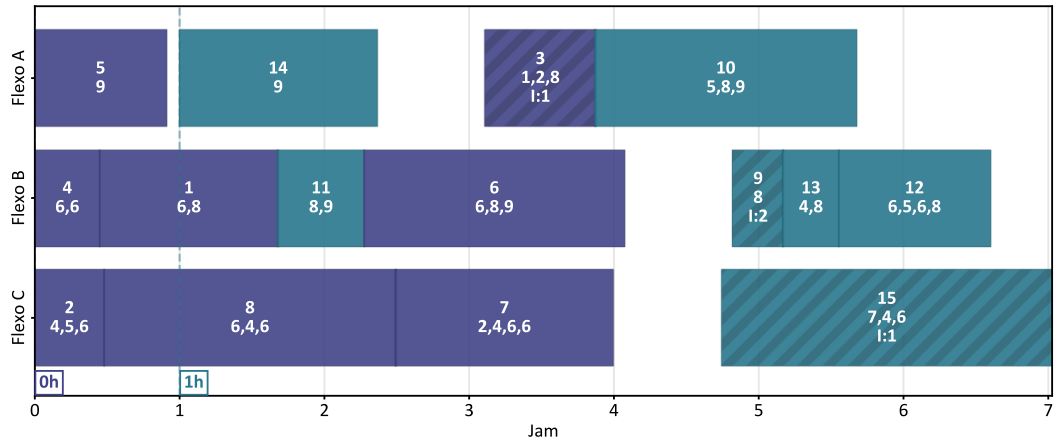
Metrik	Nilai
Fitness	3.562
Penalti Tinta	4.0
Penalti Tardy	0.0
Makespan (h)	7.03
Runtime (s)	5.02

Tabel 6.28 Metrik Utilisasi Mesin Skenario 4B

Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Flexo A	4.83	5.67	85.15	80.73
Flexo B	5.85	6.60	88.63	93.90
Flexo C	6.28	7.03	89.33	100.00

Tabel 6.29 Metrik Keuangan Skenario 4B

Metrik Keuangan	Nilai
Pendapatan	Rp 265,434,000
Penalti Tinta	Rp 3,200,000
Laba Bersih	Rp 262,234,000



Gambar 6.22 *Gantt Chart* Jadwal Skenario 4B

Tabel 6.30 Jadwal Skenario 4B

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Deadline	Penalti
Flexo A	1	5	0h	9	0.00	0.91	4.53	Tidak ada
	2	14	1h	9	1.00	2.36	7.29	Tidak ada
	3	3	0h	1, 2, 8	3.11	3.87	4.53	1 Tinta
	4	10	1h	5, 8, 9	3.87	5.67	7.29	Tidak ada
Flexo B	1	4	0h	6, 6	0.00	0.45	4.53	Tidak ada
	2	1	0h	6, 8	0.45	1.68	4.53	Tidak ada
	3	11	1h	8, 9	1.68	2.27	7.29	Tidak ada
	4	6	0h	6, 8, 9	2.27	4.07	4.53	Tidak ada
	5	9	1h	8	4.82	5.17	7.29	2 Tinta
	6	13	1h	4, 8	5.17	5.55	7.29	Tidak ada
	7	12	1h	6, 5, 6, 8	5.55	6.60	7.29	Tidak ada
Flexo C	1	2	0h	4, 5, 6	0.00	0.48	4.53	Tidak ada
	2	8	0h	6, 4, 6	0.48	2.49	4.53	Tidak ada
	3	7	0h	2, 4, 6, 6	2.49	3.99	4.53	Tidak ada
	4	15	1h	7, 4, 6	4.74	7.03	7.29	1 Tinta

Pada Skenario 4B, *batch* kedua tiba lebih awal sehingga sebagian informasi pekerjaan telah tersedia pada tahap penjadwalan yang lebih dini. Mengacu pada Tabel 6.27-Tabel 6.29, jadwal terbaik pada skenario ini menghasilkan penalti tinta yang lebih rendah dibanding S4A, sementara *makespan* dan ketepatan waktu tetap terjaga. Visualisasi pada Gambar 6.22 menunjukkan bahwa sebagian pekerjaan *batch* kedua dapat disisipkan lebih awal dan lebih strategis, sehingga pola warna antar pekerjaan menjadi lebih konsisten. Dampak langsung dari perbaikan ini terlihat pada penurunan biaya penalti tinta dan peningkatan laba bersih. Dengan demikian, kedatangan informasi yang lebih cepat memberikan

keuntungan operasional nyata, terutama dari sisi efisiensi setup dan kinerja finansial, tanpa mengorbankan durasi penyelesaian jadwal.

6.5.6 Hasil Pengujian Jadwal Terbaik Skenario 4C

Tabel 6.31 Metrik Optimasi Skenario 4C

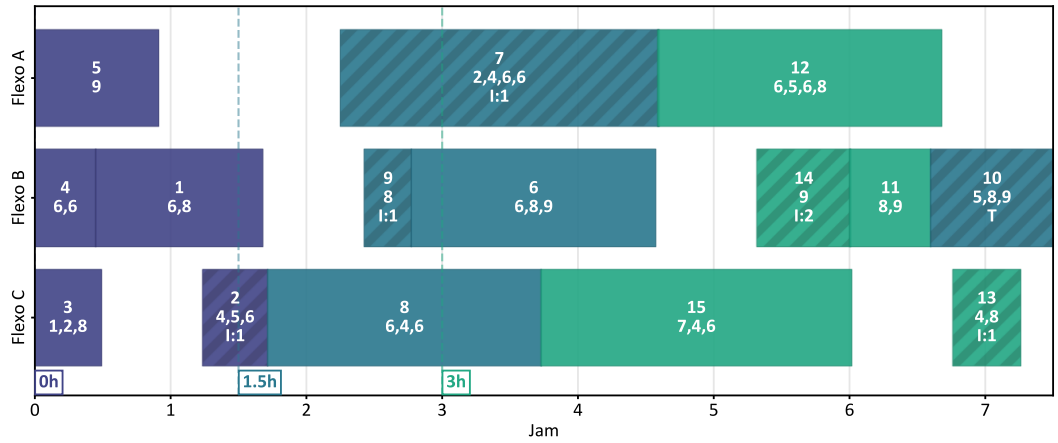
Metrik	Nilai
Fitness	4.425
Penalti Tinta	6.0
Penalti Tardy	1.0
Makespan (h)	7.50
Runtime (s)	5.94

Tabel 6.32 Metrik Utilisasi Mesin Skenario 4C

Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Flexo A	5.34	6.68	79.91	89.07
Flexo B	6.00	7.50	80.00	100.00
Flexo C	5.76	7.26	79.33	96.77

Tabel 6.33 Metrik Keuangan Skenario 4C

Metrik Keuangan	Nilai
Pendapatan	Rp 265,434,000
Penalti Tinta	Rp 4,800,000
Laba Bersih	Rp 260,634,000



Gambar 6.23 *Gantt Chart* Jadwal Skenario 4C

Tabel 6.34 Jadwal Skenario 4C

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Deadline	Penalti
Flexo A	1	5	0h	9	0.00	0.91	2.00	Tidak ada
	2	7	1.5h	2, 4, 6, 6	2.25	4.59	5.01	1 Tinta
	3	12	3h	6, 5, 6, 8	4.59	6.68	7.61	Tidak ada
Flexo B	1	4	0h	6, 6	0.00	0.45	2.00	Tidak ada
	2	1	0h	6, 8	0.45	1.68	2.00	Tidak ada
	3	9	1.5h	8	2.43	2.77	5.01	1 Tinta
	4	6	1.5h	6, 8, 9	2.77	4.57	5.01	Tidak ada
	5	14	3h	9	5.32	6.00	7.61	2 Tinta
	6	11	3h	8, 9	6.00	6.60	7.61	Tidak ada
	7	10	1.5h	5, 8, 9	6.60	7.50	5.01	Tardy
Flexo C	1	3	0h	1, 2, 8	0.00	0.49	2.00	Tidak ada
	2	2	0h	4, 5, 6	1.24	1.71	2.00	1 Tinta
	3	8	1.5h	6, 4, 6	1.71	3.73	5.01	Tidak ada
	4	15	3h	7, 4, 6	3.73	6.01	7.61	Tidak ada
	5	13	3h	4, 8	6.76	7.26	7.61	1 Tinta

Skenario 4C merupakan kasus paling kompleks dengan tiga *batch* kedatangan. Mengacu pada Tabel 6.31 dan Tabel 6.32, jadwal terbaik menunjukkan utilisasi mesin yang tinggi namun disertai dengan peningkatan pergantian tinta dan munculnya satu pekerjaan terlambat. Visualisasi pada Gambar 6.23 memperlihatkan transisi warna yang sulit dihindari serta tekanan beban yang mendekati batas kapasitas sistem. Pola ini mengindikasikan bahwa pada struktur kedatangan seperti S4C, penghilangan keterlambatan sepenuhnya kemungkinan memerlukan penambahan kapasitas atau pelonggaran *due date*, bukan sekadar perubahan urutan penjadwalan.

6.5.7 Ringkasan dan Analisis Hasil Pengujian Jadwal Terbaik

Pada skenario statis, jadwal terbaik membentuk spektrum perilaku yang jelas, mulai dari fokus kecepatan pada S1 dengan pembagian beban yang sangat merata, ekstrem minimalisasi pergantian tinta pada S2 yang menghasilkan utilisasi mesin timpang, hingga kompromi paling seimbang pada S3 yang mampu menekan penalti tinta sambil mempertahankan *makespan* dan pemerataan beban. Pola ini menunjukkan bahwa penambahan komponen tujuan secara bertahap secara langsung memengaruhi struktur jadwal dan memungkinkan optimasi yang lebih efektif ketika seluruh informasi *job* tersedia sejak awal.

Sebaliknya, pada skenario dinamis, kualitas jadwal sangat dipengaruhi oleh waktu kedatangan *batch* karena keputusan awal dibuat dengan informasi yang belum lengkap. Dalam pendekatan *rolling horizon*, penjadwalan dilakukan berbasis jendela prakiraan terbatas dan diperbarui ketika *job* baru tiba, sehingga solusi awal cenderung bersifat lebih miopik dibanding penjadwalan statis dengan visibilitas penuh (Ovacik & Uzsoy, 1995). Kondisi ini tercermin pada S4A yang menghasilkan tanpa keterlambatan namun dengan penalti tinta lebih tinggi, S4B yang membaik ketika *batch* kedua datang lebih awal sehingga fleksibilitas urutan meningkat, serta S4C yang mengungkap batas kapasitas sistem melalui meningkatnya pergantian tinta dan munculnya satu pekerjaan yang terlambat.

Ditinjau dari efisiensi komputasi, *runtime* meningkat dari skenario statis ke dinamis seiring bertambahnya frekuensi optimasi ulang, namun tetap stabil dan proporsional terhadap kompleksitas skenario. Perbedaan *runtime* antara Tabel 6.35 dan Tabel 6.11 bersifat metodologis karena perbedaan konfigurasi eksperimen, dan tidak mencerminkan perubahan kinerja algoritma. Secara keseluruhan, SA yang telah di-*tuning* mampu menghasilkan jadwal yang konsisten dan layak secara operasional, serta menjadi dasar bagi pembandingan dengan jadwal industri pada subbab berikutnya.

Tabel 6.35 Ringkasan Metrik Inti Pengujian Jadwal Terbaik Setiap Skenario

Skenario	Fitness	Makespan (h)	Penalti Tinta	Penalti Tardy	Runtime (s)
S1	5.774	5.77	-	-	3.24
S2	2.000	-	2	-	3.31
S3	2.428	6.28	2	-	3.18
S4A	3.662	7.03	5	0	4.36
S4B	3.562	7.03	4	0	5.02
S4C	4.425	7.50	6	1	5.94

6.6 Perbandingan Jadwal Hasil Optimasi dengan Jadwal Industri

Perbandingan dilakukan antara jadwal terbaik pada Skenario 3 dan jadwal aktual industri. Kedua jadwal menggunakan himpunan *job* dan mesin yang sama

sehingga perbedaan langsung mencerminkan potensi peningkatan kinerja jika perusahaan menerapkan pendekatan optimasi.

6.6.1 Hasil Perbandingan Jadwal Hasil Optimasi dengan Jadwal Industri

Tabel 6.36 Perbandingan Metrik Utama Jadwal Optimasi dengan Jadwal Industri

Metrik	Jadwal Optimasi	Jadwal Industri
Fitness	2.428	16.877
Penalti Tinta	2.0	16.0
Makespan (h)	6.28	8.77

Hasil ini menunjukkan bahwa jadwal optimasi memiliki performa yang lebih baik pada seluruh metrik utama. Perbedaan terbesar terlihat pada penalti tinta dan *makespan*, yang mengindikasikan bahwa pengurutan berbasis konsistensi warna dan pembagian beban yang lebih rapat berperan penting dalam peningkatan kualitas jadwal.

Tabel 6.37 Perbandingan Utilisasi Mesin Jadwal Optimasi dengan Jadwal Industri

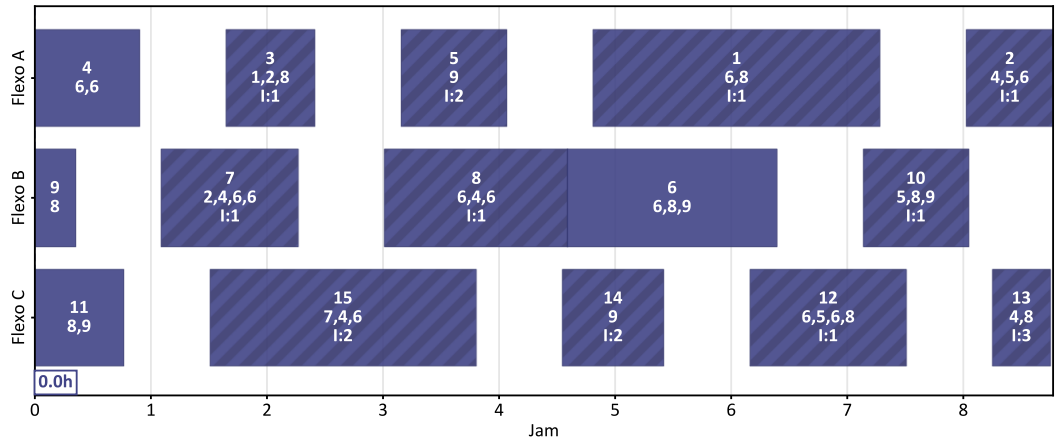
Jadwal	Mesin	Waktu Kerja (h)	Waktu Rentang (h)	Efisiensi (%)	Cakupan (%)
Optimasi	Flexo A	5.51	6.26	88.02	99.59
	Flexo B	5.50	6.25	88.00	99.47
	Flexo C	6.28	6.28	100.00	100.00
Industri	Flexo A	5.77	8.77	65.81	100.00
	Flexo B	5.79	8.04	72.02	91.64
	Flexo C	5.75	8.75	65.70	99.69

Utilisasi mesin pada jadwal optimasi lebih kompak dan efisien, sedangkan jadwal industri menunjukkan rentang waktu yang lebih panjang dengan celah *idle* yang lebih banyak.

Tabel 6.38 Perbandingan Metrik Keuangan Hasil Optimasi dengan Jadwal Industri

Metrik Keuangan	Jadwal Optimasi	Jadwal Industri
Pendapatan	Rp 265,434,000	Rp 265,434,000
Penalti Tinta	Rp 1,600,000	Rp 12,800,000
Laba Bersih	Rp 263,834,000	Rp 252,634,000

Secara finansial, perbedaan utama berasal dari penalti tinta yang jauh lebih rendah pada jadwal optimasi.



Gambar 6.24 *Gantt Chart* Jadwal Industri

Tabel 6.39 Jadwal Industri

Mesin	Urutan	ID	Batch	Level Tinta	Mulai	Selesai	Penalti
Flexo A	1	4	0h	6, 6	0.00	0.90	Tidak ada
	2	3	0h	1, 2, 8	1.65	2.41	1 Tinta
	3	5	0h	9	3.16	4.07	2 Tinta
	4	1	0h	6, 8	4.82	7.28	1 Tinta
	5	2	0h	4, 5, 6	8.03	8.77	1 Tinta
Flexo B	1	9	0h	8	0.00	0.35	Tidak ada
	2	7	0h	2, 4, 6, 6	1.10	2.27	1 Tinta
	3	8	0h	6, 4, 6	3.02	4.59	1 Tinta
	4	6	0h	6, 8, 9	4.59	6.39	Tidak ada
	5	10	0h	5, 8, 9	7.14	8.04	1 Tinta
Flexo C	1	11	0h	8, 9	0.00	0.76	Tidak ada
	2	15	0h	7, 4, 6	1.51	3.80	2 Tinta
	3	14	0h	9	4.55	5.42	2 Tinta
	4	12	0h	6, 5, 6, 8	6.17	7.50	1 Tinta
	5	13	0h	4, 8	8.25	8.75	3 Tinta

Visualisasi jadwal industri memperlihatkan perubahan tinta yang sering dan celah *idle* yang memperpanjang rentang waktu penyelesaian. Sebaliknya, jadwal optimasi pada S3 mampu mengelompokkan *job* berdasarkan level tinta sekaligus menyeimbangkan beban antar mesin.

6.6.2 Analisis Perbandingan Jadwal Optimasi dengan Jadwal Industri

Perbandingan antara jadwal optimasi pada Skenario 3 dan jadwal industri menunjukkan peningkatan kinerja yang konsisten pada seluruh metrik utama. Penalti tinta turun dari 16 menjadi 2 atau berkurang sebesar 87,%, sementara

makespan memendek dari 8,77 jam menjadi 6,28 jam sehingga waktu produksi berkurang lebih dari 2,5 jam. Peningkatan ini juga tercermin pada utilisasi mesin, yang naik dari kisaran 65%-72% pada jadwal industri menjadi 88%-100% pada jadwal optimasi, dengan celah *idle* yang jauh lebih kecil dan distribusi beban yang lebih rapat.

Peningkatan tersebut tidak terjadi secara kebetulan, melainkan merupakan konsekuensi langsung dari kemampuan metode optimasi dalam mengurutkan dan menugaskan pekerjaan secara lebih terkoordinasi dibandingkan praktik penjadwalan industri. Pada jadwal industri, urutan *job* cenderung dibentuk berdasarkan kebiasaan operasional atau pertimbangan lokal per mesin, sehingga pergantian tinta sering terjadi meskipun masih tersedia *job* lain dengan komposisi tinta yang lebih kompatibel. Sebaliknya, jadwal optimasi secara eksplisit mempertimbangkan penalti pergantian tinta dalam fungsi objektif, sehingga *job* dengan kemiripan komposisi tinta cenderung dikelompokkan dan dijalankan secara berurutan. Hal ini secara langsung menekan frekuensi dan intensitas pergantian tinta, yang tercermin pada penurunan nilai penalti yang signifikan.

Selain itu, penurunan *makespan* dan peningkatan utilisasi mesin menunjukkan bahwa metode optimasi mampu memanfaatkan kapasitas mesin secara lebih efisien. Penugasan *job* ke mesin dilakukan dengan mempertimbangkan keseimbangan beban antar mesin, sehingga waktu menganggur dapat diminimalkan dan *bottleneck* dapat dikurangi. Distribusi *job* yang lebih merata ini menyebabkan aliran produksi menjadi lebih kontinu, tanpa jeda panjang antar operasi sebagaimana yang ditemukan pada jadwal industri.

Secara finansial, kedua jadwal menghasilkan pendapatan yang sama karena himpunan *job* yang diproses identik. Namun, laba bersih pada jadwal optimasi meningkat dari Rp 252.634.000 menjadi Rp 263.834.000. Selisih ini sepenuhnya berasal dari penurunan biaya penalti tinta, yang turun dari Rp 12.800.000 pada jadwal industri menjadi Rp 1.600.000 pada jadwal optimasi. Dengan demikian, peningkatan kinerja finansial dicapai bukan melalui peningkatan volume produksi, melainkan melalui efisiensi operasional yang lebih baik, khususnya dalam pengendalian pergantian tinta dan pemanfaatan waktu mesin.

Hasil ini sejalan dengan penelitian terdahulu pada konteks flexografi dengan dataset yang sama. Hutomo (2022) menunjukkan bahwa pengendalian pergantian warna mampu menurunkan penalti dari 14 menjadi 0 dan meningkatkan nilai optimal dari Rp 253.114.000 menjadi Rp 265.434.000. Hanif et al. (2024) juga melaporkan bahwa seluruh eksperimen menghasilkan pendapatan yang sama sebesar Rp 265.434.000, sehingga perbedaan kualitas jadwal terutama tercermin pada aspek operasional terkait pergantian tinta. Perbedaan nilai penalti minimum yang dicapai pada penelitian ini dapat dijelaskan oleh perbedaan pemodelan tinta, karena Hutomo (2022) dan Hanif et al. (2024) memodelkan satu warna per *job*, sedangkan penelitian ini menggunakan komposisi *multi-ink per job*. Struktur *multi-ink* tersebut secara alami membatasi kemungkinan membentuk urutan *job* dengan kesamaan warna penuh, sehingga pencapaian penalti nol menjadi lebih sulit, meskipun arah dan mekanisme perbaikannya tetap konsisten.

6.7 Ringkasan Pengujian

Dari rangkaian pengujian yang dilakukan, diperoleh gambaran menyeluruh mengenai pengaruh konfigurasi SA, bobot tujuan, dan penetapan *due date* terhadap kinerja penjadwalan mesin cetak flexografi pada lingkungan statis dan dinamis. Pengujian parameter pada Skenario 3 menghasilkan konfigurasi terbaik dengan *max iteration* 250.000, *alpha* 0,995, p_o 0,8, *floor ratio* 0,05, dan *critical machine bias* 1,0. Konfigurasi ini memberikan nilai *fitness* terendah, stabilitas solusi yang tinggi, serta *runtime* yang relatif efisien, sehingga dipilih sebagai *baseline* untuk seluruh pengujian selanjutnya.

Dari pengujian bobot tujuan, terlihat bahwa karakter *trade off* berbeda antara lingkungan statis dan dinamis. Pada skenario statis, bobot yang menekankan minimalisasi penalti tinta (0,90) menghasilkan kinerja terbaik karena pengurangan perubahan tinta secara simultan menurunkan SDST dan memperbaiki *makespan*. Sebaliknya, pada skenario dinamis, kombinasi bobot (0,45; 0,10; 0,45) menjadi yang paling efektif karena mampu menghilangkan keterlambatan sekaligus menjaga penalti tinta tetap pada tingkat moderat. Sementara itu, pengujian *due date multiplier* menunjukkan bahwa nilai 0,8 terlalu ketat dan 1,2 terlalu longgar, sehingga *multiplier* 1,0 menjadi konfigurasi paling seimbang dan layak dijadikan *baseline*.

Dari hasil pengujian skenario, diperoleh pola kinerja yang konsisten. Skenario statis (S1-S3) menunjukkan stabilitas yang sangat tinggi dengan variasi antar *seed* mendekati nol. Pada skenario dinamis (S4A-S4C), ketidaklengkapan informasi dan kedatangan *batch* secara bertahap meningkatkan kompleksitas masalah, yang tercermin pada kenaikan *makespan* dan penalti tinta, serta kebutuhan komputasi yang lebih besar. Di antara skenario dinamis tersebut, S4B memberikan performa terbaik karena struktur informasi yang lebih menguntungkan, menghasilkan penalti tinta rendah dan *tardy* nol, sedangkan S4C menunjukkan batas kapasitas sistem melalui satu pekerjaan yang secara konsisten terlambat.

Ditinjau dari efisiensi komputasi, *runtime* pada penelitian ini dibedakan antara eksekusi tunggal untuk menghasilkan satu jadwal dan pengujian statistik berbasis multi *run*. Pada eksekusi tunggal, skenario statis memerlukan sekitar 3.18 hingga 3.31 detik, sedangkan skenario dinamis berada pada kisaran 4.36 hingga 5.94 detik akibat optimasi ulang pada setiap *horizon* ketika *batch* baru datang. Pada pengujian statistik, *runtime* meningkat karena eksekusi berulang dan overhead *multiprocessing*, dengan rata-rata sekitar 5.52 hingga 6.05 detik untuk skenario statis dan 7.51 hingga 10.50 detik untuk skenario dinamis. Peningkatan ini bersifat metodologis dan tidak mencerminkan kebutuhan operasional, karena dalam praktik *runtime* yang relevan adalah eksekusi tunggal pada momen kedatangan *batch*, yang berada pada orde detik dan sangat kecil dibandingkan durasi produksi pada orde jam, sehingga pendekatan yang diusulkan tetap praktis sebagai sistem pendukung keputusan di lantai produksi.

Dari perbandingan dengan jadwal industri, diperoleh potensi peningkatan kinerja yang signifikan. Penalti tinta berhasil diturunkan dari 16 menjadi 2 atau

87,5%, *makespan* berkurang dari 8,77 jam menjadi 6,28 jam atau sekitar 2,5 jam, efisiensi mesin meningkat dari sekitar 65%-72% menjadi 88%-100%, dan laba bersih naik dari Rp 252,634,000 menjadi Rp 263,834,000 atau meningkat sekitar Rp 11,200,000. Secara keseluruhan, hasil pengujian menunjukkan bahwa pendekatan SA yang telah di-*tuning* mampu menghasilkan jadwal yang lebih efisien secara operasional, stabil, dan layak secara komputasional dibandingkan praktik penjadwalan manual.

6.8 Keterbatasan Model

Keterbatasan model terletak pada efisiensi konvergensi SA pada fase temperatur rendah. Meskipun SA efektif dalam eksplorasi ruang solusi pada tahap awal, proses pencarian pada fase akhir masih didominasi pembangkitan *neighborhood* yang bersifat stokastik, sehingga perbaikan solusi berlangsung gradual dan berpotensi menimbulkan osilasi nilai *fitness*. Untuk mengurangi fenomena ini, penelitian ini menerapkan *critical machine bias* yang mengarahkan *move* ke mesin *bottleneck* penyumbang *makespan* terbesar. Hasil pengujian menunjukkan bahwa mekanisme ini menurunkan variasi solusi dan mempercepat stabilisasi, meskipun pada temperatur rendah perbaikan solusi akhir masih berlangsung lama.

Keterbatasan lain berkaitan dengan konsistensi efek penggunaan *due date multiplier* statis pada berbagai pola dinamika kedatangan pekerjaan. Nilai pengali satu digunakan sebagai *baseline* kondisi tanpa keterlambatan, dengan penetapan due date yang telah mempertimbangkan estimasi beban kerja melalui aturan TWK yang dimodifikasi. Namun, ketika diterapkan pada skenario dinamis dengan karakteristik kedatangan berbeda, tingkat kelonggaran *due date* dapat bervariasi, yang mencerminkan keterbatasan pendekatan statis dalam merepresentasikan kelonggaran yang seragam pada lingkungan produksi yang kompleks.

BAB 7 PENUTUP

7.1 Kesimpulan

Berdasarkan tujuan dan rumusan masalah yang telah ditetapkan, maka kesimpulan penelitian ini adalah sebagai berikut:

1. Permasalahan penjadwalan pencetakan flexografi dapat diformulasikan secara efektif sebagai UPMSP-SDST dan diselesaikan menggunakan algoritma SA yang disesuaikan dengan karakteristik produksi. Model penjadwalan yang dikembangkan mengintegrasikan tiga tujuan operasional utama, yaitu penalti pergantian tinta, *makespan*, dan *tardiness*, sehingga mampu menangani kondisi statis maupun dinamis berbasis *rolling horizon*. Kerangka SA yang digunakan, yang mencakup operator *neighborhood*, mekanisme penerimaan probabilistik, jadwal pendinginan, serta *critical machine bias*, terbukti stabil untuk penjadwalan multiobjektif pada sistem produksi flexografi.
2. Hasil pengujian menunjukkan bahwa penjadwal berbasis SA memiliki *robustness* yang tinggi dengan variabilitas hasil yang rendah pada seluruh skenario. Dibandingkan jadwal industri, solusi SA memberikan peningkatan kinerja yang signifikan, ditunjukkan oleh penurunan nilai *fitness* dari 16.877 menjadi 2.428, pengurangan penalti pergantian tinta dari 16 menjadi 2, serta penurunan *makespan* dari 8.77 jam menjadi 6.28 jam. Pencapaian ini setara dengan penurunan sekitar 87.5% jumlah pergantian tinta dan percepatan waktu produksi sekitar 2.5 jam, disertai peningkatan pemanfaatan mesin dan penurunan biaya pergantian tinta dari Rp 12,800,000 menjadi Rp 1.600.000. Dari sisi efisiensi komputasi, *runtime* algoritma berada pada orde detik, sehingga overhead komputasi tidak signifikan dibandingkan durasi produksi aktual.

7.2 Saran

Berdasarkan hasil penelitian, masih terdapat beberapa keterbatasan yang perlu diperhatikan sebagai dasar pengembangan penelitian selanjutnya. Salah satu keterbatasan utama terletak pada penggunaan dataset yang masih terbatas, sehingga penelitian lanjutan disarankan untuk menggunakan dataset yang lebih besar dan lebih beragam guna menguji *scalability* serta meningkatkan validitas generalisasi penjadwal pada berbagai karakteristik produksi. Selain itu, proses pencarian pada suhu rendah masih menunjukkan osilasi yang relatif lama karena perbaikan *makespan* memerlukan *move* spesifik sementara pembangkitan *neighbor* bersifat stokastik, penelitian berikutnya disarankan meningkatkan efektivitas pembangkitan *neighbor* melalui *tuning* probabilitas pemilihan *move* dan penyusunan set operator *neighborhood* yang lebih sesuai agar konvergensi lebih cepat dan stabil. Selanjutnya, mengingat penetapan *due date* dengan *baseline multiplier* bernilai 1 masih menghasilkan keterlambatan pada kasus tertentu, penelitian lanjutan disarankan mengembangkan heuristik penetapan *due date* berbasis *workload* yang lebih adaptif.

DAFTAR REFERENSI

Alimian, M., Ghezavati, V., Tavakkoli-Moghaddam, R. & Ramezanian, R., 2022. Solving a parallel-line capacitated lot-sizing and scheduling problem with sequence-dependent setup time/cost and preventive maintenance by a rolling horizon method. *Computers & Industrial Engineering*, 168, p.108041.

Allahverdi, A., Ng, C.T., Cheng, T.C.E. & Kovalyov, M.Y., 2008. A survey of scheduling problems with setup times or costs. *European Journal of Operational Research*, [online] 187(3), pp.985–1032. https://doi.org/https://doi.org/10.1016/j.ejor.2006.06.060.

Baker, K.R. & Scudder, G.D., 1990. Sequencing with earliness and tardiness penalties: a review. *Operations research*, 38(1), pp.22–36.

Delahaye, D., Chaimatanan, S. & Mongeau, M., 2019. Simulated Annealing: From Basics to Applications. Dalam: M. Gendreau and J.-Y. Potvin, eds. *Handbook of Metaheuristics*. [online] Cham: Springer International Publishing. pp.1–35. https://doi.org/10.1007/978-3-319-91086-4_1.

Hanif, H., Daud, N., Kurnianingtyas, D., Hutomo, I.K. & Gunawan, I., 2024. Cost-effective job sequence optimization for minimizing downtime in flexographic printing with puma optimizer algorithm. Dalam: *2024 IEEE International Conference on Communication, Networks and Satellite (COMNETSAT)*. IEEE. pp.15–20.

Hutomo, I.K., 2022. *Model Optimasi Penjadwalan Pesanan pada Proses Pencetakan Gambar untuk Kemasan Kotak Karton Gelombang dengan Mempertimbangkan Tingkatan Warna*. Report. Universitas Katolik Widya Mandala Surabaya.

Iori, M., Locatelli, A. & Locatelli, M., 2021. Scheduling of parallel print machines with sequence-dependent setup costs: a real-world case study. Dalam: *IFIP International Conference on Advances in Production Management Systems*. Springer. pp.637–645.

Iori, M., Locatelli, A. & Locatelli, M., 2023. A GRASP for a real-world scheduling problem with unrelated parallel print machines and sequence-dependent setup times. *International Journal of Production Research*, 61(21), pp.7367–7385.

Kipphan, H., 2014. *Handbook of print media: technologies and production methods*. Springer.

Kirkpatrick, S., Gelatt Jr, C.D. & Vecchi, M.P., 1983. Optimization by simulated annealing. *science*, 220(4598), pp.671–680.

Lee, Y.H. & Pinedo, M., 1997. Scheduling jobs on parallel machines with sequence-dependent setup times. *European Journal of Operational Research*, [online] 100(3), pp.464–474. https://doi.org/https://doi.org/10.1016/S0377-2217(95)00376-2.

Li, H., Zheng, Y., Sun, B. & Du, B., 2024. A Job Sequence Optimization Approach for Parallel Machine Scheduling Problem in Printing Manufacturing Systems. *IEEE Access*, 12, pp.63462–63476. https://doi.org/10.1109/ACCESS.2024.3396455.

Li, K., Shi, Y., Yang, S. & Cheng, B., 2011. Parallel machine scheduling problem to minimize the makespan with resource dependent processing times. *Applied Soft Computing*, [online] 11(8), pp.5551–5557. https://doi.org/https://doi.org/10.1016/j.asoc.2011.05.005.

Moser, M., Musliu, N., Schaerf, A. & Winter, F., 2022. Exact and metaheuristic approaches for unrelated parallel machine scheduling. *Journal of Scheduling*, [online] 25(5), pp.507–534. https://doi.org/10.1007/s10951-021-00714-6.

Ouazene, Y., Yalaoui, F., Chehade, H. & Yalaoui, A., 2014. Analysis of Different Criteria for Workload Balancing on Identical Parallel Machines. Dalam: *Proceedings of the 2014 International Conference on Industrial Engineering and Operations Management*. pp.1822–1829.

Ovacik, I.M. & Uzsoy, R., 1995. Rolling horizon procedures for dynamic parallel machine scheduling with sequence-dependent setup times. *International journal of production research*, 33(11), pp.3173–3192.

Pinedo, M.L., 2022. *Scheduling: Theory, Algorithms, and Systems*. [online] Springer International Publishing. Tersedia di: <https://books.google.co.id/books?id=O_ChEAAAQBAJ>.

Ramis Ferrer, B., Muhammad, U., Mohammed, W.M. & Martínez Lastra, J.L., 2018. Implementing and visualizing ISO 22400 key performance indicators for monitoring discrete manufacturing systems. *Machines*, 6(3), p.39.

Said, A.A., 2006. Dasar Desain Dwimatra. Dalam: *DIMENSI WARNA*. [online] Makassar: Badan Penerbit UNM Makassar. pp.91–116. Tersedia di: <http://eprints.unm.ac.id/id/eprint/4228> [Diakses 30 September 2025].

Schuurman, J. & Vuuren, J., 2016. Scheduling sequence-dependent colour printing jobs. *South African Journal of Industrial Engineering*, 27. https://doi.org/10.7166/27-2-1119.

Shirazi, H., Kia, R., Javadian, N. & Tavakkoli-Moghaddam, R., 2014. An archived multi-objective simulated annealing for a dynamic cellular manufacturing system. *Journal of Industrial Engineering International*, [online] 10(2), p.58. https://doi.org/10.1007/s40092-014-0058-6.

Suman, B. & Kumar, P., 2006. A survey of simulated annealing as a tool for single and multiobjective optimization. *Journal of the operational research society*, 57(10), pp.1143–1160.

Talbi, E.G., 2009. *Metaheuristics: From Design to Implementation*. Wiley Series on Parallel and Distributed Computing. [online] Wiley. Tersedia di: <https://books.google.co.id/books?id=SIsa6zi5XV8C>.

LAMPIRAN

Lampiran A Cuplikan Hasil Mentah Pengujian Parameter SA

Parameter	Nilai	Seed	Fitness	Makespan	Penalti Tinta	Penalti Tardy	Runtime
max_iter	5000	42	2.45	6.47	2	0	0.11
max_iter	5000	43	2.44	6.43	2	0	0.13
max_iter	5000	44	2.46	6.62	2	0	0.12
max_iter	5000	45	2.44	6.39	2	0	0.14
max_iter	5000	46	2.46	6.62	2	0	0.13
max_iter	5000	47	2.44	6.39	2	0	0.13
max_iter	5000	48	2.46	6.62	2	0	0.13
max_iter	5000	49	2.44	6.43	2	0	0.12
max_iter	5000	50	2.45	6.51	2	0	0.11
max_iter	5000	51	2.44	6.39	2	0	0.14
max_iter	5000	52	2.48	6.81	2	0	0.12
max_iter	5000	53	2.44	6.39	2	0	0.12
max_iter	5000	54	2.43	6.28	2	0	0.13
max_iter	5000	55	2.44	6.40	2	0	0.12
max_iter	5000	56	2.45	6.51	2	0	0.12
max_iter	5000	57	2.45	6.51	2	0	0.13
max_iter	5000	58	2.44	6.39	2	0	0.15
max_iter	5000	59	2.47	6.66	2	0	0.13
max_iter	5000	60	2.44	6.39	2	0	0.10
max_iter	5000	61	2.44	6.39	2	0	0.12
max_iter	5000	62	2.46	6.56	2	0	0.15
max_iter	5000	63	2.45	6.47	2	0	0.14
max_iter	5000	64	2.46	6.62	2	0	0.11
max_iter	5000	65	2.46	6.55	2	0	0.10
max_iter	5000	66	2.46	6.57	2	0	0.11
max_iter	5000	67	2.44	6.39	2	0	0.10
max_iter	5000	68	2.45	6.51	2	0	0.11
max_iter	5000	69	2.44	6.39	2	0	0.13
max_iter	5000	70	2.47	6.66	2	0	0.13
max_iter	5000	71	2.45	6.53	2	0	0.13
...	...	...	...	...	...	...	...

Lampiran B Cuplikan Hasil Mentah Pengujian Bobot Tujuan

Skenario	wt	wi	wm	Seed	Fitness	Makespan	Penalti Tinta	Penalti Tardy
S3	0	0.05	0.95	42	6.35	6.47	4	0
S3	0	0.05	0.95	43	6.47	6.60	4	0
S3	0	0.05	0.95	44	6.44	6.57	4	0
S3	0	0.05	0.95	45	6.48	6.71	2	0
S3	0	0.05	0.95	46	6.42	6.54	4	0
S3	0	0.05	0.95	47	6.48	6.66	3	0
S3	0	0.05	0.95	48	6.57	6.70	4	0
S3	0	0.05	0.95	49	6.52	6.65	4	0
S3	0	0.05	0.95	50	6.53	6.61	5	0
S3	0	0.05	0.95	51	6.23	6.40	3	0
S3	0	0.05	0.95	52	6.56	6.70	4	0
S3	0	0.05	0.95	53	6.62	6.76	4	0
S3	0	0.05	0.95	54	6.58	6.66	5	0
S3	0	0.05	0.95	55	6.47	6.60	4	0
S3	0	0.05	0.95	56	6.53	6.66	4	0
S3	0	0.05	0.95	57	6.43	6.50	5	0
S3	0	0.05	0.95	58	6.48	6.56	5	0
S3	0	0.05	0.95	59	6.50	6.69	3	0
S3	0	0.05	0.95	60	6.38	6.56	3	0
S3	0	0.05	0.95	61	6.37	6.49	4	0
S3	0	0.05	0.95	62	6.38	6.55	3	0
S3	0	0.05	0.95	63	6.55	6.68	4	0
S3	0	0.05	0.95	64	6.63	6.71	5	0
S3	0	0.05	0.95	65	6.53	6.66	4	0
S3	0	0.05	0.95	66	6.57	6.70	4	0
S3	0	0.05	0.95	67	6.51	6.70	3	0
S3	0	0.05	0.95	68	6.61	6.70	5	0
S3	0	0.05	0.95	69	6.55	6.74	3	0
S3	0	0.05	0.95	70	6.54	6.73	3	0
S3	0	0.05	0.95	71	6.61	6.70	5	0
...	...	...	...	...	...	...	...	...

Lampiran C Cuplikan Hasil Mentah Pengujian *Due Date Multiplier*

Skenario	Seed	Due Date Multiplier	Fitness	Makespan	Penalti Tinta	Penalti Tardy	Runtime
S4A	42	0.8	5.17	7.15	6	3	7.27
S4A	43	0.8	5.17	7.15	6	3	7.33
S4A	44	0.8	5.26	8.36	6	2	6.96
S4A	45	0.8	5.17	7.15	6	3	7.26
S4A	46	0.8	5.26	8.36	6	2	7.12
S4A	47	0.8	5.26	8.36	6	2	7.15
S4A	48	0.8	5.26	8.36	6	2	5.86
S4A	49	0.8	5.26	8.36	6	2	5.95
S4A	50	0.8	5.17	7.15	6	3	5.98
S4A	51	0.8	5.17	7.15	6	3	5.94
S4A	52	0.8	5.26	8.36	6	2	5.78
S4A	53	0.8	5.17	7.15	6	3	6.13
S4A	54	0.8	5.17	7.15	6	3	6.01
S4A	55	0.8	5.17	7.15	6	3	6.06
S4A	56	0.8	5.17	7.15	6	3	6.08
S4A	57	0.8	5.17	7.15	6	3	5.87
S4A	58	0.8	5.17	7.15	6	3	6.00
S4A	59	0.8	5.26	8.36	6	2	6.29
S4A	60	0.8	5.26	8.36	6	2	5.72
S4A	61	0.8	5.26	8.36	6	2	5.66
S4A	62	0.8	5.17	7.15	6	3	5.87
S4A	63	0.8	5.17	7.15	6	3	5.81
S4A	64	0.8	5.26	8.36	6	2	5.71
S4A	65	0.8	5.17	7.15	6	3	6.06
S4A	66	0.8	5.26	8.36	6	2	5.66
S4A	67	0.8	5.26	8.36	6	2	5.78
S4A	68	0.8	5.17	7.15	6	3	5.86
S4A	69	0.8	5.17	7.15	6	3	5.76
S4A	70	0.8	5.26	8.36	6	2	5.75
S4A	71	0.8	5.26	8.36	6	2	6.12
...	...	...	...	...	...	...	...

Lampiran D Cuplikan Hasil Mentah Pengujian Skenario

Skenario	Seed	Fitness	Makespan	Penalti Tinta	Penalti Tardy	Runtime
S1	42	5.78	5.78	14	0	4.57
S1	43	5.78	5.78	12	0	4.64
S1	44	5.77	5.77	10	0	4.53
S1	45	5.78	5.78	11	0	4.70
S1	46	5.78	5.78	11	0	4.67
S1	47	5.78	5.78	12	0	4.51
S1	48	5.77	5.77	11	0	4.67
S1	49	5.77	5.77	12	0	4.79
S1	50	5.77	5.77	14	0	4.64
S1	51	5.77	5.77	10	0	4.76
S1	52	5.77	5.77	14	0	5.08
S1	53	5.77	5.77	10	0	4.87
S1	54	5.77	5.77	10	0	4.65
S1	55	5.77	5.77	13	0	4.71
S1	56	5.78	5.78	13	0	4.58
S1	57	5.77	5.77	12	0	4.60
S1	58	5.77	5.77	14	0	4.74
S1	59	5.77	5.77	12	0	5.04
S1	60	5.78	5.78	11	0	4.64
S1	61	5.78	5.78	11	0	4.64
S1	62	5.77	5.77	12	0	4.66
S1	63	5.78	5.78	12	0	4.73
S1	64	5.77	5.77	11	0	4.94
S1	65	5.77	5.77	11	0	4.87
S1	66	5.78	5.78	13	0	4.93
S1	67	5.77	5.77	11	0	5.02
S1	68	5.77	5.77	10	0	5.04
S1	69	5.77	5.77	12	0	4.96
S1	70	5.77	5.77	9	0	4.92
S1	71	5.77	5.77	15	0	4.93
...	...	...	...	...	...	...

Lampiran E Statistik Hasil Pengujian Bobot Pada Skenario 4B

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.05	0.05	0.9	6.61	3.83	2.03	8.56	1.57
0.05	0.1	0.85	6.70	3.10	1.67	7.10	1.13
0.05	0.15	0.8	6.80	3.67	1.03	6.41	0.51
0.05	0.2	0.75	6.81	3.63	1.00	6.31	0.47
0.05	0.25	0.7	7.03	3.00	1.03	5.77	0.37
0.05	0.3	0.65	7.03	3.00	1.00	5.70	0.00
0.05	0.35	0.6	7.03	3.00	1.00	5.70	0.00
0.05	0.4	0.55	7.03	3.00	1.00	5.70	0.00
0.05	0.45	0.5	7.03	3.00	1.00	5.70	0.00
0.05	0.5	0.45	7.03	3.00	1.00	5.70	0.00
0.05	0.55	0.4	7.03	3.00	1.00	5.70	0.00
0.05	0.6	0.35	7.03	3.00	1.00	5.70	0.00
0.05	0.65	0.3	7.03	3.00	1.00	5.70	0.00
0.05	0.7	0.25	7.03	3.00	1.00	5.70	0.00
0.05	0.75	0.2	7.03	3.00	1.00	5.70	0.00
0.05	0.8	0.15	7.03	3.00	1.00	5.70	0.00
0.05	0.85	0.1	7.03	3.00	1.00	5.70	0.00
0.05	0.9	0.05	7.03	3.00	1.00	5.70	0.00
0.1	0.05	0.85	6.68	4.27	0.90	6.73	0.98
0.1	0.1	0.8	6.70	3.57	1.20	6.64	1.19
0.1	0.15	0.75	6.82	3.63	1.00	6.31	0.54
0.1	0.2	0.7	6.80	3.67	1.00	6.35	0.46
0.1	0.25	0.65	7.03	3.00	1.00	5.70	0.00
0.1	0.3	0.6	7.03	3.00	1.00	5.70	0.00
0.1	0.35	0.55	7.03	3.00	1.00	5.70	0.00
0.1	0.4	0.5	7.03	3.00	1.00	5.70	0.00
0.1	0.45	0.45	7.03	3.00	1.00	5.70	0.00
0.1	0.5	0.4	7.03	3.00	1.00	5.70	0.00
0.1	0.55	0.35	7.03	3.00	1.00	5.70	0.00
0.1	0.6	0.3	7.03	3.00	1.00	5.70	0.00
0.1	0.65	0.25	7.03	3.00	1.00	5.70	0.00

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.1	0.7	0.2	7.03	3.00	1.00	5.70	0.00
0.1	0.75	0.15	7.03	3.00	1.00	5.70	0.00
0.1	0.8	0.1	7.03	3.00	1.00	5.70	0.00
0.1	0.85	0.05	7.03	3.00	1.00	5.70	0.00
0.15	0.05	0.8	6.69	4.40	0.83	6.74	0.69
0.15	0.1	0.75	6.69	3.80	0.83	6.14	1.20
0.15	0.15	0.7	6.84	3.63	1.00	6.32	0.48
0.15	0.2	0.65	6.83	3.57	1.00	6.25	0.49
0.15	0.25	0.6	7.03	3.00	1.00	5.70	0.00
0.15	0.3	0.55	7.03	3.00	1.00	5.70	0.00
0.15	0.35	0.5	7.03	3.00	1.00	5.70	0.00
0.15	0.4	0.45	7.03	3.00	1.00	5.70	0.00
0.15	0.45	0.4	7.03	3.00	1.00	5.70	0.00
0.15	0.5	0.35	7.03	3.00	1.00	5.70	0.00
0.15	0.55	0.3	7.03	3.00	1.00	5.70	0.00
0.15	0.6	0.25	7.03	3.00	1.00	5.70	0.00
0.15	0.65	0.2	7.03	3.00	1.00	5.70	0.00
0.15	0.7	0.15	7.03	3.00	1.00	5.70	0.00
0.15	0.75	0.1	7.03	3.00	1.00	5.70	0.00
0.15	0.8	0.05	7.03	3.00	1.00	5.70	0.00
0.2	0.05	0.75	6.70	4.30	0.83	6.64	0.77
0.2	0.1	0.7	6.82	4.23	0.67	6.25	0.55
0.2	0.15	0.65	6.84	3.67	0.93	6.22	0.49
0.2	0.2	0.6	6.81	3.67	1.00	6.35	0.46
0.2	0.25	0.55	7.03	3.00	1.00	5.70	0.00
0.2	0.3	0.5	7.03	3.00	1.00	5.70	0.00
0.2	0.35	0.45	7.03	3.00	1.00	5.70	0.00
0.2	0.4	0.4	7.03	3.00	1.00	5.70	0.00
0.2	0.45	0.35	7.03	3.00	1.00	5.70	0.00
0.2	0.5	0.3	7.03	3.00	1.00	5.70	0.00
0.2	0.55	0.25	7.03	3.00	1.00	5.70	0.00
0.2	0.6	0.2	7.03	3.00	1.00	5.70	0.00

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.2	0.65	0.15	7.03	3.00	1.00	5.70	0.00
0.2	0.7	0.1	7.03	3.00	1.00	5.70	0.00
0.2	0.75	0.05	7.03	3.00	1.00	5.70	0.00
0.25	0.05	0.7	6.75	5.13	0.37	6.54	1.17
0.25	0.1	0.65	7.03	4.23	0.00	4.94	0.43
0.25	0.15	0.6	7.03	3.60	0.50	5.30	0.50
0.25	0.2	0.55	7.03	3.67	0.33	5.04	0.48
0.25	0.25	0.5	7.03	3.27	0.73	5.44	0.45
0.25	0.3	0.45	7.03	3.00	1.00	5.70	0.00
0.25	0.35	0.4	7.03	3.00	1.00	5.70	0.00
0.25	0.4	0.35	7.03	3.00	1.00	5.70	0.00
0.25	0.45	0.3	7.03	3.00	1.00	5.70	0.00
0.25	0.5	0.25	7.03	3.00	1.00	5.70	0.00
0.25	0.55	0.2	7.03	3.00	1.00	5.70	0.00
0.25	0.6	0.15	7.03	3.00	1.00	5.70	0.00
0.25	0.65	0.1	7.03	3.00	1.00	5.70	0.00
0.25	0.7	0.05	7.03	3.00	1.00	5.70	0.00
0.3	0.05	0.65	6.88	4.93	0.00	5.62	1.31
0.3	0.1	0.6	7.03	4.37	0.00	5.07	0.49
0.3	0.15	0.55	7.03	4.23	0.03	5.00	0.47
0.3	0.2	0.5	7.03	3.43	0.57	5.27	0.50
0.3	0.25	0.45	7.03	3.70	0.30	5.00	0.47
0.3	0.3	0.4	7.03	3.27	0.73	5.44	0.45
0.3	0.35	0.35	7.03	3.00	1.00	5.70	0.00
0.3	0.4	0.3	7.03	3.00	1.00	5.70	0.00
0.3	0.45	0.25	7.03	3.00	1.00	5.70	0.00
0.3	0.5	0.2	7.03	3.00	1.00	5.70	0.00
0.3	0.55	0.15	7.03	3.00	1.00	5.70	0.00
0.3	0.6	0.1	7.03	3.00	1.00	5.70	0.00
0.3	0.65	0.05	7.03	3.00	1.00	5.70	0.00
0.35	0.05	0.6	6.84	5.57	0.00	6.25	1.48
0.35	0.1	0.55	7.03	4.40	0.00	5.10	0.50

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.35	0.15	0.5	7.03	4.27	0.00	4.97	0.45
0.35	0.2	0.45	7.03	3.67	0.33	5.04	0.48
0.35	0.25	0.4	7.03	3.60	0.40	5.10	0.50
0.35	0.3	0.35	7.03	3.60	0.40	5.10	0.50
0.35	0.35	0.3	7.03	3.20	0.80	5.50	0.41
0.35	0.4	0.25	7.03	3.00	1.00	5.70	0.00
0.35	0.45	0.2	7.03	3.00	1.00	5.70	0.00
0.35	0.5	0.15	7.03	3.00	1.00	5.70	0.00
0.35	0.55	0.1	7.03	3.00	1.00	5.70	0.00
0.35	0.6	0.05	7.03	3.00	1.00	5.70	0.00
0.4	0.05	0.55	6.81	5.53	0.00	6.21	1.50
0.4	0.1	0.5	7.03	4.30	0.00	5.00	0.47
0.4	0.15	0.45	7.03	4.47	0.00	5.17	0.51
0.4	0.2	0.4	7.03	4.43	0.07	5.27	0.50
0.4	0.25	0.35	7.03	3.73	0.27	4.97	0.45
0.4	0.3	0.3	7.03	3.70	0.30	5.00	0.47
0.4	0.35	0.25	7.03	3.63	0.37	5.07	0.49
0.4	0.4	0.2	7.03	3.17	0.83	5.54	0.38
0.4	0.45	0.15	7.03	3.00	1.00	5.70	0.00
0.4	0.5	0.1	7.03	3.00	1.00	5.70	0.00
0.4	0.55	0.05	7.03	3.00	1.00	5.70	0.00
0.45	0.05	0.5	6.83	4.93	0.00	5.62	1.23
0.45	0.1	0.45	7.03	4.17	0.00	4.87	0.38
0.45	0.15	0.4	7.03	4.53	0.00	5.24	0.51
0.45	0.2	0.35	7.03	4.50	0.00	5.20	0.51
0.45	0.25	0.3	7.03	3.47	0.53	5.24	0.51
0.45	0.3	0.25	7.03	3.43	0.57	5.27	0.50
0.45	0.35	0.2	7.03	3.47	0.53	5.24	0.51
0.45	0.4	0.15	7.03	3.47	0.53	5.24	0.51
0.45	0.45	0.1	7.03	3.27	0.73	5.44	0.45
0.45	0.5	0.05	7.03	3.00	1.00	5.70	0.00
0.5	0.05	0.45	6.89	4.37	0.00	5.06	0.50

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.5	0.1	0.4	7.03	4.50	0.00	5.20	0.51
0.5	0.15	0.35	7.03	4.57	0.00	5.27	0.50
0.5	0.2	0.3	7.03	4.33	0.00	5.04	0.48
0.5	0.25	0.25	7.03	4.30	0.13	5.27	0.50
0.5	0.3	0.2	7.03	3.43	0.57	5.27	0.50
0.5	0.35	0.15	7.03	3.57	0.43	5.14	0.50
0.5	0.4	0.1	7.03	3.43	0.57	5.27	0.50
0.5	0.45	0.05	7.03	3.67	0.33	5.04	0.48
0.55	0.05	0.4	6.91	4.37	0.00	5.06	0.50
0.55	0.1	0.35	7.03	4.37	0.00	5.07	0.49
0.55	0.15	0.3	7.03	4.37	0.00	5.07	0.49
0.55	0.2	0.25	7.03	4.47	0.00	5.17	0.51
0.55	0.25	0.2	7.03	4.47	0.00	5.17	0.51
0.55	0.3	0.15	7.03	3.50	0.50	5.20	0.51
0.55	0.35	0.1	7.03	3.47	0.53	5.24	0.51
0.55	0.4	0.05	7.03	3.43	0.57	5.27	0.50
0.6	0.05	0.35	7.03	4.43	0.00	5.14	0.50
0.6	0.1	0.3	7.03	4.43	0.00	5.14	0.50
0.6	0.15	0.25	7.03	4.30	0.00	5.00	0.47
0.6	0.2	0.2	7.03	4.60	0.00	5.30	0.50
0.6	0.25	0.15	7.03	4.50	0.00	5.20	0.51
0.6	0.3	0.1	7.03	4.37	0.07	5.20	0.51
0.6	0.35	0.05	7.03	3.40	0.60	5.30	0.50
0.65	0.05	0.3	7.03	4.30	0.00	5.00	0.47
0.65	0.1	0.25	7.03	4.53	0.00	5.24	0.51
0.65	0.15	0.2	7.03	4.30	0.00	5.00	0.47
0.65	0.2	0.15	7.03	4.57	0.00	5.27	0.50
0.65	0.25	0.1	7.03	4.40	0.00	5.10	0.50
0.65	0.3	0.05	7.03	4.50	0.00	5.20	0.51
0.7	0.05	0.25	7.03	4.33	0.00	5.04	0.48
0.7	0.1	0.2	7.03	4.40	0.00	5.10	0.50
0.7	0.15	0.15	7.03	4.37	0.00	5.07	0.49

Bobot			Rata-rata				Standar Deviasi (2T + I + 0.1M)
Penalti Tardy	Penalti Tinta	Makespan	Makespan	Penalti Tinta	Penalti Tardy	(2T + I + 0.1M)	
0.7	0.2	0.1	7.03	4.27	0.00	4.97	0.45
0.7	0.25	0.05	7.03	4.60	0.00	5.30	0.50
0.75	0.05	0.2	7.03	4.30	0.00	5.00	0.47
0.75	0.1	0.15	7.03	4.47	0.00	5.17	0.51
0.75	0.15	0.1	7.03	4.30	0.00	5.00	0.47
0.75	0.2	0.05	7.03	4.37	0.00	5.07	0.49
0.8	0.05	0.15	7.03	4.43	0.00	5.14	0.50
0.8	0.1	0.1	7.03	4.43	0.00	5.14	0.50
0.8	0.15	0.05	7.03	4.63	0.00	5.34	0.49
0.85	0.05	0.1	7.03	4.53	0.00	5.24	0.51
0.85	0.1	0.05	7.03	4.53	0.00	5.24	0.51
0.9	0.05	0.05	7.03	4.50	0.00	5.20	0.51